

大模型分布式训练面试通关手册

从 AllReduce 到 DualPipe , 从 ZeRO 到 RLHF Rollout

duo.an

2026-09-03

目录

1	前言	9
1.1	这本书是写给谁的	9
1.2	这本书讲什么，不讲什么	9
1.3	怎么读	10
1.4	参考文献与致谢	10
1.5	记号约定	10
1.6	一句话导航	11
2	PyTorch 与 autograd：分布式训练的底盘	12
2.1	Tensor: storage / view / stride	12
2.2	Autograd: 图、Function、hook	12
2.3	Hook 类型速查	13
2.4	<code>torch.compile</code> : JIT 边界与分布式	14
2.5	DTensor: 分布式 tensor 抽象	14
2.6	CUDA stream / event / graph	15
2.7	<code>torch.distributed</code> : 从 backend 到 process group	16
2.8	常用调试/性能 API	16
2.9	面试考点	17
3	现代 LLM 架构：从 GPT-2 到 DeepSeek-V3	19
3.1	Transformer 骨架的 8 个模块	19
3.2	Pre-LN vs Post-LN：为什么现在都是 Pre-LN	19
3.3	LayerNorm vs RMSNorm	19
3.4	SwiGLU 与其他 FFN 变体	20
3.5	位置编码：从绝对到 RoPE	20
3.6	Attention 变体：MHA $\rightarrow$ MQA $\rightarrow$ GQA $\rightarrow$ MLA	21
3.7	MoE 层结构	21
3.8	Embedding tying / Untied	22
3.9	各模型的架构 diff 表	22
3.10	初始化 (init)	23
3.11	面试考点	23
4	Optimizer：从 SGD 到 Muon	25
4.1	更新规则通用形式	25
4.2	SGD + momentum	25
4.3	Adam / AdamW	25
4.4	LAMB	26
4.5	Adafactor	26
4.6	Lion (EvoLved Sign Momentum)	27
4.7	Muon (Kimi K2)	27
4.8	Sophia	28
4.9	Distributed Shampoo	28
4.10	Optimizer 与 ZeRO/FSDP 的相互作用	28
4.11	Optimizer 状态 offload	29
4.12	心算 optimizer 显存	29

4.13	面试考点	29
5	学习率调度：从 Cosine 到 WSD	31
5.1	为什么需要 LR schedule	31
5.2	warmup: linear 上升到 target LR	31
5.3	Cosine Annealing	31
5.4	Warmup-Stable-Decay (WSD)	32
5.5	Rewarming	33
5.6	Cyclical LR / SGDR	33
5.7	与 Batch Size 的关系：linear scaling / sqrt scaling	33
5.8	与 Model Size 的关系： μ P (Muon-friendly)	33
5.9	面试考点	33
6	数值稳定性 & Loss 病症学	36
6.1	训练三种典型故障 & 曲线特征	36
6.2	NaN 的 7 个主要来源	36
6.3	Loss 病症对照表	38
6.4	经典 loss spike 故事	39
6.5	BF16 vs FP16 vs FP8	39
6.6	具体调试脚本	40
6.7	Loss 病症学：面试答题模板	40
6.8	Grad clip 详解	41
6.9	面试考点	41
7	主流 recipe 速查：Llama 3 / DeepSeek-V3 / Qwen 3 / Kimi K2	44
7.1	Llama 3 (Meta, 2024)	44
7.2	DeepSeek-V3 (2024)	44
7.3	Qwen 3 (Alibaba, 2025)	45
7.4	Kimi K2 (Moonshot, 2025)	46
7.5	老一代 recipe 对比	46
7.6	快速对比表	47
7.7	你要背的 5 个数字	48
7.8	面试考点	48
8	集合通信与带宽模型	50
8.1	硬件层次：从 SM 到 IB	50
8.2	集合通信原语	50
8.3	带宽项 vs 延迟项：Message size 决定一切	53
8.4	通信量的三种度量	53
8.5	延迟分解	53
8.6	简易 profile：nsys 里怎么看通信	54
8.7	NCCL 调优 checklist	54
8.8	拓扑感知的进程排布	55
8.9	面试考点	55
9	显存、算力与三堵墙	57
9.1	参数量速查	57
9.2	显存组成：单卡训练的四头	57
9.3	算力速查	58
9.4	Roofline：一个 kernel 值不值得优化	59

9.5	三堵墙：训练系统的诊断框架	60
9.6	一个“能不能塞下”的心算 checklist	60
9.7	面试考点	61
10	Data Parallel：从 DP 到 DDP 到 gradient accumulation	63
10.1	<code>nn.DataParallel</code> 为什么被淘汰	63
10.2	DDP 的三大改进	63
10.3	Gradient Accumulation：显存换 batch	65
10.4	DP 的通信量分析	65
10.5	DDP 与 gradient scaling / AMP 的互动	66
10.6	一个“看似 DP，实际不是 DP”的坑	66
10.7	DDP 与 batch size：drop_last 陷阱	66
10.8	DDP vs FSDP：什么时候还用 DDP？	67
10.9	面试考点	67
11	ZeRO 与 FSDP：把模型切进 DP 组	69
11.1	ZeRO 的三个 stage	69
11.2	ZeRO 的通信增量	70
11.3	FSDP：PyTorch 原生实现	70
11.4	FSDP2 (torch/titan / PyTorch 2.4+)	72
11.5	ZeRO-Offload / ZeRO-Infinity：CPU / NVMe 扩展	73
11.6	与 gradient accumulation / activation checkpoint 的组合	73
11.7	ZeRO 与 TP 的分工	73
11.8	FSDP 常见坑	74
11.9	一个 FSDP 完整训练脚本骨架	74
11.10	面试考点	75
12	Tensor Parallel 与 Sequence Parallel	77
12.1	TP 的核心思想：一层 GEMM 里就切	77
12.2	通信量与效率	78
12.3	Sequence Parallel (SP)：把 LN 也切了	78
12.4	Async TP / TP overlap	80
12.5	Expert Tensor Parallel (ETP)	80
12.6	TP × PP × DP × EP 的拓扑排布	80
12.7	与 FSDP 组合	81
12.8	TP 的坑	81
12.9	面试考点	82
13	Pipeline Parallel：从零造一个出来	84
13.1	第一步：在进程边界上手工接链式法则	84
13.2	第二步：把 schedule 变成数据	85
13.3	第三步：P2P 的三个工程问题	87
13.4	第四步：把 backward 拆成 B 和 W	88
13.5	第五步：接进真实训练循环	89
13.6	调度族谱	92
13.7	PP 通信量	93
13.8	PP 与其他并行的组合	93
13.9	PP 的坑	94
13.10	面试考点	94

14	Context Parallel : Ring Attention, Ulysses 与长序列训练	97
14.1	为什么需要 CP : attention 的 quadratic 显存	97
14.2	Ulysses : All-to-All 沿 head 维交换	97
14.3	Ring Attention : K/V 沿环流动	98
14.4	USP: Unified Sequence Parallel (Hybrid)	99
14.5	FlashAttention v3 + CP	100
14.6	通信量对比表	100
14.7	Long-context 训练的其他关键点	100
14.8	SP/CP 的全栈适配 : attention 之外的所有 cross-token 模块	100
14.9	完整 CP 训练配置样例	109
14.10	面试考点	109
15	Expert Parallel 与 MoE 训练概览	111
15.1	EP 的核心 : 把 expert 沿 rank 切	111
15.2	EP $\times$ DP 的关系	111
15.3	通信量与 dense 的对比	112
15.4	与 CP 的组合 : MoE 长序列训练	112
15.5	EP 通信量公式速查	112
15.6	ETP: Expert Tensor Parallel	113
15.7	与 PP 的互动: MoE 层 vs Dense 层	113
15.8	常见配置模板	113
15.9	面试考点	114
16	精度 : 从 AMP 到 BF16 到 FP8 / FP4	115
16.1	数值格式速览	115
16.2	混合精度训练的 hierarchy	115
16.3	FP16 与 GradScaler	116
16.4	Transformer Engine 的 mixed precision	117
16.5	FP8 训练 : Hopper 的杀手锏	117
16.6	FP4 训练 (Blackwell)	118
16.7	BF16 训练的精度坑	118
16.8	Loss Scale (BF16 版)	119
16.9	面试考点	119
17	Activation Checkpoint 与 Offload	121
17.1	Activation 从哪来	121
17.2	Activation Checkpoint (recompute) 的基本思想	121
17.3	Selective Recompute	121
17.4	Fine-grained Recompute (DeepSeek / Megatron-Core MoE)	122
17.5	Activation Offload	122
17.6	Optimizer Offload	123
17.7	Precision-Aware Optimizer (Megatron-Core)	123
17.8	组合策略	123
17.9	Recompute vs Offload : 怎么选	124
17.10	面试考点	125
18	Overlap: 通信与计算的重叠	126
18.1	Overlap 的物理基础	126
18.2	Overlap 的分层 : 从 op 到 pipeline	126

18.3	逐 level 详解	127
18.4	Overlap 值不值得做	128
18.5	Overlap 的常见 bug	129
18.6	用 nsys profile overlap	129
18.7	一份 overlap checklist	129
18.8	面试考点	130
19	Data Loader, Packing 与 Sample Balancing	132
19.1	一个隐藏的 bottleneck	132
19.2	数据集抽象	132
19.3	Packing : 变长序列的显存/算力救星	133
19.4	Padding vs Packing vs No-pad (Bucket)	134
19.5	多源数据混合	134
19.6	Streaming: 从 S3 / HDFS 拉数据	134
19.7	长文档 vs 短文档 : DDP 里的 sample-level imbalance	135
19.8	开了 SP/CP 之后的 DataLoader	135
19.9	Multimodal Data Loader 的特殊难度	135
19.10	一个可用的 DataLoader 骨架	136
19.11	面试考点	137
20	多模态训练的特殊优化	139
20.1	多模态的结构	139
20.2	变长图像/视频的处理	139
20.3	Vision Encoder 与 LLM Decoder 的分工	140
20.4	Packing 在多模态里的困难	141
20.5	图像预处理的 dataloader 特殊坑	141
20.6	跨模态 sample balancing	142
20.7	Sequence length variance 在 attention 里	142
20.8	一个 VLM 分布式训练配置样例	142
20.9	多模态特有的稳定性问题	143
20.10	面试考点	143
21	RL 训练的分布式 : 从 PPO 到 GRPO 到 rollout/train 分离	145
21.1	从 PPO 说起	145
21.2	GRPO (DeepSeek): 干掉 value model	146
21.3	Rollout 是 inference workload : 与 training 完全不同	146
21.4	Rollout / Train 分离架构	146
21.5	Weight Sync: 从 training 到 rollout	147
21.6	主流 RL 框架架构	148
21.7	Long-context / Reasoning RL 的挑战	148
21.8	Reward Model 的分布式	149
21.9	KL divergence 的分布式计算	149
21.10	显存分析 : 一次 PPO iteration 的 GPU 占用	149
21.11	一个 GRPO 训练配置样例 (verl-like)	150
21.12	常见 RL 训练坑	151
21.13	面试考点	151
22	RLVR 与 Agentic RL 的分布式 : verifier 集群、多轮 rollout、长尾 straggler	153
22.1	一. RLVR : 把 reward model 换成 verifier	153

22.2	二. RLVR 的算法层：二值 reward 带来的新问题	156
22.3	三. Agentic RL 的 rollout 引擎	158
22.4	四. Agentic RL 的训练层	160
22.5	五. 环境作为分布式服务	162
22.6	六. 工业案例速查	163
22.7	七. 面试题	163
22.8	八. STAR 故事：把 RLVR + Agentic 串起来	165
22.9	九. 配套代码	166
23	工程稳定性：Checkpoint, 容错, 慢卡诊断	167
23.1	稳定性问题的规模	167
23.2	Checkpoint: 从秒级到分钟级	167
23.3	容错：从“节点挂了”到“训练继续”	168
23.4	训练发散 / loss spike 的检测和处理	169
23.5	监控指标 (Metrics)	170
23.6	一个 healthy training run 的 “signature”	170
23.7	MegaScale 的关键工程 tricks	171
23.8	面试考点	171
24	面试数学：从模型配置到 step time 的完整推导	173
24.1	记号约定	173
24.2	Estimator 1：参数量、显存	173
24.3	Estimator 2：FLOPs 和 step time (compute 部分)	175
24.4	Estimator 3：通信量 & 通信时间	176
24.5	组合：完整 step time 模型	178
24.6	Roofline 判定	179
24.7	端到端：从 model config 到 iter 时长	179
24.8	8 个 worked examples 一览	180
24.9	面试脚本模板	180
24.10	附：estimator 完整代码位置	181
25	附录 A：数字速查表	182
25.1	硬件带宽 / 算力	182
25.2	参数量与显存	183
25.3	FLOPs	183
25.4	通信量速查	184
25.5	Bubble & Overlap 收益	184
25.6	训练成本参考	184
25.7	长上下文常见数字	185
25.8	训练稳定性阈值	185
25.9	常用环境变量	185
26	附录 B：核心参考文献	187
26.1	集合通信与硬件	187
26.2	Data Parallel	187
26.3	ZeRO / FSDP	187
26.4	Tensor / Sequence Parallel	187
26.5	Pipeline Parallel	187
26.6	长上下文 / Context Parallel	188

26.7	MoE	188
26.8	精度 / FP8	188
26.9	Activation / Recompute	189
26.10	大规模系统	189
26.11	数据 / Packing	189
26.12	Multimodal	189
26.13	RL / RLHF	190
26.14	RLVR / Agentic RL	190
26.15	Scaling Laws	190
27	附录 D : 130 题面试题库	192
27.1	基础 / 通信 (Q1-Q15)	192
27.2	DDP / FSDP / ZeRO (Q16-Q30)	193
27.3	TP / SP / PP (Q31-Q50)	195
27.4	长上下文 / CP (Q51-Q60)	197
27.5	MoE / EP (Q61-Q70)	199
27.6	精度 / 优化 (Q71-Q80)	200
27.7	Overlap / 系统 (Q81-Q90)	201
27.8	多模态 / RL / 稳定性 (Q91-Q100)	202
27.9	RLVR (Q101-Q110)	203
27.10	Agentic RL (Q111-Q120)	205
27.11	SP/CP 全栈适配 (Q121-Q130)	206
28	附录 E : 面试故事 —— “框架都做了，你还能加什么”	209
28.1	STAR 框架：一个技术故事的结构	209
28.2	20 个可复用的“差异化优化”案例	209
28.3	“框架 X 已做，我加了 Y” delta 对比表	212
28.4	面试里怎么回答“MFU 只 30%，你怎么优化”	213
28.5	面试故事的 anti-pattern	214
28.6	一个完整的 STAR 故事示例	214
28.7	结语	214
29	附录 F : Megatron 上的改造 —— 教你回答“你们框架改了什么”	215
29.1	全景图：四个维度的改造清单	215
29.2	一. 算子层改造	216
29.3	二. 算法层改造	219
29.4	三. 稳定性改造	223
29.5	四. 监控改造	226
29.6	五. Infra + 数据/系统接入改造	230
29.7	六. RL 训练栈上的改造	232
29.8	七. 完整 STAR 故事：把四个维度串起来	234
29.9	八. 面试 anti-pattern：什么情况不能这么答	235
29.10	九. 一页速记：改造维度总表	236

前言

这本书是写给谁的

这本书是给那些已经能独立训通一个 **dense** 中等模型 (**1-10B**) 想拿分布式训练岗位 **offer** 的人写的面试通关手册。

在阅读之前你应该：

- 熟悉 PyTorch，能自己写一个 Transformer training loop。
- 对 CUDA、cuBLAS、NVLink、Infiniband 这些名词不陌生（不需要写 kernel）。
- 用过 `torch.distributed` 或至少跑过 `torchrun --nproc-per-node`。
- 知道 AMP / gradient accumulation / checkpoint 是什么，但可能没深挖。

如果你从来没接触过分布式，也可以直接读——第 1 章会把集合通信 + 带宽模型从零讲一遍。但节奏是“面试拿分”而不是“科普”，所以每一章都会直接给数字、公式、坑。

这本书讲什么，不讲什么

讲的：

1. 分布式训练的第一性原理：内存怎么算、通信怎么算、compute/comm 怎么 overlap，Roofline 拉到哪里。
2. 每种并行策略的核心动机 + 数学 + 通信模式 + **Megatron/DeepSpeed/FSDP** 的具体实现差别——DP, ZeRO 1/2/3, FSDP, TP, SP, PP, CP, EP。
3. 长序列并行：Ring Attention, Ulysses, USP hybrid，怎么支持 1M ctx。
4. **MoE** 训练：EP + all-to-all + DualPipe + DeepEP（本书简讲，深入部分请看姊妹卷《Sparse MoE 训练实战》）。
5. 精度：AMP / BF16 / FP8 (Transformer Engine, DeepSeek-V3 recipe) / FP4，loss scaling 与 outlier 处理。
6. 激活优化：selective recompute, activation offload, TE checkpoint, DeepSeek 的模块级 recompute。
7. **Overlap**：DDP bucket, ZeRO param gather, TP-SP overlap, PP overlap (1F1B / ZeroBubble / DualPipe / Megatron FWD-BWD merged)。
8. **Data loader**：packing (BFD / FFD / streaming), variable-length batching, IterableDataset shard 均衡, mosaicml streaming。
9. 多模态特殊问题：变长图像/视频/音频、encoder/decoder 隔离、跨模态负载不均、data-packing 的特殊难度。
10. **RL** 训练：RLHF/PPO/GRPO 的分布式，rollout/train 分离，vLLM/SGLang 集成，weight sync 的三种方案。
11. 工程稳定性：MegaScale 级 checkpoint、容错、慢卡诊断、监控指标。
12. 面试题库 + 面试故事模板：100 道高频题 + 20 个“框架都做了这些，你还能加什么”式差异化优化案例。

不讲的：

- CUDA kernel 手写（去看姊妹卷《CUDA Kernel 优化实战》）
- MoE 内部 router / gating / grouped GEMM 的细节（去看姊妹卷《Sparse MoE 训练实战》）

- 具体框架 API 的完整教程 (Megatron-LM / DeepSpeed 的官方文档已经写得很好, 我们只讲“怎么在面试里说清楚它做了什么以及为什么”)
- 推理 (vLLM / SGLang) 的深入优化——RL 章会讲一点, 但推理是另一本书

怎么读

- 顺序读: 第 1-2 章是所有后续章节的公共基础。第 3-8 章是并行策略, 可以按顺序深入。第 9-11 章是优化技巧, 正交于并行策略。第 12-15 章是工程实战。
- 对着 **profile** 读: 所有优化都对应 nsys/torch profiler 上的具体现象。手边打开一个 profiler screenshot 会更快。
- 把面试考点框读透: 每章末尾紫框是高频追问, 答案在正文里都有依据。
- 面试故事(黄框)是“你可以怎么用这个知识点在面试里讲一个 STAR 故事”的模板, 不是照抄的话术。核心的话你必须真的做过。

参考文献与致谢

本书内容主要综合了以下工作, 具体引用见附录 B:

- **DP / ZeRO** 家族: DeepSpeed (Rajbhandari 2020), ZeRO-Offload, ZeRO-Infinity, FSDP (Zhao 2023)
- **Megatron** 家族: Megatron-LM (Shoeybi 2019), Megatron-Turing NLG, Megatron-Core MoE (2026), Reducing Activation Recomputation (2022, sequence parallel)
- **Pipeline**: GPipe (Huang 2018), PipeDream (Narayanan 2019), Interleaved 1F1B (Narayanan 2021), ZeroBubble (Qi 2023), DualPipe (DeepSeek 2024)
- 长上下文: Ring Attention (Liu 2023), DeepSpeed-Ulysses (Jacobs 2023), USP (Fang 2024), Striped Attention (Brandon 2024)
- **MoE**: GShard (Lepikhin 2020), Switch (Fedus 2021), Mixtral, DeepSeek-V3 (2024), Megablocks (Gale 2022), Tutel (Hwang 2023), DeepEP
- 精度: Transformer Engine, DeepSeek-V3 FP8 recipe, MS-AMP
- 系统: MegaScale (Jiang 2024), MegaScale-MoE, Alpa (Zheng 2022), veScale, torchtitan
- **RL**: InstructGPT (Ouyang 2022), OpenRLHF, verl (ByteDance), TRL, DeepSpeed-Chat, NeMo-Aligner
- **Data**: Mosaicml Streaming, MosaicBench, Nemotron data pipeline, torchdata

记号约定

- B : micro-batch size; GBS: global batch size
- S / L : sequence length
- H : hidden dim; I : FFN intermediate dim
- A : attention heads; $d_h = \frac{H}{A}$: per-head dim
- L (在 layer 上下文里) 或 N_{layers} : Transformer layer 数
- N : 模型参数量 ($B = \text{billion}$ 参数)
- DP / TP / PP / CP / EP / SP: 各种并行的 world size
- $W = \text{world_size} = \text{DP} \times \text{TP} \times \text{PP} \times \text{CP} \times \text{EP}$ (视配置)
- bs: 以 bytes 计的精度大小 ($\text{BF16} = 2, \text{FP32} = 4, \text{FP8} = 1$)
- 硬件默认 H100 SXM5 80GB (HBM 3.35 TB/s, NVLink4 900 GB/s bidi, BF16 989 TFLOPS Tensor Core); 性能数字是量级估算, 实测请以 profile 为准。

一句话导航

如果你只有 30 分钟，先读：

1. 第 2 章 §“Roofline + 三堵墙”
2. 第 4 章 §“ZeRO-3 = FSDP，全书重点公式”
3. 第 6 章 §“1F1B vs DualPipe bubble 对比表”
4. 第 11 章 §“什么时候 overlap 值得做”
5. 附录 D 前 20 道高频题

翻页开始。

PyTorch 与 autograd：分布式训练的底盘

这一章讲面试里高频、但常常被“直接跳过”的 PyTorch 底盘知识。不重复官方 tutorial，只讲与分布式训练/性能强绑定的部分：Tensor 存储模型、autograd 图、hook 时机、`torch.compile` 边界、DTensor、CUDA stream/event、`memory_format`。理解这些，才能读懂 Megatron、FSDP、`torch.distributed` 的源代码。

Tensor: storage / view / stride

一个 Tensor 是“底层 Storage 的视图”。理解这一点能省下 90% 的“shape 对但值不对”的调试时间。

```
x = torch.arange(12).reshape(3, 4)      # shape=(3,4), stride=(4,1), storage=[0..11]
y = x.T                                  # shape=(4,3), stride=(1,4), storage 同上,
无 copy                                  # False
y.is_contiguous()                        # False
y.contiguous()                           # 触发 copy → 新 storage
```

为什么要 `.contiguous()`：NCCL send/recv、`all_gather` 底层都直接读取连续 storage；非连续张量会 `assertion fail` 或悄悄传错。生产代码里 P2P 前的 `.contiguous()` 都是必需。

view vs reshape：view 只做 stride 变换，要求连续；reshape 自动决定是否 copy。

```
x = torch.zeros(4, 8)
x.view(2, 16)          # OK
x.T.view(8, 4)         # RuntimeError: not contiguous
x.T.reshape(8, 4)      # OK (silent copy)
```

memory_format：channels-last (NHWC) 用于 CNN；LLM 场景不涉及。分布式训练里唯一相关的是 `.contiguous(memory_format=torch.contiguous_format)`——多数场景默认已是 contiguous。

面试考点. 面试题：“为什么 `all_gather` 前要 `.contiguous()`？非连续 tensor 会怎样？”答：NCCL 直接读 base pointer + stride=1 假设，non-contiguous 会传出乱序数据。PyTorch 新版本会 `assert`，旧版本静默出错。

进阶：“a2a 时输入 shape (B, S/W, A, d) 你怎么保证是 contiguous 的？”答：`.chunk(dim=2)` 出来的每片不连续（因为切的是 stride 大的维度），需要显式 `.contiguous()`；否则 `all_to_all_single` 会失败。

Autograd: 图、Function、hook

Autograd 是一张动态构建的有向图。每个 Tensor 有 `.grad_fn`（若是运算得到）指向创建它的 Function；Function 有 `.next_functions` 指向输入。

核心 API：

- `torch.autograd.Function` — 自定义前反向
- `Tensor.register_hook(fn)` — 该 tensor 有 grad 时触发（backward 阶段）

- `Module.register_forward_hook`, `register_full_backward_hook` — 模块级
- `Tensor.retain_grad()` — 非叶子 tensor 保留 `.grad` (默认 backward 完就丢)

hook 触发顺序 (面试常问):

```
backward starts (from loss)
→ for each Function in reverse:
    run Function.backward() → produce grad w.r.t. inputs
    for each input tensor:
        if it has registered hook → run hook with grad
    grad accumulates into input.grad (if leaf)
    if input is not leaf, propagate to input.grad_fn
```

DDP 的 **backward hook** 就是绑在参数 (叶子 **tensor**) 的 `.grad` 上。参数 `p` 的 grad 一算出, `p.register_hook(lambda g: bucket.mark_ready(p))` 立即触发, 然后 bucket 满就发 async AR。这也是为什么 `find_unused_parameters=True` 会破 overlap——DDP 无法确定某个 bucket 何时“齐全”, 只能等 backward 全结束。

自定义 **Function** 的正确写法:

```
class RingAllReduce(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, group):
        ctx.group = group
        dist.all_reduce(x, group=group)
        return x
    @staticmethod
    def backward(ctx, dy):
        dist.all_reduce(dy, group=ctx.group)
        return dy, None # 每个 forward 输入对应一个 grad
```

常见陷阱:

- `forward` 里 in-place 修改输入 tensor → autograd 报错, 除非在 `mark_dirty()`
- `backward` 返回的数量必须与 `forward` 输入的数量相同 (None 占位不需要 grad 的参数)
- `ctx.save_for_backward` 只能存 tensor; 其他用 `ctx.some_attr = value`
- `forward` 里的 `ctx.save_for_backward(x)` 会让 `x` 一直保活到 `backward`, 若 `x` 是大 activation → 显存占满

完整例子: **Ring AllReduce** 前向 = 后向对称 是 DP 通信的关键——正因为 `forward AR` 会让 grad 也需要 AR, DDP 才能只在 bucket 里发一次通信就完成“所有 rank 的 grad 求和”。

Key insight. 面试深挖: “如果你不显式做 **backward AR**, 会怎样?” 答: DDP 里就是靠 `Function` 的对称性——forward 时 replicate input (每卡拿全 batch 的分片), backward 时 grad 自动 AR。若在自定义 `Function.forward` 里做了 AR, 就必须在 `backward` 里也做——否则不同 rank 的 grad 会不一致, params drift。

Hook 类型速查

Hook	触发	常见用途
------	----	------

<code>Tensor.register_hook</code>	该 tensor 的 grad 计算好时	DDP、grad 监控、grad-clip unscale
<code>Module.register_forward_pre_hook</code>	forward 前	activation offload、input shape trace
<code>Module.register_forward_hook</code>	forward 后	activation checkpoint、 mem tracker
<code>Module.register_full_backward_hook</code>	该 module 的输入 grad 算好后	gradient magnitude tracker
<code>Module.register_state_dict_pre_hook</code>	<code>state_dict()</code> 保存前	checkpoint 剥离 buffer
<code>Module.register_load_state_dict_post_hook</code>	<code>load_state_dict</code> 后	FSDP 参数重构

torch.compile : JIT 边界与分布式

`torch.compile()` 用 TorchDynamo 抓取 python 字节码, 用 AOTAutograd + PrimTorch 展开成 op 序列, 用 Inductor 编译成 Triton kernel。收益: kernel fusion (10-30% speedup on H100)、消除 Python overhead。

与分布式的相互作用 (面试重点):

1. 通信不能 **trace**: `dist.all_reduce` 是 Python-side call, TorchDynamo 遇到会 **graph break** (切断编译图)。Torch 2.2+ 用 `torch._C._distributed_c10d._register_process_group` 让 NCCL op 变成 traceable, Torch 2.4+ 有 `torch._inductor.config.reorder_for_compute_comm_overlap` 可以在编译图里自动 reorder, 让 compute/comm overlap。
2. **DDP + compile**: 先 wrap DDP 再 compile → 每个 bucket 是一个 subgraph; 先 compile 再 DDP → 一个大图。生产用第一种。
3. **FSDP + compile**: Torch 2.3+ 支持, 需要 `use_orig_params=True`。仍有若干边界问题 (graph break at unshard), 预期 2026 稳定。
4. **TP + compile**: `torch.distributed.tensor.parallel` 的 DTensor 与 compile 一起可用 (torchitan 主线支持)。
5. 动态 **shape**: `torch.compile(dynamic=True)` 才能编译一次跑不同 shape, 否则每种 shape 都要重编——分布式训练 packing 场景常见的坑。

```
# 生产推荐 idiom
model = MyModel().cuda()
model = FSDP(model, use_orig_params=True, sharding_strategy=FULL_SHARD)
model = torch.compile(model, mode="reduce-overhead", dynamic=False)
```

Warning. `mode="reduce-overhead"` 使用 CUDA graphs, 与 CUDA stream 上的动态 P2P 通信不兼容。PP 场景要用 `mode="default"`。

DTensor : 分布式 tensor 抽象

Torch 2.1 引入。DTensor 把 sharding meta(“这个 tensor 在 dim=1 沿 tp 组切”)附加到 tensor 上, 让 op 自动推断需要什么 collective。

```

from torch.distributed.tensor import DeviceMesh, distribute_tensor, Shard,
Replicate

mesh = DeviceMesh("cuda", torch.arange(8).view(2, 4),
                  mesh_dim_names=("dp", "tp"))

W = torch.randn(4096, 4096)
W_dt = distribute_tensor(W, mesh["tp"], placements=[Shard(0)])
# W_dt: DTensor sharded on dim 0 across 4 TP ranks; each rank holds (1024, 4096)

y = W_dt @ x_dt # DTensor 会自动推断 Shard(0) @ Replicate → 需要 AllGather

```

为什么 **DTensor** 是趋势：

- Megatron/DeepSpeed 里 TP 是手写 `ColumnParallelLinear` / `RowParallelLinear` 每个 op；DTensor 让“把 tensor 沿某维切”变成声明式
- torchtitan、torchtn、FSDP2 全部基于 DTensor
- 与 `torch.compile` 集成好——因为 sharding meta 是 tensor 属性，编译期就能拿到

面试考点：“**DTensor** 与 **FSDP/TP** 手写实现相比，性能怎样？”答：目前 (2026 Q3) DTensor 的 latency 略高 (5% due to placement resolution)，但 torchtitan 已达到 Megatron 90% MFU。长期看 DTensor 会成为主流，因为工程复杂度低太多。

CUDA stream / event / graph

Stream：CUDA 内的执行队列，同一 stream 内串行，不同 stream 之间并行。默认 stream 是 `torch.cuda.default_stream()`。

Event：同步点。 `event.record(stream)` 打点， `stream2.wait_event(event)` 让 stream2 等到该 event 触发。

用法：让通信与计算 **overlap**：

```

comm_stream = torch.cuda.Stream()

# 主 stream 做计算 y = A @ B
# comm stream 做 AllReduce x

evt_a = torch.cuda.Event()
evt_a.record() # 记录 default stream 上当前进度

with torch.cuda.stream(comm_stream):
    comm_stream.wait_event(evt_a) # 等 default stream 到达此点
    dist.all_reduce(x, async_op=True)

y = A @ B # 与 AR 并行

torch.cuda.default_stream().wait_stream(comm_stream) # 用 x 前等 AR 完

```

这就是 Ch11 里 overlap 的物理机制。手写实现见 `src/distributed_training/11_overlap_2stream.py`。

CUDA graph : 把一段 stream 上的 kernel launch 序列固化成“图”，之后每次 replay 只需 1 次 launch overhead。收益：小 batch/短 seq 时 CPU launch overhead 可占 20-30%，graph 消灭它。torch.compile(mode="reduce-overhead") 底层用 CUDA graph。局限：**shape** 必须静态、控制流不能变。所以动态 packing / PP boundary 都不能上 graph。

torch.distributed : 从 backend 到 process group

torch.distributed.init_process_group(backend="nccl") 做的事：

1. 从 RANK / WORLD_SIZE / MASTER_ADDR / MASTER_PORT 读参数 (torchrun 帮你设好)
2. 与其他 rank rendezvous (TCP store)
3. 初始化 NCCL/GLOO backend

Process group : 一个通信组的抽象。默认组 = 全部 world。要做 sub-group AR(比如仅 TP 组内):

```
tp_group = dist.new_group(ranks=[0, 1, 2, 3])
dist.all_reduce(x, group=tp_group) # 仅在这 4 卡内 AR
```

Megatron/FSDP 内部维护多个 group : TP group, DP group, PP group, EP group , Rank ↔ 各 group 的映射由 mesh 决定。

NCCL communicator 内存 : 每个 process group 会创建一个 NCCL communicator , 占 100 MB HBM。你有 TP+DP+PP+EP+CP 5 个维度 → 5 个 comm × 5 个 group per dim = 25 个 group , 占几 GB。这就是“过多 sub-group 吃显存”的原因，实际会 lazy init。

超时 : init_process_group(timeout=timedelta(minutes=30)) ——默认 10 min ,长 checkpoint 保存或大 batch 慢 rank 场景需拉大。

常用调试/性能 API

API	用途
torch.cuda.memory_allocated()	当前 HBM 占用
torch.cuda.max_memory_allocated()	step 峰值, OOM 分析必用
torch.cuda.memory_summary()	详细 breakdown by size bin
torch.cuda.reset_peak_memory_stats()	每 step 前清零, 追踪峰值来源
torch.profiler.profile(...)	kernel timeline + memory + comm
torch._C._cuda_setSyncDebugMode(1)	任何隐式 sync 报错, 抓 dataloader 阻塞
TORCH_NCCL_ASYNC_ERROR_HANDLING=1	NCCL hang 时 raise 而不是死等
TORCH_DISTRIBUTED_DEBUG=DETAIL	打印每个 collective 的 shape/rank
torch.cuda.set_per_process_memory_fraction(0.9)	限制单进程显存, 防跑飞

生产模板：

```
export TORCH_NCCL_ASYNC_ERROR_HANDLING=1
export TORCH_DISTRIBUTED_DEBUG=DETAIL # 只在 debug 时
export NCCL_DEBUG=WARN # 平时; hang 时 =INFO
```



```
export NCCL_ASYNC_ERROR_HANDLING=1
export CUDA_LAUNCH_BLOCKING=0 # =1 只在 debug crash 时
```

面试考点

面试考点. Q1 : `register_hook` 何时触发? 与 `register_full_backward_hook` 有何不同?

A: `register_hook` 是 tensor 级, backward 走到该 tensor 时 (其 grad 被计算) 触发。
`register_full_backward_hook` 是 module 级, 输入 grad 全部算好后触发。DDP 用前者绑参数 grad; activation offload 用后者绑 module 输入。

面试考点. Q2 : `.view()` 和 `.reshape()` 什么区别? 分布式代码里用哪个?

A: `view` 要求连续, 只改 stride 不 copy; `reshape` 自动 fallback 到 copy。分布式代码 (NCCL) 强制要求连续, 一般显式 `.contiguous().view(...)`, 避免 `reshape` 里的“看似省事但会 silent copy 占显存”的坑。

面试考点. Q3 : `torch.compile` 遇到 `dist.all_reduce` 会发生什么? 如何 workaround?

A: 老版本 (< 2.2) 触发 graph break, 把编译图切成两段。新版本注册了 functional collective ops (`torch._C._distributed_c10d`), 可以 inline 进图。生产用 `torch.compile(mode="default")` + 让 comm 在 Python 层 (不 compile 的 wrapper) 里发; 或用 Torch 2.4+ 的 `reorder_for_compute_comm_overlap`。

面试考点. Q4 : CUDA stream 和 CUDA graph 的关系?

A: Stream 是执行队列 (decorate op 顺序 + 并行), graph 是“把 stream 上的一串 kernel launch 序列快照下来”。graph replay 消除 CPU-side launch overhead, 但要求 shape 静态。`torch.compile(mode="reduce-overhead")` 底层用 graph。

面试考点. Q5 : DTensor 与 Megatron 里的 `ColumnParallelLinear` 相比?

A: 语义等价。DTensor 是声明式 (“这个 tensor 沿 tp 切”, op 自动推断需要 AG/RS/AR); Megatron 是命令式 (每个 op 手写通信)。DTensor 更易读、易组合, 但当前 latency 略高, 正在追赶。torch titan 是 DTensor path 的旗舰实现。

面试考点. Q6 : `init_process_group(timeout=...)` 为什么重要?

A: 默认 10 min。如果 rank 之间 barrier / collective 未在 10 min 内会合, 会 raise。大规模训练里 checkpoint save 可能几分钟, dataloader 卡顿也常见 → 需要拉到 30-60 min。同时开 `TORCH_NCCL_ASYNC_ERROR_HANDLING=1`, 避免整个 job 死锁等 hang。

面试考点. Q7 : 你怎么知道你的 DDP overlap 生效了?

A: 三种验证:

1. `nsys profile` 看 timeline: `ncclAllReduce` kernel 与前面 `sgemm` kernel 在不同 stream + 时间叠加

2. `torch.profiler` 看 “GPU busy time” : 若 $\text{busy} \approx \text{wall-clock}$, comm 全 hidden
3. 手动实验 : `comm_stream + Event` 显式 overlap 与不 overlap 对比 iter time。

现代 LLM 架构 : 从 GPT-2 到 DeepSeek-V3

分布式训练面试常见问法：“你训过 Llama，那 DeepSeek 的 MLA 有什么不同？”或“为什么现在都不用 Multi-Head Attention，改 GQA / MLA 了？”答不上这些，就会被认为“只会调库”。这一章不教你怎么从零推导 attention（那是 Andrej 的视频），而是快速梳理业界主流 LLM 的关键选择、演进原因、对分布式训练的影响。

Transformer 骨架的 8 个模块

现代 dense LLM (Llama, Qwen, Mistral) 的一层长这样：

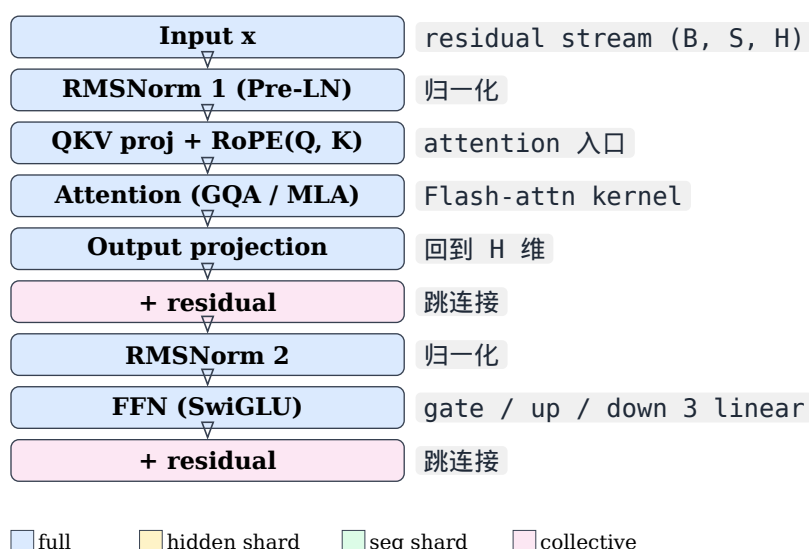


图 1 现代 dense LLM (Llama-3, Qwen-3, Mistral) 一层 transformer 的完整数据流。Pre-LN 结构：LN 在每个子层入口，残差在末尾 (add)。SwiGLU 是标准 FFN，MHA 现在少见——都换成了 GQA 或 MLA。

面试的每个模块都有可挑的点。逐个过。

Pre-LN vs Post-LN：为什么现在都是 Pre-LN

Post-LN (原 Transformer, BERT)： $x \rightarrow \text{attn} \rightarrow +x \rightarrow \text{LN}$ 。信号先加残差再归一，训练早期梯度大，需要 learning-rate warmup，深层难训。

Pre-LN (GPT-2 起)： $x \rightarrow \text{LN} \rightarrow \text{attn} \rightarrow +x$ 。归一化在残差之前，梯度更稳。GPT-2 之后所有 decoder-only LLM 全用 Pre-LN。

为什么 **Pre-LN** 好训：Xiong et al. 2020 证明 Pre-LN 的梯度 norm 在 layer 深度上保持 $O(1)$ ，Post-LN 是 $O(\sqrt{L})$ ——300 层 Post-LN 训不起来。

Sandwich Normalization (GLM-130B, Cogview)：Pre + Post 都加。缓解 Pre-LN 的 activation drift 问题（deep Pre-LN 中间层输出会累加变大）。DeepSeek-V3 用 Sub-Layer Normalization（类似 sandwich 的变体）抑制 MoE spike。

LayerNorm vs RMSNorm

LayerNorm (Ba 2016)： $y = \text{gain} \cdot \frac{x - \mu}{\sigma} + \text{bias}$ 。要计算 mean 和 var，两次 reduction。

RMSNorm (Zhang 2019) : $y = \text{gain} \cdot \frac{x}{\sqrt{\text{mean}(x^2) + \epsilon}}$ 。只算 x^2 的均值，一次 reduction。

为什么 **LLM** 全换 **RMSNorm** :

1. 速度快 10% (少一次 reduction + no bias)
2. Llama 论文实测 loss 曲线一致——没损失
3. 分布式训练里，SP 场景下 LN 需要跨 seq 组通信 mean/var，RMSNorm 只 mean(x^2)，通信更少

Llama-1 起全用 RMSNorm。Qwen, DeepSeek, Kimi K2, Mistral 全用。GPT-4 未公开但推测也是 RMSNorm。

eps 陷阱：eps=1e-5 是默认，BF16 下会出问题——因为 $\text{sqrt}(\text{var} + 1e-5)$ 在 var 特别小时精度不够。修法：eps=1e-6 甚至 1e-8，或 var 计算走 FP32 (upcast_var=True)。DeepSeek-V3 训练报告明确提到 Norm 计算强制 FP32。

SwiGLU 与其他 FFN 变体

传统 FFN : $W2 \cdot \text{GELU}(W1 \cdot x)$ 。两个矩阵乘。

GLU 家族 (Shazeer 2020) : $W3 \cdot (\text{act}(W1 \cdot x) \odot W2 \cdot x)$ 。三个矩阵乘。用 activation 选：

- ReGLU: ReLU
- GEGLU: GELU
- **SwiGLU: SiLU (= $x \cdot \text{sigmoid}(x)$)** ← 目前主流

参数量：SwiGLU 用 3 个 linear。为等参数量，Llama 把 inter_size 设为 $8\frac{H}{3}$ 而不是 $4H$ (乘 $\frac{3}{4}$ 抵消)。

为什么 **SwiGLU** 好：Shazeer 论文 empirical：同参数量下 SwiGLU 优于 ReLU FFN 约 1% perplexity。没有理论解释，就是“work”。

Llama, Qwen, Mistral, DeepSeek 全 SwiGLU。GPT-3 是 GELU。

位置编码：从绝对到 RoPE

绝对 (原 **Transformer**, **GPT-2**) : learned 或 sinusoidal position embedding，加到 token embedding 上。缺点：长度不能外推。

相对 (**T5**, **DeBERTa**) : attention bias。可以外推但慢。

RoPE (Su 2021, Llama) : 对 Q, K 应用旋转矩阵，让 attention score 天然带相对位置。

$$Q'_m = R_m \cdot Q_m, \quad K'_n = R_n \cdot K_n, \quad Q'_m \cdot K'_n = Q_m \cdot R_{n-m} \cdot K_n$$

其中 R_θ 是 2D 旋转矩阵。实际把 dim 拆成 pair，每对 $(2i, 2i+1)$ 用不同 base 频率 $\theta_i = 10000^{-2\frac{i}{d_h}}$ 。

为什么 **RoPE** 赢：

1. 无参数 (不占 gradient / memory)
2. 天然相对位置——外推可以 (虽然要调 base)
3. 与 KV cache 兼容——每个位置 rotate 一次，与 attention 内积交换律友好
4. 与 FlashAttention 完美兼容

RoPE base scaling (长上下文关键)：

- Llama-1: base = 10000, S = 2K
- Llama-2: base = 10000, S = 4K
- Llama-3: base = 500000, S = 8K

- Llama-3.1: base = 500000 + YaRN scaling, S = 128K
- Qwen 2.5: base = 1000000
- Kimi K2: NTK-aware + YaRN

长上下文 **RoPE** 三大方案：

1. Position Interpolation (Chen 2023)：把长 seq 的 position 缩放到 train 时的范围
2. NTK-aware / dynamic-NTK (bloc97)：让高频衰减，低频保留
3. **YaRN (Peng 2023)**：结合 NTK + temperature scaling + attention scaling，目前主流

Rope overflow：BF16 下 sin/cos 在 base=1e6 时精度差，需要用 FP32 计算 R 后 downcast。这是长上下文训练的隐蔽 NaN 源之一，见 P4。

Attention 变体：MHA → MQA → GQA → MLA

Multi-Head Attention (MHA)：A heads，每 head 独立 KV。参数量 = $4H^2$ 。KV cache size = $2 \cdot A \cdot d_h \cdot S \cdot b \cdot L = 2HS \cdot b \cdot L$ 。

Multi-Query Attention (Shazeer 2019)：KV 只 1 组，共 A 个 Q head 用。KV cache 缩 A 倍。快但质量下降。

Grouped Query Attention (GQA, Ainslie 2023)：折中——G 组 KV， $\frac{A}{G}$ 个 Q 共用一组 KV。 $G \in [4, 8]$ 主流。KV cache 缩 $\frac{A}{G}$ 倍。质量与 MHA 接近。

- Llama-2 34B/70B: G=8
- Llama-3 8B/70B: G=8
- Llama-3.1 405B: G=8
- Qwen 2.5 72B: G=8
- Mistral 7B: G=8

Multi-Head Latent Attention (MLA, DeepSeek-V2)：把 KV 压缩到低秩隐空间 $c_{kv} \in \mathbb{R}^{d_c}$ ， $d_c \ll H$ 。attention 时先 up-project 回 K, V 。参数减少、KV cache 极小（只存 c_{kv} ）

$$\begin{aligned} c_{kv} &= W_{DKV} \cdot x \quad (\text{down-project, cached}) \\ K &= W_{UK} \cdot c_{kv} + \text{RoPE} \quad (\text{up-project on the fly}) \\ V &= W_{UV} \cdot c_{kv} \end{aligned}$$

为什么 **DeepSeek** 选 **MLA** 而不是 **GQA**：

1. KV cache 更小（DeepSeek-V3 每 token KV = 3.4 KB, vs GQA-8 的 12 KB）
2. 训练侧无损（DeepSeek 论文对比 GQA-8 相同 KV budget，MLA 略好）
3. RoPE 需要特殊设计（K 分成 rotated 部分 + non-rotated 部分）——工程复杂

对分布式训练的影响：MLA 的 QKV projection 结构比 MHA 复杂——列并行需要特别处理（up-proj 权重也要沿 head 切）。DeepSeek 训练主要用 EP+FSDP，不用 TP（也无需处理 MLA 的 TP 切分）。

Sliding Window Attention (SWA, Mistral)：每 token 只看前 W 个 token（ $W = 4096$ ），KV cache 常数化。Mistral 7B 用；Llama/DeepSeek 不用（认为 quality drop）。

Attention Sink / Register (StreamingLLM, Xiao 2024)：观察 attention 会强烈聚焦在前几个 token（sink）。Streaming 场景保留 sink 位置 + 滑窗，实现无限长 inference。Kimi Chat 用类似机制。训练时若有 sink token（如 BOS 特殊 token）可显式建模。

MoE 层结构

Dense FFN → **MoE FFN**：每个 token 路由到 K 个 expert（K 通常 1-2），其余 expert 不激活。

模块：

1. **Gate / Router** : $\text{gate} = \text{softmax}(W_g \cdot x)$, 选 top- K 。Loss 项 : load balance loss (鼓励均匀路由)。
2. **Expert** : 普通 FFN。
3. **Dispatch / Combine** : 跨 EP rank 的 all-to-all (Ch8)。

Fine-grained expert : DeepSeek-V3 把 expert 数从常见 8 变成 256(每个更小)。经验 : **expert 更多 + 更小** 收益优于少而大。同参数量下前者 loss 更低。Kimi K2 用 384 experts。

Shared expert (DeepSeek-V2/V3) : 每层除了 routed experts 还有 1-2 个“共享 expert” , 每 token 都过。承担通用能力 , 减轻 routed expert 的负担。

Auxiliary loss :

- Load balance loss : $\text{lb-loss} = \alpha \cdot \sum_i (f_i \cdot P_i)$, f_i = expert i 被路由到的 token 频率 , P_i = 平均概率
- Router z-loss (ST-MoE) : $\log(\sum_i \exp(\text{logit}_i))^2$, 抑制 logit 幅度爆炸
- Bias-based load balance (DeepSeek-V3, no aux) : 给每 expert 加动态 bias , 无需 aux loss , 收敛更快

Embedding tying / Untied

Tied : `lm_head.weight = embedding.weight.T`。省 $V \cdot H$ 参数。Llama 系列 (7B/13B) tied ; Llama-3 起 untied (35B extra 参数 , quality gain 值得)。

分布式的坑 : tied weight 意味着 `.parameters()` 出来只有一份 , 但 grad 会累加两遍——DDP 里没问题 (都进同一个 bucket) , 但 FSDP 会 assert (同一 param 不能 shard 两次)。FSDP 用 `ignored_modules=[lm_head]` 或提前 untie。

各模型的架构 diff 表

模型	LN	FFN	Attn	PE (base) / Vocab	MoE
GPT-3 175B	Pre-LN	GELU	MHA	learned / 50K	—
Llama-2 70B	RMSNorm	SwiGLU	GQA-8	RoPE 10K / 32K	—
Llama-3 70B	RMSNorm	SwiGLU	GQA-8	RoPE 500K / 128K	—
Llama-3.1 405B	RMSNorm	SwiGLU	GQA-8	RoPE+YaRN / 128K	—
Mistral 7B	RMSNorm	SwiGLU	GQA-8+SWA	RoPE / 32K	—
Mixtral 8×22B	RMSNorm	SwiGLU	GQA-8	RoPE 1M / 32K	E=8, top-2
Qwen 2.5 72B	RMSNorm	SwiGLU	GQA-8	RoPE 1M / 152K	—
Qwen 3 235B	RMSNorm	SwiGLU	GQA-8	RoPE+YaRN / 152K	E=128, top-8

DeepSeek-V3 671B	RMS + Sub- LN	SwiGLU	MLA	RoPE+YaRN / 129K	E=256 + 1 shared, top-8
Kimi K2	RMSNorm	SwiGLU	MLA	RoPE+YaRN+NTK 163K	E=384, MuP scaled

Key insight. 面试快速识别 model family : “用了 **MLA** 且 **shared expert** = **DeepSeek** 系 ; 用 **SWA** = **Mistral** 系 ; **GQA-8 + RoPE-500K** = **Llama 3** 系”。这三点能覆盖 80% 模型。

初始化 (init)

Xavier / He : 早期 CNN 时代。LLM 不用。

Fixed std (GPT-2) : `init_std = 0.02`。所有 linear 都用此。

Scaled residual init (GPT-2, Megatron) : 残差路径上的 `W_o`, `W_2` 用 $\text{std} = \frac{0.02}{\sqrt{2L}}$ 缩放, 防止 activation norm 随层数爆炸。

Wang init (Llama) : `W_o` init $\text{std} = \frac{0.02}{\sqrt{2L}}$; 其他 linear $\text{std} = 0.02$ 。

Muon init (Kimi K2, 2025) : 与 Muon optimizer 配套。所有权重 near-orthogonal (SVD $s \approx 1$)。

Embedding init : `init_std = 1.0` (Llama-2 论文), 比其他小; 也有用 $\sqrt{\frac{1}{H}}$ 的。

位置 init (learned PE) : uniform 或 sinusoidal。RoPE 无 param 免疑。

RMSNorm gain init : `1.0`。Post-init 后每层 gain 会自然增长到 1.5-2.0。

为什么 **init** 重要 : GLM-130B 训练最初 loss spike 就是 init std 太大 + Pre-LN 累加 → 深层 activation 幅度爆炸。改小 init std 后稳定。

面试考点

面试考点. **Q1** : Pre-LN vs Post-LN , 为什么现在都是 Pre-LN ?

A : Xiong 2020 证明 Post-LN 梯度 norm 随深度 $O(\sqrt{L})$ 增长, Pre-LN $O(1)$ 。所以 Post-LN 必须 warmup + 精细 LR, Pre-LN 更 robust。GPT-2 起全部 Pre-LN。缺点 : activation 会累加 (sandwich LN 解决)。

面试考点. **Q2** : GQA 和 MLA 都省 KV cache , 什么时候选谁 ?

A : GQA 简单 (只是 replicate K/V head), 与 MHA 训练几乎无差别, 工程零成本; 缺点 : KV cache 只能缩到 $\frac{A}{G}$ 倍。MLA 用 low-rank cache 缩到极小, 缺点 : 需要 RoPE 特殊设计 (rotated + non-rotated 部分), 训练/推理代码复杂。想 push KV cache 极限 (比如 128K 长上下文 inference) 就上 MLA。

面试考点. **Q3** : 为什么 RMSNorm 能替换 LayerNorm ? RMSNorm 的 eps 为什么不能太小 ?

A : LN 减 mean 只是为了对齐 activation 分布, 实测在 residual 网络里“减 mean 效果不明显” (因为 residual 保持零均值) ——所以 RMSNorm 只除 std 就够。速度快 10%, quality

不降。eps 太小会让 BF16 下 $\sqrt{\text{var} + \text{eps}}$ 精度差，出 NaN。推荐 $\text{eps}=1\text{e-}5$ 时把 var 计算 upcast 到 FP32。

面试考点. Q4：RoPE 怎么外推到长上下文？三种方案？

A：

1. Position Interpolation (Chen 2023): 直接把 pos 除以 factor
 2. NTK-aware: 只缩小高频，保低频
 3. YaRN (主流): NTK + temperature scaling on attention logits + length-scale
- 外加改 RoPE base (10K $\rightarrow$ 500K $\rightarrow$ 1M) 从 pretrain 阶段就支持长上下文。

面试考点. Q5：DeepSeek-V3 为什么不用 TP？

A：MLA + fine-grained expert 结构下，TP 收益递减。DeepSeek 采用 $\text{EP}=64 + \text{FSDP} + \text{DualPipe}$ ：EP 切 expert 权重，FSDP 切 attention/embedding 权重，DualPipe 切层。避免了 TP 内的 AllReduce (TP AR 是所有并行策略里最频繁的通信)。同时全部 comm 走 NVLink+DeepEP (自研 all-to-all lib)，效率高。

面试考点. Q6：Shared expert 是什么？为什么有效？

A：MoE 层里除了 K 个 routed expert，还有 1-2 个“永远激活”的 shared expert，每个 token 都走。作用：承担通用能力 (通用语法、常识)，让 routed expert 专注于“细分能力”。DeepSeek 论文数据：加 1 个 shared expert 减少 routed expert 的 collapse，同参数量下 loss 更低。

面试考点. Q7：MoE 里 auxiliary loss 是什么？DeepSeek-V3 的“no aux loss”怎么实现的？

A：aux loss = load balance loss，鼓励 token 均匀分到各 expert。lb-loss = $\alpha \sum f_i P_i$ 。缺点：与 language loss 冲突，超参 α 敏感。DeepSeek-V3 用 **bias-based balancing**：每 expert 给一个动态 bias (EMA 更新)，bias 让“最近少被选的 expert”更容易被选，无需 aux loss。收敛更稳。

面试考点. Q8：tied embedding 和 untied embedding 各有什么好处？

A：Tied：省 $V \cdot H$ 参数 (Llama-7B 省 260M $\approx$ 3.7%)。Untied：embedding 与 lm_head 各自优化，quality 略高 (Llama-3 起改 untied)。分布式：tied 在 FSDP 里麻烦 (同 param 不能 shard 两次)，需 ignored_modules 或 untie。

面试考点. Q9：SwiGLU 相比 GELU 好在哪？为什么参数量看似多但要缩 inter_size？

A：SwiGLU 三个 linear (gate, up, down) vs GELU 两个 (up, down)，参数量 $1.5\times$ 。为等参数量，inter_size 从 $4H$ 缩到 $8\frac{H}{3}$ 。同参数量下 SwiGLU quality 略好 (+1% perplexity)。工程上多一个 linear 意味着多一次 gemm——但 fused kernel (如 xformers 里的 swiglu) 能压平差距。

Optimizer : 从 SGD 到 Muon

面试常问：“为什么 LLM 不用 SGD？”、“AdamW 里 weight decay 和 L2 有什么区别？”、“你听说过 Muon 吗？”—— 这章把工业界见过的 optimizer 都过一遍，重点在选择理由和与分布式的相互作用，代码实现见 `src/distributed_training/12_optimizers.py` (AdamW / Lion / Muon 手写并与 `torch.optim` 对拍)

更新规则通用形式

所有 first-order optimizer 长这样：

$$\theta_{t+1} = \theta_t - \eta_t \cdot \text{update}(g_t, \text{state}_t)$$

差异在于：

1. 用不用 momentum？
2. 用不用 second moment (Adam-style adaptive LR)？
3. update 是“raw gradient”还是“归一化后的 direction”？
4. weight decay 加在哪？(loss 里 $\rightarrow$ L2；update 里 $\rightarrow$ decoupled = “W” 后缀)

SGD + momentum

```
v_t = beta * v_{t-1} + g_t
theta_{t+1} = theta_t - lr * v_t
```

状态： v 1 份，与参数同 shape。总显存 = $2Pb$ (param + momentum)

为什么 **LLM** 不用：

1. 深 transformer 的 loss landscape 非凸/病态 (ill-conditioned)——不同参数的 gradient scale 差 10^3 级，SGD 无法自适应
2. 无 second moment 意味着 lr 需要精调每层每参数——不现实
3. CNN 时代 SGD+momentum + LR schedule 精调可以打 Adam，但 transformer 已经明确 “Adam 家族赢”

仍在用 **SGD** 的场景：(很少)

- 精调 CV 模型
- Segment Anything (Meta) 用 AdamW pretrain + SGD finetune
- 无 second moment 时显存少一半，工业界不太在意 (AdamW 显存也扛得住)

Adam / AdamW

```
m_t = beta1 * m_{t-1} + (1-beta1) * g_t          # 一阶
v_t = beta2 * v_{t-1} + (1-beta2) * g_t^2         # 二阶
m_hat = m_t / (1 - beta1^t)                       # bias correction
v_hat = v_t / (1 - beta2^t)
theta_{t+1} = theta_t - lr * m_hat / (sqrt(v_hat) + eps)  # Adam
# AdamW 追加: decoupled weight decay
theta_{t+1} = theta_{t+1} - lr * wd * theta_t
```

状态： m, v 各 1 份 FP32 = $8P$ bytes。加 master weight FP32 ($4P$) = **12 P**。这就是 Ch4 里“opt state = 12 P”的来源。

bias correction 的意义： $m_0 = 0$ ，所以 $m_1 = 0.1g_1$ ——第一步 update 幅度太小。除以 $(1 - \beta_1^t)$ 消除 warmup。生产代码里若开 LR warmup，bias correction 可省 (Adam-w-no-bias-correction 有人试过)。

AdamW 关键：decoupled weight decay：

- L2 regularization： $\text{loss} = \text{loss} + \text{wd} * ||\text{theta}||^2 \rightarrow \text{grad} + 2 \text{wd} * \text{theta} \rightarrow$ 进 m, v ，被 $\text{lr}/\text{sqrt}(v)$ 调节
- decoupled wd (AdamW, Loshchilov 2019)： $\text{theta} -= \text{lr} * \text{wd} * \text{theta}$ 直接减，不经过 m, v

为什么 **decoupled** 更好：L2 时 wd 会被 v 缩放——大 gradient 参数被少 decay (错的)；AdamW 让 wd 独立生效，公平地 shrink 所有参数。GPT-3 起 LLM 全用 AdamW。

hyper 建议 (LLM 默认)：

- $\text{betas} = (0.9, 0.95)$ (不是 0.999！LLM 数据非 iid， $\beta_2 = 0.999$ 收敛慢)
- $\text{eps} = 1\text{e-}8$ (BF16 下用 $1\text{e-}6$ 也可)
- $\text{wd} = 0.1$ (Llama), 0.01-0.05 (Qwen), 0.001 (small model)

为什么 $\beta_2 = 0.95$ 不是 **0.999**：LLM batch 内数据分布快速变化 (curriculum、packing)， $\beta_2 = 0.999$ 会让 v 反映远古的 gradient scale——不 adaptive。0.95 意味着 v 半衰期 ≈ 14 步，足够 track 局部 scale。

LAMB

You et al. 2020。为解决“大 batch 训练时 Adam 崩溃”设计。

```
m_t, v_t = Adam(...) # 同 Adam
r_t = m_t / (sqrt(v_t) + eps) + wd * theta_t # Adam-style direction
# LAMB 特色: layer-wise scaling
phi = ||theta_t|| / ||r_t|| # trust ratio
theta_{t+1} = theta_t - lr * phi * r_t
```

核心思想：每一层的 update 大小 = weight norm $\times$ direction。避免大 batch 时“小 layer 的 update 相对 weight 太大”。

适用：BERT 8K batch $\rightarrow$ LAMB 让 lr 大 $10\times$ 而不崩；Llama-1 报告曾试 LAMB，最终选 AdamW (他们没用极大 batch)。

现在的实际使用：Google TPU pod BERT 训练；Anthropic Claude 训练报告提到 LAMB-like；Meta Llama 系列都用 AdamW。工业界 LLM 主流仍 AdamW，LAMB 是备选。

Adafactor

Shazeer & Stern 2018。为省 optimizer memory 设计——Adam 需要 $O(P)$ 的 v ，Adafactor 用 rank-1 factorization 把 v 变成 $O(\sqrt{P})$ 。

```
V_t 存为 outer(r_t, c_t) # r_t: (rows,), c_t: (cols,)
```

显存节省：Adam 的 v 从 $4P$ 降到 $4\sqrt{P}$ ，对大 model 显著。T5 (11B on $4\times$ TPU) 因此可行。

代价：quality 稍降 (0.5% perplexity)，一些场景需要 external LR schedule。

现状：LLM 时代 ZeRO/FSDP 分片了 v ，Adafactor 显存优势不再突出。Google T5, PaLM 用；Meta/OpenAI 不用。

Lion (EvoLved Sign Momentum)

Chen et al. 2023 (Google Brain, symbolic search 找出来的)。极简。

```
# 只用一阶 momentum, 且只用 sign
c_t = beta1 * m_{t-1} + (1-beta1) * g_t
theta_{t+1} = theta_t - lr * (sign(c_t) + wd * theta_t)
m_t = beta2 * m_{t-1} + (1-beta2) * g_t      # 更慢的 momentum 用于下步
```

特点：

1. 只存一阶 m ，显存 = $8P$ （省一半 vs AdamW 的 $12P$ ；无 master weight 就只 $4P$ ）
2. `sign()` 让每次 update 的每维幅度都是 `lr`，天然稳
3. 无 $v \rightarrow$ 不 adaptive，但 `sign` 起了类似作用

超参：`(beta1, beta2) = (0.9, 0.99)`，`lr` 需比 **AdamW** 小 **3-10** 倍（因为 `sign` 输出 unit norm，AdamW 的 direction 是 $O(\frac{1}{\sqrt{v}})$ ，量级不同）。

实际效果：Google 报告 Lion 在 BERT / ViT / language modeling 上打平或略优 AdamW。工业界目前 few use，因为已有 AdamW 且 ZeRO 分片解决了显存问题。

Muon (Kimi K2)

Bernstein et al. 2024, Jordan et al. 2024 (Muon)。 μ -P 家族里的 optimizer，Kimi K2 (2025) 全量 pre-train 用。

```
# 只对"矩阵"权重 (linear.weight, embedding), 维度  $\geq 2$ 
m_t = beta * m_{t-1} + g_t          # heavy-ball momentum
U = orthogonalize(m_t)              # Newton-Schulz iteration on m_t
theta -= lr * U

# 对"向量"权重 (norm gain, bias, 1D): 仍用 AdamW
```

核心：orthogonalize 的 update。用 Newton-Schulz 5 次迭代把 momentum matrix m 归一化到 SVD 值全 1（近正交）。这让 update 每一层每一维的 scale 一致，不需 second moment。

$$X_{k+1} = 1.5X_k - 0.5X_kX_k^TX_k$$

（大约 5 次迭代收敛到 orthogonal）。计算成本： $O(H^3)$ per layer per step——用 BF16 GEMM 在 GPU 上飞快，Kimi K2 报告开销 <5%。

为什么“orthogonalize”work：直觉是 spectral norm bound——把 update 的 spectral norm 限制到 1，避免任何方向的 update 太大（Adam 的 second moment 本质也在做类似事，但更粗糙）。

显存：只需一阶 m + BF16 param = $4P + 2P = 6P$ ，比 AdamW 少一半。

适用：仅 2D 及以上权重（linear/embedding）。1D (LN gain, bias) 仍 AdamW。

- 参数 90%+ 是 linear $\rightarrow$ Muon 覆盖了大部分
- Muon 的效率增益随 model size 增长

Kimi K2 报告：Muon vs AdamW 同 flops 达到相同 loss，且 **loss curve** 更平滑（fewer spikes）。这是 2025 年 optimizer 领域的重要突破。

开源实现：https://github.com/KellerJordan/Muon —— Meta/Anthropic 未公开使用，DeepSeek/Qwen 还在 AdamW。

Sophia

Liu et al. 2023 (Stanford)。用 Hessian diagonal 估计做 second moment，取代 Adam 的 $v = g^2$ 。

核心：Adam 的 $\sqrt{v}$ 逼近 Hessian 对角线的一个不精确估计——为什么不直接估 Hessian？Sophia 用 Hutchinson trick 每 k 步 ($k = 10$) 估一次 Hessian diag，其他步复用。

```
h_t = hutchinson_hessian_diag(loss)  # every k steps
theta -= lr * clip(m_t / (h_t + eps), rho)
```

收益：Sophia 报告 $2\times$ faster than AdamW (reach same loss with half tokens)，但未复现——multiple labs 尝试都没能复现 $2\times$ 增益，Meta Llama 团队试后放弃。

现状：Sophia 是学术上很有吸引力的方向，但生产未用。面试遇到“你听说过 Sophia 吗”回答“了解，但未复现，业界仍用 AdamW”即可。

Distributed Shampoo

Anil 2020 (Google)。全 Hessian 二阶方法。为参数矩阵 $W \in \mathbb{R}^{m \times n}$ 分别维护 $L \in \mathbb{R}^{m \times m}$ 和 $R \in \mathbb{R}^{n \times n}$ 两个“factor”，update 时用 $L^{-\frac{1}{4}}gR^{-\frac{1}{4}}$ 。

显存： $O(m^2 + n^2)$ per matrix，可跨 DP shard。成本：矩阵求逆开销大——只更新 factor 每 $k = 100$ 步一次。

使用：Google 内部用 (PaLM 报告提到)，未开源标准实现。DeepMind 有类似。

现状：与 Sophia 类似——理论优雅，实际 few 使用。

Optimizer 与 ZeRO/FSDP 的相互作用

ZeRO-1：shard optim state (m, v , master weight)。每 rank 只更新自己那份 shard 的 param。AllGather 更新后的 param 回来 (每 step)。

ZeRO-2：还 shard grad。每 rank 只 ReduceScatter 自己那份 grad → 更新自己那份 param → AllGather param。

ZeRO-3 / FSDP FULL_SHARD：还 shard param。每层 forward/backward 前 AllGather，用完 free。optim step 仍在 shard 上做。

关键点：optimizer.step() 总是在 **shard** 上跑。这意味着：

1. Lion/Muon 等新 optimizer 若无 shard-aware 实现，会 assert
2. Muon 的 Newton-Schulz 需要整个矩阵——不能只在 shard 上跑！所以 Muon 与 ZeRO-3/FSDP 不兼容。Kimi K2 用 ZeRO-1 + Muon on full matrix per rank
3. 一些老 optimizer (Shampoo, K-FAC) 也有同样问题

Muon + FSDP 的 workaround：

1. Muon-DP：在 FSDP shard 内做 Newton-Schulz，效果打折
2. 定期全参 AllGather → Muon on rank 0 → broadcast 回来 (通信爆炸)
3. ZeRO-1 only (Kimi K2 选择) ——放弃 param/grad shard，只 shard optim state

Optimizer 状态 offload

Adam/AdamW 12P bytes 是训练最大显存消耗。生产手段：

1. **ZeRO-Offload** (DeepSpeed) : optim state 放 CPU , 反向后 stream 上来
2. **CPU-Adam** : 在 CPU 上做 Adam update (因为 update 逻辑 memory-bound , CPU 也能扛)
3. **NVMe-Offload** : NVMe SSD 存 optim state (DeepSpeed ZeRO-Infinity)
4. **8-bit optimizer** (bnb, Dettmers) : 把 m, v 量化到 8-bit , quality 无损 , 显存缩 4×

生产选择 : AdamW 8-bit + ZeRO-1 + gradient checkpointing 已经能训 70B on 8×80G。

心算 optimizer 显存

16P bytes = AdamW BF16 mixed precision 常见总数：

- 2P param BF16
- 2P grad BF16 (或 4P FP32 grad accum)
- 4P master weight FP32
- 4P m FP32
- 4P v FP32

若上 8-bit optim (m, v int8) : $= 2P + 2P + 4P + P + P = 10P$, 省 37.5%。若上 Muon : $= 2P + 2P + 4P + 4P = 12P$ (无 v)。若上 Lion : $= 2P + 2P + 4P + 4P = 12P$ (无 v , 同 Muon)。

面试考点

面试考点. **Q1** : AdamW 和 Adam+L2 有什么区别 ? 为什么 LLM 用 AdamW ?

A : L2 把 $wd * ||\theta||^2$ 加进 loss , wd 项进 grad 后被 $\sqrt{v}$ 归一化——大 gradient 的参数被少 decay (错的)。AdamW 把 $-lr * wd * \theta$ 独立减 , 让 decay 公平作用于所有参数。LLM 训练 $wd = 0.1$ 时二者相差显著。GPT-3 起全部 AdamW。

面试考点. **Q2** : 为什么 LLM 用 $\beta_2 = 0.95$ 而不是 0.999 ?

A : LLM 数据分布快速变化 (curriculum, packing) , $\beta_2 = 0.999$ 半衰期约 700 步 , 让 v 太“老” , 不 adaptive。0.95 半衰期 14 步 , track 局部 gradient scale。GPT-3, Llama, DeepSeek 全用 0.95。

面试考点. **Q3** : Muon 是什么 ? 为什么能打 AdamW ?

A : Muon 只用一阶 momentum + Newton-Schulz orthogonalize。update matrix 的 spectral norm 被限制到 1 , 等价于 per-direction 幅度均匀——比 Adam 的 diagonal second moment 更精细。显存少一半 (无 v) , loss curve 更平滑 (Kimi K2 报告)。缺点 : 只对 2D+ 权重生效 , 1D 仍需 AdamW ; 与 FSDP 不兼容 (Newton-Schulz 要整矩阵)。

面试考点. **Q4** : LAMB 相比 AdamW 好在哪 ? 现在还用吗 ?

A : LAMB = AdamW + layer-wise trust ratio。让每层 update 与 weight norm 比例一致 , 适合大 batch (BERT 8K batch 时 LAMB 让 lr 大 10× 而不崩)。现在 Llama/GPT 用适中 batch (1-4M tokens) + AdamW 已够 ; LAMB 更多在 TPU pod 训 BERT 场景。

面试考点. Q5 : 8-bit optimizer 怎么做到显存 4× 缩减而 quality 无损 ?

A : m 和 v 是低精度容忍的 (本身就是 exponential average)。bnb 用 block-wise quantization : 每 2048 元素一 block , 用一个 FP32 scale。Adam update 时先 dequantize、update、requantize。Dettmers 2021 论文实测 GPT-2 / GLUE 无损。工业界 (HF transformers , torchtitan) 主流选项。

面试考点. Q6 : ZeRO 分片 optim state 时 , 如果我用 Lion / Muon 需要注意什么 ?

A : Lion 只需一阶 m , shard 完全 OK , 与 ZeRO-1/2/3 兼容。Muon 的 Newton-Schulz 需整个 **matrix**——ZeRO-3/FSDP FULL_SHARD 下 param 分散在多卡 , 无法做 orthogonalize。Kimi K2 用 ZeRO-1 (只 shard optim state , param 复制)。若必须 FSDP , 只能用 Muon-DP 变体 (shard 内 approximate)

面试考点. Q7 : 为什么 Sophia 论文说 2× 更快但没人用 ?

A : Sophia 论文报告 2× faster to reach same loss。Meta / Anthropic / DeepSeek 内部尝试都未复现 2× (只 1.1-1.3×)。可能原因 : (1) 论文的 baseline AdamW 未调优 ; (2) Hutchinson 估 Hessian 噪声大 , 需要非常大 batch 才稳。目前无 open source 生产实例。面试遇到答“了解 , 未复现”即可。

面试考点. Q8 : 算算 70B 模型 AdamW BF16 optim state 显存 ? 8-bit 呢 ? Muon 呢 ?

A :

- AdamW BF16: $16 \times 70B = 1120 \text{ GB}$ (aggregate; ZeRO-3 分 100 卡后 11 GB / GPU)
- 8-bit AdamW: $2P + 2P + 4P + 1P + 1P = 10P = 700 \text{ GB}$
- Muon (BF16 param + FP32 master + FP32 m , no v): $12P = 840 \text{ GB}$

Muon 比 AdamW 少 25% , 比 8-bit AdamW 多 20%。8-bit + FSDP 是最激进的省显存组合。

学习率调度：从 Cosine 到 WSD

学习率是“训练里唯一的超参”——夸张但接近事实。数据、模型、optimizer 定了之后，schedule 的选择直接决定 loss 曲线形状。这一章从经典 warmup+cosine 讲到 2024 出的 WSD，配 `src/distributed_training/13_lr_schedules.py` 的可视化。

为什么需要 LR schedule

从零开始：初始化后，网络处于混乱状态，大 LR 会让 gradient 爆炸 → loss NaN。需要 **warmup** 慢慢提到目标 LR。

收敛后期：LR 太大让模型在最优点周围振荡，无法收敛到 sharp minimum。需要 **decay** 让 LR 逐步降低。

所以 schedule 的骨架 = `warmup → main phase → decay`。差异在于 main phase 与 decay 的形状。

warmup: linear 上升到 target LR

最简单：从 0 或小 LR 线性上升到 `peak_lr`，用时 `warmup_steps`。

```
def linear_warmup(step, warmup_steps, peak_lr):
    if step >= warmup_steps:
        return peak_lr
    return peak_lr * step / warmup_steps
```

长度：LLM 常用 500-2000 步 warmup。相当于第一个 `warmup_tokens = warmup_steps × batch_tokens`，Llama 是 2000 步。

为什么：pre-LN 网络的 gradient variance 在最初几步爆炸性大（未 update 过的参数 activation 分布不 stable）。warmup 让 optimizer state (m, v) 有时间“稳定”到合理值。若开 AdamW `beta_1 = 0.9`， m 半衰期约 10 步——warmup 少于此就没意义。

Warmup 长度对 **loss** 的影响：太短 → loss spike（见 P4）；太长 → 浪费 compute。经验：500 步够小 model (< 7B)，2000 步稳大 model。

Cosine Annealing

Loshchilov & Hutter 2017。GPT-2/GPT-3 起主流 schedule。

$$\text{lr}(t) = \text{lr}_{\min} + 0.5 \cdot (\text{lr}_{\text{peak}} - \text{lr}_{\min}) \cdot \left(1 + \cos\left(\pi \cdot \frac{t - t_{\text{warm}}}{T - t_{\text{warm}}}\right) \right)$$

形状：warmup 上到 peak → 沿 cosine 曲线降到 `lr_min`（通常 = $0.1 \times \text{peak_lr}$ 或 0）。

为什么用 **cosine** 不用 **linear decay**：Cosine 早期降得慢（保留学习强度），后期降得快（精细化）。Empirical 上 cosine 比 linear 稍好（0.5% loss）。原因不清，可能与 loss landscape 曲率相关。

总步数 **T** 必须提前定 这是 cosine 的最大缺点——若训练中途想“再多训 20%”，需要重跑 schedule 或接 rewarming。

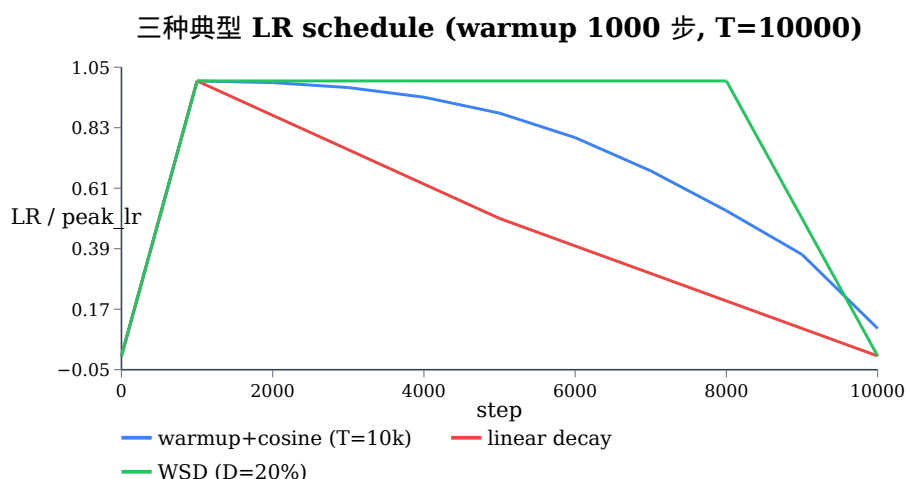


图 2 warmup+cosine 是 2020-2023 主流；WSD 是 2024 后新宠（保持 peak 更久，最后短 decay）。生成脚本 `src/distributed_training/13_lr_schedules.py`。

Warmup-Stable-Decay (WSD)

Hu et al. 2024 (MiniCPM), 后被 Kimi K2 / Qwen 3 / DeepSeek-V3 采用。

三段式：

1. **Warmup**：与 cosine 相同，比如 2000 步
2. **Stable**：保持 `peak_lr` 不变，占总步数的 80-90%
3. **Decay**：在最后 10-20% 步内 rapid decay 到 0 (linear 或 sqrt)

为什么 **WSD** 赢 **cosine**：

1. 不需提前定 **T**：stable 期间随时可决定“再训 100B token”——continual pre-training 友好
2. 中间 **checkpoint** 可用：cosine 的中间 ckpt LR 已很低，不适合做 base model；WSD 的 stable ckpt 是 fully-trained-at-peak 状态，可作 branch pretrain 起点
3. **loss** 略优：MiniCPM 论文 empirical: WSD 比 warmup+cosine 低 0.005-0.01 perplexity

decay 形状：Kimi K2 用 linear，MiniCPM 试过 $1 - \sqrt{x}$ ，二者接近。核心是“快速降到接近 0”，不像 cosine 一路缓降。

WSD-Rewarm (阶段续训)：

- 训了 100B tokens WSD (warmup 2K + stable 80B + decay 20B) → 得 model_1
- 继续训 100B tokens：从 stable 阶段 checkpoint 起 → warmup 短一点 (500 步) → 新一段 stable + decay → model_2
- 关键：从 **stable ckpt** 恢复，不是 decay ckpt——decay 完的 loss landscape 已“sharpen”，rewarming 效果差

现状：

- Kimi K2 (2025): WSD, 20% decay
- Qwen 3: WSD-like，加 curriculum
- DeepSeek-V3: WSD, 分阶段 decay
- Llama 3: 仍 cosine (保守)
- Mistral: cosine
- OLMo 2: WSD

WSD 是 2024-2025 事实标准替代 cosine。

Rewarming

若已训模型 M，想 continual pretrain 或 fine-tune，从 M 的最终 LR (通常很低) 直接继续 → loss 不动或退化。需要 **rewarming**：短 warmup 上到新的 peak (比原 peak 低 3-10×) → decay。

为什么需要：原训练结束时 optim state (m, v) 已适应“低 LR 小 update”。突然拉高 LR → gradient scale 陡增 → v 需要重新 adapt → loss spike。Warmup 500 步让 m, v 平滑过渡。

Rewarming 的 LR 选择：

- Continual pretrain (10% 原 token 量) : $\text{new_peak} = 0.3 \times \text{original_peak}$
- Fine-tune (SFT, 1B tokens) : $\text{new_peak} = 0.1 \times \text{original_peak}$
- RLHF : $\text{new_peak} = 0.01 \times \text{original_peak}$ (RL 目标不同, LR 极小)

Cyclical LR / SGDR

Smith 2017。周期性重启 cosine：一段 cosine annealing → 突然回到 peak_lr → 再 cosine annealing...

用途：CV 领域证明有效 (周期性重启帮助跳出 local min)，LLM 不常用 (loss landscape 平坦，重启效果差)。

Kimi K2 的 **mini-restart**：训到 60% 时做一次小 rewarming，实测有效防止 loss 停滞——这是 SGDR 的变体。

与 Batch Size 的关系：linear scaling / sqrt scaling

Linear scaling (Goyal et al. 2017, “Accurate large minibatch SGD”)：batch 增大 k 倍，lr 也要增大 k 倍，才保持训练动态一致。这在 CNN (ImageNet) 上被证实。

Sqrt scaling (LLM 领域)：batch k 倍 → lr 增大 $\sqrt{k}$ 。为什么不是 linear？

- LLM optimizer 是 Adam-style adaptive，本身对 batch size 变化有 buffer
- Linear scaling 在 LLM 大 batch (>4M tokens) 下会崩

实际 empirical: LLM lr $\propto \text{batch}^{0.5 - 0.7}$ 。GPT-3 / Chinchilla 都用类似经验规则。

Critical batch size (Kaplan et al. 2020)：超过某个 batch B_{crit} ，linear scaling 完全失效。 B_{crit} 随 model 大小、训练进度增大。Llama-3 405B 训练 batch 从 4M 逐步涨到 16M tokens。

与 Model Size 的关系： μP (Muon-friendly)

Yang & Hu 2022 (μP)。传统训练 LR 与 model size 有关 (大 model 通常小 LR)， μP ：一套 parametrization 让 LR 与 model size 完全无关——可以在 tiny model (300M) 上调 LR，直接迁移到 huge model (100B) 用相同 LR。

μP 需要的修改：

1. Init std scaling : $\propto \frac{1}{\sqrt{\text{fan-in}}}$
2. Attention scale : logits = $Q \frac{K^T}{d_h}$ (不是 $\sqrt{d_h}$)
3. Output projection 有额外 $\frac{1}{\text{width_mult}}$

现状：Cerebras, Kimi K2 报告用 μP 。Llama/DeepSeek 未明确用。

面试考点

面试考点. Q1：warmup 的作用是什么？没有会怎样？

A : AdamW 的 m, v 初始为 0 ,前几步 update 幅度小 + v 估计不准。若直接用 peak LR ,gradient scale 与 v 不匹配 $\rightarrow$ update 过大 $\rightarrow$ loss spike。Warmup 让 optim state 平滑升到“合理估计”。500-2000 步是 LLM 常规。

面试考点. Q2 : 为什么 WSD 逐渐取代 cosine ?

A :

1. Cosine 需提前定总步数 T ——continual pretrain / 中途扩训不友好
2. WSD stable 阶段 checkpoint 可作 base model ; cosine 中间 ckpt LR 太低
3. Empirical WSD loss 略优
4. Decay 阶段只 10-20% 步 , 快

MiniCPM 2024 first, Kimi K2, DeepSeek-V3, Qwen 3 全用 WSD 或变体。

面试考点. Q3 : Rewarming 是什么 ? 为什么直接从 M 的低 LR 继续训不行 ?

A : Rewarming = 短 warmup 到新 peak LR + decay。直接用 M 最终 LR 继续训 : optim state 已适应“小 update”, 新数据到来 gradient 会重新变大—— v 需要重新 adapt , 期间 loss spike / drop。Rewarming 短 warmup (200-500 步) 让 v 平滑升级。

面试考点. Q4 : batch size 从 1M 涨到 4M , LR 怎么调 ?

A : sqrt scaling : $\text{new_lr} = \text{old_lr} \times \sqrt{4} = 2 \times$ 。LLM 实测更接近 $\text{batch}^{0.6}$, 比 linear 保守。若 batch 超过 critical batch (Kaplan 2020) , 则完全失效——不再增益。Llama-3 405B batch 从 4M 涨到 16M 就是接近 critical。

面试考点. Q5 : LR 太大会怎样 ? 太小呢 ? 如何 monitor ?

A : 太大 $\rightarrow$ loss spike / NaN ; 具体表现 : grad_norm 突然 10-100 $\times$, weight_norm 也涨。太小 $\rightarrow$ loss 缓降但停滞 , grad_norm 稳定但小。Monitor :

1. grad_norm (每 step 记录) : spike $> 10 \times$ median 即 alarm
2. weight_norm per layer : 应该缓慢单调增长
3. loss 5-step moving average 与 baseline 对比

见 `src/distributed_training/l5_training_monitor.py`。

面试考点. Q6 : cosine 里 lr_min 通常设多少 ? 为什么不是 0 ?

A : $\text{lr_min} = 0.1 \times \text{peak_lr}$ 是主流。设成 0 会让最后几步 update 为 0——但如果训练还需继续(continual 或 SFT) , LR 为 0 意味着彻底停止学习。留 10% 让后续可以温柔续训。DeepSeek 用 $\text{lr_min} = 0.1 \text{ peak}$, Llama-3 也是。

面试考点. Q7 : μP 是什么 ? 为什么大厂开始关注 ?

A : μP (Yang & Hu 2022) 让最优 LR 与 model width 无关。传统训练里每次改 model size 都要重新扫 LR (grid search $O(N)$ runs) , 成本极高。 μP 允许在 300M model 扫好 LR , 直接迁移到 100B。省 $10^3 \times$ 的 hyper search compute。Cerebras 首推 , Kimi K2 采用。Llama/DeepSeek 未公开使用 , 可能内部已在用但没写论文。

面试考点. **Q8** : 训到 80% 时 loss 突然平了, 你怎么办?

A : 分层排查 :

1. 是否 data distribution shift ?(curriculum 到新阶段)
2. 是否 LR 已太小 ?(cosine tail 会 stuck) — 尝试 mini rewarming
3. 是否 optimizer state 已 saturate ? — 尝试重置 v 到 EMA of recent grad^2
4. Batch 是否太小已达 Chinchilla convergence ?
5. Model 是否已“记住”数据, 进入 overfit ? — 加 wd, dropout, 或增数据

典型 rewarming 方案 : $\text{LR} \times 0.3 \rightarrow \text{warmup } 200 \text{ 步} \rightarrow \text{继续 stable}$, 一般 loss 立即恢复下降。

数值稳定性 & Loss 病症学

面试真实场景：“上次训练 loss 突然 fly up，跳到 20，然后过一会又回来了，你觉得是什么原因？”这一章系统整理 loss 病症 (spike / drop / NaN / plateau)，每种病症都给出：成因 → 如何诊断 → 如何修复 → 经典案例。配 `src/distributed_training/14_nan_reproducer.py` (可复现 5 种 NaN) 和 `15_training_monitor.py` (监控工具)。

训练三种典型故障 & 曲线特征

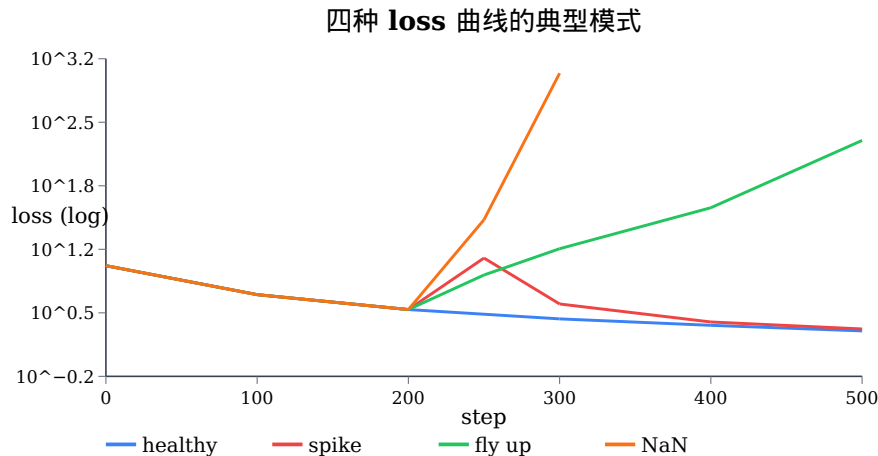


图 3 Healthy: 单调下降。Spike: 尖峰后回落 → gradient outlier 或 data corruption，通常自愈。Fly up: 持续上涨 → LR 太大 / init 差 / 优化 diverge，不可自愈。NaN: 突然 999+ → BF16 overflow / softmax overflow / grad explode / div-by-zero。

NaN 的 7 个主要来源

1. Softmax overflow

`softmax(x)` 计算 $\exp(x - \max(x)) / \sum(\dots)$ 。若 `max(x)` 特别大 (比如 attention 里 $Q \frac{K^T}{\sqrt{d_h}}$ 出 500)，BF16 里 `exp(500)` 早已溢出到 inf。

触发场景：

- Attention logit 出 outlier
- LM head logit outlier (未 pre-norm)
- Router logit outlier (MoE)

诊断：hook 到 softmax 输入，看 max value 是否 > 60 (BF16 exp 上限 ≈ 88 ，但 sum 之后除法可能 inf/inf = NaN)。

修复：

- Attention: FlashAttention 内部就有 online softmax，天然稳。若用 SDPA 也有 safe softmax
- QK-norm (Chameleon, Cogview): 对 Q, K 分别 LayerNorm，把 QK^T 幅度控在 $[-10, 10]$
- Router: z-loss (ST-MoE): 抑制 logit 幅度
- LM head: 训练时 upcast to FP32 (with `torch.autocast(enabled=False)`)

2. RMSNorm/LN 的 sqrt 域

`sqrt(mean(x^2) + eps)`。若 `mean(x^2)` 特别小 ($\ll \text{eps}$) 但也不算 0 $\rightarrow$ BF16 下 `sqrt(1e-8)` 精度问题；若 `mean(x^2)` 特别大 $\rightarrow$ BF16 sum overflow $\rightarrow \text{inf} \rightarrow \text{sqrt}(\text{inf}) = \text{inf}$ 。

诊断：训练开始时 hook LN 输入，看每层 `mean(x^2)` 分布。

修复：

- var 计算 upcast FP32：`var = x.float().pow(2).mean()`
- Sub-Layer Normalization (DeepSeek-V3)：activation 累加后再 LN，抑制 activation drift
- Weight decay 稍大点 (0.1)：防止 weight 无限膨胀让 activation 变大

3. Attention $Q \cdot K^T$ 的 BF16 accumulation

Q, K 是 BF16。 QK^T 里内积 = sum 上千项 BF16 乘积。BF16 mantissa 7 位，累加时后加项被 truncate。真值 $100 + 0.01 \rightarrow$ 存 100 (0.01 丢失)。累积几百步就漂移。

触发：QK-norm 未开，且序列长 ($S > 8k$)。

诊断：nsys 里看 `flash_fwd_kernel` 是否用 FP32 accumulator。

修复：

- FlashAttention v2/v3 内部 accumulator = FP32 (默认已开)
- 手写 attention: Q @ K.T 前 `Q = Q.float(), K = K.float()`
- 或用 `torch.matmul(Q, K.T, out_dtype=torch.float32)` (Torch 2.4+)

4. Gradient explosion / vanishing

反向路径 gradient 每层缩放 $g_{l-1} = J_l \cdot g_l$ 。若 spectral norm $> 1 \rightarrow$ 逐层爆炸 $\rightarrow \text{grad_norm}$ 数千 $\rightarrow \text{NaN}$ 。

触发：

- init 太大 (Xavier 应用错误)
- 深度 Pre-LN 但无 residual scale (用 Wang init 修)
- Activation function 有 exponential region

诊断：

- `torch.nn.utils.clip_grad_norm_` 返回值 $\rightarrow$ 每 step 记录，画曲线
- 应该在 median 0.5-2, spike 时 20-100
- `grad_norm > 1000 \rightarrow \text{NaN}` 边缘

修复：

- Grad clip: `clip_grad_norm_(model.parameters(), max_norm=1.0)` ——LLM 标配
- Warmup 拉长
- Init std 缩小
- Wang init on residual output projection

5. `sqrt(v) + eps` division

Adam 里 $m / (\text{sqrt}(v) + \text{eps})$ 。若 $v = 0$ 且 $m \neq 0$ (第一步 update，且 grad 巧合非零但为 rare 值)， $\frac{m}{\text{eps}}$ 会极大。

触发：

- 训练第一步 (bias correction 只能补救不完全)
- Resume 时 optim state 损坏

修复：

- 用 `eps = 1e-8` (default) 通常够
- Warmup 前几步不做 param update (可选)
- Resume 时验证 optim state hash

6. Loss function 的 $\log(0)$

Cross-entropy: $-\log(p)$ 。若 model 对 target 极度 unconfident $\rightarrow p \approx 0 \rightarrow \log(0) = -\text{inf}$ 。

触发：

- Label smoothing 未开且 model 出错
- Vocab 太大且 logit distribution 无 mass 在 target 上

修复：

- PyTorch `nn.CrossEntropyLoss` 内部用 `log_softmax (safe)`, 不会真的 $\log(0)$
- 若手写 loss, 一定用 `F.log_softmax` 而不是 `torch.log(F.softmax(x))`

7. RoPE 里的 $\sin/\cos$ overflow

RoPE base = $1e6$ 时, $\theta = \text{pos} * \text{base}^{(-2i/d)}$ 在 $d=128$ 中 θ 覆盖 $[1e-6, 1]$ 。
 $\sin/\cos$ 在 BF16 下精度 3 位有效数字——position 大时相位精度不够。

触发：长上下文训练 ($S > 32k$) + base 大 ($1M+$)

诊断：hook RoPE 输出，看 `Q_rot` 是否有异常大值

修复：

- RoPE 部分 FP32: `Q_rot = (Q.float() * cos.float() + rotate_half(Q).float() * sin.float()).to(Q.dtype)`
- YaRN scale 会自动缓解 (本来就要 upcast)

Loss 病症对照表

病症	何时出现	grad_norm 特征	可能原因 (按概率)	快速修复
Spike (自愈)	任意	1 step 20-100×	data outlier; 数值 rare event	skip update; grad clip 1.0
Fly up	warmup 后	持续 10× median	LR 太大; init 差; 优化 diverge	重启 + LR $\times 0.5$ + warmup $\times 2$
NaN 短时	任意	NaN 前一步 100×	softmax overflow; grad explode	grad clip + upcast attn + skip step
NaN 持续	restart 之后	重启即 NaN	ckpt 损坏; optim state 坏; precision 变	从更早 ckpt 恢复
Loss 平坦不降	训练后期	稳定但很小	LR 太小; 数据重复; capacity 满	rewarming; 换数据; 扩 model
Loss 慢升	训练中期	median 缓涨	BF16 累积 drift; weight explode	wd 拉大; norm eps $\uparrow$; upcast LN
Loss 阶段跳变	curriculum 换	瞬间 2-3×	数据阶段切换; SFT 起步	rewarming 500 步; 查 data mix

经典 loss spike 故事

GLM-130B (2022) : 训练前期频繁 spike。原因 pre-LN 深层 activation drift + init std 偏大。Fix: Sandwich Norm + emb layer normalization + init std 缩到 0.02 → stable。

OPT-175B (Meta 2022) : 训练日志里 45+ 次手动 restart。原因不同 : hardware failure, loss spike, dataloader stall。Meta 论文 “Training the OPT 175B” 完整记录。教训 : **checkpoint** 频繁 (每 500 step) + 自动 **restart** 机制。

PaLM (Google 2022) : 出现 loss “step function” 上跳。原因未公开细节 , Google 用 “skip and retry” 策略 : spike 前 200 步的 ckpt 恢复 + skip 出问题的 batch。

Llama 2 (Meta 2023) : 报告全程 loss curve smooth , 但内部承认前几次尝试有 spike。修复 = data cleaning (删损坏 batch) + grad clip 1.0。

DeepSeek-V3 (2024) : 宣称 “training was remarkably stable”。作者归因 : Sub-Layer Norm + no-aux-loss balancing + Muon-style init scaling + BF16 with FP8 for weight only。

Kimi K2 (2025) : 宣称 Muon optimizer 让 loss curve 显著更平。对比图里 AdamW 有 3 次可见 spike , Muon 0 次。

BF16 vs FP16 vs FP8

format	exp	mantissa	range	LLM 用途
FP32	8	23	1e-38..1e38	master weight, m, v, LN var, loss
FP16	5	10	6e-5..65504	训练主 dtype (V100 时代)
BF16	8	7	1e-38..1e38	训练主 dtype (A100+)
FP8 E4M3	4	3	1e-9..448	forward activation (H100+)
FP8 E5M2	5	2	1e-15..57344	backward gradient (H100+)
MXFP4	—	—	block-scaled 4-bit	推理为主 (2025)

BF16 vs FP16 : BF16 range 同 FP32 (exponent 8 bit) , 精度差 (mantissa 7 bit)。FP16 range 小 (5 bit exp) 但精度稍好。LLM 全用 BF16 : range 广不需要 loss scaling。

FP16 陷阱 :

- Range 只到 65504——activation 大就 overflow
- 需要 loss scaling (GradScaler) : `loss.backward()` 前先乘 2^{16} , 让 grad 落在 FP16 表示范围内

BF16 陷阱 :

- Mantissa 只 7 bit——大数累加时小加项被 truncate
- LN 里 `sum(x^2)` 累加 4k 项时会 drift (用 FP32 upcast 修)
- Adam 里 `sqrt(v)` 精度差

FP8 陷阱 (H100+ 才有):

- range 更小 ; activation 需要 dynamic scaling
- Transformer Engine 用 “delayed scaling” 或 “current scaling” 追踪
- Backward gradient 常用 E5M2 (range 大) , forward 用 E4M3 (精度略高)
- 长训练需要 scale statistics stability , 容易出 hidden NaN

具体调试脚本

Step 1: 定位到 layer

```
def register_nan_hook(model):
    for name, module in model.named_modules():
        def hook(module, input, output, name=name):
            if isinstance(output, torch.Tensor):
                if torch.isnan(output).any() or torch.isinf(output).any():
                    print(f"NaN/Inf at {name}, input stats:")
                    for i, inp in enumerate(input):
                        if isinstance(inp, torch.Tensor):
                            print(f"  input[{i}]: min={inp.min()}, max={inp.max()},
                                "
                                    f"has_nan={torch.isnan(inp).any()}")
                    raise RuntimeError(f"NaN at {name}")
            module.register_forward_hook(hook)
```

跑一步就能定位到哪个 module 先出 NaN。

Step 2: 决定是 forward 还是 backward

- forward hook 抓不到 → NaN 在 backward。
- 加 `torch.autograd.set_detect_anomaly(True)` (慢 5-10×, 只 debug 时开), 能定位反向哪个 op 出问题。

Step 3: 决定是 数值 vs 数据

- 用同 seed 重跑相同 batch → 若 reproduce → 数值问题
- 若不 reproduce → data corruption / race condition

Step 4: 保存 poisoned batch

```
try:
    loss = model(batch)
    loss.backward()
except RuntimeError as e:
    torch.save(batch, "poisoned_batch.pt")
    torch.save(model.state_dict(), "poisoned_model.pt")
    raise
```

后续在小 machine 上 reproduce + 修。

Loss 病症学：面试题模板

面试常见问法：“上次训练 loss fly up, 你怎么办？” 结构化答法：

Step 1: 观察阶段 — 描述当时看到的现象

- 具体数值 (loss 从 3 跳到 20, 5 steps 内)
- grad_norm 表现 (是否也跳)
- 是否 1 次事件还是持续
- 训到了哪 step, 什么阶段

Step 2: 假设分层 — 按概率列可能原因

- LR/optimizer (最可能：warmup 不够 / LR 太大 / batch 变了没调 LR)

- 数据 (batch corrupt / curriculum 切换)
- 数值 (softmax overflow / BF16 drift / RoPE overflow)
- 硬件 (ECC error / NCCL 通信错乱)

Step 3: 诊断动作

- 立即 grep 日志 : `grep "grad_norm" log`
- 查 checkpoint 之前的 loss 曲线
- 保存 poisoned batch
- 试 reproduce (同 seed + 同 batch)

Step 4: 修复动作

- 短期 : 从前 200 step 的 ckpt 恢复 + skip 出问题的 batch + grad clip
- 中期 : 如果 fly up 复发 $\rightarrow$ $LR \times 0.5 + \text{warmup} \times 2 + \text{upcast attention}$
- 长期 : 加 monitor (grad_norm, act stats), 改用 WSD 更 robust

面试官关心的是你的思维结构——不是背下所有可能原因，而是给出可复用的分层排查框架。

Grad clip 详解

```
# 标准 idiom
total_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
if total_norm.isnan() or total_norm.isinf() or total_norm > 100:
    # skip this update
    optim.zero_grad()
    logger.warning(f"skip step: grad_norm={total_norm}")
    continue
optim.step()
```

max_norm 选值 :

- LLM pretrain : 1.0 (Llama)
- Fine-tune : 1.0 或 0.5
- RLHF : 0.5 (梯度更 noisy)

clip 的实现 : `clip_grad_norm_` 计算所有 param 的 grad L2 全局 norm , 若 $> \text{max_norm}$, 等比例缩到 max_norm 。不是 per-param clip。

clip on unscaled grad : AMP + GradScaler 时 , clip 必须在 `scaler.unscale_(optim)` 之后 :

```
scaler.scale(loss).backward()
scaler.unscale_(optim)           # ← 先 unscale
clip_grad_norm_(model.parameters(), 1.0) # ← 再 clip
scaler.step(optim)              # scaler 会 skip 若有 inf
scaler.update()
```

面试考点

面试考点. **Q1** : Loss NaN , 你从哪里开始查 ?

A : 分四步 : (1) 定位 layer : forward hook 抓第一个出 NaN 的 module ; (2) forward vs backward: forward hook 不触发就 `set_detect_anomaly(True)` ; (3) 数值 vs 数据 : 同 seed 重跑相同 batch reproduce ? (4) 保存 poisoned batch 到小机器上 debug。

面试考点. **Q2** : BF16 与 FP16 有什么区别? 为什么 LLM 都用 BF16?

A : BF16 exp=8 bit (同 FP32 range), mantissa 7 bit; FP16 exp=5 bit (小 range), mantissa 10 bit. LLM activation 会很大 (attention logit 100+), FP16 溢出需要 loss scaling; BF16 天然覆盖 → 不需要 GradScaler. 代价: BF16 精度低, 长累加会 drift (LN var 用 FP32 fix). A100+ 有硬件 BF16 支持后成主流。

面试考点. **Q3** : Loss 突然 spike 然后回落, 正常吗?

A : 偶发 **spike** 正常——data 里有 outlier batch 或数值 rare event. 表现: 1 step 内 grad_norm $100\times$, loss $5-10\times$, 下一步就恢复. 修: 加 grad clip 1.0, 或加“若 grad_norm > 100 skip step”逻辑. 持续 **spike** (每 100 step 一次) → 系统性问题 (LR 太大 / init 差 / data 有 systematic corruption).

面试考点. **Q4** : LR 突然 fly up (不 NaN, 但持续涨), 怎么办?

A : 立即 rollback 到 spike 前 200 step 的 ckpt, 然后:

1. Diagnose : grad_norm 是否也持续涨? (是→优化 diverge; 否→数据/loss function 问题)
2. LR $\times 0.5$, warmup 步数 $\times 2$ 重训
3. 若仍 fly up: 换 init (Wang init on residual)
4. 若仍 fly up: 换 optim (AdamW → Muon or Lion)
5. 若仍 fly up: 换 model 结构 (Sub-LN 抑制 activation drift)

面试考点. **Q5** : 为什么 attention 里要用 QK-norm?

A : $Q \frac{K^T}{\sqrt{d_h}}$ 的每 entry = sum d_h 项 dot product. BF16 下 sum 会 drift → 出现异常大 entry (500+) → softmax overflow → NaN. QK-norm (Chameleon): 对 Q, K 分别 LayerNorm, 把 QK^T 幅度控在 $[-10, 10]$. Meta / Chameleon / Grok 都用. DeepSeek-V3 不用 (改 upcast MLA 内部矩阵).

面试考点. **Q6** : 训了 500B tokens 后 loss 突然开始缓慢上涨 (不 spike), 怎么办?

A : 典型“BF16 累积 drift”: weight norm 慢慢涨→activation 慢慢涨→LN var 慢慢涨→BF16 sqrt 精度慢慢差. 诊断: monitor per-layer `weight.norm()`, 若持续单调涨且已 $> \text{init } 3\times$ → drift. 修: (1) 加 weight decay (0.1); (2) LN var 计算 upcast FP32; (3) 若已上 FP8, 检查 scaling factor 是否 stale.

面试考点. **Q7** : FP8 训练稳定吗? 你会开吗?

A : H100+ FP8 训练是 2024-2025 生产成熟技术, DeepSeek-V3、Kimi K2 都用 (weight 存 BF16, compute 用 FP8). 关键:

1. E4M3 for forward activation, E5M2 for backward grad
2. Delayed scaling (每 step 追一次 max) 或 current scaling
3. Transformer Engine (TE.Linear) 已封装
4. Loss curve 与 BF16 几乎一致 (差 $< 0.5\%$ perplexity)

开 FP8 的收益 : H100 上 forward 快 1.5-1.8×。风险 : hidden NaN 需 monitor scale statistics 稳定性。

面试考点. **Q8** : `clip_grad_norm_` 应该在 `scaler.unscale_` 之前还是之后 ?

A : 之后。因为 GradScaler 把 $\text{loss} \times 2^{16}$, grad 也 $\times 2^{16}$ 。若 clip 在 `unscale` 之前, 你比的是 `scaled grad norm`——完全错误。正确顺序 : `scale.backward` → `scaler.unscale_(optim)` → `clip_grad_norm_` → `scaler.step` → `scaler.update`

BF16 场景不需要 scaler, 直接 clip 即可。

面试考点. **Q9** : checkpoint 恢复后 loss NaN, checkpoint 前一步 loss 正常, 什么原因 ?

A : 几个可能 : (1) 保存 checkpoint 时 optim state 未 sync ; (2) 恢复时未 restore RNG state → dataloader 出不同的 batch ; (3) precision 变了 (BF16 存的但恢复时用 FP16); (4) sharding 变了 (原来 DP=8 恢复到 DP=4 , optim state 无法重组)。修 : 从更早 ckpt 恢复 ; 验证 ckpt integrity (每保存后 loss forward reproduce); 用 `torch.distributed.checkpoint` 而不是 `torch.save` (支持 rank change)

面试考点. **Q10** : loss stuck 不动, `grad_norm` 也稳定, 什么问题 ?

A : 几种 :

1. LR 已太小 (cosine tail) : mini rewarming
2. optim state saturate (v 太大, effective step size 微小) : reset v 到 EMA of recent
3. 数据 curriculum 已“记住” : 换 batch mixing / 加新 domain
4. Capacity 用尽 : 加 model size / expert 数
5. Numerical: BF16 update 太小被 round 掉 (每 step $\text{lr} \times \frac{m}{\sqrt{v}} < 1e-4$ 就危险)

Kimi K2 报告在 60% training 时 mini restart, 就是防这种 stagnation。

主流 recipe 速查：Llama 3 / DeepSeek-V3 / Qwen 3 / Kimi K2

面试问：“你训过 Llama 3 他们的 recipe 你能背几个？” 这章把 2024-2025 主流开放技术报告里的具体训练配方汇总成一张速查表，涵盖 optimizer、LR、init、batch scaling、precision、schedule、并行策略。目的是让你能当场对比不同模型的选择差异，而非死背某一个。

Llama 3 (Meta, 2024)

Model: 8B / 70B / 405B. Dense (无 MoE). RMSNorm + SwiGLU + GQA-8 + RoPE 500K → YaRN 128K.

Data: 15T tokens (405B 版), 数据配比 heavy web + code + math.

Training:

- Optimizer: AdamW, $\text{betas}=(0.9, 0.95)$, $\text{eps}=1\text{e-}8$, $\text{wd}=0.1$
- LR schedule: warmup 8000 步 → cosine to $\text{lr_min} = 0.1 \times \text{peak}$
- Peak LR: $3\text{e-}4$ (8B), $1.5\text{e-}4$ (70B), $8\text{e-}5$ (405B) — sqrt scale by width
- Batch size: 8B → 4M tokens; 405B → 4M 16M ramp
- Init: Wang-style (residual output projection $\text{std} = \frac{0.02}{\sqrt{2L}}$)
- Precision: BF16 mixed, master FP32
- Grad clip: 1.0

Parallelism (405B on 16K H100):

- TP=8 (intra-node NVLink)
- PP=16 (across nodes)
- CP=8 (long ctx 128K stage)
- DP=16 (with FSDP HYBRID)
- SP on
- 40 days per full run

Long context stage:

- 主 pretrain 完成在 8K → 追加 800B tokens 在 128K
- RoPE base 从 500K rewarm 到 8M (YaRN)

RLHF stage:

- SFT → DPO (chosen: rejected pair) 而不是 PPO
- 迭代式 5 rounds

报告独特之处:

- “no significant loss spikes” - 主要靠 grad clip 1.0 + 保守 LR
- 论文明确列 hardware failure 每 3 hours 一次
- Checkpoint 每 10 min , 可 recover

DeepSeek-V3 (2024)

Model: 671B total / 37B active. MLA + fine-grained MoE (256 routed + 1 shared, top-8). RoPE + YaRN.

Data: 14.8T tokens.

Training:

- Optimizer: AdamW, `betas=(0.9, 0.95)`, `wd=0.1`
- LR schedule: WSD variant
 - Warmup: 2000 steps
 - Stable: 大部分 (至 90% training)
 - Decay: 分两段 cosine-like, 最终 $lr_min = 0.1 \times peak$
- Peak LR: $4.2e-4$ (small tuning)
- Batch: $3072 \text{ seq} \times 4096 \rightarrow 12.5M \text{ tokens}$
- Init: MuP-inspired scaling
- Precision: **FP8 mixed**, weight BF16, forward FP8 E4M3, backward FP8 E5M2
- No aux loss (bias-based load balance)

Parallelism (2048 × H800):

- TP=1 (显式不用!)
- PP=16 (DualPipe)
- EP=64 (MoE dispatch, DeepEP for a2a)
- FSDP on non-expert params
- ZeRO-1 on expert params

Stability tricks:

- Sub-Layer Normalization (类似 sandwich Norm)
- MLA 内 up-project matmul upcast FP32
- Router logit z-loss (虽然 no-aux)
- 训练 2.788M H800 hours (79 days on 2048 GPUs) with “remarkably stable” claim

报告独特之处:

- FP8 训练 : H800 上 forward compute $1.5\times$
- DualPipe: 让 PP bubble 逼近 0 (通过 EP a2a 与 PP compute overlap)
- Multi-token prediction (MTP): 加 speculative decoding target 头, Loss 略优

Qwen 3 (Alibaba, 2025)

Model: 4B / 32B dense; 30A3B / 235A22B MoE. RMSNorm + SwiGLU + GQA-8. RoPE 1M + YaRN.

Data: 36T tokens (235B MoE 版).

Training:

- Optimizer: AdamW `betas=(0.9, 0.95)`, `wd=0.1`
- LR schedule: WSD-like with 3-stage curriculum
 - Warmup 2000 步
 - Stage 1: general web + code (peak LR)
 - Stage 2: math heavy ($LR \times 0.7$)
 - Stage 3: high quality + reasoning ($LR \times 0.3$)
 - Final decay 5%
- Peak LR: $1e-4$ (MoE 235B)
- Batch: 8M tokens
- Precision: BF16 mixed

- Aux loss: 保留 (small α)

Parallelism (235B on 4096 H800):

- TP=1 (MoE 版; dense 32B 用 TP=2)
- PP=8 (interleaved 1F1B)
- EP=8 or 16
- DP with FSDP2

Notes:

- Thinking mode training: RL 阶段特殊 loss , 加上思考 token 的 mask
- Long ctx stage: 128K $\rightarrow$ 256K $\rightarrow$ 1M rewarm

Kimi K2 (Moonshot, 2025)

Model: 1T total (架构类似 DeepSeek). MLA + 384 experts + shared. YaRN long ctx.

Data: 15T tokens.

Training (最独特!):

- **Optimizer: Muon** (2D 权重), AdamW (1D)
 - Muon: `beta=0.95` , Newton-Schulz 5 iterations
 - AdamW: `betas=(0.9, 0.95)` , `wd=0.1`
- LR schedule: WSD + mini-restart at 60%
 - Warmup 2000 $\rightarrow$ stable 78% $\rightarrow$ mini rewarm (LR $\times$ 0.5 $\rightarrow$ warmup 500 $\rightarrow$ stable) $\rightarrow$ final decay
- Peak LR: 2e-3 (Muon 允许比 AdamW 大 3-5 $\times$)
- Init: near-orthogonal (配合 Muon)
- Batch: 8M tokens
- Precision: BF16 (Muon 与 FP8 集成尚未稳定)

Parallelism:

- ZeRO-1 (must! Muon needs full matrix per rank)
- EP=32
- PP=8
- No TP (like DeepSeek)

报告独特之处:

- Muon vs AdamW ablation: Muon loss curve 明显更平滑 , 无 spike
- Mini restart at 60%: 防 stagnation
- MuP scaling: 300M model 调 LR 迁移到 1T

老一代 recipe 对比

GPT-3 (2020):

- Adam `betas=(0.9, 0.95, 1e-8)`, `wd=0.1`
- Warmup 375M tokens $\rightarrow$ cosine to 10% peak $\rightarrow$ const
- 8 $\times$ 64 batch $\rightarrow$ 3.2M tokens

OPT-175B (2022, Meta):

- AdamW, warmup 2K $\rightarrow$ linear decay
- 45+ manual restarts due to spike (recorded in paper)

- 教训: WSD/rewarming 尚未普及, cosine 硬训

GLM-130B (2022, THUDM):

- Adafactor + Adam mixed
- Sandwich Norm (fix Pre-LN activation drift)
- Embedding layer norm gradient shrink

Chinchilla (2022, DeepMind):

- 定义了 $20 \times \text{params tokens} = \text{compute-optimal}$
- AdamW cosine, wd=0.1

PaLM (2022, Google):

- Adafactor (TPU friendly)
- Loss “step function” spike; skip-and-retry restart

快速对比表

算法侧配方 (optim / LR / batch / precision)

model	optim	LR sched	peak LR	batch	prec
Llama-3 8B	AdamW	warmup+cos	3e-4	4M	BF16
Llama-3 70B	AdamW	warmup+cos	1.5e-4	4M	BF16
Llama-3 405B	AdamW	warmup+cos	8e-5	4-16M	BF16
DeepSeek-V3	AdamW	WSD	4.2e-4	12.5M	FP8
Qwen3 235B	AdamW	WSD 3-stage	1e-4	8M	BF16
Kimi K2	Muon+AdW	WSD+restart	2e-3	8M	BF16
Mixtral 8×22B	AdamW	cosine	2e-4	4M	BF16
GPT-3 175B	Adam	warmup+cos+const	6e-5	3.2M	FP16
OPT-175B	AdamW	warmup+linear	1.2e-4	2M	FP16

系统侧配方 (parallelism)

model	parallelism
Llama-3 8B	FSDP2 only
Llama-3 70B	TP=4, PP=4, FSDP over DP
Llama-3 405B	TP=8, PP=16, CP=8, FSDP over DP (16K H100)
DeepSeek-V3	no TP, PP=16 (DualPipe), EP=64, FSDP non-expert + ZeRO-1 expert
Qwen3 235B	no TP, PP=8, EP=8, FSDP2
Kimi K2	ZeRO-1 only (Muon needs full matrix), PP=8, EP=32
Mixtral 8×22B	PP, EP=4, FSDP
GPT-3 175B	Megatron TP + PP
OPT-175B	Megatron TP + PP

你要背的 5 个数字

面试常问“你们训 X 用什么超参”，报这 5 个至少 hit 一个：

1. **AdamW betas**: (0.9, 0.95) — 不是 0.999
2. **Weight decay**: 0.1
3. **Warmup steps**: 2000
4. **Grad clip**: 1.0
5. **lr_min / peak_lr**: 0.1

其他随 model 变，但这 5 个是全行业主流。

面试考点

面试考点. **Q1** : DeepSeek 和 Llama 训练最大的三个差异？

A :

1. **Precision**: DeepSeek 用 FP8 (H800 上 forward 1.5×), Llama 保守 BF16
2. **TP**: DeepSeek 明确 不用 TP (MLA + fine-grained MoE 让 TP 收益差), Llama 405B 用 TP=8
3. **LR schedule**: DeepSeek WSD, Llama cosine

背后逻辑：DeepSeek 追极致 training efficiency (H800 受限), Llama 保守求稳定。

面试考点. **Q2** : Kimi K2 用 Muon 相对 AdamW 什么优势？

A :

1. Loss curve 更平滑 (Kimi 报告 0 vs 3 次可见 spike)
2. LR 可 3-5× 大 ($2e-3$ vs $4e-4$)
3. Optim state 少一半 (无 second moment v)
4. 与 μP scaling 集成好

代价：与 FSDP 不兼容 (Newton-Schulz 要 full matrix), 只能 ZeRO-1 ; 1D 权重仍 AdamW ; FP8 集成不成熟。

面试考点. **Q3** : 为什么 GPT-3 用 (0.9, 0.95) 而不是默认的 (0.9, 0.999) ?

A : LLM 数据分布快速变化 (curriculum, packing 组合不断变化), $\beta_2 = 0.999$ 让 v 半衰期 700 步——反映了远古的 gradient scale, 不 adaptive。0.95 半衰期 14 步, 紧跟当前 scale。这个默认改动被 GPT-3 论文推广, 之后所有 LLM 遵循。

面试考点. **Q4** : LLM 训练里 $wd=0.1$ 是不是有点太大？

A : 不大, 正好。Decoupled weight decay 独立于 gradient, $\theta \leftarrow \theta - lr * wd * \theta$ 每 step 缩 $lr * wd = 3e-4 * 0.1 = 3e-5$ ——很小。1M steps 才 shrink 3%。目的是防止 weight 无限膨胀 (activation 也跟着涨 $\rightarrow$ BF16 drift)。CV 时代 $wd=1e-4$ 是因为 L2 (被 grad scale 缩), AdamW decoupled 后 wd 才敢开大到 0.1。

面试考点. **Q5** : 4M tokens 的 batch size 你会开吗？为什么不再大？

A : 4M 是 2024 主流 (Llama-3, Qwen-3)。DeepSeek-V3 到 12.5M, Llama-3 405B 涨到 16M。为什么不无限大 :

1. Critical batch size (Kaplan) : 超过后 loss 不再随 batch 增大同比降低
2. LR 只能 sqrt-scale , 实际收益递减
3. 单 iter 太慢 → 反馈周期长 , 问题发现晚 (data corruption, spike 判断)

经验 : 越大 model 越大 batch , 8B → 4M, 70B → 4-8M, 400B+ → 8-16M。

面试考点. **Q6** : 训练中间 checkpoint , continual pretrain 应该从哪个 ckpt 起 ?

A : **WSD** 的 **stable** 阶段 **ckpt** , 不是 decay ckpt。因为 decay 完 LR 已很低 , loss landscape 已“sharpen”到局部极值 , rewarm 破坏这个。Stable ckpt 是 fully-trained-at-peak , loss landscape 平坦 , rewarm 到新 peak 平滑过渡。这是 WSD 相对 cosine 的一大优势——cosine 无“稳定态 ckpt”可用。

面试考点. **Q7** : 你从 Kimi K2 recipe 里学到什么可以用到自己项目 ?

A : 可迁移的 :

1. WSD + mini restart at 60% (防 stagnation)
2. Grad clip 1.0 + monitor grad_norm
3. BF16 mixed precision , LN var FP32
4. MuP scaling 让 hyper search 在 small model 上做

不宜盲抄 :

1. Muon (需 ZeRO-1 , 与现有 FSDP 生产不兼容)
2. 384 experts (需自研 dispatch lib)
3. 1T model scale (硬件不够)

面试考点. **Q8** : 面试官说“我们最近 loss spike 挺多” , 你怎么问下一层 ?

A : 结构化提问 :

1. 频率 : 每 100 步、每 1000 步、还是随机偶发 ?
2. 复发病规律 : 与 LR schedule 阶段有关吗 ? 与 curriculum 换阶段有关吗 ?
3. 阶段 : warmup 阶段 vs stable 阶段 vs decay 阶段 ?
4. 严重度 : 能自愈还是需要 restart ?
5. Monitor : grad_norm 是否也 spike ? weight_norm 是否累积增长 ?

这些答案能锁定问题 :

- 偶发 + 自愈 → data outlier (改 grad clip + skip step)
- warmup 阶段 + 系统性 → LR 太大 or init 差 (改 warmup step 或 init)
- decay 阶段 + curriculum 相关 → 数据阶段切换 (rewarming)
- 已 checkpoint 但 restart 后立即 spike → ckpt 损坏或 precision 变化

集合通信与带宽模型

分布式训练的一切性能分析，最终都能落到一次集合通信要几字节 × 走什么带宽 × 有没有被 **overlap**。这一章把公式和数字讲清楚，后面每章都会用。

硬件层次：从 **SM** 到 **IB**

一个训练集群的通信媒介，按带宽从高到低：

介质	典型带宽 (单向)	延迟	作用域
SM ↔ HBM	1.7 TB/s (H100)	400 cycles	单卡
NVLink 4 (H100)	450 GB/s (uni) / 900 GB/s (bidi)	1 μs	同一 NVL8 / NVL72 域
NVSwitch (GB200 NVL72)	900 GB/s per link × 18	1 μs	72 卡 flat 域
PCIe Gen5 x16	63 GB/s (uni)	2 μs	CPU↔GPU / GPU↔NIC
Infiniband NDR (400 Gb/s)	50 GB/s (uni)	2 μs	跨节点
Infiniband HDR (200 Gb/s)	25 GB/s	2 μs	老集群
Ethernet RoCEv2 400G	40 GB/s effective	5 μs	跨节点

表 1 常见通信媒介带宽。注意 NVLink 与 IB 差 **10×**，跨节点通信永远是瓶颈；GB200 NVL72 通过把 NVLink 域扩到 72 卡缓解了这一点。

一个关键比例：H100 单卡 BF16 算力 989 TFLOPS，HBM 3.35 TB/s，NVLink 双向 900 GB/s，IB 单向 50 GB/s。也就是说：

$$\text{compute} : \text{HBM} : \text{NVLink} : \text{IB} \approx 1000 : 3.35 : 0.9 : 0.05 \quad (\text{TB/s vs TFLOPS})$$

Arithmetic intensity (每字节多少 FLOPs) 在 HBM 层需要 295 FLOP/byte 才能 roof，NVLink 层需要 1100，IB 层需要 20000。这是所有 overlap 策略的物理基础。

集合通信原语

八个 NCCL 提供的原语，是所有分布式训练框架的字面构件。

Op	语义	每卡 send	每卡 recv
Broadcast	rank 0 广播 V 给所有	V	V
Reduce	所有 rank 数据 reduce 到 rank 0	V	V
AllReduce	reduce 且结果广播到所有	$2V \frac{W-1}{W}$	$2V \frac{W-1}{W}$
AllGather	rank i 有 $\frac{V}{W}$, 最终每 rank 拥有完整 V	$\frac{V(W-1)}{W}$	$\frac{V(W-1)}{W}$
ReduceScatter	reduce 后每 rank 拿 $\frac{V}{W}$ 分片	$\frac{V(W-1)}{W}$	$\frac{V(W-1)}{W}$
Scatter	rank 0 把 $\frac{V}{W}$ 分片发给每 rank	$\frac{V(W-1)}{W}$ (0 号)	$\frac{V}{W}$
Gather	rank 0 收集所有 rank 的 $\frac{V}{W}$	$\frac{V}{W}$	$\frac{V(W-1)}{W}$ (0 号)
All-to-All	rank i 的第 j 段发给 rank j	$\frac{V(W-1)}{W}$	$\frac{V(W-1)}{W}$

表 2 八种集合通信及每卡 volume (V = 单卡起始/结束数据量, W = 世界大小)。AllReduce 的 volume 是 AllGather 或 RS 的 $2\times$, 这是 ZeRO/FSDP 分片省 comm 的公式基础。

两个关键身份, 面试常考:

$$\text{AllReduce} = \text{ReduceScatter} + \text{AllGather}$$

$$\text{vol}_{\text{AllReduce}} = 2 \times \text{vol}_{\text{AllGather}} = 2 \times \text{vol}_{\text{ReduceScatter}}$$

第一条是算法层面 (NCCL Ring AllReduce 就是这两步 fuse); 第二条是通信量层面 (AllReduce 走两遍数据, AG/RS 只走一遍)。ZeRO-2 (grad AR) $\rightarrow$ ZeRO-3 (param AG + grad RS) 看似省了 comm, 其实通信量相同——只是切换了通信模式。

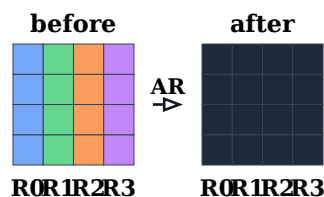


图 4 AllReduce: 每卡持有一份数据, op 后每卡都得到“全体归约”的结果 (深色 = 归约后)。



图 5 AllGather: 每卡持自己那一份 shard, op 后每卡拿到全部 shard 的拼接。

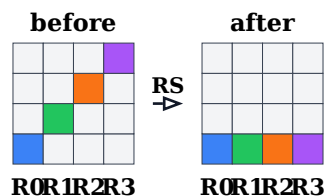


图 6 ReduceScatter: 每卡有 W 份数据, op 后每卡只留自己“该负责的那一份”, 且该份已被归约。



图 7 All-to-All : 转置。rank i 的第 j 块发给 rank j 。带宽压力最大——常成为 MoE 训练瓶颈。

Ring AllReduce 算法

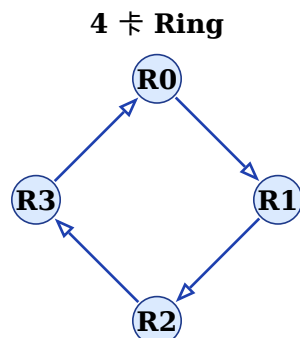


图 8 Ring AllReduce 的物理拓扑：所有 rank 组成一个环，每一步只与相邻 rank 通信。共 $2(W-1)$ 步：前 $W-1$ 步 ReduceScatter，后 $W-1$ 步 AllGather。手写实现见 `src/distributed_training/01_collectives.py::ring_all_reduce()`，与 NCCL 结果 bit-for-bit 对齐。

NCCL 默认在中大规模 (> 8 rank) 用 Ring 算法。分 $W-1$ 步 ReduceScatter + $W-1$ 步 AllGather，每步每卡发送 $\frac{V}{W}$ ：

$$T_{\text{ring}} = 2(W-1) \times \frac{V}{B} + 2(W-1) \times \alpha$$

- B ：单向带宽
- α ：每步延迟（NCCL 里主要是 kernel launch + IB QP setup）
- $\frac{V}{W}$ ：每步 payload

当 W 大时， T 主要由 $2\frac{V}{B}$ 主导（与 W 无关），加 $2W\alpha$ 的延迟项。这就是 ring 好的原因：带宽项与 W 无关，只有延迟随 W 线性增长。

Tree AllReduce

Ring 的延迟项 $O(W)$ 在小 payload 上会主导。NCCL 提供 Tree 算法：

$$T_{\text{tree}} = 2 \log_2(W) \times \frac{V}{B} + 2 \log_2(W) \times \alpha$$

带宽项从 $O(V)$ 变成 $O(V \log W)$ ——大 payload 时更差，但延迟项只有 $O(\log W)$ ——小 payload 更快。NCCL 自动根据 payload 选。手动切换：NCCL_ALGO=Tree 或 Ring。

Key insight. 面试爱问“为什么 ring allreduce 通信量与世界大小无关”。回答：每步每卡传 $\frac{V}{W}$ ，共 $2(W-1)$ 步 $\rightarrow$ total per-GPU volume = $2\frac{V(W-1)}{W} \rightarrow 2V$ 。这个“ $2V$ ”就是 AllReduce 的黄金常数——不管多少卡都是它。

NCCL Ring vs NVLS

从 NCCL 2.19 起 H100/H200 支持 **NVLS (NVLink SHARP)** : AllReduce 里把 reduction 逻辑下放到 NVLink switch 芯片里做, 节省一次 GPU 参与。带来 **1.3×** AllReduce 带宽。开启: `NCCL_NVLS_ENABLE=1` (部分 setup 需要显式打开)。

带宽项 vs 延迟项: Message size 决定一切

一次 AllReduce 的时间:

$$T = \alpha_{\text{latency}} + \frac{V}{B_{\text{effective}}}$$

V 小的时候 (< 1 MB) 延迟项主导, V 大的时候带宽项主导。转折点约 1-8 MB (依赖 NCCL 参数)。这决定了:

1. **DDP bucket size** 要 ≥ 25 MB (PyTorch 默认), 太小会被延迟吃掉
2. **ZeRO-3 flat param** 要 shard 得够大, `FULL_SHARD` 每个 flat group ≥ 100 MB 是好的
3. **TP AllReduce** 一层 activation 通常 10-100 MB, 天然合适
4. **EP All-to-All** 单层 100 MB - 1 GB, 永远带宽项主导

通信量的三种度量

面试里经常混淆, 务必区分:

1. **Per-GPU volume** (`sendrecv`): 一张卡自己发出/收到多少字节。上面表格给的是这个。
2. **Aggregate volume**: W 张卡加起来。 $\text{agg} = W \times \text{per-GPU}$ 。
3. **Bisection bandwidth**: 网络切成两半时穿过 cut 的总流量。跨节点集群里 bisection 决定 all-to-all 的性能。

Ring AllReduce 的 per-GPU 是 $2V$ (bandwidth 侧), aggregate 是 $2VW$ (跨所有 rank 的总数据流)。IB fabric 的 bisection 通常是 $O(\text{nodes})$, 所以 all-to-all 在跨 100+ 节点会退化。

延迟分解

一次跨节点 send/recv 的延迟组成:

阶段	典型耗时	说明
CUDA kernel launch	5-20 μs	每次 collective 都要 launch NCCL kernel
NCCL 内部同步	10-30 μs	rank 之间的同步握手
PCIe DMA (GPU→NIC)	1 μs	大 message 里可忽略
IB fabric (Switch hops)	1-3 μs	每 hop 500 ns, spine-leaf 通常 2-3 hops
NIC→PCIe→GPU (对端)	1 μs	
接收方 kernel launch	5-20 μs	
Total (small msg)	30-80 μs	α 约等于这个数

表 3 跨节点集合通信延迟分解。这就是为什么小 message 会被延迟吃掉——payload 只有几 KB 时, 30 μs 的启动开销让有效带宽掉到几 GB/s。

IBGDA (Infiniband GPUDirect Async): 让 GPU 直接下发 IB verb, 绕过 CPU, 把 launch 相关的 20+ μs 干到 < 5 μs 。DeepEP、UCX 都在用。

简易 **profile** : **nsys** 里怎么看通信

一个典型的 nsys trace 里：

```
[cuda kernel] gemm_kernel_0          2.34 ms
[cuda kernel] ncclKernel_AllReduce_.. 0.85 ms    <- 通信
[cuda kernel] elementwise_add        0.12 ms
```

关键看的三点：

1. **NCCL kernel** 与其他 **kernel** 是否 **overlap** (不同 stream)
2. **NCCL kernel** 内的 **gap** (是不是 IB 卡了)
3. 通信占总 **step** 时间的 %

Blackwell / Hopper 上还能用 NCCL profiler `NCCL_DEBUG_SUBSYS=COLL,PROFILE`。

NCCL 调优 **checklist**

生产训练必调的环境变量：

```
# 通用
export NCCL_DEBUG=WARN          # 或 INFO 排错时用
export NCCL_ASYNC_ERROR_HANDLING=1
export CUDA_DEVICE_MAX_CONNECTIONS=8  # 允许多 stream 并行 (Hopper 上建议 ≥4)
export TORCH_NCCL_HIGH_PRIORITY=1

# Ring / Tree
export NCCL_ALGO=Auto           # 通常自动选好
export NCCL_MIN_NCHANNELS=8     # 更多 SM channel
export NCCL_NTHREADS=512

# IB
export NCCL_IB_HCA=mlx5_0,mlx5_1,mlx5_2,mlx5_3  # 用足所有 IB HCA
export NCCL_IB_GID_INDEX=3      # RoCEv2 常用
export NCCL_IB_TIMEOUT=22
export NCCL_IB_RETRY_CNT=13
export NCCL_IB_SL=1            # Service Level

# H100 NVLS
export NCCL_NVLS_ENABLE=1

# Buffer
export NCCL_BUFFSIZE=8388608    # 8MB, 大 message 稳定

# Debug 慢通信时
export NCCL_P2P_DISABLE=0
export NCCL_SHM_DISABLE=0
```

每个集群都要用 **nccl-tests** 实测选参数——不同拓扑差别巨大。生产建议跑：

```
mpirun -np 64 --hostfile hosts \
  ./build/all_reduce_perf -b 8 -e 8G -f 2 -g 1
```

看输出的 bus BW 列，理想 H100 8 卡是 370 GB/s (NVLink) 或 30-40 GB/s (跨节点)。低于这个就要 debug。

拓扑感知的进程排布

错误的做法：

```
torchrun --nproc-per-node=8 --nnodes=8 ...
# 默认 rank 0-7 在 node 0, rank 8-15 在 node 1, ...
# TP=8, DP=8: TP 组默认取连续 rank -> TP 组内是 NVLink OK
# 但如果 TP=2, DP=32: TP 组 = (0,1),(2,3),... 都在同一 node OK
# 反过来 PP=2 时 stage 之间会跨 node
```

建议：手动设置 process group 时确认三件事：

1. **TP** 组必须在同一 **NVLink** 域内 (NVL8 就是同一 node, NVL72 允许一个 rack)
2. **DP** 组的 **AllReduce** 可以跨节点 (带宽敏感度低)
3. **PP** 组的 **P2P** 尽量在同 **node** 或同 **spine**
4. **EP** 组根据规模：**EP ≤ 8** 尽量同 **node**，**EP > 8** 只能跨

Megatron-LM 的 `torch.distributed.new_group` 排列方式看 `megatron/core/parallel_state.py`；FSDP 用 `DeviceMesh` 更直观。

面试考点

面试考点. **Q1**: AllReduce 的通信量为什么是 $2 \frac{V(W-1)}{W}$ ，“2” 从哪来？

A: Ring AllReduce = ReduceScatter + AllGather。两步各走 $\frac{V(W-1)}{W}$ ，总 $2 \frac{V(W-1)}{W}$ 。 W 大时约等于 $2V$ 。这就是 AllReduce “两倍数据”的直觉。

面试考点. **Q2**: DDP 的 gradient AllReduce 与 ZeRO-3 的 param AllGather + grad ReduceScatter 通信量对比？

A: 假设模型参数 P 字节：

- DDP : $2P$ 字节 (grad AR)
- ZeRO-3 : forward 时 P (param AG) + backward 时 P (param AG) + P (grad RS) = $3P$ 字节

ZeRO-3 通信量多 **50%** 但内存少得多 (optim + grad + param 都切)。这是显存-通信 tradeoff。

面试考点. **Q3**: 什么时候 Tree AllReduce 比 Ring 好？

A: 小 payload + 大 world size。Ring 的延迟项是 $2(W-1)\alpha$ ， $W=1024$ 时约 60 ms 只算延迟。Tree 只 $2 \log_2(W)\alpha \approx 0.6$ ms。GPT-3 训练的 embedding sync 之类小 message 会自动切 Tree。

面试考点. **Q4**: 为什么 NVLS (NVLink SHARP) 能加速 AllReduce？

A: SHARP 让 reduction 在 NVSwitch 芯片里做——每卡只需 send 一次数据到 switch, switch 里做 sum 后广播回, 把 $2\frac{V(W-1)}{W}$ 的 per-GPU volume 降到 $\frac{V}{W} + \frac{V}{W} = 2\frac{V}{W}$, 即 **1.5-2x** 提速。类似 InfiniBand SHARP 在网络上做。

面试考点. **Q5**: bisection bandwidth 是什么? 为什么 all-to-all 特别敏感?

A: 网络切成任意等分两半, 穿过切面的总带宽。all-to-all 里每对 rank 都要通信, 跨节点流量 $\propto n^2$; 只有 bisection $\propto n$ 的 fat-tree 才能不阻塞。100+ 节点的 IB fabric 里 bisection 常常 oversubscribe, all-to-all 掉到理论 1/3。

面试考点. **Q6**: NCCL 的 NCCL_MIN_NCHANNELS 是什么? 调高有代价吗?

A: NCCL 用多个 CUDA channel (每 channel 2 SM) 来并行传输。channel 越多带宽越接近峰值, 但吃 **SM**, 与 kernel compute 争资源。H100 上 8 channel 大约 16 SM (12% of 132), 能拿到 90%+ 带宽; DeepEP V1 吃 20 SM 让 GEMM 掉 20% 效率——所以 V2 降到 4-6。这是通信-计算的 SM 分配 tradeoff。

面试考点. **Q7**: 如果 NCCL AllReduce 突然变慢一半, 你怎么排查?

A: 按顺序:

1. `NCCL_DEBUG=INFO` 看是不是切换到 Tree/Ring 了 (payload 边界);
2. `nccl-tests` 测 raw 带宽, 排除模型代码;
3. `ibstat` / `ethtool` 看 IB/Eth 是不是有 error 或 rate 掉;
4. `nvidia-smi topo -m` 看 GPU 与 NIC 的 affinity 是不是错了;
5. 排查慢卡: `NCCL_DEBUG_SUBSYS=COLL` + 每 rank 打时间戳, 找拖后腿的 rank;
6. Node NIC 硬件重启 (IB 偶尔会陷入 slow-mode)。

显存、算力与三堵墙

在讲任何具体并行策略之前，你必须能心算一个模型在特定 setup 下的显存和算力占用。这一章给三张速查表 + Roofline + 三堵墙模型，为后续所有优化提供诊断框架。

参数量速查

一个标准 GPT-style Transformer，参数量：

$$N \approx L \times (12H^2) + VH$$

(L 层， H hidden dim， V vocab size；不含 LN/bias 的小项)

其中每层的 $12H^2$ 来自：

- $QKVO$ 四个 projection： $4H^2$
- FFN 两个 linear ($H \rightarrow 4H, 4H \rightarrow H$)： $8H^2$

GLU-family (SwiGLU / GeGLU) 用三个 linear ($H \rightarrow (\frac{8}{3})H, H \rightarrow (\frac{8}{3})H, (\frac{8}{3})H \rightarrow H$)，参数量还是约 $8H^2$ 。

典型模型速查：

Model	L	H	A	V	Params
GPT-2 (small)	12	768	12	50257	124M
GPT-3	96	12288	96	50257	175B
LLaMA-2 7B	32	4096	32	32000	7B
LLaMA-2 70B	80	8192	64	32000	70B
LLaMA-3 405B	126	16384	128	128256	405B
Mixtral 8×7B	32	4096	32	32000	47B (13B active)
DeepSeek-V3	61	7168	128	129280	671B (37B active)
Qwen3-235B (MoE)	94	4096	64	151936	235B (22B active)

表 4 常见模型参数速查。MoE 括号里是每 token 激活量。

显存组成：单卡训练的四头

不做任何并行时，训练一个模型显存 = weight + grad + optim state + activation：

组成	字节 / 参数	说明
Weight (BF16)	2	模型主权重，混合精度下 fwd/bwd 用这个
Weight (FP32 master)	4	Adam 更新用；如果不用 master 则省掉
Grad (BF16 或 FP32)	2 或 4	和 update 精度一致
Adam m (FP32)	4	一阶动量
Adam v (FP32)	4	二阶动量
Total (标准 AdamW BF16 混合精度)	16	$2 + 4 + 4 + 4 + 2 \approx 16$ per param
Total (Precision-aware, m/v BF16)	10	DeepSeek/Megatron-Core 用
Total (SGD momentum)	8	$2 + 4 + 2 = 8$

表 5 每参数字节数。标准 AdamW 混合精度是 **16 bytes / param**，这是所有显存讨论的基线。一句话公式（AdamW 混合精度）：

$$\text{mem}_{\text{model}} \text{ (GB)} = N \text{ (B)} \times 16$$

所以：

- LLaMA-2 7B : $7 \times 16 = 112 \text{ GB} \rightarrow$ 单卡 **A100 80GB** 装不下
- LLaMA-2 70B : $70 \times 16 = 1120 \text{ GB} \rightarrow$ 需要 **14+** 卡
- DeepSeek-V3 671B : $671 \times 16 = 10736 \text{ GB} \rightarrow$ 需要 **135+** 卡起步

Activation 更麻烦：与 batch/seq 强相关。粗估（BF16，不 recomp）：

$$\text{act} \approx s \cdot L \cdot B \cdot S \cdot H$$

s 大约在 30-60 之间（Megatron 2022 activation recomp paper 里给了精确公式）， L 是层数， B micro-batch， S seq len， H hidden dim。

LLaMA-2 7B， $L = 32$, $B = 1$, $S = 4096$, $H = 4096$ ：

$$\text{act} \approx 34 \times 32 \times 1 \times 4096 \times 4096 \times 2 \approx 34 \text{ GB}$$

Recomp 之后可以降到 $2LBSH$ （每层只存 input）约 2 GB。

$$\text{act}_{\text{recomp}} \approx 2LBSH$$

算力速查

Forward FLOPs per token（Chinchilla 公式简化版）：

$$\text{FLOPs}_{\text{fwd}} \text{ per token} \approx 2N$$

（ N = 参数量。系数“2”因为一个 matmul 是 $2MNK$ FLOPs，而每个参数在 fwd 中恰好乘 1 次。忽略 attention 的 SH 项——在 $S \ll H$ 时成立。）

Forward + Backward：

$$\text{FLOPs}_{\text{train}} \text{ per token} \approx 6N$$

（fwd 一次、bwd 输入梯度一次、bwd 权重梯度一次；每次都是 $2N$ FLOPs。）

Attention 修正： S 大时 attention 的 $O(S^2H)$ 不可忽略。精确：

$$\text{FLOPs}_{\text{train}} \text{ per token} \approx 6N + 12LSH$$

举例 ,LLaMA-3 70B 在 $S = 8192 : 6 \times 70 \cdot 10^9 + 12 \times 80 \times 8192 \times 8192 \approx 4.2 \cdot 10^{11} + 6.4 \cdot 10^{10} \approx 4.9 \cdot 10^{11}$, attention 项占 13%。

训练一个模型总 **FLOPs** : $C = 6ND$ ($D = \text{tokens}$) GPT-3 175B on 300B tokens = 3.14×10^{23} FLOPs。

MFU (Model FLOPs Utilization) :

$$\text{MFU} = \frac{\text{achieved FLOPs}}{\text{peak FLOPs of hardware}}$$

H100 BF16 peak = 989 TFLOPS。如果 step time 里每卡处理 BS tokens :

$$\text{achieved TFLOPS} = \frac{6NBS}{\text{step_time} \times 10^{12}}$$

行业参考 :

- Dense LLM 训练 : 40-55% MFU 是好的 (GPT-4-class)
- MoE : 35-50% MFU (多了 dispatch overhead)
- FP8 训练 : 30-45% MFU (peak 变 2× , 绝对 TFLOPS 更高但 utilization 降)

Roofline : 一个 kernel 值不值得优化

Roofline 说 kernel 性能受两个屋顶限制 : 算力 (compute) 或带宽 (memory)。

$$\text{achievable perf} = \min(\pi_{\text{peak}}, \beta_{\text{peak}} \times I)$$

- π_{peak} : 硬件峰值 (H100 BF16: 989 TFLOPS)
- β_{peak} : HBM 带宽 (H100: 3.35 TB/s)
- I : arithmetic intensity (FLOPs / byte)

ridge point : $I^* = \frac{\pi}{\beta} = \frac{989 \cdot 10^{12}}{3.35 \cdot 10^{12}} \approx 295$ FLOPs/byte。

意思 : kernel 每读一 byte 至少要做 295 FLOPs ,才能跑到峰值算力。低于这个就是 memory-bound , 高于就是 compute-bound。

常见 **kernel** 的 **intensity** :

Kernel	Intensity (FLOP/byte)	状态
Element-wise (add, gelu)	0.25	memory-bound
LayerNorm	1-2	memory-bound
Softmax	3-5	memory-bound
Attention (MHA, S=4K)	50-100	memory-bound (无 Flash)
FlashAttention v2 (S=4K)	100-200	compute-bound 50%
GEMM (M=N=K=1024)	340	compute-bound
GEMM (M=N=K=4096)	1300	compute-bound
GEMM (M=8, N=K=8192)	15	memory-bound (small-M GEMM)

表 6 常见 kernel 的算术强度。GEMM 的 intensity 随 $\min(M, N, K)$ 线性增长——小 M(decode / MoE per-expert) 就是 memory-bound , 这是 MoE 训练 compute wall 的根源。

Key insight. 面试常问：“为什么 batch size 越大 MFU 越高？”答：MFU 直接对应 arithmetic intensity。BS 大意味着 attention 和 FFN 的 GEMM M 维大，越过 ridge point，越接近 compute-bound。BS 小时是 memory-bound，无论怎么优化算法都拿不到峰值。

三堵墙：训练系统的诊断框架

我们借用 Megatron-Core MoE Tech Report 的“三堵墙”框架，扩展到所有大规模训练。任何一次训练卡顿最终都能归为：

1. **Memory Wall**：显存装不下。症状：OOM，或被迫用小 batch。
2. **Communication Wall**：通信占太多时间。症状：nsys 里 comm 占 step > 30%。
3. **Compute Efficiency Wall**：算力没用满。症状：MFU < 40%，SM 利用率低。

三堵墙互相耦合：解决内存墙常常增加通信（切分权重需要 AR/AG）；overlap 通信需要更多 batch（增加显存）；小 M GEMM 让 compute 掉率，恰恰是切碎权重的副产物。

每堵墙武器谱系，本书对应章节：

墙	武器	本书章节
Memory	ZeRO / FSDP; TP; PP; Activation recompute; Activation offload; Precision-aware optim; FP8 activation; CPU offload; Selective recompute	第 4-6, 9-10 章
Communication	DDP bucket; ZeRO overlap; TP-SP; Ring/Ulysses attention; Hierarchical a2a; DualPipe; FWD-BWD merged; FP8 dispatch; NCCL tuning	第 1, 3-8, 11 章
Compute Efficiency	Grouped GEMM; FlashAttention; TE fused kernel; CUDA graph; Sync-Free MoE; SM carve-out policy; batch size / seq packing 让 M 变大	第 8-11, 12 章

表 7 三堵墙的武器谱系。定位问题时先 profile 判定属于哪堵墙，再挑对应工具。反过来“这个技术很酷”式的堆叠通常没收益。

一个“能不能塞下”的心算 checklist

拿到一个新模型 + 新集群时，30 秒决定“能不能跑起来”：

1. 模型 **param** 量 N ：查论文或 `sum(p.numel() for p in model.parameters())`
2. 单卡显存： $16N$ (AdamW BF16 混精)；MoE 记 N_{total} 不是 active
3. 总卡数需求（纯 **DP**，不切模型）： $\text{cards} = \lceil 16 \frac{N}{80} \text{ GB} \rceil$ ；这是绝对下限，任何切分方案都能省
4. 切完之后 **activation**： $B \times S \times H \times L \times \text{factor} \times 2$ ，其中 factor 视 recompute 策略
5. 算 **step time**： $6NB \frac{S}{\text{MFU} \times 989 \text{ TFLOPS}}$ 每卡，加通信估算
6. 总训练时间： $6N \frac{D}{\text{cards} \times \text{MFU} \times 989 \text{ TFLOPS}}$

例：LLaMA-2 70B on $512 \times \text{H100}$, $D=1\text{T}$ tokens, MFU=50%:

$$\frac{6 \times 70 \cdot 10^9 \times 10^{12}}{512 \times 0.5 \times 989 \cdot 10^{12}} = 1.66 \cdot 10^6 \text{ s} \approx 19.2 \text{ 天}$$

一个数量级估算 30 秒能得出。这个能力是分布式训练面试的“入场券”。

典型 70B/TP4/DP4/PP4 一次 step 时间构成 (估算, 秒)

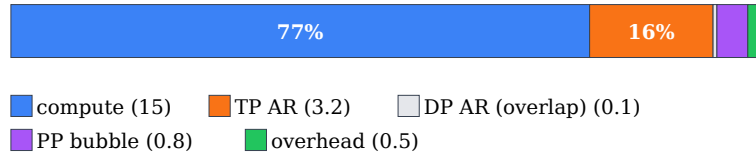


图 9 一次 step 时间的组成。TP AR 是不能 overlap 的显式通信；DP AR 100% 被 backward 隐藏；PP bubble 与 (P,m) 有关。完整推导见「面试数学」章 (M)。

Key insight. 拿到面试题时先按上面这张 stacked-bar 的四项报数。90% 情况下你能给出 $\pm 30\%$ 的答案，比背公式实用。完整 estimator: `src/distributed_training/estimators.py`。

面试考点

面试考点. **Q1:** 为什么 AdamW 混合精度是 16 bytes/param ?

A: BF16 weight (2) + FP32 master weight (4) + BF16/FP32 grad (2 或 4) + FP32 momentum (4) + FP32 variance (4)。用 grad=BF16 就是 $2 + 4 + 2 + 4 + 4 = 16$ 。如果不保留 FP32 master 是 12。Precision-aware optimizer (m/v BF16) 是 10。

面试考点. **Q2:** FLOPs = 6 ND 里的“6”是怎么来的？

A: 每个参数在 forward 中乘 1 次 activation (2 FLOPs : 乘 + 累加)。Backward 里对 input 和 weight 各求一次导 = $2 \times 2 = 4$ FLOPs。总 6 FLOPs/param/token $\rightarrow 6ND$ tokens 训完。

面试考点. **Q3:** 一个 70B 模型 MFU 40%，你觉得“正常”吗？

A: 依赖 setup。dense 70B on 512 H100 with BF16, TP=8/PP=2/DP=32, S=8K : 40% 偏低 (GPT-3 era 已经能 50%+)。可能原因: 通信没 overlap 好、activation recomp 太激进、data loading 慢。可以问 profile 看 comm / compute ratio。

面试考点. **Q4:** 为什么 MoE 训练比 dense MFU 低 5-10%？

A: 三个来源: (1) all-to-all 通信 overlap 不完全; (2) grouped GEMM 的 M 维小 (每 expert 只有 $1/E$ 的 tokens), 小-M GEMM 未饱和 TensorCore; (3) permute / bincount / cross-rank sync 的 launch overhead。用 fine-grained MoE + DeepEP + capacity padding 能把差距压到 3%。

面试考点. **Q5:** Roofline 上一个 kernel 是 memory-bound，你说“用更大 batch”——那 attention 呢？attention 的复杂度是 $O(S^2)$ ，batch 大 memory 反而爆了。

A: 分两层看: **Q/K/V** 的 **projection matmul** 是 batch-dominated，BS 大 $\rightarrow$ M 大 $\rightarrow$ compute-bound。**QK^T** 和 **PV** 的 **attention matmul** 是 seq-dominated (每 head 的 $M = S$)，S 大就有 M 大 $\rightarrow$ 也 compute-bound。所以 FlashAttention 的 tile 沿 S 而非 BS 切——单个 tile 里 $M = \text{block_size}$ 已经越过 ridge point，与 BS 无关。这解释了为什么 FA 对小 batch decode 也很快。

面试考点. **Q6**: activation 显存怎么估? 给一个 LLaMA 7B, $B=1$, $S=8192$ 的数字。

A: 精确要看 Megatron 2022 paper 公式 5-6。简易估: $\text{act} \approx (s \cdot L \cdot B \cdot S \cdot H)$, $s \approx 34$ (unfused) 或 17 (fused, TE)。7B: $L = 32, H = 4096$ 。 $s = 17$ (TE): $17 \times 32 \times 1 \times 8192 \times 4096 \times 2 = 36 \text{ GB}$ 。开 selective recompute 后 attention 部分省 10 – 15 GB, 剩 20 GB。

面试考点. **Q7**: 什么是“three walls”? 举一个案例说明它们互相耦合。

A: Memory / Communication / Compute Efficiency Wall。案例: 为解决 memory wall 引入 EP=64 切 MoE → 暴露 communication wall (a2a 变主要开销) → 上 DualPipe overlap → 需要 2× 权重副本 → 又拉高 memory wall → 用 FP8 activation 压缩 → 引入 quantize/dequantize kernel + 小 group 计算 → 拉低 compute efficiency wall → 用 grouped GEMM + CUDA graph 缓解。整个链条就是“三堵墙互相制约”的具体展开。

Data Parallel : 从 DP 到 DDP 到 gradient accumulation

Data parallel 是最简单也最基础的并行——每卡持有完整模型，各处理不同 batch 分片，梯度 AllReduce。看起来平淡无奇，但里面 DDP overlap、gradient accumulation 语义、last-batch handling 这些细节是面试高频问点。

`nn.DataParallel` 为什么被淘汰

`torch.nn.DataParallel` (简称 DP, 注意与广义“data parallel”区分):

1. Master GPU (rank 0) 把 batch scatter 到其他 GPU
2. 每 GPU forward, output gather 回 rank 0
3. Loss 在 rank 0 计算, backward
4. 梯度 gather 回 rank 0, rank 0 更新
5. 权重 broadcast 回其他 GPU

问题:

- **rank 0 是 bottleneck**: 显存、算力、通信都不均衡
- **Python GIL**: 单进程多线程, 被 GIL 卡
- 无法跨机: 只支持单进程 (single node)
- 无 **overlap**: 所有通信都在 backward 结束后同步做

`DataParallel` 从 PyTorch 1.x 开始就不推荐了。所有生产用 **DDP (DistributedDataParallel)**。

DDP 的三大改进

1. 每 GPU 一进程: 绕开 GIL, 跨节点原生支持
2. 梯度 **AllReduce** 而非 **gather**: 每卡拿到完整梯度, 各自更新, 权重天然一致
3. **Bucket + backward hook overlap**: 这是核心

Bucket 机制

DDP 在 `__init__` 时把所有 parameter 按顺序切成“bucket” (默认 25 MB 一个), 每个 bucket 是一次 AllReduce 的单位。

Backward 走的时候按相反顺序产生梯度 (output layer 最先), DDP 注册 backward hook: 一个 bucket 内所有 param 的 grad 都算完 → 立刻起一次 **AllReduce (async)**, **backward** 继续往前走。

DDP bucket + backward hook 让 AR overlap 反向计算



图 10 naive DDP 等 backward 全部完成后一次大 AR；bucket DDP 让每个 bucket 就绪就发起 async AR，与后续 backward 计算 overlap。手写实现：`src/distributed_training/02_ddp_from_scratch.py::MyDDP`，与 `torch.nn.parallel.DDP` 输出对齐。

伪代码：

```
class DDP:
    def __init__(self, module, bucket_cap_mb=25):
        self.buckets = split_params_into_buckets(module.parameters(),
                                                  bucket_cap_mb)

        for p in module.parameters():
            p.register_hook(self._grad_ready)

    def _grad_ready(self, grad):
        bucket = self.bucket_of(grad)
        bucket.mark_ready()
        if bucket.all_ready():
            # 起 async AllReduce, 不 block backward
            handle = dist.all_reduce(bucket.buffer, async_op=True)
            self.pending.append(handle)

    def finish_backward(self):
        for h in self.pending:
            h.wait() # 等所有 bucket 的 AR 完成
        for p in self.parameters():
            p.grad = p.grad / self.world_size # 除以 W (平均而非求和)
```

关键：AllReduce 与前面层的 backward compute 时间重叠。理想情况下最后一个 bucket AllReduce 完成的时刻恰好是 backward 结束。

Bucket size 的 tradeoff

- 太小：多次 AllReduce，每次都有启动延迟 α ，总时间 $= n \times \alpha + \frac{V}{B}$ ， α 主导
- 太大：backward 到最后 bucket 还没起 AR，overlap 少

PyTorch 默认 25 MB 是通用值。大模型 (7B+) 可以调到 200-500 MB 减少 overhead。

调节：

```
model = DDP(model, bucket_cap_mb=200, gradient_as_bucket_view=True)
```

`gradient_as_bucket_view=True` 让 `param.grad` 直接 view 到 bucket buffer，省一次 copy——所有 7B+ 训练都建议开。

`find_unused_parameters` 的坑

如果模型里有 parameter 在某些 batch 不参与 forward (例如 MoE 的部分 expert、条件分支), DDP 默认假设所有 param 都会有 grad, 等不到会 hang。

解决: `DDP(model, find_unused_parameters=True)`。

代价: 每步做一次 dtype+shape 的额外遍历判定, **step time** 增加 **10-30%**。所以能不开就不开——重写模型让所有 param 都参与更好。

MoE 常见做法: aux loss 里加一个 tiny term 让所有 expert weight 有梯度 (即使为 0), 避免 `find_unused_parameters`。

Gradient Accumulation : 显存换 batch

现在你想训 GBS=1024, 但每卡显存只够 micro batch (MBS) = 4。累积梯度:

```
model.zero_grad()
for i in range(accum_steps):
    x = next(data_iter)          # MBS = 4
    with model.no_sync() if i < accum_steps - 1 else nullcontext():
        loss = model(x).loss / accum_steps
        loss.backward()
optimizer.step()
```

关键点:

1. `loss / accum_steps` — 让梯度是平均而非求和。或等价: `loss.backward()` 累积后 `p.grad /= accum_steps`。
2. `model.no_sync()` — 前 $n-1$ 步不做 **AllReduce**, 只在最后一步同步。DDP 支持这个 context manager, 省下 `accum_steps - 1` 次 AllReduce。

公式: $GBS = MBS \times accum_steps \times DP_world_size$ 。

一个 512 卡 setup:

- $MBS = 1, accum = 2, DP = 512 \rightarrow GBS = 1024$ (简单)
- $MBS = 4, accum = 4, DP = 64 \rightarrow GBS = 1024$ (更多 DP, 更少 accum, 通信更多但 kernel 更满)

Gradient Accumulation 的隐藏坑

1. **LayerNorm/BatchNorm** 的统计: 如果模型用 BatchNorm, micro-batch stats 只在当前 micro-batch 上算, 不等价于 GBS。所以大模型都用 LayerNorm/RMSNorm 而不 BN。
2. **Dropout**: 每 micro-batch 独立采样, 等价于 GBS 里每个样本各自 dropout — 这是 desired behavior。
3. 学习率 **warmup**: warmup 按步算而非样本。GBS 加倍不代表 warmup 加倍。经验规则 $LR \propto \sqrt{GBS}$ (Chinchilla 建议) 或 linear scaling (Goyal 2017)。
4. 梯度 **clip** 顺序: 应该在 AllReduce 之后 (每步一次), 不是 micro-batch backward 之后。累积期间 grad 是 partial 的, clip 无意义。

DP 的通信量分析

每步 AllReduce 一次全模型梯度:

$$\text{vol}_{\text{DP}} = 2P \frac{W-1}{W} \approx 2P$$

P = 模型参数字节数。LLaMA-2 7B (BF16 grad): $P = 14$ GB, per-step comm = **28 GB**。

带宽 400 GB/s (NVLink) 下要 70 ms ; 50 GB/s (IB) 下 560 ms。如果 step compute 只要 500 ms , DP 通信在跨节点不 overlap 就把 step 拖成 1 秒——2× 慢。

所以 **DDP overlap** 至关重要：把这 28 GB 的 AR 完全隐藏在 backward compute 里。

何时 **DDP overlap** 会失败？

1. Backward 太快，AR 起不完：小模型 + 大集群
2. 通信带宽极差：老 Ethernet
3. Bucket size 不匹配：太小或太大

诊断：torch.profiler 看 AR kernel 是不是 overlap 在 compute 里。

DDP 与 gradient scaling / AMP 的互动

用 torch.cuda.amp.GradScaler (FP16 训练) 时：

```
scaler = GradScaler()
for x in data:
    with autocast(dtype=torch.float16):
        loss = model(x).loss
    scaler.scale(loss).backward()           # scale 后的 loss 反传
    scaler.unscale_(optimizer)              # 取 scale
    torch.nn.utils.clip_grad_norm_(model.parameters(), 1.0)
    scaler.step(optimizer)
    scaler.update()
```

要点：

1. scaler.scale(loss).backward() 里的 scale 值广播 → 每 rank 用相同 scale
2. unscale_ 是本地操作，各 rank 独立除 → 结果一致
3. scaler.update() 里的 found_inf 需要跨 **rank AllReduce** (DDP 自动做，FSDP 要手动)

BF16 不需要 GradScaler (BF16 exponent 范围与 FP32 相同)。所以 H100+ 时代基本弃用 FP16 + GradScaler，直接 BF16。

一个“看似 DP，实际不是 DP”的坑

Batch Norm 在 DP 下会因为各卡独立算 stats 而不等价于单卡大 batch。修复：`nn.SyncBatchNorm.convert_sync_batchnorm(model)` ——跨 rank 同步 mean/var。但每层 BN 会加两次 AllReduce (一次 mean，一次 var)，overhead 显著。大模型不用 BN。

Sequence-level RNG state : dropout 随机 seed。如果每 rank 用同一 seed，DP 里同一个 batch 的 dropout mask 相同 → 变相减少数据多样性。修复：seed = rank 相关。torch 默认 DataLoader 已经处理，但自己写 dataset 时要注意。

DDP 与 batch size：drop_last 陷阱

`DataLoader(dataset, batch_size=B, shuffle=True)` 在 `DistributedSampler` 下：

```
sampler = DistributedSampler(dataset, num_replicas=W, rank=r,
                             shuffle=True, drop_last=False)
loader = DataLoader(dataset, batch_size=B, sampler=sampler)
```

- `drop_last=False` : 最后一个 batch 可能 $< W$ tokens。DDP AllReduce 期待每 rank 都有梯度, 少梯度 **rank** 会 **hang**。修复: `drop_last=True` 或 `pad`。
- `set_epoch(epoch)` 必须每 epoch 调用, 否则 shuffle seed 每 epoch 相同 $\rightarrow$ data 完全一样。这是新手常见 bug。
- 多个 dataset 混合时用 `WeightedRandomSampler` 或 `ConcatDataset` ——但 `DistributedSampler` 只能包一层, 需要自己写。

DDP vs FSDP : 什么时候还用 DDP ?

DDP 是“每卡完整模型”, FSDP 是“参数切片”。DDP 显存更差, 通信更少 (2P vs 3P)。适用:

1. 模型小: $< 1\text{B}$ 参数, 一张卡装得下 $\rightarrow$ DDP 更快
 2. 调试期: DDP 简单, debug 快
 3. 推理时的 **DP**: 如果只是 forward, 无需 grad AR, DDP 相当于 replica broadcast
- 7B+ 训练基本弃 DDP 用 FSDP/ZERO-3。1-7B 之间看具体情况。 $< 1\text{B}$ dense 模型 DDP 最快。

面试考点

面试考点. **Q1**: DDP 里 backward 和 AllReduce 怎么 overlap ?

A: DDP 把 parameter 分 bucket (25 MB), 每 bucket 所有 param 的 grad 完成后立刻起 async AllReduce。因为 backward 是从 output 层往 input 层反向计算, “最靠近 output 的 bucket 最先 ready”, 早期 bucket 的 AR 与后期 bucket 的 backward compute 时间重叠。理想情况 overlap 率 $> 90\%$ 。

面试考点. **Q2**: `no_sync()` 什么时候用? 为什么能省时间?

A: gradient accumulation 时用。累积期间 grad 是“partial”, 不需要同步。DDP 的 `no_sync()` context 暂停 AR, 只在最后一个 micro-batch 后同步一次。`accum_steps=8` 时省 7 次 AR, 直接省 87.5% comm time。

面试考点. **Q3**: `bucket_cap_mb` 调大调小的 tradeoff?

A: 大: 单次 AR 效率高 (带宽项), 但 overlap 空间小 (要等更多 grad ready)。小: 多次 AR, 延迟项累积。7B+ 模型建议 100-500 MB。跨节点 (IB 慢) 时更大更好; 单机 NVLink 时 25 MB 默认够。

面试考点. **Q4**: DDP 的 loss 是每卡各自算的, 最终优化器用的 grad 是“平均”还是“求和”?

A: 平均。`dist.all_reduce(..., op=SUM)` 之后 DDP 内部自动 `grad /= world_size`。所以 loss 直接算不用手动除。用 `all_reduce op=AVG` (NCCL 支持) 更快但依赖 NCCL 版本。

面试考点. **Q5**: `find_unused_parameters=True` 为什么慢?

A: DDP 要跟踪哪些 param 参与了 forward autograd graph, 需要遍历所有 param + hook counting, 每 **step** 都要跑一遍。CPU overhead 显著 (10-30% step time)。而且 finalize_backward 会做额外 AR 判定谁 ready。改进: 重构模型让所有 param 参与 forward (哪怕加 0)。

面试考点. **Q6**: 如果 GBS=1024, 你会选 (DP=64, accum=4, MBS=4) 还是 (DP=256, accum=1, MBS=4)?

A: 主要看通信 overhead 与 kernel 效率的平衡。DP=256 一步一次 AR (跨节点很贵); DP=64+accum=4 只每 4 步 AR 一次 (省 4 $\times$), 但每步 kernel 里 MBS=4 是够大的。跨节点带宽有限时选前者, 同 NVLink 域选后者。实测: H100 8 卡节点上 DP=8 nested DP=8 (across nodes) with accum=4 通常最快。

面试考点. **Q7**: DDP 用 BF16 训练, 为什么不需要 GradScaler?

A: FP16 exponent 只有 5 bit, 动态范围 $\sim 6 \times 10^{-5}$ 到 6×10^4 ; 梯度 underflow 到 0 是常态。GradScaler 把 loss scale $\times 65536$ 让梯度进入 FP16 表示范围。BF16 exponent 8 bit, 动态范围与 FP32 相同 (10^{-38} 到 10^{38}), 无 underflow 问题, 直接省掉 GradScaler。

ZeRO 与 FSDP：把模型切进 DP 组

ZeRO (Zero Redundancy Optimizer, Rajbhandari 2020) 和 FSDP (Fully Sharded Data Parallel, Zhao 2023) 是同一个思想的两个实现：把 **optimizer state / grad / parameter** 沿 DP 组分片，让每卡只持有 $1/W$ 。这是 7B+ 训练的入门砖。

ZeRO 的三个 stage

标准 AdamW 混合精度显存 = 16 bytes/param，分三块：

- Optimizer state (m, v, master weight) : 12 bytes = 75%
- Gradient : 2 bytes = 12.5%
- Weight : 2 bytes = 12.5%

ZeRO 的三 stage 依次分片这三块：

Stage	Optim state	Grad	Weight	每卡 mem
DDP (baseline)	replicate	replicate	replicate	$16P$
ZeRO-1	shard	replicate	replicate	$4P + 12\frac{P}{W}$
ZeRO-2	shard	shard	replicate	$2P + 14\frac{P}{W}$
ZeRO-3 (= FSDP)	shard	shard	shard	$16\frac{P}{W}$

表 8 ZeRO 三 stage 分片对象。 P 是参数量。 $W = \text{DP world size}$ 。ZeRO-3 显存线性降到 $16\frac{P}{W}$ ——1000 卡就能装 6250B 模型的 state。

心算：LLaMA-2 70B 在 8 卡 DDP 每卡 $16 \times \frac{70}{8} = 140 \text{ GB}$ (放不下，会 OOM)；ZeRO-3 每卡 $16 \times \frac{70}{64} = 17.5 \text{ GB}$ (64 卡)；再加 activation，能塞进 80GB H100。

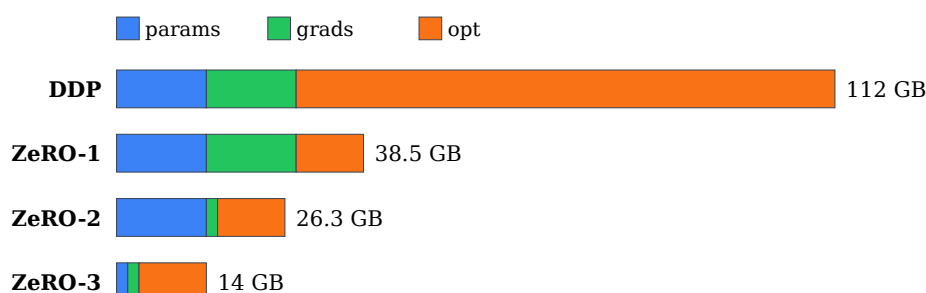


图 11 7B 模型、8 卡、BF16+Adam 的 per-GPU 显存对比 (不含 activation)，DDP 占 112 GB → OOM；ZeRO-3 只 14 GB。

同一个模型换到 64 卡：

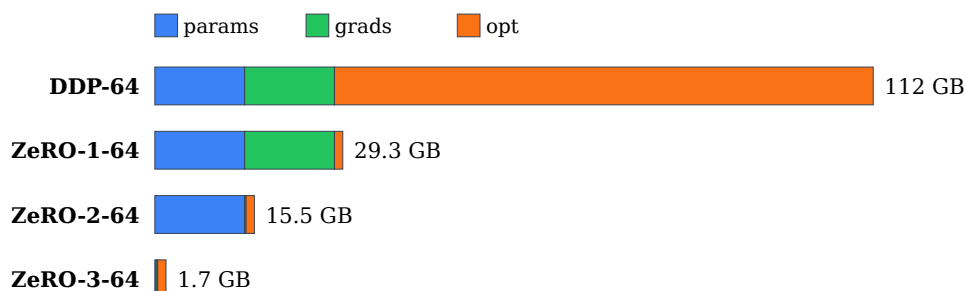


图 12 64 卡场景。ZeRO-3 每卡只需 1.7 GB 存 param/grad/opt，activation 反而成为主导。

估算器：`src/distributed_training/estimators.py::mem_estimate()` ——传入 `zero_stage 0/1/2/3` 即可打印每项 breakdown。

ZeRO 的通信增量

ZeRO 显存换通信。总量分析：

DDP：backward 完做一次 grad AllReduce = $2P$ 字节。

ZeRO-1：

- Backward grad 仍需 AllReduce ($2P$)——因为每卡要有完整 grad ,才知道自己那份 optim state 该 update 什么
- 优化：可以改成 **ReduceScatter** (每卡只需自己 shard 的 grad) = P
- Update 后：每卡只更新自己的 optim state → 只有自己 shard 的 weight 新值 → 需要 AllGather 广播 = P
- Total = $P + P = 2P$ (同 DDP 通信量)

ZeRO-2：

- Backward：ReduceScatter grad = P (每卡拿到自己 shard 的 grad)
- Update：本地做，只更新 shard 的 weight
- Forward 下一步：需要 AllGather weight = P
- Total = $2P$ ✓ 同 DDP

ZeRO-3：

- Forward：每层前 AllGather weight → 层结束 discard = P
- Backward：每层前 AllGather weight → 层结束 discard = P
- Backward end：ReduceScatter grad = P
- Total = $3P$ — 比 DDP 多 50%

总结：

策略	Mem / GPU	Comm / step
DDP	$16P$	$2P$
ZeRO-1	$4P + 12\frac{P}{W}$	$2P$
ZeRO-2	$2P + 14\frac{P}{W}$	$2P$
ZeRO-3	$16\frac{P}{W}$	$3P$

表 9 ZeRO 显存 vs 通信。三 stage 里唯有 stage-3 增加通信量 —— 但换来的显存收益让训 100B+ 成为可能。

Key insight. 面试常问：“ZeRO-3 比 DDP 通信量多 50%，为什么大家还用它？”答：因为你本来就装不下。DDP 在 70B 模型上根本跑不起来，多 50% 通信换来能训——这是没得选。若模型足够小 (< 5B)，DDP 更快是对的，实际上 PyTorch 的 FSDP `NO_SHARD` 模式就是 DDP。

FSDP：PyTorch 原生实现

FSDP 是 ZeRO-3 的 PyTorch 官方实现，语义等价但 API 更 pythonic。基本用法：

```
from torch.distributed.fsdp import FullyShardedDataParallel as FSDP
from torch.distributed.fsdp import (
```

```

MixedPrecision, ShardingStrategy, BackwardPrefetch,
CPUOffload,
)
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
from functools import partial

mp = MixedPrecision(
    param_dtype=torch.bfloat16,
    reduce_dtype=torch.bfloat16,
    buffer_dtype=torch.bfloat16,
)
wrap_policy = partial(transformer_auto_wrap_policy,
                      transformer_layer_cls={LlamaDecoderLayer})

model = FSDP(
    model,
    sharding_strategy=ShardingStrategy.FULL_SHARD,    # = ZeRO-3
    auto_wrap_policy=wrap_policy,
    mixed_precision=mp,
    backward_prefetch=BackwardPrefetch.BACKWARD_PRE,
    device_id=torch.cuda.current_device(),
    use_orig_params=True,                          # 推荐
)

```

auto_wrap_policy 是最重要的旋钮

FSDP 是“逐 unit AllGather”。unit 太大：一次 AG 很多参数，显存峰值高。unit 太小：AG 次数多，overhead 高。

正确做法：每 Transformer layer 一个 FSDP unit。这样每层 AG 140 MB (7B / 32 层)，够大不掉带宽。

错误做法 1：整个 model 一个 unit → 相当于 DDP，没切 错误做法 2：每个 `nn.Linear` 一个 unit → AG 每次 40 MB，一层 forward 6 次 AG，overhead 爆炸

ShardingStrategy 四种

Strategy	ZeRO 等价	Weight / Grad / Optim	用途
NO_SHARD	DDP	replicate / replicate / replicate	小模型
SHARD_GRAD_OP	ZeRO-2	replicate / shard / shard	模型能装下，只切 grad+optim
FULL_SHARD	ZeRO-3	shard / shard / shard	大模型标配
HYBRID_SHARD	HSDP	intra-node shard, inter DDP	多节点大模型
<code>_HYBRID_SHARD_ZER02</code>	HSDP+ZeRO-2	intra-node shard weight, inter DDP grad	新选项

表 10 FSDP sharding strategy。HYBRID_SHARD 是跨节点场景的关键——见下节。

HYBRID_SHARD：跨节点场景的最优解

问题：FULL_SHARD 在 1024 卡的集群上做每层 AllGather，跨节点带宽 (IB 50 GB/s) 慢一个数量级。

思路：节点内 **FULL_SHARD (ZeRO-3)**，节点间 **DDP**。每节点各持完整模型的 1/8 (节点内 8 卡共享)，节点间只做 grad 的 AllReduce。

```
FSDP(
    model,
    sharding_strategy=ShardingStrategy.HYBRID_SHARD,
    device_mesh=DeviceMesh("cuda", (num_nodes, 8),
                           mesh_dim_names=("dp", "shard")),
)
```

通信分析 (每 step, 每卡):

- Intra-node AG (NVLink): $2\frac{P}{8} = \frac{P}{4}$ (fwd + bwd)
- Intra-node RS (NVLink): $\frac{P}{8}$
- Inter-node AR (IB): $2\frac{P}{8} = \frac{P}{4}$ (因为 grad 已经 shard, 只 AR 自己那份)

对比 pure FULL_SHARD (1024 卡): 所有 AG/RS 都跨节点 = $3P$ IB volume. HSDP 只 $\frac{P}{4}$ 。wall-clock 5× 加速。

这是 Llama-3 405B 训练用的方案。

use_orig_params=True 是什么？

FSDP 内部把多个 param flatten 成一个大 buffer ("FlatParameter")。旧版 API 里 optimizer.param_groups 看到的是 FlatParameter 而不是原 name——想按层设 lr / weight decay 会难。

use_orig_params=True (推荐): optimizer 仍然看到原始的 named_parameters(), backward 时 FSDP 内部处理 flatten。付出小的 index bookkeeping 开销, 换来 API 一致性。

BackwardPrefetch

Backward 里第 l 层的 AllGather 什么时候起？

- **BACKWARD_PRE**: 在第 l 层的 backward 开始前 prefetch → overlap 与第 $l+1$ 层 backward compute (激进, 最快, 但显存峰值高)
- **BACKWARD_POST**: 在第 l 层 backward 结束后起 → overlap 更保守 (少 overlap, 显存低)
- **None**: 不 prefetch, 最省内存最慢

大模型训练建议 BACKWARD_PRE; OOM 再降到 POST。

FSDP2 (torch.titan / PyTorch 2.4+)

FSDP1 有几个痛点: FlatParameter、composability 差、动态 shape 支持差。FSDP2 (torch.distributed._composable.fsdp.fully_shard) 重写了:

1. **Per-parameter sharding**: 每 param 独立 shard, 不再 flatten
2. **DTensor** 后端: sharding meta 用 DTensor 表达, 与 DeviceMesh 完全集成
3. 更好的 **optimizer** 交互: optim state 自动 shard
4. **TP × FSDP composability**: FSDP2 与 TP 组合更自然


```

from torch.distributed._composable.fsdp import fully_shard
from torch.distributed._tensor import DeviceMesh

mesh = DeviceMesh("cuda", (num_nodes, 8), mesh_dim_names=("dp_replicate",
"dp_shard"))
for layer in model.layers:
    fully_shard(layer, mesh=mesh["dp_shard"])
fully_shard(model, mesh=mesh["dp_shard"]) # root last

```

torchtitan 的 Llama 3 训练用 FSDP2 + TP + PP。FSDP2 是未来。

ZeRO-Offload / ZeRO-Infinity : CPU / NVMe 扩展

DeepSpeed 独有：

- **ZeRO-Offload** (Ren 2021)：把 optim state + FP32 master weight 放 CPU，update 在 CPU 用 optimized Adam。GPU 只保留 weight/grad。
- **ZeRO-Infinity** (Rajbhandari 2021)：更进一步 offload 到 NVMe，理论支持万亿参数模型。

收益：单卡训 10B 模型（原本装不下）。代价：每步 optim update 走 PCIe (63 GB/s)，慢一个数量级。

生产训练现在很少用（有钱直接多买卡）。学术复现 / 单机大模型 fine-tune 场景仍常见。

FSDP 也支持 `CPUOffload(offload_params=True)`——把 param shard 存 CPU，AG 前拉到 GPU。7B fine-tune 单卡 4090 24GB 常用。

与 gradient accumulation / activation checkpoint 的组合

ZeRO-3 + gradient accumulation：

1. FSDP `no_sync()` 支持——但要小心，`no_sync` 里 grad 是完整 unshard 保存的，显存翻 **W** 倍（本来 shard 存的现在每卡完整）
2. 更好：直接不用 `no_sync`，让每 micro-batch 都做 RS——反正 RS 与 backward compute overlap，与不做 RS 差不多

ZeRO-3 + activation checkpoint：完全兼容。checkpoint 层里 backward recompute 时需要 AllGather 参数——FSDP 自动处理，一样 prefetch。

ZeRO-3 + TP：FSDP2 支持组合。TP 组内切 tensor，TP 组间 FSDP 切 param。DeviceMesh 二维 (tp, dp) 表达。

ZeRO 与 TP 的分工

面试常问：“既然 ZeRO-3 已经把 weight 切 W 份，为什么还要 TP？”

答：**ZeRO-3** 切的是“权重存储”，不切“权重使用”——forward 时还要 AllGather 回来算，然后 discard。TP 是切“权重使用”——GEMM 直接在切分的 weight 上算，输入/输出走 AllReduce。

区别：

- ZeRO-3：显存减少，**compute** 不变（因为算之前 gather 回全 weight）
- TP：显存减少，**compute** 也切分（每卡只算一部分 GEMM）

所以：

- ZeRO-3 一层的 activation 与不 **shard** 时一样大（forward 时 param 是完整的）

- TP 一层的 activation 沿 **TP** 维切分 (GEMM output 是切分的)

对小模型 + 大 **batch** :ZeRO-3 activation 打满显存,TP 反而不会。对大模型 + 小 **batch** :ZeRO-3 够用,TP 增加通信开销不划算。

一个经验规则: 模型 < 20B 时 FSDP + PP 就够; 模型 > 20B 或 seq > 4K, 加 TP。

FSDP 常见坑

1. `optim.step` 后需要 `optim.zero_grad(set_to_none=True)`: FSDP2 里 grad 存在 shard buffer, `set_to_none` 让 FSDP 内部知道可以释放
2. `torch.save(model.state_dict())` 是 **shard** 的: 加载需要 same world size, 或用 `torch.distributed.checkpoint` (DCP) 保存 rank-agnostic 格式
3. 混合精度 **buffer** 的 **dtype**: BN running_mean 之类 buffer 默认 FP32, 会跟 param BF16 类型不匹配。手动 `MixedPrecision(buffer_dtype=torch.bfloat16)`
4. **AC (activation checkpoint)** 与 **FSDP** 的组合顺序: 正确顺序是 `apply_ac(module)` → `FSDP(module)`。反过来 FSDP 内部 AllGather 与 AC 的 detach 不能配合
5. **frozen parameters**: 不参与训练的 param 也需要 FSDP unit 处理, 否则 AllGather 语义错。用 `torch.no_grad()` 里的 param 要放独立 FSDP wrap 或者不 wrap

一个 FSDP 完整训练脚本骨架

```
import torch, torch.nn as nn
import torch.distributed as dist
from torch.distributed.fsdp import (
    FullyShardedDataParallel as FSDP,
    MixedPrecision, ShardingStrategy,
    BackwardPrefetch,
)
from torch.distributed.fsdp.wrap import transformer_auto_wrap_policy
from torch.distributed.device_mesh import init_device_mesh
from functools import partial

def main():
    dist.init_process_group(backend="nccl")
    rank = dist.get_rank(); world = dist.get_world_size()
    torch.cuda.set_device(rank % 8)

    # HSDP: 8 卡节点内 shard, 节点间 replicate
    mesh = init_device_mesh("cuda", (world // 8, 8),
                           mesh_dim_names=("dp_replicate", "dp_shard"))

    from transformers import AutoModelForCausalLM
    model = AutoModelForCausalLM.from_pretrained("meta-llama/Llama-2-70b-hf",
                                                  torch_dtype=torch.bfloat16)

    # 打 gradient checkpoint
    model.gradient_checkpointing_enable()

    mp = MixedPrecision(param_dtype=torch.bfloat16,
                        reduce_dtype=torch.bfloat16,
                        buffer_dtype=torch.bfloat16)
```

```

from transformers.models.llama.modeling_llama import LlamaDecoderLayer
wrap = partial(transformer_auto_wrap_policy,
                transformer_layer_cls={LlamaDecoderLayer})

model = FSDP(model,
              sharding_strategy=ShardingStrategy.HYBRID_SHARD,
              device_mesh=mesh,
              auto_wrap_policy=wrap,
              mixed_precision=mp,
              backward_prefetch=BackwardPrefetch.BACKWARD_PRE,
              use_orig_params=True,
              device_id=rank % 8)

optim = torch.optim.AdamW(model.parameters(), lr=1e-4, fused=True)

for step in range(100000):
    batch = next(loader)
    loss = model(**batch).loss
    loss.backward()
    # optional: grad clip
    model.clip_grad_norm_(1.0)          # FSDP-aware
    optim.step()
    optim.zero_grad(set_to_none=True)
    if step % 1000 == 0 and rank == 0:
        print(f"step {step} loss {loss.item():.3f}")

```

面试考点

面试考点. Q1: ZeRO-3 通信量比 DDP 多 50%，为什么还用？

A: DDP 装不下大模型。ZeRO-3 显存 $16 \frac{P}{W}$ 让 100B+ 模型可训。多 50% 通信换 W 倍显存减少，量纲不对等，在“不切装不下”的场景是没得选。

面试考点. Q2: HSDP 相比 pure FSDP 收益在哪？

A: FSDP AllGather 是每层都做，跨节点走 IB (50 GB/s)。HSDP 让 AG 只在节点内 (NVLink 400 GB/s)，跨节点只做 grad 的 AllReduce (一次，且 grad 已经 shard)。跨节点通信从 $3P$ 降到 $2 \frac{P}{n}$ (n 节点数)，wall-clock 5-10× 加速。Llama-3 训练用的。

面试考点. Q3: FSDP auto_wrap_policy 怎么选？

A: 每 Transformer layer 一个 unit。unit 太大退化到 DDP (一次 AG 打满显存)，太小 AG 次数爆炸 (overhead 大)。用 `transformer_auto_wrap_policy(transformer_layer_cls={YourLayer})` 是通用做法。

面试考点. Q4: FSDP + activation checkpoint 的顺序对吗？

A: 先 `apply AC` 再 `FSDP wrap`。反过来 `FSDP` 内部会把整层 `AllGather` 出来后 `checkpoint discard`, `backward` 时需要 `re-AllGather` ——语义乱。`torch.distributed.checkpoint_wrapper` 提供 `apply_activation_checkpointing(model, ...)`, 之后再 `FSDP`。

面试考点. **Q5**: 用 `use_orig_params=True` 有什么好处?

A: `optimizer.param_groups` 看到的是原始 `named_parameters()` 而不是 `FlatParameter`, 可以按 `name` 设 `lr/wd/frozen`。小的 `bookkeeping` 开销, 几乎所有生产训练都开。

面试考点. **Q6**: `FSDP` 里 `MixedPrecision.reduce_dtype=bf16` 会不会导致数值问题?

A: 有可能。BF16 mantissa 只 7 bit, 大 `world_size` (1024+) 下 `AllReduce` 累加误差累积。DeepSeek/Megatron 的做法: `grad` 用 BF16 通信 AR 后 upcast 到 FP32 累加, 或者 `reduce_dtype=torch.float32` ——通信量翻倍但精度稳。经验: < 512 卡 BF16 AR 一般 OK, > 1024 卡建议 FP32 `grad AR`。

面试考点. **Q7**: ZeRO-3 forward 时 `AllGather param` 之后 `discard`, 反向又要 `AllGather` 一次——为什么不缓存?

A: 因为 `activation` 才是“每层被 `fwd/bwd` 都用到”的东西, `param` 缓存下来就等于没 `shard`。的确 `fwd` 和 `bwd` 各 AG 一次, 比 DDP 多一次 AG。可以做的 mitigation 是 `limit_all_gathers=True` 里预取更远的层, 让 AG 与前面 `compute overlap`; 但绝对通信量省不了。这是 ZeRO-3 与 DDP 的固有 P 差。

面试考点. **Q8**: ZeRO-3 vs TP, 哪个更能扩到大 `world_size`?

A: ZeRO-3 (FSDP)。TP `AllReduce` 每层 `activation`, 与 `batch/seq` 相关; `world_size` 大了 `activation` 无法充分利用 TP 组带宽。ZeRO-3 通信量与 `param` 相关 ($3P$, W 无关), 带宽利用率高。所以 Llama-3 405B 用 FSDP2 主切, TP 只用 8 (NVLink 域内)。

Tensor Parallel 与 Sequence Parallel

Tensor Parallel (TP) 由 Megatron-LM (Shoeybi 2019) 提出，把单个 matmul 沿某一维切给多张卡，在层内完成一次 AllReduce 拼回结果。Sequence Parallel (SP, Korthikanti 2022) 补上 TP 没切的 LN / Dropout activation。两者结合是 dense LLM 训练标配。

TP 的核心思想：一层 GEMM 里就切

考虑一个 FFN: $Y = XW_1W_2$, $W_1 \in \mathbb{R}^{H,4H}$, $W_2 \in \mathbb{R}^{4H,H}$ 。

Column parallel W_1 : 把 W_1 沿 output (4H) 维切成 $W_1 = [W_1^{(1)}, W_1^{(2)}]$ 。

$$XW_1 = [XW_1^{(1)}, XW_1^{(2)}]$$

每卡各自算，得到 activation 沿 output 维切分。不需要通信。

Column-parallel weight (out dim sharded across TP=4)

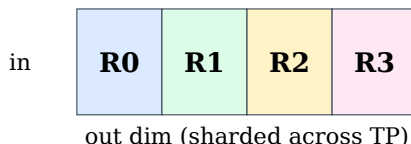


图 13 列并行: W 沿 out 维 (4H) 切，输入 X 复制。每卡独立算 $XW^{(i)}$ ，输出天然沿 out 维切分，不需通信。

Row parallel W_2 : 把 W_2 沿 input (4H) 维切成 $W_2 = \begin{pmatrix} W_2^{(1)} \\ W_2^{(2)} \end{pmatrix}$ 。

$$Y = XW_2 = X^{(1)}W_2^{(1)} + X^{(2)}W_2^{(2)}$$

每卡拿到自己切分的 input 与 row，各算部分和；最后 **AllReduce** 求和拼回。

Row-parallel weight (in dim sharded across TP=4)

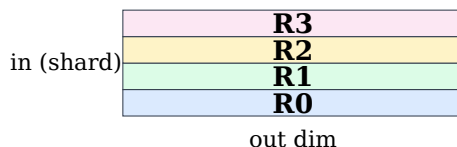


图 14 行并行: W 沿 in 维切，输入必须已按 in 维切分。每卡算部分和，最后一次 AllReduce 求和。

连起来: $X \rightarrow XW_1$ (column, 无通信) $\rightarrow$ activation (Y_1) 沿 4H 切分 $\rightarrow$ 送 GELU (elementwise, 无通信) $\rightarrow$ 送 W_2 (row) $\rightarrow$ **AllReduce**。整个 FFN 只 1 次 AllReduce (forward) + 1 次 (backward)。

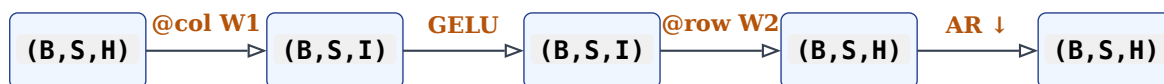


图 15 Megatron FFN 的 column-then-row 数据流。整个 FFN 只在最后一次 AllReduce (forward), backward 再一次。

代码见 `src/distributed_training/04_tp_column_row.py`——用 200 行实现了 `ColumnParallelLinear + RowParallelLinear + TP FFN + TP Attention`，并与单卡 baseline 数值对齐 (`torch.allclose`)。

这就是 Megatron 的 “column-then-row” pattern。所有 Transformer 组件 (attention, MLP) 都能这样组织。

Attention 的 TP 分解

Multi-head attention 天然按 head 切：

- W_Q, W_K, W_O 沿 head 维 column parallel (每 head 独立)
- Attention compute (QK^T , softmax, PV) 本地做，不需要通信
- W_O row parallel + AllReduce

每个 attention 层 forward 1 次 AllReduce，backward 1 次。加上 FFN 的两次，一整层 4 次 AllReduce。

GQA / MQA：head 数少 (Grouped Query) 时 K/V 沿 head 切会遇到 head 不够分的问题。做法：K/V 复制到多 TP rank (重复存储)，Q 正常切。或者反过来 KV 沿 head 切 Q 复制——各 TP 组内做。

通信量与效率

每层 forward + backward 4 次 AllReduce。activation 大小 = $BSH \times bs$ 。TP=8 时每次 AR：

$$\text{vol}_{\text{TP}} = 2 \frac{W-1}{W} \times BSH \times bs \approx 2BSH \text{ bs}$$

$B = 1, S = 4096, H = 8192, bs = 2$ (BF16)：= 128 MB per AR。32 层 $\times$ 4 次 $\times$ 128 MB = 16 GB per step。

TP=	每 AR bytes	每层 AR 数	400GB/s NVLink 每层耗时
2	$\frac{BSH}{2} \cdot b \cdot 2$	4	0.16 ms
4	$\frac{3}{4} \cdot BSH \cdot b \cdot 2$	4	0.24 ms
8	$\frac{7}{8} \cdot BSH \cdot b \cdot 2$	4	0.28 ms
16	$\frac{15}{16} \cdot BSH \cdot b \cdot 2$	4	0.30 ms

注意随 TP 翻倍通信 bytes 增速 $\rightarrow$ per-GPU compute 减半，通信/计算比翻倍。这就是 TP 不能拉太大的根本原因。参考 `src/distributed_training/estimators.py::comm_volumes()` 里 TP 部分。

TP 只能在高带宽域内用——AllReduce 频率是 DP 的 $32\times+$ (每层 vs 每 step)。跨节点 IB 不够快，会把 step time 拖垮。

铁律： $\text{TP} \leq$ 同 NVLink 域大小。H100 NVL8 只能 TP=8，GB200 NVL72 可以 TP 到 72 (但没意义，见下)。

为什么 **TP > 8** 通常不划算

1. **AllReduce** 时间： $T_{\text{AR}} \propto \frac{W-1}{W}$ ， $W = 8 \rightarrow 87.5\%$ ， $W = 16 \rightarrow 94\%$ ， $W = 64 \rightarrow 98\%$ 。加一倍卡通信只增 6%——但 **compute** 每卡少一半。带宽利用率骤降。

2. **Activation** 也在 **TP** 组内切：TP=8 $\rightarrow$ activation 每卡 $\frac{1}{8}$ ；TP=64 $\rightarrow$ 每卡 $\frac{1}{64}$ 。但每次 AllReduce 都要传送近 H 的完整 activation $\rightarrow$ 通信量与 TP 无关，即 **TP** 越大越不划算。

DeepSeek-V3 明确不用 **TP**，全用 EP + FSDP。GB200 NVL72 上 Megatron 也建议 $\text{TP} \leq 8$ 而不是拉到 72。

Sequence Parallel (SP)：把 LN 也切了

Megatron TP 里，一层 forward 的顺序：

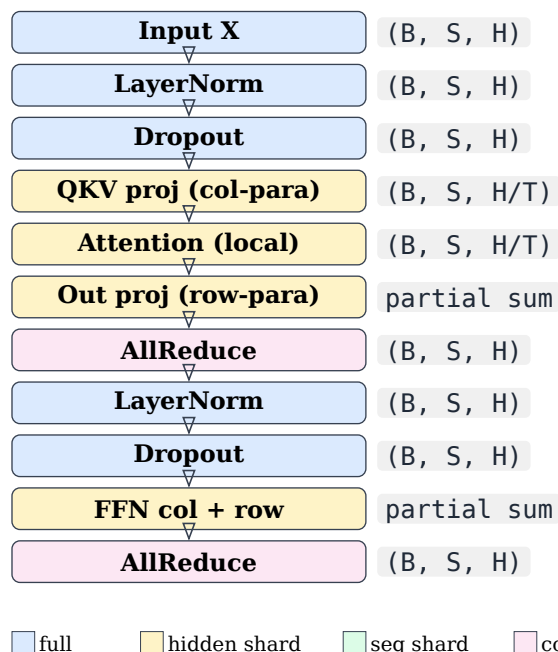


图 16 Megatron TP 一层 forward 顺序。黄色格表 hidden 维已切分（每 rank 只留 $\frac{1}{T}$ ），蓝色格表张量仍是完整 (B, S, H) —— 这就是 LN/Dropout 显存痛点：TP 完全没帮上忙。粉色格是每层的两次 AllReduce。

痛点：LN 和 dropout 的 activation 是 BSH （完整），TP 完全没帮上忙。对 $B=1, S=32K, H=8192$ ：一个 layer 的 LN activation = 512 MB。80 层 = 40 GB，光 LN activation 就爆显存。

SP 的解决：LN + Dropout 沿 **sequence** 维切分。这段 $X \in (B, \frac{S}{TP}, H)$ ，每卡处理 $\frac{1}{TP}$ 长度的序列。

通信改造：

- LN 出来 $(B, S/TP, H) \rightarrow$ 进 QKV projection 前需要 AllGather 到 (B, S, H)
- Output projection 出来 $(B, S, H, \text{partial}) \rightarrow$ 直接 ReduceScatter 到 $(B, S/TP, H)$

原来 forward 一层的 2 次 AllReduce 变成 2 次 AllGather + 2 次 ReduceScatter。回顾第 1 章：AllReduce = AG + RS，通信量完全相同！



图 17 SP + TP 数据流。SP 区间张量沿 seq 维切成 $(B, \frac{S}{T}, H)$ （LN/Dropout 只吃 $\frac{1}{T}$ 显存）；进入 TP 区间前 AllGather 到 (B, S, H) ；TP compute 完 ReduceScatter 回 SP 区间。

收益：LN/Dropout activation 显存从 $2BSH$ 降到 $2BS\frac{H}{TP}$ ，别的照常。TP=8 时 activation 省 87.5%。

代价：0（通信量不变），仅代码复杂化。所有 Megatron 生产训练都开 SP。

Megatron flag: `--sequence-parallel`。

Note. SP 把 activation 沿 seq 维切开之后，模型里所有跨 **token** 计算的模块都被切断了——loss 的分母、MoE 的 router aux loss、pooling 头、grad norm、以及 label shift 这类只差一个 token 的位移。这些改造与 CP 完全共用一套规则（SP 和 CP 本质都是 sequence sharding），统一放在 §7 讲。

这里只强调一点：LN/Dropout 本身不需要任何通信就能在 seq-shard 上正确执行，因为它们沿 hidden 维归一化、逐 token 独立——这正是 SP 能零通信成立的原因。

TP-only 与 TP+SP 数据流对比

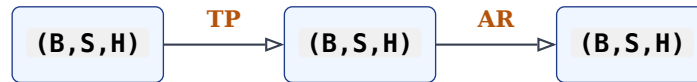


图 18 **TP-only** :整条链上张量都是完整 (B, S, H) ,每层一次 AllReduce。LN/Dropout activation 显存无节省。



图 19 **TP + SP** : SP 区间张量沿 seq 维切成 $\frac{S}{T}$,只在 TP 区间还原为完整 (B, S, H) 。通信量与 TP-only 完全相同 ($AG + RS = AR$) ,但 LN/Dropout 显存降到 $\frac{1}{T}$ 。

AG + RS 与原来的 AR 完全等价 , NCCL 也可以 fuse。

Async TP / TP overlap

TP 的 AllReduce 是同步的——AG 完才能 compute。Megatron-Core 2024 加了 async TP :

- `--tp-comm-overlap` :把 AR 拆成“partial + delayed” , 与 GEMM overlap
- 依赖 Transformer Engine (TE) 的 fused `LayerNormMLP` / `LayerNormLinear` ——kernel 里 chunked GEMM , 边算边通信

收益 : TP=8 通信占 15-20% → 5-8%。生产 Llama 70B 训练一定开。

Expert Tensor Parallel (ETP)

MoE 场景下 : expert 内的 GEMM ($H \rightarrow I \rightarrow H$) 也可以 TP 切。Megatron-Core flag: `--expert-tensor-parallel-size 2`。

用法 : 如果 expert 权重太大 (I 很大) , 单卡装不下。等价于把 EP 组内的每个 expert 再切 ETP 份 , 需要在 expert 内再做 AllReduce。

DeepSeek-V3 用 EP=64 + ETP=1 (不切 expert) , 因为 fine-grained expert 权重本来就小。Mixtral 8×22B 有时用 EP=8 + ETP=2。

TP × PP × DP × EP 的拓扑排布

Rank 分配的物理意义 (Megatron 默认顺序):

```
world = TP × CP × EP × DP × PP
        (最内)           (最外)
```

即 : TP 组是相邻 rank (同 node 内的 NVLink 高带宽) , DP 组跨 node , PP 组更外层。

例 : world=64, TP=8, PP=2, DP=4:

- TP 组: {0-7}, {8-15}, ..., 每组同 node
- DP 组: {0, 8, 16, 24}, ..., 跨 node
- PP 组: {0, 32}, {1, 33}, ..., 跨 node

TP=8 组: 每行的 8 张卡 (同 NVLink 域)

G0	G1	G2	G3	G4	G5	G6	G7
G8	G9	G10	G11	G12	G13	G14	G15

图 20 world=64 TP=8 PP=2 DP=4 的物理布局。每 8 张卡组成一个 TP 组 (需要同 NVLink 域); 两行是 PP 两个 stage; DP 沿多个 node 复制。

调整原则:

1. TP 组必须同 **NVLink** 域 (NVLink 最多 TP=8)
2. EP 组尽量同 **NVLink** 域, 否则 all-to-all 会慢 (DeepSeek 用 hierarchical a2a 缓解)
3. DP 组可以跨节点 (AllReduce 对带宽宽容)
4. PP 组可以跨节点 (P2P send 只在阶段过渡时发生)
5. CP 组同 **NVLink** 域最好 (下章)

与 FSDP 组合

Megatron-Core 传统 TP+PP+DP 是自己实现的三维网格。torch-titan 用 FSDP2 + TP:

```
mesh = init_device_mesh("cuda", (dp, tp), mesh_dim_names=("dp", "tp"))

for layer in model.layers:
    parallelize_module(layer, mesh["tp"], {
        "attention.wq": ColwiseParallel(),
        "attention.wk": ColwiseParallel(),
        "attention.wv": ColwiseParallel(),
        "attention.wo": RowwiseParallel(),
        "mlp.w1": ColwiseParallel(),
        "mlp.w3": ColwiseParallel(), # SwiGLU 的 gate
        "mlp.w2": RowwiseParallel(),
    })
    fully_shard(layer, mesh=mesh["dp"])

fully_shard(model, mesh=mesh["dp"])
```

用 DTensor 的 `ColwiseParallel` / `RowwiseParallel` 标注每个 module, PyTorch 底层自动插入 AllGather/AllReduce。torch-titan Llama-3 70B 用这个跑到 40%+ MFU。

TP 的坑

1. **LayerNorm weight/bias** 是 **replicated**: 每 TP rank 都有一份, 梯度合并要 AR (DDP 自动)。不合并会 drift
2. **nn.Embedding** 的 **TP**: Vocab 大 (128K+) 时 embedding 是大头。Megatron 用 “vocab-parallel embedding”: vocab 沿 TP 切, index 到不同 rank, output 用 AllReduce
3. **loss** 计算: cross_entropy 的 logits 是 (B, S, V), V 大时是巨大 activation。Megatron 有 fused vocab-parallel cross entropy, 直接在切分的 logits 上算。TE 也有 `parallel_cross_entropy`
4. 确定性 **seed**: TP 组内 dropout mask 必须完全一致 (因为不同 rank 处理同一 sample 的不同切片, dropout 语义要求跨 rank 一致)。set seed by (rank)
5. 梯度合并顺序: TP replicated param (LN weight) 的梯度需要在 TP 组内 AR + DP 组间 AR。顺序: 先 TP, 再 DP。DDP 里 `process_group=tp_group` 单独 wrap

6. **checkpoint save/load** : TP 权重是 shard 的, checkpoint 要按 TP rank 分片存。Megatron 有 dist ckpt, torchtitan 用 `torch.distributed.checkpoint`

面试考点

面试考点. **Q1**: 为什么 Megatron 的 FFN 是“column-then-row”, 反过来能行吗?

A: 反过来 (row-then-column) 每层需要 2 次 AllReduce (row 之后一次, column 之后又一次)——但 column 出来的 activation 无需 AR。column-then-row 只需 row 之后一次 AR。语义等价但通信量差 2×。所以选前者。

面试考点. **Q2**: TP=8 一层 forward 通信几次? activation 显存怎么算?

A: FFN 1 次 AR, attention 1 次 AR, 总 2 次 forward + 2 次 backward = 4 次。开 SP 后是 2 次 AG + 2 次 RS = 4 次 (volume 相同)。activation 显存: TP 只切 QKV/FFN 中间 activation ($\frac{1}{TP}$), LN 部分不切。开 SP 后 LN activation 也切 → 全部 $\frac{1}{TP}$ 。

面试考点. **Q3**: Sequence Parallel 通信量与只用 TP 一样, 那 SP 为什么“值得做”?

A: SP 不改变通信量 (AG+RS = AR), 但把 LN / Dropout 的 activation 显存从 BSH 降到 $BS\frac{H}{TP}$ 。对长序列 (S=32K+) 这一项能省 GB 级显存, 允许更大 batch 或更长 seq。零通信代价的免费显存。

面试考点. **Q4**: 为什么 TP 不超过 8?

A: (1) 物理: NVLink 域 8 卡, 跨域走 IB 太慢; (2) AR 通信量随 TP 增长: $2\frac{W-1}{W}$ 逼近 2 后不再省, 但 compute 每卡还是 $\frac{1}{W}$ 降; (3) activation 沿 TP 切但 AR 通信量不变——TP 越大越不划算。GB200 NVL72 上 Megatron 建议 $TP \leq 8$ 而非 72。

面试考点. **Q5**: Async TP 是怎么 overlap 的?

A: `--tp-comm-overlap` (Megatron + TE)。把 GEMM 拆成 tile, AR 也拆成 chunked ReduceScatter+AllGather。GEMM tile k 算完后立刻起 chunk k 的通信, tile $k+1$ 继续算, 通信与后续 tile compute overlap。要 kernel 支持 (TE `LayerNormLinear` / `LayerNormMLP fused`)。TP 通信占比 15-20% → 5-8%。

面试考点. **Q6**: GQA (num_kv_heads=8) 在 TP=8 情况下 K/V 怎么切?

A: K/V 只有 8 个 head, 正好每卡 1 个, OK。如果 TP=16 而 num_kv_heads=8: K/V 每卡 0.5 head 不行——要么复制 K/V (每 2 个 TP rank 存一份完整 KV head), 要么调 TP=8。Llama-2/3 里 GQA=8 是特意为 TP=8 设的。

面试考点. **Q7**: TP + ZeRO-3 一起用有意义吗?

A: 有。ZeRO-3 沿 DP 组切 param, TP 沿 TP 组切 tensor——正交维度。TP=8 且 DP=64 时 param 切 512 份。torchtitan 就是这样跑 Llama-3 70B。FSDP1 + TP 组合有 bug, 用 FSDP2。

面试考点. **Q8**: 为什么 embedding layer 也要 TP 切 ?

A: Llama-3 vocab=128256 , embedding weight = $128256 \times 8192 \times 2 \text{ bytes} = 2 \text{ GB}$ 。TP=8 切 4 份不算很省 , 但 loss 计算里 logits (B, S, V) 是 4 GB (S=4K) , 不切显存爆。Megatron 的 vocab-parallel embedding 把 V 沿 TP 切 , logits 也切分 , 最后 fused cross_entropy 直接在切分 logits 上算 , 省下 loss activation。

Pipeline Parallel : 从零造一个出来

PP 的理论只有三个公式，一页纸讲得完：bubble 是 $\frac{P-1}{m+P-1}$ ，1F1B 省显存，interleaved 把 bubble 除以 V 。真正的难点全在实现，而实现里最关键的那件事，任何一张时空图都不会告诉你：

Warning. autograd 不跨进程。loss 只在最后一个 stage 上存在，stage 0 上调 `loss.backward()` 是不可能的——计算图在 `send / recv` 那里就断了。所以 PP 的核心不是“怎么排 F 和 B”，而是手工把链式法则在网络边界上接起来。排程只是接好之后的调度问题。

这一章按“造轮子”的顺序走：先接 autograd 边界，再把 schedule 变成数据，再解决 P2P 的死锁与形状协商，再拆 B/W，最后接进真实训练循环。每一步都有可运行、可验证的代码：

文件	内容
<code>common/pipeline.py</code>	引擎本体：4 个 schedule 生成器、B/W 拆分、P2P 层、约 40 行的解释器
<code>21_pp_from_scratch.py</code>	autograd 边界、真实复现 P2P 死锁、形状协商、四种 schedule 对齐单卡参考
<code>22_pp_schedule_sim.py</code>	依赖图模拟：实测 bubble 对闭式公式、ASCII 时空图、activation 峰值
<code>23_pp_integration.py</code>	rank 排布、按耗时分层、首尾权重共享、loss 缩放与 DP 同步时机

表 11 本章代码。四种 schedule 的梯度与单进程参考的最大误差是 `0.00e+00` ——不是“接近”，是逐位相同。

第一步：在进程边界上手工接链式法则

单卡上 `loss.backward()` 之所以能工作，是因为 autograd 有一张从 loss 一直连回第一层权重的图。`send / recv` 是普通的数据拷贝，不是 autograd op，所以图在 stage 边界断成 P 段。你要做的是给每一段单独提供“起点”。

协议只有四行，但每一行都有讲究：

```
# forward
x = recv(...).detach().requires_grad_(True)  # 造一个新的 leaf
y = stage(x)
send(y)

# backward
gy = recv(...)                                # 下游告诉你 dL/dy
torch.autograd.backward(y, gy)                 # 同时填好 W.grad 和 x.grad
send(x.grad)                                   # 把 dL/dx 交给上游
```

三个容易写错的地方：

1. `.detach().requires_grad_(True)` 不是防御性代码。recv 出来的 buffer 没有 `grad_fn`、也没有 `.grad`。不把它变成 leaf，backward 之后 `x.grad` 是 `None`，你就没有任何东西可以往上游发——而且不会报错，只是上游所有层的梯度恒为零。

2. 用 `torch.autograd.backward(y, gy)` , 不是 `y.backward()` 。只有最后一个 stage 手上有标量 loss ;其他每个 stage 都是“拿到一个梯度,从这里继续往回传”。`y.backward()` 对非标量会直接抛异常。
3. 第一个 **stage** 的输入不要 `requires_grad_` 。stage 0 拿到的是真实数据而不是 activation , 没有上游可以接收梯度。更实际的原因: 它往往是 int64 的 token id , PyTorch 会直接报 `only Tensors of floating point dtype can require gradients` 。我自己写引擎时就踩了这个——在纯 float 的玩具模型上完全看不出来,一接上 embedding 就炸。

各 stage 手上有什么、没有什么,是这套协议的全部信息:

	stage 0	中间 stage	最后 stage
输入	真实数据	recv 的 activation	recv 的 activation
输入要 grad 吗	不要	要 (新 leaf)	要 (新 leaf)
标量 loss	没有	没有	有
backward 起点	收到的 <code>gy</code>	收到的 <code>gy</code>	<code>loss</code> (无需 <code>gy</code>)
往上游发	不发	<code>x.grad</code>	<code>x.grad</code>
需要 label 吗	不需要	不需要	需要

表 12 PP 的每个 stage 都在跑同一段代码,但边界条件三种。绝大多数“梯度全零”或“loss 不降”的 PP bug 都是某一格填错了。

`21_pp_from_scratch.py` 的 Part 1 把这件事做成了可验证的最小例子: 两卡切开 $f(x) = W_2 \cdot \text{gelu}(W_1 x)$, stage 0 从未见过 **loss** ,但它算出的 $\partial L / \partial W_1$ 与单进程完全一致。

Key insight. 面试里被问“PP 怎么实现”,从这里开始讲,而不是从时空图开始讲。能说清“我要手工构造 leaf、手工提供 `grad_outputs`、把 `x.grad` 当消息发出去”,就说明你真的写过;只会画 F/B 方格图的人讲不到这一层。

第二步: 把 **schedule** 变成数据

朴素写法是给每种 schedule 写一套 for 循环,于是 GPipe、1F1B、interleaved、zero-bubble 就是四份互相抄的代码,改一个 bug 要改四处。

更好的做法——也是 `torch.distributed.pipelining` 和 Megatron-Core 背后的结构——是让 schedule 变成一个指令列表,然后写一个解释器:

```
@dataclass(frozen=True)
class Instr:
    op: str          # F / B / BI / BW
    mb: int          # micro-batch id
    chunk: int = 0   # 这张卡上的第几个 virtual chunk
```

四种 schedule 于是各自只有几行。1F1B 是最典型的:

```
def sched_1f1b(rank, P, m):
    n_warm = min(P - 1 - rank, m)
    out = [Instr("F", i) for i in range(n_warm)]          # warm-up
    for i in range(m - n_warm):                             # steady
```

```

    out += [Instr("F", n_warm + i), Instr("B", i)]
    out += [Instr("B", i) for i in range(m - n_warm, m)]    # cool-down
    return out

```

`n_warm = P - 1 - rank` 这一行就是 1F1B 的全部内容：stage 0 要先跑 $P - 1$ 个 forward 才等到第一个梯度回来，最后一个 stage 只跑 1 个。

打印出来 ($P = 4, m = 8$):

```

stage 0: F0 F1 F2 F3 B0 F4 B1 F5 B2 F6 B3 F7 B4 B5 B6 B7
stage 1: F0 F1 F2 B0 F3 B1 F4 B2 F5 B3 F6 B4 F7 B5 B6 B7
stage 2: F0 F1 B0 F2 B1 F3 B2 F4 B3 F5 B4 F6 B5 F7 B6 B7
stage 3: F0 B0 F1 B1 F2 B2 F3 B3 F4 B4 F5 B5 F6 B6 F7 B7

```

这样做的好处不只是省代码。schedule 成了数据之后，你可以在不跑任何 **GEMM** 的前提下把它打印、diff、灌进模拟器算 bubble、检查它是否会死锁。22_pp_schedule_sim.py 全部指标都是从这些列表算出来的。

interleaved 的 (chunk, micro-batch) 映射

interleaved 1F1B 唯一复杂的地方是“第 k 步该算哪个 chunk 的哪个 micro-batch”。Megatron 的映射是：

$$\text{chunk}(k) = \lfloor (k \bmod PV) / P \rfloor, \quad \text{mb}(k) = \lfloor k / PV \rfloor \cdot P + (k \bmod P)$$

backward 方向把 chunk 反过来： $V - 1 - \text{chunk}(k)$ 。全局 chunk 编号按 $g = v \cdot P + \text{rank}$ 排布，也就是说 rank r 持有 chunk $r, r + P, r + 2P, \dots$ ，micro-batch 在环上绕 V 圈。

Note. 这两个映射式与 **rank** 无关——每张卡算出来的 (chunk, mb) 序列完全相同，只是各自处在 warm-up/steady/cool-down 的不同位置。这正是为什么各 rank 的 send 和 recv 能自动对上而不需要任何全局协调。想清楚这一点，interleaved 就不神秘了。

warm-up 深度是 $2(P - 1 - r) + (V - 1)P$ ：前一项是普通 1F1B 的往返，后一项是“micro-batch 还要再绕 $V - 1$ 圈才回到你手上”。

解释器与 FIFO 纪律

解释器本体就是一个 dispatch，加上每个 **chunk** 一个 **FIFO** 队列：

```

def run(self, schedule, inputs, labels):
    for ins in schedule:
        if ins.op == "F": self._forward(ins.mb, ins.chunk, inputs, labels)
        elif ins.op == "B": self._backward(ins.mb, ins.chunk, weight=True)
        elif ins.op == "BI": self._backward(ins.mb, ins.chunk, weight=False)
        elif ins.op == "BW": self._weight(ins.mb, ins.chunk)
    self._drain_sends()

```

forward 时把 (mb, x_leaf, y) push 进队尾，backward 时从队首 popleft。FIFO 不是随手选的：本章四种 schedule 对同一个 chunk 都是按 **forward** 的顺序 backward 的，而 P2P 的配对也依赖这个顺序。引擎里留了一行断言，它能在几十行之内定位一个写错的 schedule：

```
mb_q, x, y = self.live[v].popleft()
assert mb_q == mb, f"FIFO violated on chunk {v}: expected {mb_q}, got {mb}"
```

顺带一个可以直接背的结论，四种 schedule 通用：

activation 峰值 = 该 stage 在第一次 **backward** 之前跑掉的 forward 个数

代入即得：GPipe 是 m （它根本没有 steady 阶段），1F1B 是 $P - \text{rank}$ ，interleaved 是 $2(P - 1 - r) + (V - 1)P + 1$ 。文件 22 模拟出来的值与文件 21 实测的值逐位相同：

schedule	stage 0 峰值	stage 3 峰值	P2P 消息数（每 stage）
GPipe	8	8	16 / 32 / 32 / 16
1F1B	4	1	16 / 32 / 32 / 16
zero-bubble	4	1	16 / 32 / 32 / 16（另有 6 份 W 张量滞留）
interleaved ($V = 2$)	11	5	48 / 64 / 64 / 48

表 13 $P = 4, m = 8$ 实测。interleaved 的 P2P 消息数是 $3\times$ ，这是它换 bubble 付的真实代价；显存也比 1F1B 高而不是低。

第三步：P2P 的三个工程问题

死锁：1F1B 稳态一定会撞上

这是最经典的 PP bug。1F1B 稳态下，stage s 正要把 activation 往下发，而 stage $s + 1$ 正要把 gradient 往上发。如果两边都写成“先 send 再 recv”：

```
stage s    : send(act -> s+1)    阻塞，等 s+1 来收
stage s+1 : send(grad -> s)      阻塞，等 s 来收
```

两边都卡在 send 里，等着对方去执行那个永远不会到达的 recv。21_pp_from_scratch.py 的 Part 2 用一个 2 秒超时的进程组把它真实复现出来（不是画图说明），然后对比三种解法：

做法	为什么能解	代价 / 风险
奇偶定序	偶数 rank 先 send 后 recv，奇数 rank 反过来，用 rank 奇偶性打破对称	脆。每加一种消息都要重新排一遍；interleaved 打破了“每步恰好一收一发”的前提，这套就不成立了
batch_isend_irecv	把这一步所有 send/recv 先全部 post，再一起 wait，顺序约束直接消失	真实框架的做法。还顺带让两个方向的传输在线路上重叠
send 永不阻塞	send 一律 post 后把 handle 存起来，只有 recv 才阻塞。循环等待需要至少有一个 rank 卡在 send 上，所以环不可能形成	本章引擎的做法。要自己管 buffer 生命周期，且要在 step 结束前 drain

表 14 三种解法都对，选一种并且知道为什么。第三种最容易讲清楚——“没人阻塞在 send 上”是一个可以一句话证明的不变量。

Megatron 里第二种做法对应的函数就叫 `send_forward_recv_backward` (以及 `send_backward_recv_forward`)——看到这个函数名, 你就知道它存在的唯一理由是躲开上面那个环。

Warning. 一个只有踩过才知道的细节: gloo 的 P2P 超时会关闭整条 **pair**, 超时之后这个进程组就不能再用了, 后续操作直接报 `Application timeout caused pair closure`。生产上也是这个性质——P2P 超时不会给你留一个还能重试的进程组, 只能重建。所以文件 21 里死锁演示用的是独立的短超时进程组, 后面三种解法各用干净的组。

还有一个 buffer 生命周期的坑: `isend` 立刻返回, 但它持有那块内存直到传输完成。把待发的 activation 原地改掉, 或者让它被回收, 收到的就是垃圾。引擎里这一行的 `clone()` 不是随手加的:

```
def _send(self, x, dst, tag):
    buf = x.detach().contiguous().clone()      # isend 期间必须一直有效
    work = dist.isend(buf, dst=dst, tag=tag, group=self.group)
    self.pending_sends.append((work, buf))     # 连 buf 一起存, 否则会被 GC
```

形状协商: 接收方得先分配才能接收

`dist.recv` 需要一个已经分配好的 tensor 来写入, 所以接收方必须在数据到达之前就知道 **shape** 和 **dtype**。固定形状时硬编码就行, 一旦上了 packing 或变长序列就不行了, 而猜错的后果是静默截断或者挂住。

做法是先发一个小 header:

```
def send_meta(x, dst, tag, group):
    hdr = torch.tensor([x.dim(), DTYPE_CODE[x.dtype]] + list(x.shape) + pad,
                       dtype=torch.int64)
    dist.send(hdr, dst=dst, tag=tag, group=group)
```

代价是每个 tensor 多一条极小的消息, 所以框架都把它做成开关: Megatron 的 `--variable-seq-lengths` 默认关闭。引擎同理——传 `act_shape` 就跳过协商, 传 `None` 就每次协商。

tag 还是位置配对

同—对 rank 之间怎么区分“这条是 activation”和“这条是 gradient”?

多数情况下不用区分: activation 走 $r \rightarrow r+1$, gradient 走 $r+1 \rightarrow r$, 方向不同即 `(src, dst)` 不同, 天然不会混。但 $V > 1$ 且 $P = 2$ 时会退化——rank 1 到 rank 0 这条链上同时跑着 activation (`chunk 1  $\rightarrow$  2`) 和 gradient (`chunk 3  $\rightarrow$  2`, `chunk 1  $\rightarrow$  0`), 单靠顺序区分不了。

两条出路:

- **tag**: $\text{tag} = 2(m \cdot N_g + g) + \text{dir}$, 其中 N_g 是全局 chunk 总数, 简单直接。但 NCCL 忽略 **tag**, 只有 gloo 支持——这就是本章 demo 把 backend 钉在 gloo 的原因。
- 位置配对: 每一步把该步所有 op 塞进一个 `batch_isend_irecv`, 靠“两边步骤结构相同”来保证第 j 个 op 对上第 j 个 op。这是真实框架的做法, 也是它们不需要 tag 的原因。

第四步: 把 **backward** 拆成 **B** 和 **W**

zero-bubble 的前提是把一次 backward 拆成两半:

- **B (input-grad)** : $\partial L / \partial X$, 会阻塞上游 stage , 必须尽早算
- **W (weight-grad)** : $\partial L / \partial W$, 不阻塞任何人 , 可以塞到任意空隙里

问题是 PyTorch 的 `backward()` 一次遍历就把两个都算了。怎么拆？

天真的做法是调两次 `torch.autograd.grad` , 一次求 `inputs=[x]` 、一次求 `inputs=params` ——但那是把整个反向图遍历了两遍 , 白算一倍。

正确做法是写一个拒绝干完一半活的 autograd Function :

```
class _LinearSplitBW(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, w, tape):
        ctx.save_for_backward(x, w)
        ctx.tape = tape
        return x @ w.t()

    @staticmethod
    def backward(ctx, gy):
        x, w = ctx.saved_tensors
        ctx.tape.push(w, gy.detach(), x.detach()) # W: 记账, 先不算
        return gy @ w, None, None                # B: 现在就算
```

于是 **BI** 就是普通的 `torch.autograd.backward(y, gy)` (它会把整段的 input-grad 算完 , 并把所有 W 的活记在 tape 上) , **BW** 就是把 tape 排空 :

```
for w, gy, x in work:
    gw = gy.reshape(-1, gy.shape[-1]).t() @ x.reshape(-1, x.shape[-1])
    w.grad = gw if w.grad is None else w.grad + gw
```

没有 autograd 补丁 , 没有 monkey patch , 两个 GEMM 各算一次。

Key insight. 代价在显存 , 而且账要算清楚 : **BI** 释放了 activation , 但 tape 上还压着 x 和 g_y 等着 **BW** 用。 $P=4, m=8$ 时 stage 0 的 activation 峰值和 1F1B 一样是 4 , 但另外滞留 6 份 W 张量 (文件 22 的 Part 4 实测) 。所以 zero-bubble 是用显存换 **bubble** , 不是白拿。

第五步：接进真实训练循环

到这里引擎能跑了 , 但离能训练还差五件事 , 而这五件事一张时空图都不会画。全部在 `23_pp_integration.py` 里 , 并且端到端验证过 : $PP \times DP +$ 权重共享的梯度必须等于单进程跑完整 global batch。

rank 排布 : 你的邻居不是 $rank \pm 1$

Megatron 默认排布是 TP 变化最快、PP 最慢 :

$$\text{global_rank} = pp \cdot (DT) + dp \cdot T + tp$$

这个顺序不是随便定的。TP 每层要来回搬两次完整 activation , 所以 TP 组必须落在一个 NVLink 域里 , 也就是连续的 **global rank** ; PP 每个 stage 边界只搬一次 activation , 所以给它最大的 stride , 让它去跨节点。

$\text{world} = 8, T = 2, D = 2, P = 2 :$

rank	tp	dp	pp	node	TP 组	PP 组
0	0	0	0	0	0,1	0,4
1	1	0	0	0	0,1	1,5
2	0	1	0	0	2,3	2,6
3	1	1	0	0	2,3	3,7
4	0	0	1	1	4,5	0,4
5	1	0	1	1	4,5	1,5
6	0	1	1	1	6,7	2,6
7	1	1	1	1	6,7	3,7

表 15 TP 组永远是连续 rank，不跨节点；PP 组 stride = $DT = 4$ ，正好跨节点。

Warning. 手写 PP 最常见的 bug：pipeline 邻居是 $\text{rank} \pm DT$ ，不是 $\text{rank} \pm 1$ 。而且 `dist.send/recv` 即使传了子进程组，`src/dst` 要的仍然是全局 rank，不是组内序号。

我自己的引擎就中过这一枪：文件 21 里 PP 组恰好是整个 world，组内序号与全局 rank 相同，四种 schedule 全部通过；一到文件 23 开了 $DP=2$ 、PP 组变成 `[1, 3]`，立刻 `ValueError: Global rank 0 is not part of group`。引擎里现在有一层显式翻译：

```
def _peer(self, pipe_rank):
    if self.group is None:
        return pipe_rank
    return dist.get_global_rank(self.group, pipe_rank)
```

更阴的情况是它不报错：如果错算出的那个 rank 恰好存在，你就在往一张持有相同层的卡发 activation，表现是挂住而不是异常。

分层：层数相等不等于工作量相等

32 层、 $h = 4096$ 、vocab 128K 的模型，按模块相对耗时（一个 transformer block = 1.0）：embedding 约 0.5（查表很便宜），lm_head 约 5.0——head 的 GEMM 本身就是 $hV/12h^2 \approx 2.6$ 个 block，再加上在 $(B, S, 128K)$ logits 上做 cross-entropy。

按层数平均切 4 段：负载 `[7.5, 9.0, 9.0, 12.0]`， $\text{max}/\text{mean} = 1.28\times$ 。按耗时贪心切：`[9.5, 10.0, 9.0, 9.0]`， $1.07\times$ 。

Key insight. 流水线跑在最慢那一段的速度上，所以 $1.28\times$ 的不均衡是每个 micro-batch 都要付的 $1.28\times$ 减速——它和 bubble 相乘，而不是被 bubble 藏起来。

常见解法：给首尾 stage 少分几个 block，或者把 loss 单独放一个 virtual chunk。Megatron 有手动指定分层的 flag。

还有一个约束容易忽略：不能先建完整模型再切。70B 以上没有任何一张卡装得下，每张卡只能构造自己那几层——也就是说分层方案必须在任何权重被分配之前就定下来，只能靠一张成本表，不能靠跑一遍 profile。

权重共享：**embedding** 和 **lm_head** 落在不同 **stage** 上

embedding 在 stage 0, lm_head 在 stage $P - 1$, 两者 tie 在一起——但它们在不同的进程上。每一端只算到梯度的一部分：stage 0 从查表那条路径拿到一部分，最后一个 stage 从投影那条路径拿到另一部分。

所以首尾两个 rank 要单独组一个进程组 (Megatron 就叫 embedding group), backward 之后在组内 all-reduce：

```
emb_group = dist.new_group([first_pp_rank, last_pp_rank])  # 每个 dp 坐标一个
...
dist.all_reduce(emb.grad, op=SUM, group=emb_group)
```

文件 23 实测：每一端各自只持有 tied 梯度的一部分 (该配置下约 69%), all-reduce 之后两端都拿到完整的。

Warning. 漏掉这个 all-reduce 不会崩。两份 embedding 副本会在几个 step 内漂开，于是模型用一张表编码、用另一张表解码。现象只是 loss 不再下降——这类 bug 极难查，因为所有 assert 都过、所有 shape 都对。

两个反直觉的补充，都是文件 23 里验证过的：

1. 两次 **reduce** 的顺序无关紧要。embedding group 上的 SUM 和 DP 组上的 AVERAGE 作用在不相交的 rank 集合上，两者都是线性的，可交换。真正要求的是它们都在 clip 之前完成。
2. 但 **grad-norm** 会把共享参数数两遍。all-reduce 之后首尾两端持有的是同一份完整梯度，在 pipeline 组上求平方和就重复计入了。实测膨胀 $1.27\times$ (embedding 占主导时最坏是 $\sqrt{2}$)。norm 被放大意味着 clipping 比你设定的更早介入，等效学习率悄悄变小。规则和第 7 章 CP 那里完全一样：每个被复制的参数只在一张卡上计入 **norm**。

loss 缩放与 DP 同步时机

两件事必须同时对：

- 每个 micro-batch 的 loss 除以 m ，这样 m 次累加之后等于对整个 batch 求平均
- 梯度 all-reduce 每个 **step** 一次，不是每个 micro-batch 一次——后者是 m 倍通信量。原生 DDP 会在每次 backward 都触发 all-reduce，所以前 $m - 1$ 个 micro-batch 必须包在 `no_sync()` 里

文件 23 把两件事一起验证了：引擎内部除以 m ，之后在 DP 组上做一次 SUM 再除以 D ，结果与单进程跑完 $D \times m$ 个 micro-batch 的 global batch 逐位一致。

数据路由

stage 0 要 token 但从不需要 label，最后一个 stage 要 label 但从不需要 token。中间 stage 两个都不要。

```
inputs = toks if pp_rank == 0 else [None] * m
labels = labs if pp_rank == pp_size-1 else [None] * m
```

给每个 stage 都发全套不会算错，但白占 host-to-device 带宽，而且会掩盖“某个 stage 用错了张量”这类 bug。真实实现里 dataloader 只在 PP 组的两端起作用，而同一个 **PP** 组内所有 **rank** 必须拿到同一个样本——这条和第 7 章 CP 组的要求是同一回事。

调度族谱

引擎搭好之后，各种 schedule 就只是不同的指令生成器了。下面是它们的定位，配文件 22 的实测值 ($P = 4, m = 8, V = 2$ ，全部 schedule 总工作量相同均为 24 个单位——这一栏是防止“我的优化其实只是少算了”的护栏)。

schedule	实测 bubble	闭式	换来 / 付出
GPipe	27.3%	$(P - 1)/(m + P - 1)$	基线。收敛性与不开 PP 完全一致
1F1B	27.3%	$(P - 1)/(m + P - 1)$	bubble 完全相同，activation 从 m 份降到 $P - \text{rank}$ 份
interleaved (V)	15.8%	$(P - 1)/(Vm + P - 1)$	bubble 除以 V ；P2P 次数 $\times 3$ ，显存反而更高
ZeroBubble ZB-H1	14.3%	—	W 填满 cool-down；warm-up 那一半填不掉，另需显存压 W 张量
DualPipe	—	$(P/2 - 1)(F \& B + B - 3W)$	双向注入 + 4 相 overlap； $2\times$ 权重副本

表 16 $P = 4, m = 8$ 。前三行的实测值与闭式公式吻合到小数点后若干位（文件 22 里是断言，不是打印）。

三个结论值得单独记

GPipe 和 **1F1B** 的 **bubble** 完全一样。这与直觉相反——大多数人以为 1F1B 更快。它不更快，它只是更省显存。 $P = 4, m = 8$ 时两者都是 27.3%，文件 22 里断言了这一点。

那为什么所有生产系统都用 1F1B？因为 GPipe 的 activation 是 m 份而 1F1B 是 $P - \text{rank}$ 份，而缩小 **bubble** 唯一最便宜的手段就是加大 m 。GPipe 把“降 bubble”和“爆显存”绑在了一起，1F1B 解开了这个耦合——这才是它赢的地方。

	m=2	m=4	m=8	m=16	m=32	m=64
1F1B, P=4	60.0%	42.9%	27.3%	15.8%	8.6%	4.5%

zero-bubble 到不了零。27.3% $\rightarrow$ 14.3%，砍掉一半。W 能填满 cool-down，但 warm-up 阶段还没有任何梯度回来，根本没有 **W** 可做。要填那一半得上 ZB-2P，代价是更多 activation 滞留。

时空图上三者的差别一眼就看出来（每格一个时间单位，F=forward，B=fused backward，b=B-only，w=W-only，.=bubble）：

gpip (bubble 27.3%)		1f1b (bubble 27.3%)	
s0	FFFFFFFF.....BBBBBBBBBBBBBBBBB	FFFF.....BBFBBFBBFBBFBB.BB.BB.BB	
s1	.FFFFFFF.....BBBBBBBBBBBBBBBBB..	.FFF.....BBFBBFBBFBBFBBFBB.BB.BB..	
s2	..FFFFFFF...BBBBBBBBBBBBBBBBB....	..FF...BBFBBFBBFBBFBBFBBFBB.BB....	
s3	...FFFFFFFBBBBBBBBBBBBBBBBB.....	...FBBFBBFBBFBBFBBFBBFBBFBBFBB.....	

```

zero-bubble (bubble 14.3%)
s0 |FFFF...bFbFbFbFb.bwbwbwwwww|
s1 |.FFF...bFbFbFbFbFb.bwbwwwww|
s2 |..FF.bFbFbFbFbFbFb.bwwwwwww|
s3 |...FbFbFbFbFbFbFbFbwwwww.|

```

读形状,不只读数字。GPipe 的空闲是中间一整块——每个 stage 把 m 个 forward 全跑完,然后一起等反向波传过来。1F1B 空闲总量一模一样,但分布在 warm-up 的等待加 cool-down 的小缝里。注意没变的那个量: bubble。1F1B 赢的是“第一次 backward 只需等 $P-1-\text{rank}$ 个 forward 而不是 m 个”,这是显存性质,不是时间性质。zero-bubble 则把右侧斜坡用 `w` 填掉,左侧原样留着——正如 warm-up 那个论证预测的。

两个降 **bubble** 的旋钮代价不同。加大 m 在 1F1B 下不花额外显存,所以永远先动它。加大 V 要付 P2P 往返和 activation,所以它是“ m 已经被你想要的 global batch 卡住了”之后才用的。

```

interleaved, m=8, P=4:   V=1: 27.3%   V=2: 15.8%   V=4: 8.6%   V=8: 4.5%

```

DualPipe 与 Megatron FWD-BWD merged

DeepSeek-V3 的 DualPipe 是最激进的方案: pipeline 两端同时注入 micro-batch, 中间 rank 同时跑正反两个方向, 再把每个 chunk 拆成 attention / dispatch / MLP / combine 四相与通信交叉。bubble 系数从 $P-1$ 降到 $P/2-1$ 。代价是 $2\times$ 权重副本, 外加要自己 hack autograd 层的调度、与 DeepEP 深度耦合。

Megatron 2024 给了个折中: 不改 **schedule**(仍 1F1B/interleaved), 但在同一 iteration 里合并相邻 micro-batch 的 fwd 和 bwd, 两个 CUDA stream 并行——stream 0 跑 forward compute, stream 1 跑 backward compute 加 a2a 通信, 配合 `--delay-wgrad-compute` 的 B/W 拆分打破依赖。需要 `CUDA_DEVICE_MAX_CONNECTIONS>1`。MoE 场景下 EP a2a 占比从 30-40% 压到 $< 5\%$, overlap 93%, 且不需要 $2\times$ 权重。

什么时候值得上 DualPipe: > 1000 卡、权重吃得下 $2\times$ 副本、且 comm 与 compute 接近 1:1。绝大多数场景 Megatron 的 merged 方案就够——90% 的收益, 不用 $2\times$ 权重。

PP 通信量

每个 micro-batch 每个 stage 边界过一次 P2P activation: $B \times S \times H \times \text{bytes}$ 。

一个 step 总量 $\approx m \times (P-1) \times 2 \times B \times S \times H \times 2$ (forward + backward, bf16), $B=1, S=8K, H=8192, m=32, P=8$: 约 60 GB per step。

但 **P2P** 只在 **pair** 之间, 不占整个 fabric。跨节点走 IB, 节点内走 NVLink, 每对 pair 各自一条管道, 与 collective 的性质完全不同。所以 PP 通信量看着大, 实际很少成为瓶颈。

要避免的 pattern: PP 组跨节点且 P2P 消息又小又多时, 延迟主导而非带宽主导。合并消息或者 chunked send。

PP 与其他并行的组合

PP 与 TP/DP/EP 完全正交。生产配置:

Llama-3 405B on 16K H100: $TP=8$ (NVL 域内) $CP=16, PP=16, DP=8$ 。16K = $8 \times 16 \times 16 \times 8$ ✓

DeepSeek-V3 671B on 2048 H800 : TP = 1 (不用 TP) , EP = 64, PP = 16 (DualPipe) , DP = 2。

选 PP 深度的经验 : 让每 stage 层数 $L/P \approx 4-8$ 。太少 (每 stage 1-2 层) bubble 大且 P2P overhead 占比高 ; 太多则 stage 内 compute 远大于通信 , overlap 空间小。

PP 的坑

1. 邻居算错 : rank $\pm DT$ 而非 ± 1 ; 子进程组里 send/recv 仍要全局 rank
2. 第一个 stage 的输入设了 `requires_grad` : token id 是 int64 , 直接报错
3. 忘了把 recv 出来的 **activation** 变成 **leaf** : 上游梯度恒零 , 且不报错
4. **tied embedding** 漏掉 **embedding-group all-reduce** : 不崩 , 只是 loss 不降
5. **grad-norm** 重复计入 **tied** 参数 : norm 膨胀最多 $\sqrt{2}$, clipping 提前介入
6. 每个 **micro-batch** 都做 **DP all-reduce** : m 倍通信量 , 忘了 `no_sync()`
7. **Layer** 不均衡 : embedding 与 output head 比中间层重 , 按耗时分而非按层数分
8. **LN / dropout seed** : PP 组内每个 micro-batch 用不同 seed , 但跨 stage 要保持一致
9. **isend** 的 **buffer** 被改写或回收 : 收到垃圾数据 , 且是间歇性的
10. 变长序列没做形状协商 : 静默截断或挂住
11. **KV cache in generation** : inference 时每 stage 维护自己的 stage-local KV
12. **ckpt** 的 **PP** 层分 : save 按 PP 存、load 要匹配 ; 改 PP 数需要重新切分
13. `micro_batch_size = 1` 的 **attention** : $B = 1$ 时 GEMM 的 M 维太小 , 掉利用率 , 可以 pack seq

面试考点

面试考点. Q1: PP 里 `loss.backward()` 为什么不能用 ? 你怎么办 ?

A: 计算图被 `send / recv` 切成了 P 段 , `loss` 只在最后一个 stage 上 , stage 0 上没有任何从 `loss` 连回自己权重的路径。做法是手工提供每一段的起点 : forward 时把 `recv` 到的 `activation detach().requires_grad_(True)` 变成新 leaf ; backward 时 `recv` 到下游给的 `grad_output` , 调 `torch.autograd.backward(y, gy)` (不是 `y.backward()` , 非标量会抛异常) , 它会同时填好本段的 `W.grad` 和 `x.grad` ; 再把 `x.grad` 发给上游。注意第一个 stage 的输入不要 `requires_grad_` ——它是数据不是 activation , 而且通常是 int64 token id。

面试考点. Q2: 1F1B 相对 GPipe 的核心收益是什么 ?

A: 不是 **bubble**——两者 bubble 完全相同 , 都是 $\frac{P-1}{m+P-1}$ 。收益是 activation 从 m 份降到 $P - \text{rank}$ 份。为什么这很重要 : 降 bubble 最便宜的手段是加大 m , 而 GPipe 的显存正比于 m , 把“降 bubble”和“爆显存”绑死了 ; 1F1B 解开这个耦合 , 所以你能放心把 m 开到 $4P$ 以上。

面试考点. Q3: 手写 1F1B 时 P2P 怎么排才不死锁 ?

A: 稳态下 stage s 要往下发 activation , 同时 stage $s + 1$ 要往上发 gradient ; 两边都“先 send 后 recv”就双向卡死。三种解法 : 奇偶定序 (脆 , interleaved 下不成立) ; `batch_isend_irecv` 把该步所有 op 先 post 再一起 wait (Megatron 的 `send_forward_recv_backward`) ; 或者让 send 永不阻塞——只 post 不 wait , 只有 recv 阻塞 , 这样循环等待需要至少一个 rank 卡在 send 上 , 环不可能形成。第三种要自己管 `isend` 的 buffer 生命周期 (必须 clone , 且要连 buffer 一起持有到 wait)。

面试考点. Q4: 变长序列下 PP 怎么传 activation ?

A: `recv` 需要预先分配好的 tensor , 所以接收方必须先知道 shape/dtype。发一个小 header (`ndim + dtype code + dims`) 再发数据。代价是每个 tensor 多一条极小消息, 所以框架做成开关 (Megatron `--variable-seq-lengths` , 默认关)。猜错形状的后果是静默截断或挂住, 不是报错。

面试考点. Q5: ZeroBubble 怎么把 B 和 W 拆开? 为什么不能直接调两次 `autograd.grad` ?

A: 调两次会把反向图遍历两遍, 白算一倍。正确做法是写一个自定义 `autograd Function` : `backward` 里只算 `gy @ w` 返回 `input-grad` , 把 `weight-grad` 需要的 (w, g_y, x) 推到一个 tape 上不算。于是 BI 就是普通 `backward()` (顺带把所有 W 的活记账), BW 就是排空 tape 做 `gy.T @ x`。两个 GEMM 各算一次, 不需要 patch `autograd`。代价是显存: BI 释放了 activation , 但 tape 上压着 x 和 g_y 等 BW , 所以 zero-bubble 是拿显存换 bubble。

面试考点. Q6: 为什么 ZeroBubble 到不了零 bubble ?

A: W 只能填 cool-down。warm-up 阶段还没有任何梯度回来 , 手上没有 W 可做 , 那一半 bubble 填不掉。实测 $P = 4, m = 8$ 从 27.3% 降到 14.3% , 正好一半左右。要填 warm-up 得上 ZB-2P , 代价是更多 activation 滞留。

面试考点. Q7: interleaved 1F1B 每一步该算哪个 chunk 的哪个 micro-batch ?

A: $\text{chunk}(k) = \lfloor \frac{k \bmod PV}{P} \rfloor$, $\text{mb}(k) = \lfloor \frac{k}{PV} \rfloor \cdot P + (k \bmod P)$, `backward` 方向 chunk 取 $V - 1 - \text{chunk}(k)$ 。全局 chunk 按 $g = vP + \text{rank}$ 排 , rank r 持有 $r, r + P, \dots$ 。关键是这两个映射与 **rank** 无关——每张卡算出的序列相同, 只是处在不同阶段, 所以 `send/recv` 自动对上, 不需要全局协调。warm-up 深度 $2(P - 1 - r) + (V - 1)P$ 。

面试考点. Q8: PP+TP+DP 一起开, pipeline 邻居是谁 ?

A: Megatron 默认 $\text{rank} = \text{pp} \cdot (DT) + \text{dp} \cdot T + \text{tp}$, 所以邻居是 $\text{rank} \pm DT$, 不是 ± 1 。这个顺序的理由: TP 每层搬两次完整 activation , 必须在 NVLink 域内, 也就是连续 rank ; PP 每个边界只搬一次, 给最大 stride 让它跨节点。另外 `dist.send/recv` 即使传了子组, `src/dst` 仍要全局 rank (用 `dist.get_global_rank(group, i)` 翻译)。算错了如果那个 rank 恰好存在, 表现是挂住而非报错。

面试考点. Q9: embedding 和 `lm_head` tie 在一起, 但在首尾两个 stage 上, 怎么处理 ?

A: 首尾两个 rank 单独组一个进程组 (embedding group) , `backward` 之后在组内 all-reduce 这个梯度——因为每一端只算到一部分 (stage 0 走查表路径, 末 stage 走投影路径)。漏掉不会崩, 两份副本几个 step 内漂开, 模型用一张表编码另一张表解码, 现象只是 loss 不降。补充两点: 这次 reduce 与 DP 的 all-reduce 顺序无关 (都是线性、作用在不相交 rank 集上, 可交换), 但都必须在 clip 之前完成; 而 `grad-norm` 会把 tied 参数数两遍, norm 最多膨胀 $\sqrt{2}$, 导致 clipping 提前介入、等效 LR 变小, 所以要保证每个复制参数只在一张卡上计入。

面试考点. Q10: PP 下 loss 怎么缩放, DP 梯度什么时候同步 ?

A: 每个 micro-batch 的 loss 除以 m , m 次累加后等于整个 batch 的均值。DP 梯度 all-reduce 每 **step** 一次, 不是每 micro-batch 一次, 否则是 m 倍通信量——用原生 DDP 就要把前 $m - 1$ 个 micro-batch 包在 `no_sync()` 里。验证方法: PP $\times$ DP 的结果应当与单进程跑完 $D \times m$ 个 micro-batch 的 global batch 逐位一致。

面试考点. **Q11**: PP 分层为什么不能按层数平均分?

A: 模块耗时不等。32 层 $h = 4096$ vocab 128K 时, embedding 约 0.5 个 block、lm_head 约 5 个 block (head 的 GEMM 就是 $h \frac{V}{12} h^2 \approx 2.6$ 个 block, 再加 $(B, S, 128K)$ logits 上的 CE)。按层数平均切 4 段是 $1.28\times$ 不均衡, 按耗时切是 $1.07\times$ 。流水线跑在最慢那段的速度上, 所以这个 $1.28\times$ 是每个 micro-batch 都要付的减速, 与 bubble 相乘而不是被 bubble 吸收。另外注意: 不能先建完整模型再切 (70B+ 装不下), 分层必须在分配权重之前从成本表决定。

面试考点. **Q12**: PP 组该跨节点还是同节点?

A: 跨节点 OK, 而且应该让它跨。PP 用 P2P, 只有 pair-to-pair 通信, 占 fabric 少; 且通信频率低 (每 stage 边界一次, 不像 TP 每层多次)。跨节点 IB 带宽足够。TP/EP 才必须在 NVLink 域内。

面试考点. **Q13**: PP 训练最后一层为什么特别慢?

A: 最后一个 stage 是 LM head + loss, vocab 128K 时 (B, S, V) 的 logits 在 compute 和 activation 上都远超一个 transformer block, 而且 backward 从这里起步。做法: 把 embedding 与 LM head tie 到同一 stage (省参数), 或给最后一个 stage 少分几个 transformer layer 给 vocab head 留余量, 或把 loss 单独放一个 virtual chunk。

面试考点. **Q14**: PP=16 时选 DualPipe 还是 Megatron FWD-BWD merged?

A: 看 comm/compute 比。DeepSeek-V3 那种 setup (comm:compute $\approx 1:1$ 、跨节点 MoE a2a 主导) 用 DualPipe 值得。Megatron 官方推荐大多数场景用 FWD-BWD merged (1F1B + delayed wgrad + 双 stream) ——90% 收益, 不需要 $2\times$ 权重。除非你训 500B+ MoE 且 IB 是瓶颈, 否则 DualPipe overkill。

Context Parallel : Ring Attention, Ulysses 与长序列训练

Context Parallel (CP), 或“sequence parallel” (与 Megatron 那个 SP 是不同东西——命名混乱), 把 attention 沿 sequence 维切分到多张卡。1M+ context 训练必需, 也是 2023-2024 最火的方向。

澄清命名:

- **Megatron SP (Sequence Parallel)**: LN/Dropout 沿 seq 切, 与 TP 组合, 不动 attention (第 5 章)
- **Context Parallel (CP)**: attention 也沿 seq 切, 需要跨卡通信 K/V (本章)
- **Ulysses / Ring Attention**: CP 的两种实现路径

为什么需要 CP: attention 的 quadratic 显存

Standard attention 显存 $\propto BS^2A$ (存 attention matrix)。FlashAttention 把 matrix on-the-fly 算, 显存降到 $O(BSH)$ 。但 **compute** 仍然 $O(S^2H)$ ——64K $\rightarrow$ 1M 序列 compute 增加 256 $\times$ 。

Activation: 一层 forward 存 Q, K, V, output = $4BSH$ 。 $B=1, S=10^6, H=8192, bs=2$: 单层 activation 64 GB。80 层 = 5 TB。单卡当然不行。

方案: 把 S 切成 CP 份, 每卡持 $\frac{S}{CP}$ 长的 Q/K/V。activation 降到 $4BS\frac{H}{CP}$ 。但 attention 是“每 Q 都要看所有 K/V”——需要跨卡通信。

Ulysses : All-to-All 沿 head 维交换

Jacobs et al. 2023 (DeepSpeed-Ulysses)。核心思路:

1. Input: 每卡 $Q_i, K_i, V_i \in \mathbb{R}^{B, \frac{S}{CP}, A, d_h}$ (seq 切分)
2. **All-to-All**: 沿 seq 维 gather + 沿 head 维 scatter。每卡拿到 $Q_i^{a2a}, K_i^{a2a}, V_i^{a2a} \in \mathbb{R}^{B, S, \frac{A}{CP}, d_h}$ (head 切分)
3. 本地 **attention**: $O_i = \text{attention}(Q_i^{a2a}, K_i^{a2a}, V_i^{a2a})$ 。每卡处理 $\frac{A}{CP}$ 个 head 的完整 seq。attention 计算完全本地
4. **All-to-All** 逆: 从 head-shard 回 seq-shard, 得 $O_i \in \mathbb{R}^{B, \frac{S}{CP}, A, d_h}$

All-to-All (Ulysses): 每 rank 把自己 4 份中的第 j 份发给 rank j

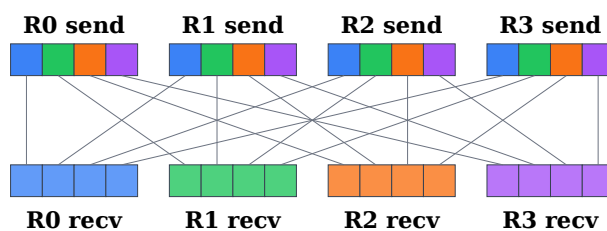


图 21 Ulysses 用一次 all-to-all 把“seq-shard、all-heads”重排成“full-seq、head-shard”, attention 完全本地, 反向再一次 a2a 换回来。

通信量: 每卡 2 次 all-to-all (forward) + 2 次 (backward), 每次 $BS\frac{H}{CP}$ 数据。

$$\text{vol}_{\text{ulysses}} = 4 \times BSH \times \frac{\text{bs}}{\text{CP}} \times \frac{\text{CP} - 1}{\text{CP}}$$

对 $B = 1, S = 32K, H = 8192, \text{CP} = 8$: 每卡 per-layer per-step $\approx 4 \times 512\text{MB} \times 7/8 = 1.75 \text{ GB}$ 。

限制：

1. $A \geq \text{CP}$: head 数必须 $\geq \text{CP}$, 否则 head shard 分不下
2. Head 少的模型 (Llama 7B $A=32$) 只能 $\text{CP} \leq 32$; GQA (KV heads 少) 更受限
3. All-to-all 通信量 $\propto SH$, S 大到 1M 时通信量单层几 GB

优势 :attention compute 全本地——kernel 效率高 ,与 FlashAttention 完全兼容 ,无需改 kernel。

Ring Attention : K/V 沿环流动

Liu et al. 2023。另一种切法：

1. Input: 每卡 $Q_i, K_i, V_i \in \mathbb{R}^{B, \frac{S}{\text{CP}}, A, d_h}$
2. 不做 **all-to-all** , 而是把 K/V 沿 ring 逐步旋转
3. 每一步：本卡 Q 与当前持有的 K/V 段做 attention , 累加到 output
4. 同时 async send K/V 到下一 rank
5. CP 步后每 Q 都看过了所有 K/V

算法：

```
# 每卡拥有 Q_i, K_i, V_i (本地 shard)
K_cur, V_cur = K_i, V_i      # 起点是自己
O_acc, lse_acc = init()      # accumulator (Flash-style online softmax)

for step in range(CP):
    # 与当前 K, V 做局部 attention
    O_i, lse_i = flash_attention(Q_i, K_cur, V_cur, causal=causal_mask(i, step))
    O_acc, lse_acc = merge_flash(O_acc, lse_acc, O_i, lse_i)

    # async 送 K, V 到下一 rank; 从上一 rank recv 下一段
    K_next = dist.p2p_recv_and_send(K_cur, next_rank, prev_rank)
    V_next = dist.p2p_recv_and_send(V_cur, next_rank, prev_rank)
    K_cur, V_cur = K_next, V_next
```

Flash-style merge : 每步局部 attention 输出 $(O^{(s)}, \text{lse}^{(s)})$, 用 online softmax 的合并规则累加。这是 Ring Attention 的关键 trick , 让分步计算等价于一次完整 attention。

Ring Attention: 4 卡各持 1/4 的 Q/K/V, K/V 沿 ring 旋转

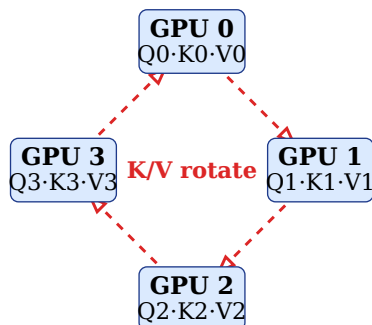


图 22 每 rank 用自己的 Q 和“当前持有的 K/V”做局部 attention , 然后把 K/V async 送到下一 rank ; 4 步后每个 Q 都看过了所有 K/V。总通信量 $2BSH \cdot b$, 与 CP 无关。

代码见 `src/distributed_training/07_ring_attention.py` ——包含 causal mask、Flash-lse merge、单卡 SDPA 对拍。

通信量：每步 P2P send $BS \frac{H}{CP}$ (K + V 一起)。CP 步 total per-GPU：

$$\text{vol}_{\text{ring}} = 2 \times CP \times \left(BS \frac{H}{CP} \right) = 2BSH \text{ bs}$$

与 **Ulysses** 相比：Ring 的每步 P2P 更小 (50 MB range) 但次数多 (CP 次)；Ulysses 是 1-2 次大 collective。Ring 的通信量与 CP 无关 ($2BSH$ 恒定)，Ulysses 与 CP 有关。

限制：

1. 与 **head** 数无关——CP 可以任意大 (up to seq_len)
2. 但每步 attention kernel 的 K 维小 ($= \frac{S}{CP}$)，效率低
3. P2P ring 的延迟累积——CP 大时通信步数多

优势：

1. CP 可以扩到 1024 (匹配序列长度)
2. 通信量 fixed
3. 与 FlashAttention 兼容 (通过 flash-attn 的 lse 合并接口)

Causal Mask 的处理

Autoregressive LM 里 attention 是 causal 的——position i 的 Q 只看 position $j \leq i$ 的 K/V。Ring 里每 rank 拿到不同 shard 的 K/V，需要正确 mask。

问题：如果 rank 0 持 tokens $[0, S/CP)$ ，rank 1 持 $[S/CP, 2S/CP)$ ，做 attention 时 rank 1 的 K/V 对 rank 0 的 Q 是未来——应该跳过。

naive 做法：只在 K/V shard 的 index $\leq$ Q shard index 时做 attention，其他 step 跳过。但这样 rank 0 一直闲、rank last 一直忙——**load imbalance 2x**。

Striped Attention (Brandon et al. 2024)：把 seq 重新排列，让每 rank 处理“从头到尾均匀的一带”，负载均衡。

Zigzag：类似思路，rank r 持有 pattern $[r, 2N - 1 - r]$ ——头尾对称，每 rank 都持有前半 + 后半。

torch-loong / vLLM / OpenDiT 都用 Zigzag 或 Striped。Megatron-LM `context_parallel_size` flag 用类似方案。

USP: Unified Sequence Parallel (Hybrid)

Fang et al. 2024 (USP)。观察：Ulysses 和 Ring 各有优劣：

- Ulysses: head-limited, 通信量 $\propto S \frac{H}{CP}$ ，1-2 次大 collective
- Ring: head-free, 通信量 $\propto SH$ ，CP 小 P2P

USP：CP = $CP_{\text{ulysses}} \times CP_{\text{ring}}$ ，先 Ulysses 沿 head 切一层，再 Ring 沿 seq 切一层。

例： $A = 64, S = 256K$ ，想 $CP=32$ 。

- 纯 Ulysses：CP $\leq A = 64$ OK，通信量 $32BS \frac{H}{32} \approx BSH$ ，短 seq 强
- 纯 Ring：32 步 P2P，长 seq 时 kernel 效率低
- USP $CP_u = 4, CP_r = 8$ ：head 切 4 份 (还剩 16 head/rank)，seq 再切 8 份。综合最优

Long-context 训练里 USP 是当前 SoTA。Kimi/Moonshot 1M ctx、Gemini 1M ctx 都用类似 hybrid。

FlashAttention v3 + CP

FA3 (Shah et al. 2024) 支持 `cp_group` , 直接把 ring attention 集成到 kernel 里 :

```
from flash_attn.flash_attn_interface import flash_attn_func
out = flash_attn_func(q, k, v, causal=True,
                      cp_group=cp_process_group,
                      cp_ring=True) # ring attention inside kernel
```

kernel 内部把 P2P 与 tile 计算 overlap , 比 Python-level ring 快 20-30%。Hopper 用 async TMA 让通信基本 hidden。

通信量对比表

对 $B = 1, S = 64K, H = 8192, bs = 2, CP=8$:

Method	Comm / layer / step	Head 限制	Kernel efficiency
Ulysses	$4BSH \text{ bs} (1 - \frac{1}{CP}) = 3.5 \text{ GB}$	$A \geq CP$	high (local attn)
Ring	$2BSH \text{ bs} = 2 \text{ GB}$	无	medium (small-K)
USP	between	$A \geq CP_u$	high
FA3 fused CP	same as ring but async	无	very high

表 17 长序列 CP 方案对比。Ring 通信量与 CP 无关但 kernel 效率打折 ; Ulysses 反之 ; USP hybrid 兼顾。

Long-context 训练的其他关键点

CP 只解决 attention。长 seq 还有其他显存来源 :

1. **Position embedding cache** : RoPE 的 cos/sin cache = $S \times d_h$, $S = 1M$ 时 = 500 MB。用 cached 或 fused RoPE (TE)
2. **LN activation** : BSH 沿 seq 切 (Megatron SP)
3. **FFN activation** : BSI 沿 seq 切 (Megatron SP + activation checkpoint)
4. **KV cache in generation** : inference 时 KV cache = $2LSH \text{ bs}$ per sample , $S = 1M$ 直接 500 GB/sample , 需要 PagedAttention

Recompute : Megatron `--recompute-granularity full` 让 attention layer 全 recompute , 只存 input , backward 重算——省 activation 但 compute 加 30%。CP + full recompute 是 1M ctx 训练常用组合。

SP/CP 的全栈适配 : attention 之外的所有 cross-token 模块

前面几节把 attention 讲透了 , 但把 CP 接进一个真实训练栈 , attention 只占改动量的一小半。原因很简单 : CP 沿 sequence 维切分 , 于是任何跨 token 计算或跨 token 归约的模块都被切断了 , 而 attention 只是其中最显眼的一个。

这类 bug 有一个共同的、极其讨厌的性质 :

Warning. 它们不报错。进程不 crash，NCCL 不超时，loss 曲线看上去完全正常——只是下降得慢一点，或者某个 metric 一直不收敛。你把 CP 从 1 调到 8，吞吐涨了 6 倍，验证 loss 差了 3%，很容易归因成“长序列本来就难训”。

Ring/Ulysses 写错了会直接数值爆炸，你当天就能发现；这一节的每一个坑都能安静地活到模型交付。

本节的代码在 `src/distributed_training/19_cp_loss_and_metrics.py`（跨 token 归约）与 `20_cp_data_loader_halo.py`（数据侧与局部算子），全部用单卡全序列结果做对拍，下面引用的数字都来自这两个 demo 的实际输出（CP=4）。

先看一张判定表

接 CP 时先把模型过一遍，对每个模块问同一个问题：它的输出依赖它自己那一段以外的 **token** 吗？

模块	跨 token	需要的处理	写错的征状
RMSNorm / LayerNorm	否	无。沿 hidden 归一化，逐 token 独立	—
FFN / SwiGLU / 逐元素	否	无	—
Attention	是	Ring / Ulysses / USP（前几节）	数值直接错，立刻发现
RoPE / 位置编码	是*	必须用全局 position_ids	长序列外推变差
Causal / doc mask	是	从全局 position 与 doc id 重建	跨文档泄漏
Cross-entropy loss	是	全局 token 数作分母	loss 偏移、等效 LR 变化
梯度尺度	是	明确 CP 是副本维还是分片维	梯度差 CP 倍
Label shift / MTP	是	切分前做位移，或 halo	边界 token 配错
Sliding window / conv	是	halo 交换 $w - 1$ 个 token	接缝处输出错
Mamba / SSM scan	是	传递递归 state，或 chunked scan	接缝处输出错
MoE router aux loss	是	全局 expert 直方图	负载均衡永不收敛
Pooling / reward 头	是	定位 owner rank 后归约	分数取自序列中间
Metrics (ppl / acc)	是	按 token 数加权归约	监控读数系统性偏移
Grad norm / clipping	是	只在分片维求和	范数虚高 $\sqrt{CP}$ 倍
Dropout RNG	是*	mask 的切分方式要与张量一致	正确性与单卡不可对拍

表 18 SP/CP 适配判定表。* RoPE 与 Dropout 本身逐 token 独立，但都依赖 token 的全局身份（位置、RNG offset），所以同样要改。前两行是提醒：不要给它们加通信。

Key insight. 表里最容易被忽略的是前两行。面试里常见的过度设计是“CP 下 LayerNorm 要不要 all-reduce”——不要。LN/RMSNorm 沿 hidden 维求均值方差，每个 token 自己算自己

的，与序列怎么切完全无关。这也正是 Megatron SP 能把 LN 放在 seq-shard 上零通信执行的原因 (§5)。

会真正需要通信的是 BatchNorm 这类沿 batch/序列维统计的算子——而 LLM 里基本不用它，这本身就是原因之一。

Loss：分母必须是全局有效 token 数

这是最高频的一个。CP 下每卡只有 S/CP 个 token，而 cross-entropy 的归约(求平均)是跨 token 的。

关键在于：真实 batch 里各卡的有效 token 数差别极大。SFT 的 loss mask 把 prompt 全部屏蔽，右侧还有 padding。demo 里 $S = 64$ 、 $CP=4$ ，各卡有效 token 数是 `[0, 12, 16, 2]`——rank 0 一个都没有。同时各卡的平均 loss 也不同（序列后段上下文更多、更好预测），实测 `[--, 3.91, 3.31, 1.01]`。

四种写法，只有最后一种对：

写法	实测值	为什么
参考值（单卡全序列）	3.3984	—
本地 mean 后跨 CP 求平均	NaN	rank 0 算了 0/0，一次 all-reduce 全场变 NaN
同上，但跳过空 shard	2.7457	$0.81 \times$ 。把每卡权重强行拉平成 $1/CP$
分母用本地总位置数	1.5930	$0.47 \times$ 。 <code>numel()</code> 而非 <code>mask.sum()</code>
先归约分子分母，再相除	3.3984	唯一正确

表 19 CP 下 cross-entropy 的四种归约。第三行是最危险的：数值有限、曲线平滑、完全错误。正确写法就一句话——把 **sum** 和 **count** 分别 **all-reduce**，最后才做除法：

```
# 每卡本地：只求和，不求平均
local_sum = (per_token_loss * loss_mask).sum()
local_cnt = loss_mask.sum()

# 跨 CP 组归约分子与分母 (count 不需要梯度)
global_cnt = local_cnt.clone()
dist.all_reduce(global_cnt, group=cp_group)

loss = local_sum / global_cnt      # 本卡对全局 loss 的贡献
```

Note. 第三行“跳过空 shard”值得单独说，因为它是唯一能长期存活的版本。它把每张卡的权重都变成 $1/CP$ ，与该卡实际持有多少真 token 无关。后果不是一个常数偏移，而是样本权重被重新分配：短答案样本被系统性放大，长答案被压小。你会看到模型偏好短输出，然后去调 length penalty——治的是症状。

这和 §15 里 agentic RL 的 `mask.numel()` vs `mask.sum()` 是同一个 bug 的两种形态：一个沿 turn 维，一个沿 CP 维。

梯度尺度：CP 组到底是副本维还是分片维

上面的 loss 写对了，还有第二个坑，而且更隐蔽——它只错在梯度里，**loss** 打印出来的数完全正确。

每卡算的是 $\text{local_sum} / \text{global_cnt}$ ，即“本卡对全局 loss 的贡献”。这些贡献需要和才等于全局 loss 的梯度。但 CP 组内参数是副本，而 Megatron 把 CP 维折进 data-parallel 的梯度归约里——DP 归约是求平均。平均一堆部分和，就丢了一个 CP 因子：

CP 维的梯度归约方式	$ g / g_{\text{ref}} $
AVG (DP 默认)	$0.2500 = 1/\text{CP}$
SUM	1.0000

表 20 CP=4 实测。两种写法的 loss 打印值一模一样，只有梯度差 4 倍——看曲线永远看不出来。

征状是“感觉 LR 小了 CP 倍”：loss 下降偏慢、grad_norm 偏小、把 LR 乘 CP 倍就好了——于是很多人就真的把 LR 乘上去当作调参结论，而没意识到是归约约定错了。

两种修法都对，选一种并写进单测：

1. 保持 AVG 归约，把 loss 乘 CP
2. CP 维用 SUM 归约，只在真正的 DP 维上做 AVG

Warning. 框架之间的约定不一致，这是升级 Megatron / 换 FSDP 时最容易踩的回归。DeepSpeed-Ulysses、Megatron-CP、torchTitan 对“谁负责这个 CP 因子”的默认值都不同。接 CP 时的第一个测试就应该是：固定数据、**CP=1** 与 **CP=N** 跑一步，比对参数更新量。

位置与 **mask**：必须作为 **batch** 的字段跟着 **token** 走

14.8.4.1 为什么先要 zigzag 置换

连续切分 (rank r 拿 $[rL, (r+1)L)$) 在 causal mask 下负载严重不均：rank 0 的 query 只能看极少的 key，最后一张卡几乎要看全序列。demo 实测每卡的未被 mask 的 (q, k) 对数：

contiguous : [36, 100, 164, 228] zigzag : [132, 132, 132, 132]

collective 走最慢的那张卡，所以代价是 max/mean 而不是更夸张的 max/min——实测 $1.73 \times$ ，CP 越大越趋近 $2 \times$ 。

Zigzag 把序列切成 2CP 个 chunk，rank r 取 chunk r 与 chunk $2\text{CP} - 1 - r$ ，一前一后配对。每卡的 (q, k) 对数变成严格相等 (上面第二组数字)，这也是 Megatron、vLLM、torchTitan 的默认做法。

	c0	c1	c2	c3	c4	c5	c6	c7
rank 0	■							■
rank 1		■					■	
rank 2			■			■		
rank 3				■	■			

表 21 CP=4 的 zigzag 布局：序列切成 8 个 chunk，rank r 取 chunk r 与 $7 - r$ 。每张卡都同时持有序列的前段和后段，causal 负载因此均衡。代价是每卡的 **shard** 不再是一段连续区间——这是后面所有麻烦的根源。

14.8.4.2 三样东西必须跟着 token 一起走

Zigzag 之后，rank 0 持有的全局位置是 $[0, 1, 2, 3, 28, 29, 30, 31]$ 。任何从 `arange(local_len)` 重新推导出来的东西都是错的：

1. 全局 **position_ids**。RoPE 是相对位置编码，但它靠绝对位置作差实现——每卡用本地 $0..L-1$ 会让所有相对距离都错。这是最经典的静默 bug：短序列时误差被 attention softmax 吸收掉一部分，长序列外推能力直接崩。
2. **document ids**。packing 场景下 (§12) 一条序列里塞了多个文档，attention 不能跨文档。切分后每卡需要自己那段的 doc id 才能重建 mask，varlen kernel 的 `cu_seqlens` 也要按本地 shard 重算。
3. 置换本身。输出要还原回原始顺序才能和 label 对齐、才能给下游模块用。

mask 不能再用“下三角”或任何基于本地 index 的形状，必须从全局量重建：

```
# q 在本卡 (长度 L), k 覆盖全序列 (Ring/Ulysses 拿到的那份)
causal    = pos_q[:, None] >= pos_k[None, :]      # 全局 position 比较
same_doc  = doc_q[:, None] == doc_k[None, :]      # packing 的文档隔离
mask = causal & same_doc
```

demo 里把“本地 position + 本地 index mask”和上面的正确写法都跑了一遍，对拍单卡结果：

$$\text{wrong : } \max|\Delta| = 3.557 \quad \text{correct : } \max|\Delta| = 2.4 \cdot 10^{-7}$$

Key insight. 工程上的结论很直接：把 **position_ids**、**doc_ids**、以及置换索引都做成 **batch** 的显式字段，由 data loader 产出、和 token 一起切分，模型里任何地方都不要用 `arange` 现场生成。

这条规则听起来平淡，但它是 CP 代码能不能长期维护的分水岭。一旦允许某个模块自己算位置，之后每加一个 CP 变体 (zigzag、striped、USP 的两级切分) 都要去找一遍所有 `arange`。

Data loader：CP 组内必须拿到同一条样本

一个 CP 组合起来才持有一条序列，所以组内每张卡必须拿到同一个样本，然后沿 seq 切分。这意味着 sampler 的索引必须用 `dp_rank`，不能用 `global_rank`：

```
cp_rank, dp_rank = rank % CP, rank // CP      # global = dp_rank * CP + cp_rank

sample_id = step * n_dp + global_rank         # 错：组内两张卡拿到不同样本
sample_id = step * n_dp + dp_rank             # 对
```

demo 里 `world=4 = DP2 × CP2`，错的写法让一个 CP 组看到 `[28, 29]` 两个不同样本，对的写法看到 `[14, 14]`。

后果有两层，都不报错：

1. rank 0 持有文档 A 的前半，rank 1 持有文档 B 的后半，attention 会把两者当成一条序列来算
2. 实际 global batch size 变成配置值的 CP 倍，等效 LR 与 token 预算全部对不上

shuffle 的随机种子有完全相同的要求——`base + global_rank` 会让组内两卡打乱出不同顺序，必须用 `base + dp_rank`。同一条规则也适用于 TP 组和 PP stage：只有 **DP** 维才允许数据不同。

Note. 这个 bug 在小规模冒烟测试里几乎测不出来：CP=2、跑 100 步，loss 照样下降（模型在学两个半截文档的拼接，仍然是有信号的语言建模任务）。可靠的检查是一行断言——把 `sample_id` 在 CP 组内 all-gather，要求全相等。这类断言应该常驻在 data loader 里，代价可以忽略。

需要 **halo** 交换的模块

有一类算子只依赖邻近的几个 token ,它们不需要全局通信 ,但需要从邻居拿一小段“边缘数据”(halo)。

14.8.6.1 Label shift : 一个 **token** 的位移就跨界了

Next-token prediction 里 `labels = tokens[1:]` 。本卡最后一个 token 的 label 在下一卡上。在 shard 内部 `roll(-1)` 会静默配错 :

布局	配错的 pair	说明
contiguous	$3 / 32 = CP - 1$	每卡末尾一个, 最后一卡那个本来无 label
zigzag	$6 / 32 = 2CP - 2$	每卡两段、两个接缝; rank $CP - 1$ 的两个 chunk 恰好相邻, 它的内部接缝碰巧对了

表 22 CP=4、 $S = 32$ 实测。zigzag 下错得更多, 而且“少错一个”纯属 chunk 配对的巧合——这类偶然正确性正是让人误判问题范围的东西。

正确做法几乎不花钱: 在切分之前对整条序列做一次位移, 然后照常切。不需要任何通信。

```
labels = torch.cat([tokens[1:], tokens[-1:]]) # 切分前做, 一次搞定
labels_local = labels[perm[r * L : (r + 1) * L]]
```

MTP / speculative decoding 的头要往后看 k 个 token , 同一个技巧一次处理完所有 k 。反过来说, 如果你只能在切分后处理, 就得做宽度 k 的 halo——而 zigzag 下“下一个位置”可能在任意一张卡上, 退化成一次不规则 gather。

14.8.6.2 Sliding window / conv / SSM

这些算子的 halo 宽度由感受野决定, 且必须通信 :

- **Causal conv1d** (kernel K): 需要前 $K - 1$ 个 token。demo 里 $K = 4$ 、 $CP=4$, 零填充版本错了 $9 = (K - 1)(CP - 1)$ 个位置, halo 版本与单卡完全一致
- **Sliding-window attention** (窗口 w): $halo = w - 1$
- **Mamba / SSM** 的递归扫描: 需要的不是 token 而是前一卡的递归 **state**, 这会把各卡串行化, 除非用 chunked scan 重写
- 多模态里 ViT / audio 前端的 depthwise conv 同理 (§13)

Warning. 注意误差的形状: 它被限制在每个接缝的 $K - 1$ 行里, 占比随 **shard** 变长而下降。 $S = 32$ 、 $CP=4$ 时错 28%, $S = 128K$ 、 $CP=8$ 时错 0.002%——loss 上完全看不见, 但模型在每个 shard 边界都学到了一小段错误的卷积响应。

这也解释了为什么带局部算子的模型通常不用 **zigzag**: zigzag 下一个 rank 有两段不连续区间, 要从两个不同的 rank 各拿一次 halo。连续切分的负载不均是可以量化、可以接受的 $1.7 \times$, 而不规则 halo 是工程复杂度。这是一个真实的设计取舍, 值得在面试里主动讲出来。

MoE router : aux loss 需要全局 **expert** 直方图

Load-balancing loss 形如 $aux = E \sum_e f_e P_e$, 其中 f_e 是路由到 expert e 的 token 比例、 P_e 是 router 对 e 的平均概率。两个都是“对 token 求平均”, 所以 CP 下两个都要全局归约。

只统计本卡会得到一个荒谬的结论。demo 构造了一个全局完美均衡的 router (rank r 的所有 token 都去 expert r , 于是每个 expert 恰好拿到 $1/E$):

统计范围	aux loss	$ \nabla $
参考值（全序列）	1.0000	0.00
只统计本卡	3.4802	0.131

表 23 CP=4、 $E=4$ 实测。全局直方图下 $f_e = 1/E$ 均匀，aux loss 退化为常数 1（理论最小值），梯度恰好为零——router 已经均衡，无需修正。只统计本卡则给出 $3.48 \times$ 的惩罚和一个不该存在的梯度。

方向更糟：本卡视角看到的是“我的 token 全去了一个 expert”，梯度在自己那个 **expert** 的 **logit** 上是正的（实测 +0.0283），也就是把每张卡都往远离全局最优的方向推。aux loss 在反对它本该促成的事情。

症状是 load-balance metric 反复震荡、永不收敛，而 expert 利用率看起来又没那么差。同一处理适用于 z-loss、以及任何从 per-rank token 数推出来的 capacity / drop rate (§8)。

归约到单点的头：pooling、reward model、GAE

有些头把整条序列归约成一个向量或一个标量，CP 下这个“点”只落在某一张卡上。

Reward model 是最典型的：分数取自最后一个有效 token 的 hidden state。demo 里最后一个有效 token 的全局位置是 49，只在 rank 3 上。各卡各自取“自己 shard 的最后一个有效 token”的话：

$\max|\Delta|$ per rank = [rank 0: 空 shard, 索引越界, 3.669, 3.610, 0.000]

只有 owner (rank 3) 是对的，rank 0 因为整段都是 padding 直接崩在空索引上。

正确做法用 one-hot 选择 + 一次可微 all-reduce，非 owner 贡献严格为零：

```
sel = torch.zeros(L, device=h.device)
if rank == owner:
    sel[global_last - rank * L] = 1.0
h_last = all_reduce_sum(sel @ h_local, cp_group) # forward 求和, backward 恒等

# mean pooling 就是前面 loss 那套分子/分母分别归约
h_mean = all_reduce_sum((h_local * mask_l[:, None]).sum(0)) / global_count
```

这里的 `all_reduce_sum` 必须是可微版本（Megatron 里的 `f/g` 算子）：forward 跨组求和、backward 恒等。裸的 `dist.all_reduce` 是 in-place 的，autograd 完全不知道其他卡的存在。

同类模块还有：embedding 模型的 mean pooling、分类头的 [CLS]、以及 RL 里的 **GAE**——广义优势估计是沿序列的反向递推，CP 下要么把 reward/value 序列聚合到一张卡上算，要么做跨卡串行扫描 (§15)。

Metrics：按 token 数加权，且只 exp 一次

监控指标和 loss 是同一个问题，但更容易被放过，因为“监控偏一点无所谓”——直到你用它做早停或选 checkpoint。

指标	参考	每卡平均后再平均	加权归约
perplexity	25.027	21.647	25.027
token accuracy	0.667	0.854	0.667

表 24 CP=4 实测。accuracy 偏高 28%，因为空 shard 与短 shard 被赋予了同样的权重。

perplexity 还多一层错误：exp 是凸函数，先在每卡 **exp** 再平均会因 Jensen 不等式产生偏差。必须先归约 **log-loss**，最后 **exp** 一次：

$$\text{ppl} = \exp \left(\frac{\sum_r \sum_i m_{ri} \ell_{ri}}{\sum_r \sum_i m_{ri}} \right)$$

按数据域、按任务分别记的 loss 全部同理——每个分桶都要自己的 (sum, count) 对。

Grad norm：只在分片维求和

梯度裁剪需要全局 $\|g\|$ 。CP 组内参数是副本，梯度归约之后每张卡持有的已经是完整梯度，不能再跨 CP 求和。但“把 $|g|^2$ 在全 world 上 all-reduce”是分片参数的标准写法，照搬过来就会：

写法	$\ g\ $	clip 系数 (max = 1.0)
正确（不跨 CP 求和）	0.8465	1.000（不裁剪）
错误（跨 CP 求和）	1.6930	0.591

表 25 CP=4 实测：范数虚高 $\sqrt{CP} = 2 \times$ 。本该完全不触发裁剪的一步，被把更新压到了 59%。

这是一次静默的 LR 削减，而且 grad_norm 监控帮不了你——记下来的就是那个虚高的值，看上去一切正常。

一条规则涵盖所有并行维：

$|g|^2$ 只在参数被分片的组上求和：ZeRO/FSDP shard、TP 的 column/row shard、EP 的 expert 维。

绝不在参数是副本的组上求和：CP、DP replica、TP 下被复制的 LN / bias。

Megatron 用 `param.tensor_model_parallel` 和 `param.shared` 这两个标记来区分，自己改框架时最容易漏的就是新加的参数忘了打标记（\$F\$）。

Dropout RNG：mask 的切分要与张量一致

Dropout 逐元素独立，看起来不跨 token，但它依赖 RNG 状态，而“该不该同步 RNG”取决于它作用的张量是怎么切的：

- 张量沿 seq 切（Megatron SP 的 LN/Dropout 区、CP 的所有 activation）：各卡是不同的 **token**，mask 就应该不同——但必须由全局 offset 可复现地生成，否则你无法与单卡对拍
- 张量在 TP 组内是复制的（例如 attention 之前的 input）：mask 必须逐位相同，否则各 TP 分支算的其实不是同一个前向

Megatron 为此维护两套 RNG tracker（`get_cuda_rng_tracker`）。这也是“CP=1 与 CP=N 对拍”时必须先关掉 dropout 的原因——否则你在对比两个不同的随机前向。

验证方法论：逐模块对拍

上面每一条都是“不报错的错”，所以唯一可靠的手段是与单卡参考实现做位级对拍，而且要逐模块做，不能只看最终 loss。

一个可以直接落地的流程：

- 固定一切随机性：关 dropout、固定数据、固定初始化种子
- 从内往外逐层对拍，出错立刻定位。每一层能抓到的东西不同，跳过任何一层都会漏掉一类 bug：

对拍对象	只有这一层能抓到
attention 输出 vs 单卡 SDPA	Ring/Ulysses 的 lse 合并、causal mask 分块
加上 position + mask	本地 arange 当位置、mask 没重建、逆置换写错
完整 forward 的 hidden states	halo 缺失 (conv/window) doc 隔离失效
loss 标量	分母用了本地 count 或 numel
每个参数的梯度	梯度尺度的 CP 因子—— loss 完全正确
一步更新后的参数	grad-norm 虚高、clipping 提前触发

表 26 CP 实现的逐层对拍。倒数第二行是最容易被跳过、也最容易出问题的一层。

- 断言常驻：sample_id 在 CP 组内相等、loss_mask.sum() 全局大于零、position_ids 单调且覆盖完整区间。这些检查每步的开销可以忽略，但能挡住绝大多数回归
- CP 扫描**：同一份数据跑 $CP \in \{1, 2, 4, 8\}$ ，要求 loss 与参数更新量在数值误差内一致。这是最强的一个测试，19_*.py 与 20_*.py 就是按这个思路写的

Key insight. 第 5 步（梯度对拍）是分水岭。只对拍 loss 的团队一定会漏掉梯度尺度那个 CP 因子，因为 loss 完全正确。面试里如果被问“你怎么保证 CP 实现是对的”，回答“和单卡比 loss 一致”是不够的，说到“比对参数更新量”才说明真的踩过坑。

面试考点

面试考点. **Q1**: 开了 CP 之后，除了 attention，还有哪些模块需要改？

按“是否跨 token”分类回答，比背清单更能体现理解：跨 **token** 归约的 (loss 分母、MoE aux loss 的 expert 直方图、pooling 头、metrics、grad norm) 要把 (sum, count) 分别归约再相除；依赖全局 **token** 身份的 (RoPE position_ids、doc mask、dropout RNG offset) 要让这些量随 token 一起切分而不是本地重算；依赖邻近 **token** 的 (label shift、MTP、sliding window、conv、SSM) 要做 halo 交换。反过来，逐 token 独立的模块 (RMSNorm/LayerNorm、FFN、逐元素算子) 不需要任何通信——能主动说出这一点，说明你不是在乱加 all-reduce。

Q2: CP 下 loss 怎么算？为什么不能各卡算完平均一下？

因为各卡的有效 token 数不同 (prompt masking + padding，极端情况下某卡为 0，直接 0/0 出 NaN)。必须本地只求和，把分子 $\sum m\ell$ 和分母 $\sum m$ 分别 all-reduce，最后相除。追问“跳过空卡再平均行不行”——不行，那等于把每卡权重拉平成 $1/CP$ ，实际效果是按样本长度重新加权，模型会偏好短输出。

Q3: loss 算对了，为什么梯度还可能差 CP 倍？

每卡产出的是“对全局 loss 的贡献”（分子是本地和、分母是全局数），这些贡献需要求和。但 CP 维常被折进 DP 的梯度归约，而 DP 是求平均，于是丢掉一个 CP 因子。修法：要么 loss 乘 CP，要么 CP 维改成 SUM 归约。关键点是 **loss** 打印值完全正确，只有梯度错，所以必须靠对拍参数更新量来发现，看曲线只会误判成“LR 需要调大”。

Q4: 为什么要 zigzag？它带来了什么新问题？

连续切分在 causal mask 下负载不均，代价是 $\max / \text{mean} \rightarrow 2 \times$ 。zigzag 让 rank r 取 chunk r 与 $2CP - 1 - r$ ，每卡的 (q, k) 对数严格相等。新问题是 **shard** 不再连续 position_ids 和 doc_ids 必须按同样的置换分发、mask 要从全局 position 重建而不能用下三角、输出要逆置换、而 halo

类算子会退化成从两个不同 rank 各取一次——所以带 conv/sliding-window/SSM 的模型往往宁愿承受连续切分的 $1.7 \times$ 不均。

Q5: CP 组内各卡应该拿到相同还是不同的数据？怎么验证？

相同——一个 CP 组合起来才是一条序列。sampler 和 shuffle 种子都要用 `dp_rank` 索引，不能用 `global_rank`。写错了不会报错：attention 会把两个半截文档当成一条序列，同时 global batch size 悄悄变成配置值的 CP 倍。验证只需一行——把 `sample_id` 在 CP 组内 all-gather 后断言全相等，建议常驻。

Q6: 为什么 CP 下 grad norm 容易算大 $\sqrt{CP}$ 倍？

因为把分片参数的写法（跨组 all-reduce $|g|^2$ ）套到了副本参数上。CP 组内参数是副本，梯度归约后每卡已持有完整梯度，不该再求和。规则是：只在参数被分片的组上求和（ZeRO/FSDP、TP shard、EP），不在副本组上求和（CP、DP replica、TP 复制的 LN）。范数虚高会让裁剪提前触发，等于静默削减 LR，而且 grad_norm 日志记的正是那个虚高值，查不出来。

完整 CP 训练配置样例

Megatron-LM 训 Llama 8B, S=128K, 8 卡：

```
torchrun --nproc-per-node=8 pretrain_gpt.py \
  --num-layers 32 --hidden-size 4096 --num-attention-heads 32 \
  --seq-length 131072 --max-position-embeddings 131072 \
  \
  --tensor-model-parallel-size 1 \
  --context-parallel-size 8 \
  --sequence-parallel \
  --use-distributed-optimizer \
  \
  --micro-batch-size 1 --global-batch-size 8 \
  --recompute-granularity full \
  --recompute-num-layers 1 \
  --distributed-timeout-minutes 60 \
  \
  --bf16 --use-flash-attn --swiglu \
  --normalization RMSNorm --position-embedding-type rope \
  --rotary-base 500000
```

<-- CP=8
Megatron SP for LN
长 seq 必须
长 seq 慢
长 ctx 一般加大 base

带 CP + FlashAttention v3 + selective recompute，H100 8 卡跑 Llama-8B S=128K，MFU 35%。

面试考点

面试考点. Q1: Ulysses 与 Ring Attention 的核心区别是什么？

A: Ulysses 用 all-to-all 沿 head 维交换，一次通信后每卡拿到完整 seq 的部分 head——attention 本地做。Ring 用 P2P 沿 ring 逐步旋转 K/V——每步只做 local partial attention，累加 Flash lse。Ulysses 简单快但 $CP \leq \text{head 数}$ ；Ring 无 head 限制但 kernel 效率低。

面试考点. Q2: Ring Attention 里怎么处理 causal mask ?

A: naive 做法是 rank r 只在收到 rank $r' \leq r$ 的 K/V 时算 attention , 其他 skip。但这导致 rank 0 一直闲、rank last 一直忙 , 负载不均 $2\times$ 。改进用 Striped Attention 或 Zigzag : 把 seq permutation 让每 rank 都持“前半+后半”pattern , 负载均衡。Megatron/vLLM 生产实现都用 Zigzag。

面试考点. Q3: 训 1M context , 你会怎么组合并行 ?

A: 假设 512 卡 H100。举例 : TP=8 $\times$ CP=16 (USP hybrid = Ulysses 4 $\times$ Ring 4) $\times$ PP=2 $\times$ DP=2。CP 主承担 seq 切 ; TP 切 attention head ; FSDP2 切 weight。加 full activation recompute + FA3 with `cp_ring`。RoPE base scaled。

面试考点. Q4: 为什么 Ulysses 的通信量与 CP 有关但 Ring 与 CP 无关 ?

A: Ulysses 一次 all-to-all volume = $\frac{V(W-1)}{W} \approx V$ ($V = BSH$) , 与 CP 弱相关。Ring 每步 P2P $\frac{V}{CP}$, 共 CP 步 $\rightarrow$ total V , 与 CP 完全无关。所以 CP 大时 Ring 通信量优势明显 (尤其单步 kernel 效率恢复后)。

面试考点. Q5: 一个 GQA num_kv_heads=8 的模型 , 能用 CP=16 的 Ulysses 吗 ?

A: 不能——KV head 只有 8, 无法切 16 份。要么 (a) CP=8 (Ulysses) ; (b) 用 Ring ; (c) 用 USP 分解 : Ulysses=8 $\times$ Ring=2 = CP 16。生产常用 (c)。

面试考点. Q6: CP 之后的 output projection 需要通信吗 ?

A: 不需要额外通信。Attention output 是 $(B, \frac{S}{CP}, A, d_h)$ (seq 切分) , output projection W_O 是 row-parallel 或全 replicate。如果 W_O replicate , 直接本地 matmul 得 $(B, \frac{S}{CP}, H)$ ——顺利传给 FFN。如果与 TP 组合 , output proj 是 row-parallel , 需要 TP AllReduce (不是 CP)。

面试考点. Q7: FlashAttention 与 CP 怎么集成 ?

A: FA 内部的 online softmax 已经支持“分块累加” (LSE 合并) 。Ring Attention 只需在每步 P2P 之后调 FA + 合并 LSE 到 accumulator。FA3 直接提供 `cp_ring` kernel , 把 P2P 用 CUDA async 与 tile compute overlap , 比 Python-level ring 快。

面试考点. Q8: 长序列训练里 activation 显存的主要来源 ? CP 之外还能省什么 ?

A: 层级 BSH (LN, Dropout, residual) + BSI (FFN intermediate) + Q/K/V/O activation。CP 切 seq 只解决 attention 内的部分。剩下的靠 : (1) Megatron SP 沿 seq 切 LN/Dropout ; (2) FSDP 沿 DP 切 weight ; (3) `--recompute-granularity full` 只存 layer input ; (4) activation offload 到 CPU (DeepSpeed / TE) 。DeepSeek-V3 Table 3 里 activation 占显存 66% , 是 long-ctx 的首要优化对象。

Expert Parallel 与 MoE 训练概览

MoE 训练的详细内容 (router, dispatcher, all-to-all, DeepEP, DualPipe) 都在姊妹卷《Sparse MoE 训练实战》里。这一章只讲与其他并行策略的集成点和面试里最容易混淆的概念区分, 避免重复。

EP 的核心: 把 expert 沿 rank 切

MoE 层的稀疏性 = “每 token 只走 $\frac{K}{E}$ 的专家”。切 EP 就是把 E 个专家分给 EP 张卡, 每卡持 $\frac{E}{EP}$ 个 expert 的完整权重。

MoE dispatch: 8 tokens → 4 experts (top-1 routing)

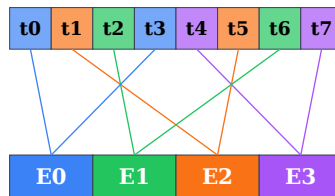


图 23 Router 给每 token 打分选 top-k expert。EP 情况下 expert 分散在多卡上, 同色 token 必须聚到对应 rank——这一步就是 all-to-all dispatch。反向再一次 all-to-all 把结果 combine 回原 rank。

代码见 `src/distributed_training/09_ep_moe.py`——用一次 `all_to_all_single` 完成 dispatch, 另一次做 combine, 与单卡 full-experts baseline 数值对齐。

四种并行在 MoE 里的分工:

并行	在 MoE 里干嘛
DP	batch 沿样本切, 非-expert 部分 grad AllReduce
TP	切 attention head + expert 内 GEMM 沿 I 维 (ETP)
PP	沿 layer 切, MoE 层与 dense attention 层交替 stage
CP	attention 沿 seq 切, MoE 部分 seq 也切
EP	MoE 专属: expert 沿 expert-id 维切, dispatch/combine 用 all-to-all

表 27 五种并行在 MoE 里的角色。EP 是 MoE 独有; 其他四个在 dense 与 MoE 里作用相同。

EP × DP 的关系

非-expert 参数 (attention, gate, LN, embedding): 跨 EP 组完全复制——因为它们本来就该完整存在每卡, 与 expert 分片无关。这些参数的 grad 用 DP AllReduce (也可能跨 EP 组)。

Expert 参数: EP 组内切分 (每 rank 持不同 expert), EP 组间也可能有 replica (若 $EP < world_size$)。这时 expert 的 grad 需要“expert-DP 组”内 AllReduce。

Megatron-Core 的 `--expert-model-parallel-size` 与 `--data-parallel-size` 组合:

- `EP=8, DP=8, world=64`: 8 个 EP 组, 每组 8 卡; 每 EP 组的 expert 有 8 份 replica, 跨 8 个 DP rank 做 AllReduce
- `EP=64, DP=1, world=64`: 单一 EP 组, 无 DP replica, 无需 expert-DP AR

第二种通信少但受限于 `batch = MBS × PP` (无 DP 加大)。生产常用 EP × DP 混合。

Parallel Folding (Megatron-Core 2026) : 打破 $EP \leq DP$ 约束, Attention 与 MoE 用独立 process group。允许配置: $attn-DP = 32, expert-EP = 64, expert-DP = 2$ 之类不对称。

通信量与 **dense** 的对比

Dense Transformer 一层 forward + backward :

- DP: 每 step $2P$
- TP: 每层 $2 \text{ AR (fwd+bwd)} \times 4 \text{ 层组件} = 8 \text{ AR/step}$, activation-sized
- PP: $P2P$

MoE 一层多出 :

- 2 次 dispatch a2a (fwd + bwd)
- 2 次 combine a2a (fwd + bwd)
- 每次 volume $\approx BSK_{EP}^H \times bs$

对 $B = 1, S = 4K, K = 2, H = 4K, EP = 8, bs = 2$: 单层 a2a = $4 \times 2 \times 4096 \times 4096 \times 2 / 8 = 32 \text{ MB}$ 。32 层 = 1 GB per step。跨节点 IB 50 GB/s 需要 20 ms —— 不 overlap 就吃掉 10% step time。

这就是为什么 MoE overlap (DualPipe / FWD-BWD merged) 比 dense 更重要。

与 **CP** 的组合 : **MoE** 长序列训练

Kimi K2, DeepSeek-V3.2 等长上下文 MoE 需要 $EP + CP + DP$ 。

注意点 :

1. MoE 层的 dispatch a2a 与 CP 的 Ring/Ulysses P2P 会争带宽
2. 一般让 EP 与 CP 走不同 device mesh 维度 : EP 在同节点 NVLink , CP 沿另一维度
3. FSDP 沿“pure DP”轴切 param , 不含 EP 组内的 expert
4. **router 的 load-balancing aux loss** 必须在 **CP** 组上归约 **expert** 直方图。 f_e (token 比例) 和 P_e (平均概率) 都是“对 token 求平均”, CP 下每卡只看到 $1/CP$ 的 token。只统计本卡会让一个全局均衡的 router 被判为极度不均衡, 梯度方向反而把各卡推离全局最优——征状是 load-balance metric 长期震荡不收敛。同理适用于 z-loss 和任何从 per-rank token 数推出的 capacity / drop rate。详见 §7 与 19_cp_loss_and_metrics.py

示例 Kimi K2 32-expert MoE, 1M ctx on 512 H100:

```
world = 512
TP = 8          (NVLink)
CP = 16         (USP: Ulysses 8 × Ring 2)
EP = 32         (32 experts, one per rank in EP group)
PP = ?          (取决于层数)
DP = auto-fill
```

CP 组与 EP 组尽量正交 (不共享 NVLink 带宽), 实测里 CP 走同 node 8 卡 NVLink , EP 跨节点 IB (EP a2a 用 hierarchical / DeepEP 缓解)。

EP 通信量公式速查

从 MoE 书里搬过来最核心的几个公式 :

Dispatch (forward) , 每卡 send:

$$\text{vol}_{\text{dispatch}} \approx N_{\text{local}} K H \text{ bs} \times \frac{\text{EP} - 1}{\text{EP}}$$

Combine (forward) : 同 dispatch , 方向反。

一层 **forward+backward total per-GPU**: $4 \times \text{dispatch} = 4N_{\text{local}} K H \text{ bs}$ (EP 大时)

与 E (总 expert 数) 无关——每 token 还是走 K 个专家, 通信量不变。这是 DeepSeek-V3 用 256 experts 却通信量与 8 experts 一样的原因。

ETP: Expert Tensor Parallel

背景: 如果单个 expert 权重太大 (I 大), 单卡装不下 → 把 expert 内的 GEMM 沿 I 维 TP 切。

Megatron flag: `--expert-tensor-parallel-size 2` (与 `--tensor-model-parallel-size` 独立)。

用法:

- Mixtral 8×22B: $I = 16384$, 单 expert weight = 128 MB, 不需要 **ETP**
- 假想 100B expert-size: 需要 ETP=2 或 4

DeepSeek-V3 用 EP=64, ETP=1 (fine-grained expert 都很小)。Qwen3-235B MoE 类似。

与 PP 的互动: MoE 层 vs Dense 层

一个 61 层的 DeepSeek-V3 里, MoE 层与 dense (attention only) 层交替:

- 前 3 层: dense (只 attention + MLP, 无 MoE)
- 中间: dense MLP + MoE FFN 组合
- 每层的 attention 部分与 MoE FFN 部分并列

PP 切分时要考虑各 stage 的 FLOPs 均衡:

- MoE 层 FLOPs $\approx 2 \times$ attention FLOPs (有 gate + expert compute)
- 单纯按层数均分 → MoE 集中的 stage 慢
- Megatron `--custom-pipeline-partitioning` 手动分: dense 密集 stage 多几层, MoE stage 少几层

常见配置模板

从 MoE 书里搬过来的两份典型配置:

Mixtral 8×7B on 64 H100 (标准 MoE):

```
TP=1, PP=4, EP=8, DP=2
Dispatcher: alltoall
overlap: --overlap-moe-expert-parallel-comm --delay-wgrad-compute
Recompute: selective
Precision: BF16
```

DeepSeek-V3-like fine-grained (256 exp) on 512 H100+:

```
TP=1, PP=16, EP=64, DP=2 (或更多 for GBS)
Dispatcher: flex + deeppep
overlap: DualPipe (or FWD-BWD merged for simpler)
```

Recompute: fine-grained modules
Precision: FP8 blockwise

详细每个 flag 的含义与调参参见 MoE 书第 8 章。

面试考点

面试考点. **Q1**: 为什么 EP 是 MoE 独有, dense 模型不用?

A: EP 切 “expert-维度”, dense 模型没有 expert——每层就是一个 FFN, 切它的方式是 TP (沿 hidden dim) 或 SP。MoE 有 E 个独立 FFN, 天然可以按 expert-id 切, 稀疏的路由让通信只对涉及的 tokens 做 (all-to-all), 比 dense TP 的全量 AllReduce 更符合稀疏语义。

面试考点. **Q2**: EP=8 与 TP=8 都能把 expert 权重切 8 份, 选哪个?

A: EP。TP 切 expert 后每卡还是有 E 个“半专家”——dense 化, 浪费稀疏性。EP 让每卡持完整的 $\frac{E}{8}$ 个专家, 通信只在 dispatch 时对被路由的 **tokens** 做, 与 expert 数无关。EP 通信 $\propto BSKH$, TP 通信 $\propto BSH$ (但每层都 AR), EP 更符合 MoE 稀疏本质。

面试考点. **Q3**: EP 组的 all-to-all 与 TP 组的 AllReduce 会争带宽吗?

A: 会。EP 组与 TP 组物理上共享 NVLink/IB。生产做法: 让 EP 与 TP 走不同 device mesh 维度, 让 TP 组同 NVLink 域 (NVL8)、EP 组尽量也同域 ($EP \leq 8$)。EP > 8 只能跨节点, 用 hierarchical a2a 或 DeepEP 减轻。DeepSeek-V3 干脆不用 TP。

面试考点. **Q4**: 为什么 MoE 训练 overlap 比 dense 更重要?

A: dense 一层 forward 只有 attention output projection 和 FFN 后的 2 次 AR (TP 组); MoE 多了 2 次 all-to-all (dispatch + combine)。a2a 是 EP 组间的大 payload 通信, 占 step 15-40% (未 overlap), overlap 到 <5% 需要 DualPipe 或 Megatron FWD-BWD merged。dense 里通信占比小, overlap 收益也小。

面试考点. **Q5**: expert-DP 与 attention-DP 分开有什么用?

A: Parallel Folding。传统 Megatron 要求 EP $\times$ 内的 DP 与 attention 的 DP 相同 (因为 world_size 拆分是全局的)。Parallel Folding 允许: “attention 部分” DP=32, “expert 部分” EP=64+DP=2——非 expert 部分获得更大有效 batch ($32 \times MBS$), expert 部分利用大 EP 减少通信。DeepSeek 类 fine-grained MoE 常用。

面试考点. **Q6**: fine-grained MoE (256 experts) 比 vanilla MoE (8 experts) 显存/通信量各是几倍?

A: 通信量: 相同 (不依赖 E)。显存: weight 相同 (相同总参数), 但 grouped GEMM 每 expert 的 M 维小 $8 \times$ (每 batch 里每 expert 平均更少 tokens) $\rightarrow$ 更容易 memory-bound。所以 fine-grained MoE 需要更好的 grouped GEMM kernel (DeepGEMM, TE GroupedLinear) 才不掉率。

精度：从 AMP 到 BF16 到 FP8 / FP4

从 2018 年的 FP16 AMP 到 2024 的 FP8 全链路训练，“精度”是每次硬件迭代都会重打一遍的战场。这一章讲清楚：每种格式的数值范围、什么会溢出/下溢、GradScaler 的作用、Transformer Engine 的 FP8 recipe、DeepSeek-V3 的 blockwise FP8。

数值格式速览

Format	Bits	Sign	Exp	Mantissa	Range
FP32	32	1	8	23	$\pm 3.4 \times 10^{38}$
TF32	19 (*)	1	8	10	FP32 range, FP16 mantissa
BF16	16	1	8	7	$\pm 3.4 \times 10^{38}$
FP16	16	1	5	10	$\pm 6.5 \times 10^4$
FP8 E4M3	8	1	4	3	± 448
FP8 E5M2	8	1	5	2	$\pm 5.7 \times 10^4$
FP4 E2M1	4	1	2	1	± 6
MX FP8	8+	1	4/5	3/2	E4M3/E5M2 + shared FP8 scale per 32-elem block
MX FP4	4+	1	2	1	E2M1 + shared FP8 scale per 32-elem block
NVFP4	4+	1	2	1	NVIDIA 变种, 16-elem block + double scale

表 28 常见数值格式。TF32 是 19-bit “假 32”（NVIDIA A100+ TensorCore 内部用，用户看到的是 FP32 tensor）。FP8/FP4 都是 block-scaled 或 tile-scaled，独立的 scalar 存 exponent。

关键 **tradeoff**：exponent 决定动态范围（会不会 overflow/underflow），mantissa 决定精度（同一 exponent 下能表示多少个不同值）。BF16 = FP32 range + FP16 mantissa：range 够用（训练稳定），但精度差 → 累加误差大。FP16 反过来：精度好但 range 小（gradient underflow）。

混合精度训练的 **hierarchy**

基本原则：以最低够用的精度做 **compute**，以最高精度做 **accumulate**。

一次 Adam 更新的完整 dtype 流：

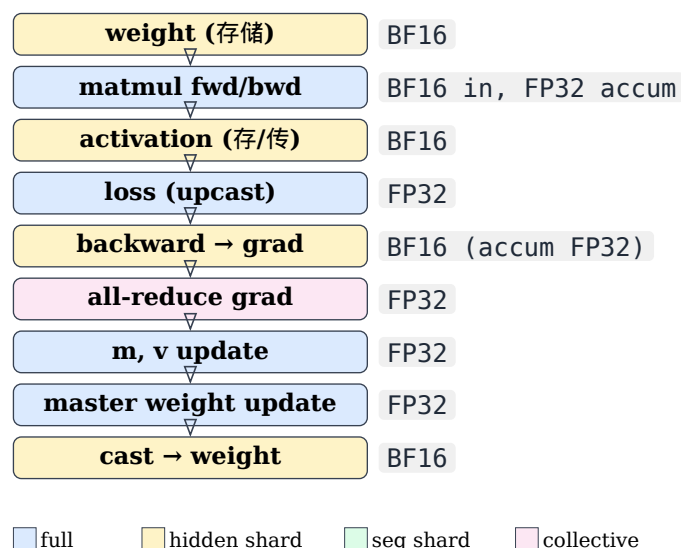


图 24 Adam step 的 dtype 流。TensorCore 天然做 BF16 → FP32 accumulate ;optim state 全 FP32 ; master weight FP32 保留低精度更新的累积精度 ; 只有 forward compute 与 activation 存储走 BF16。

optim step 伪代码 :

```
grad_fp32 = grad.to(FP32) / world_size
m = beta1 * m + (1-beta1) * grad_fp32           (FP32)
v = beta2 * v + (1-beta2) * grad_fp32.square() (FP32)
master_weight -= lr * m / (sqrt(v) + eps)       (FP32)
weight = master_weight.to(BF16)                (broadcast to BF16 replica)
```

FP32 master weight 存在的原因 : BF16 mantissa 只 7 bit , `weight - lr * update` 里 `lr * update` 通常比 `weight` 小 1000+ 倍 , 直接 BF16 减会 round 到 0。FP32 master 累积小更新 , broadcast BF16 用于 compute。

FP16 与 GradScaler

FP16 因 exponent 只 5 bit , $\exp(x)$ 里 x 稍大就 overflow (65504+ = inf)。梯度小时反过来 underflow 到 0。

GradScaler 的做法 : `loss × scale` (e.g. 65536) , 反向传播的梯度也 `×scale` , 进入 FP16 表示范围。step 前 `unscale` 回去 , 检查 inf/nan 决定是否 skip step。

```
scaler = GradScaler(init_scale=65536)
for x in data:
    with autocast(dtype=torch.float16):
        loss = model(x).loss
        scaler.scale(loss).backward()           # loss × scale
        scaler.unscale_(optim)                  # grad /= scale
        torch.nn.utils.clip_grad_norm_(...)     # 现在 grad 是真值
        scaler.step(optim)                      # 内部检查 inf, decide skip
        scaler.update()                         # dynamic scale adjust
```

BF16 完全不需要 **GradScaler**——BF16 与 FP32 exponent 相同 (8 bit) , range 一致。所以 A100/H100 时代 fp16 训练几乎绝迹。

Transformer Engine 的 mixed precision

TE 是 NVIDIA 官方 mixed-precision library，几个关键点：

1. **fused LayerNormLinear**：LN + Linear 一个 kernel，中间不落 FP32 activation
2. **LayerNormMLP**：整个 MLP (LN + Linear + GELU + Linear) 融成一个 kernel，内部保持 high precision accumulate
3. **DotProductAttention**：FlashAttention wrapper，支持 BF16 和 FP8
4. **Float8Tensor**：管 FP8 scale 的抽象，透明地处理 cast

用法：直接替换 nn.Linear：

```
import transformer_engine.pytorch as te

self.linear = te.Linear(H, H, bias=False) # replaces nn.Linear
self.norm_linear = te.LayerNormLinear(H, 4*H, eps=1e-5)
```

TE 内部 kernel 是 CUTLASS 手写，比 PyTorch 默认路径快 15-30%。生产 Llama/DeepSeek 训练都用 TE。

FP8 训练：Hopper 的杀手锏

H100 Tensor Core 支持 FP8 GEMM，peak 是 BF16 的 $2\times$ (989 TFLOPS BF16 $\rightarrow$ 1979 TFLOPS FP8)。理论算力翻倍——如果能不掉精度用起来。

难度：FP8 精度极低 (mantissa 3 bit = 8 个 unique 值 per exponent)，需要精细的 **scale** 管理。

三种 FP8 recipe

1. **TE Delayed Scaling** (default, TE v0-v1)：每 tensor 一个 scale，用上一次的 amax 估计当前 scale。简单但对 outlier 敏感。
2. **TE Current Scaling**：每次都当前 tensor 的 amax，无 lag，但每次要多一次 reduce 求 amax。
3. **DeepSeek Blockwise FP8**：weight 沿 (128, 128) tile 各存一个 FP32 scale；activation 沿 (1, 128) tile scale。granularity 更细，对 outlier 强 robust。DeepSeek-V3 训练用的 recipe。
4. **MXFP8** (Blackwell)：hardware-native block scaling，每 32 元素共享一个 FP8 scale。B200/GB200 支持，不需要软件层管理。

FP8 GEMM 的 forward-backward 全流程

标准 GEMM $Y = XW$ 在 FP8 下：

Forward：

```
X_fp8 = quantize(X_bf16, amax_X) # per-tile scale
W_fp8 = quantize(W_bf16, amax_W) # per-column scale (or blockwise)
Y_fp32 = fp8_gemm(X_fp8, W_fp8) # TensorCore FP8 in, FP32 out
Y_bf16 = Y_fp32.to(bf16) # cast for storage
```

Backward (3 个 GEMM 都可以 FP8)：

```
# dX = dY · W.T (dgrad)
dY_fp8 = quantize(dY_bf16)
```

```

dX_fp32 = fp8_gemm(dY_fp8, W_fp8.T)

# dW = X.T · dY (wgrad)
X_fp8_T = X_fp8.T
dW_fp32 = fp8_gemm(X_fp8_T, dY_fp8)

```

DeepSeek-V3 三路 GEMM (Fprop/Dgrad/Wgrad) 全 FP8 , 只有 optimizer state 和 master weight 保持 FP32/BF16。

FP8 activation 存 checkpoint

Activation 用 FP8 存 (比 BF16 省 $2\times$), backward 时反量化。DeepSeek Table 3 里 activation 12% → 用 FP8 后 6% , 配合其他技术总显存降 30%。

代价 : quantize/dequantize kernel。Hopper 上有硬件加速 , overhead < 3%。

FP8 vs BF16 精度差

DeepSeek-V3 tech report 报告 : 14T tokens 训练 , FP8 vs BF16 val loss 差 < **0.25%**。基本无损。Meta 的 Llama-3 405B 训练用 BF16 但内部实验 FP8 , 结论类似。

不适合 **FP8** 的场景 :

1. Optimizer state (要 FP32)
2. LayerNorm / softmax (中间 accumulate 用 FP32)
3. Loss 计算
4. Attention softmax 内部 (TE 里 FA-FP8 只把 GEMM 用 FP8 , softmax 是 FP32)

FP4 训练 (Blackwell)

B200 支持 FP4 GEMM , peak 是 BF16 $4\times$ 。目前只有 inference 广泛用 , 训练是研究前沿。

NVFP4 (NVIDIA 变种) : 16-elem block scaling + double scale (per-block FP8 scale + per-tensor FP32 scale) , 缓解 FP4 精度不足。Nemotron 405B 训练测试报告 FP4 可行但需要精细 recipe。

生产用 FP4 训练可能要等 2026-2027。

BF16 训练的精度坑

BF16 训练的隐藏问题 :

1. **AllReduce accumulate** 误差 : BF16 grad 在 AllReduce 时按 BF16 累加。W 大 (1000+) 时累加误差累积 , 导致 loss curve 抖动。解决 : `--grad-reduce-in-fp32` (Megatron) , 或 `FSDP MixedPrecision(reduce_dtype=torch.float32)`。通信量翻倍 , 稳定性提升。
2. **LayerNorm mean/var** : BF16 mantissa 7 bit 不够表示 $\sum \frac{x_i^2}{N}$ 的精度。TE / Megatron 的 LN 内部用 FP32 accumulate。
3. **Softmax** 内部 : $\text{softmax}(x) = \frac{\exp(x - \max)}{\sum \exp}$, $\sum$ BF16 累加会误差累积。FA/TE 内部 FP32。
4. 长序列 **attention** : QK^T 的 $\sum$ 沿 d_h 累加 , BF16 里 $d_h = 128$ 累加 128 项误差在 10^{-2} 级。FA 内部 FP32 accumulate 是标配。

所以 : “BF16 训练”实际是 “weight/activation BF16 , 中间关键操作 FP32 accumulate”。全 BF16 会训炸。

Loss Scale (BF16 版)

BF16 不需要 GradScaler 但依然会遇到 vanishing gradient——尤其 rank 大 (1000+ 卡) 时。补救：

1. **Loss shift** : `loss = loss * loss_scale`, `loss_scale = 1024` 之类常数。有效放大梯度，减小累加误差
2. **Z-loss** (PaLM 2) : `z_loss = 1e-4 * log(sum exp(logits))^2` , 防 logits 爆炸，间接稳定梯度尺度

面试考点

面试考点. **Q1**: BF16 与 FP16 都是 16 位，为什么大模型训练都选 BF16？

A: exponent 位数。BF16 8 bit exp (与 FP32 同) 动态范围 10^{-38} 到 10^{38} ；FP16 5 bit exp 只有 6×10^{-5} 到 6×10^4 。梯度小时 FP16 underflow 到 0，梯度大时 overflow 到 inf。要用 FP16 得配 GradScaler；BF16 直接开箱用，代价是 mantissa 少 3 bit (7 vs 10)，精度稍差但训练无影响。

面试考点. **Q2**: FP32 master weight 为什么必需？

A: BF16 mantissa 7 bit，`weight - lr × update` 里 `lr × update` 通常比 weight 小 1000+ 倍。BF16 直接减 round 到 0，权重不动。FP32 master 累积小更新，broadcast BF16 用于 fwd/bwd。Adam 的 m/v 也在 FP32 累加，同理由。

面试考点. **Q3**: GradScaler 里 dynamic scale 怎么调？

A: 初始 65536。每步 unscale 后检查 grad 是否含 inf/nan：

- 有 inf → skip step，`scale × 0.5`
- 连续 N 步都无 inf → `scale × 2`

目标是把 scale 维持在“最大不 overflow”附近。稳态下大概每 2000 步更新一次。

面试考点. **Q4**: FP8 训练里 Delayed Scaling 与 Current Scaling 的差别？

A: Delayed 用上一次的 amax 估计当前 scale，无额外 reduce，快但对 outlier 敏感（一个 spike 让下一步 scale 猜错）。Current 每次求当前 amax，需要一次 tensor-wide reduce，overhead 3-5%，但精度更稳。TE 里 delayed 是默认，DeepSeek 用 blockwise（每 tile 一个 scale，反正每 tile 都要 reduce）。

面试考点. **Q5**: DeepSeek 的 blockwise FP8 是什么？为什么比 tensor-wide FP8 稳？

A: Weight 按 (128,128) tile 各存一个 FP32 scale；activation 按 (1,128) tile scale。粒度细，一个 outlier 只影响自己 tile 而不是整个 tensor。tensor-wide scale 里一个 spike 让全 tensor 的 scale 猜错 → 其他 tile quantize 到很低分辨率。blockwise 完全隔离。代价：scale 存储稍多、kernel 复杂化，但 CUDA-native 支持。

面试考点. **Q6**: FSDP + BF16，什么时候要用 `reduce_dtype=torch.float32`？

A: 世界大小 > 512 卡, 或看到 loss curve 有 non-decreasing 抖动。BF16 grad AllReduce 里 W 大时累加误差 $\propto W$ 。 $W = 1024$ 时误差可能到 10^{-3} 级别, 影响 optim update。FP32 reduce 通信量翻倍但精度高。生产 Llama-3 405B 用 FP32 reduce。

面试考点. **Q7**: FP8 训练里 attention 内部为什么不用 FP8 ?

A: softmax 需要 $\max + \exp + \text{sum} + \text{divide}$, FP8 mantissa 3 bit 精度不足以做 $\exp$ 累加。FlashAttention-FP8 只把 QK^T 和 PV 两个 GEMM 用 FP8, softmax 中间 stats 全 FP32。BF16 版 FA 里 softmax stats 也是 FP32 accumulate。

面试考点. **Q8**: 一个 405B 模型用 FP8 训比 BF16 快多少? 成本降多少?

A: TensorCore FP8 peak 是 BF16 $2\times$, 但实际 MFU 会略降 (量化 overhead + 精度校验)。理论 $2\times$ wall-clock 加速, 实际 $1.5-1.7\times$ 。参数存储也少一半 (若激活也 FP8)。整体 GPU-hour 成本 -35% 到 -45% 。DeepSeek-V3 671B on 2048 H800 花了 180K GPU-hrs/T tokens = \$5.3M, 若 BF16 需要 \$8-9M。

Activation Checkpoint 与 Offload

Activation 是训练显存的大头 (DeepSeek-V3 Table 3 里占 66%)。这一章讲清楚 activation checkpoint 的原理、selective / fine-grained 版本、以及 offload 到 CPU/NVMe 的时机。

Activation 从哪来

一层 forward 计算图里, 每个中间张量都被 backward 需要 (求梯度要 input)。粗略 accounting, 一层 Transformer 的 activation:

- LN input & output : $2BSH$
- Q, K, V projection output : $3BSH$
- Attention scores (FA 后省) : BS^2A (naive) 或 0 (FA)
- Attention output : BSH
- FFN input : BSH
- FFN intermediate : $BSI \approx 4BSH$
- FFN output : BSH
- Residual : saved for grad flow

一共约 $17 - 34BSH$ per layer (因是否 fused、是否 Flash 而异)。80 层 = 上 TB。

Activation Checkpoint (recompute) 的基本思想

Chen et al. 2016 提出。核心: 只存少量 **activation** (每 layer 的 **input**) , **backward** 时重算前向。

```
from torch.utils.checkpoint import checkpoint

class TransformerLayer(nn.Module):
    def forward(self, x):
        return checkpoint(self._forward, x, use_reentrant=False)

    def _forward(self, x):
        # 完整的 attention + FFN
        ...
```

Backward 到这层时, PyTorch 会重新跑一遍 **forward** (不存中间), 拿到 activation 后算 grad, 再 free 掉。

收益: activation 从 $17BSH$ 降到 BSH (只存 input)。80 层模型省 $16\times$ activation。

代价: forward 跑 2 次 (一次原本, 一次 backward 重算), 总 compute + 30-35% (backward 一般是 forward 的 $2\times$, 加一遍 forward = 33%)。

MFU 代价: 25-30%。

Selective Recompute

Megatron 2022 (Korthikanti et al.) 提出: 只对 **attention** 层做 **checkpoint**。

Attention 的 activation 里 BS^2A (未 FA 时) 是超大项 —— 选择性 checkpoint attention 能省最大头, 同时 FFN 部分 ($4BSH$, 也不小) 不 recompute 保持 forward 效率。

Megatron flag: `--recompute-granularity selective` (默认 recompute attention only) 或 `full` (整层)。

用 FlashAttention 之后 attention activation 已经很小 (BSH 而非 BS^2A), selective 收益变小。但仍有意义: Q/K/V matmul 的 activation、softmax 的 stats 还是几十 GB per layer。

Fine-grained Recompute (DeepSeek / Megatron-Core MoE)

进一步细化: 不 recompute 整层, 只 recompute 几个特定的 **cheap** 子模块, 避免 recompute 引发的重通信。

Megatron-Core flag:

```
--recompute-granularity selective
--recompute-modules mla_up_proj layernorm moe_act moe mlp core_attn
```

各模块含义:

- `mla_up_proj`: DeepSeek MLA (Multi-head Latent Attention) 的 up-projection, 特别耗 activation
- `layernorm`: LN 内部的 mean/var stat activation
- `moe_act`: MoE expert 里的 SwiGLU intermediate ($BS \frac{I}{EP}$ 大头)
- `core_attn`: FA 内部的内积
- `mlp`: dense MLP intermediate

为什么不整层 **recompute MoE**: MoE 层含 dispatch/combine all-to-all——整层 recompute 会重触发 **a2a**, 通信量翻倍。fine-grained 只 recompute 计算部分。

DeepSeek-V3 Table 3-4 报告: fine-grained recompute 省 activation 42 GB/GPU, compute overhead < 5%。相比整层 recompute (30% overhead) 是巨大进步。

Activation Offload

不 recompute, 而是把 activation 放到 **CPU DDR** 或 **NVMe**。fwd 时 D2H, bwd 时 H2D。

PCIe 带宽: Gen4 32 GB/s, Gen5 63 GB/s。CPU DDR5 90 GB/s per socket。所以 offload 的瓶颈是 PCIe。

何时值得:

1. activation 太大装不下, 且 recompute 也没帮到显存需求
2. 有闲置 PCIe 带宽 (TP/EP 通信主要用 NVLink)
3. 单 layer activation < CPU DDR (几百 GB, 通常满足)

何时不值得:

1. recompute 已经够
2. PCIe 已经被别的用 (peer memory copy, dataloader 拉数据)
3. compute 太快, D2H 来不及

Fine-grained Offload (Megatron-Core)

Megatron 2026 tech report 提出: 不整层 **offload**, 而是选特定模块 (`expert_fc1`, `moe_act`, ...) offload。用独立 CUDA stream 与 1F1B 的 a2a 兼容 overlap。

Flag:

```
--fine-grained-activation-offloading
--offload-modules expert_fcl moe_act
```

DS-V3 场景报告 mem -10.7%, throughput -1.6%——几乎白赚 10% 显存。Qwen3-235B 换 module mapping 后甚至 +15% 吞吐 (因为 offload 让 batch 更大, kernel 效率提升)。

PyTorch Native Offload (FSDP CPUOffload)

FSDP 支持:

```
FSDP(model, cpu_offload=CPUOffload(offload_params=True))
```

粗粒度: 整个 param shard 都 offload, 需要时拉回。适合单机 fine-tune 大模型 (4090 24GB 训 7B)。不适合多卡训练 (PCIe 竞争)。

Optimizer Offload

ZeRO-Offload (DeepSpeed) / **Optimizer CPU Offload** (Megatron
--optimizer-cpu-offload)。

Optimizer state 12P 是大头。放 CPU:

- Fwd/bwd: 不需要 optim state
- Step: 把 grad 传给 CPU (D2H), CPU 更新 (fused CPU-Adam), master weight 传回 (H2D)
- Latency: 一次 optim step 从 5 ms 变 100 ms

收益: 约 12P 显存。DS-V3 场景 15-20 GB。iter time +0.1-0.2 s (小 batch 时无所谓)。

生产建议: 先 FSDP + Precision-aware optim (optim state BF16 存 GPU), OOM 再上 CPU offload。

Precision-Aware Optimizer (Megatron-Core)

DeepSeek/Megatron 观察: Adam m/v 用 FP32 存的必要性不高——m/v 只用于 update 计算, 用 BF16 存 + step 时 upcast FP32 计算, 误差可忽略。

Flag:

```
--use-precision-aware-optimizer
--exp-avg-dtype bf16
--exp-avg-sq-dtype bf16
```

收益: optim state 从 12P/param 降到 6P/param, 显存省 **50%**。这是最“零成本”的 memory 优化。

代价: fused kernel 里多几次 dtype cast, 微小 latency。精度: Megatron 官方 benchmark 显示训练曲线基本一致。

组合策略

真实训练里几种都可以叠:

Precision-Aware Optim (m/v BF16)	→ optim state -50%
+	
Selective Recompute (attn only)	→ activation -30%
+	
Fine-grained Recompute (moe_act)	→ activation 再 -20%
+	
Fine-grained Offload	→ activation 再 -15%
+	
FP8 Activation	→ activation storage -50%

DeepSeek-V3 stack 起来把 199.5 GB/GPU 压到 H100 80GB 能训。

Recompute vs Offload：怎么选

方法	Compute overhead	Bandwidth overhead	适用
Full recompute	+30%	0	简单 baseline ,短 seq 用
Selective recompute	+5-15%	0	FA 之 后 dense 训练 标配
Fine-grained recompute	+3-5%	0	MoE 训练标 配（避免重触 发 a2a)
Full offload	0	PCIe bound	PCIe 有余量 时
Fine-grained offload	0	PCIe overlap	MoE + 1F1B stream 有余 量时
FP8 activation	+2-3%	0	Hopper+, DeepSeek recipe
Precision-aware optim	≈ 0	0	几乎白赚，先 开

表 29 各种 memory 优化对比。生产训练通常混用 :Precision-aware + Fine-grained recompute + FP8 activation 组合起来能省 60%+ 显存。

选择顺序：

1. 打开 Precision-aware optim (--use-precision-aware-optimizer)
2. 打开 FSDP / ZeRO-3 切 weight
3. 打开 activation recompute (先 selective , OOM 再 fine-grained)
4. 如果还 OOM , 开 fine-grained offload
5. 如果仍 OOM , 开 FP8 activation
6. 都开满还 OOM , 减 GBS 或加卡

面试考点

面试考点. **Q1**: Activation checkpoint 的 compute overhead 为什么是 33% ?

A: 一次 backward 计算量约 forward 的 $2\times$ (对 input 求导 + 对 weight 求导)。加上 checkpoint 需要重算一次 forward $\rightarrow \text{total} = 1F + 1F(\text{重算}) + 2B = 4F$ 。原本 $= 1F + 2B = 3F$ 。overhead $= 4/3 - 1 = 33\%$ 。selective recompute 只 checkpoint 部分层, overhead 小。

面试考点. **Q2**: FlashAttention 出来后 activation checkpoint 还有必要吗 ?

A: 有。FA 只把 attention matrix BS^2A 那一项省掉; activation 还有 BSH (LN input, residual)、 BSI (FFN intermediate)、 $Q/K/V$ 各 BSH 。长 seq (8K+) 时这些累积起来仍 GB 级。selective recompute 对 attention layer 内的非-FA 部分 ($Q/K/V$ matmul) checkpoint 也有用。

面试考点. **Q3**: 为什么 MoE 场景整层 recompute 不好 ?

A: MoE 一层含 dispatch + combine all-to-all。整层 recompute 意味着 backward 时 forward 全跑一遍——包括重触发 **a2a** 通信。a2a 是 MoE 的主要通信开销, 翻倍代价太大。fine-grained recompute 只重算 compute 部分 (moe_act, expert intermediate), 跳过 dispatch/combine 的 activation。

面试考点. **Q4**: `--use-precision-aware-optimizer` 有什么代价? 为什么不默认开 ?

A: 代价 : 几乎为 0。fused kernel 里多几次 $\text{BF16} \rightarrow \text{FP32}$ cast, 微小 latency。收益 : optim state -50% 。为什么不默认 : 这是 Megatron 2024 才加的 flag, 需要 fused optimizer kernel。DS-V3 用; Llama-3 405B 训练时还没这功能。现在的新项目默认开就对。

面试考点. **Q5**: Activation offload 到 CPU 什么时候会拖慢训练 ?

A: 三个场景 : (1) 每层 activation 太大, $\text{D2H} + \text{H2D}$ 时间超过 compute ; (2) PCIe 被别的用 (peer copy for TP, dataloader batch fetch) ; (3) CPU DDR 带宽不够 (多 GPU 共享一个 socket, 聚合 $\text{D2H} > 400 \text{ GB/s}$ 超 DDR 上限)。诊断 : nsys 看 GPU 是否等 memcpy_HtoD kernel。

面试考点. **Q6**: FP8 activation 与 recompute 冲突吗 ?

A: 不冲突, 正交。FP8 activation 只是“更小的存储”, recompute 是“不存重算”。两者可以叠 : 只存少量关键 activation 用 FP8, 其他 recompute。DeepSeek-V3 stack 里都有。

面试考点. **Q7**: 你怎么决定要不要开 activation checkpoint ?

A: 先 profile 看单卡 activation 显存。如果 activation $< 30\%$ 总显存, 不用 checkpoint (compute overhead 不值)。activation $> 50\%$, 先 selective recompute。 $> 70\%$ 且 OOM, fine-grained recompute + offload + FP8。目标 : 让 batch size 尽量大 ($\rightarrow$ MFU 高), activation 恰好卡在 80% 显存以内。

Overlap: 通信与计算的重叠

这一章把散落在前几章的 overlap 技术系统整理一遍，给出选择框架和 profile 诊断方法。分布式训练 30% 以上的性能优化最终都是“让通信和计算同时跑”。

compute + comm overlap 效果对比

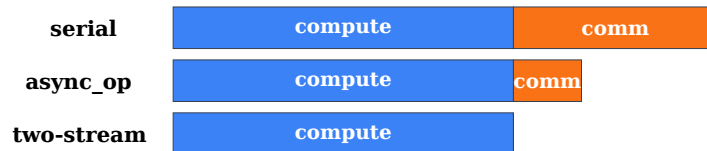


图 25 同一 workload 三种执行策略。serial 是 compute→comm；async_op 用 `async_op=True` 部分隐藏；two-stream 用独立 stream + CUDA event，通信完全隐藏在 compute 之下。见 `src/distributed_training/l1_overlap_2stream.py`。

Overlap 的物理基础

GPU 上通信 (NCCL) 与 compute (matmul) 是不同的 CUDA kernel。如果它们在不同 **CUDA stream** 上，硬件会尝试并行调度。

NVIDIA Hopper 硬件层面的限制：

1. NCCL kernel 占若干 SM (default 2-8 per channel)
2. Compute kernel 占其余 SM
3. 两者共享 HBM 带宽
4. 通信 kernel 用 PCIe / NVLink / IB，与 HBM 不冲突

所以理想 **overlap** 是 **100%** 隐藏：只要 SM 分得开、HBM 带宽够。

CUDA_DEVICE_MAX_CONNECTIONS 决定 host 到 device 的 hardware queue 数。默认 8，实际 overlap 需要 $\geq$ 通信 stream 数。生产建议 `export CUDA_DEVICE_MAX_CONNECTIONS=8`。

Overlap 的分层：从 op 到 pipeline

从最细到最粗：

1. **Level 0 — Op 内 overlap**：单 kernel 内部把 compute 和 async load overlap (Flash Attention 内部, CUTLASS pipeline)。用户不看，kernel 作者负责。
2. **Level 1 — Op 间 overlap**：AllReduce 与相邻的 GEMM overlap (Megatron async TP, DDP bucket AR)
3. **Level 2 — Layer 间 overlap**：本层 comm 与下层 compute overlap (FSDP forward prefetch, backward prefetch)
4. **Level 3 — Micro-batch 间 overlap**：本 micro-batch 的 backward 与下一 micro-batch 的 forward overlap (1F1B pipeline)
5. **Level 4 — Iteration 间 overlap**：optim step 与下 step forward overlap (fused optimizer with async grad reduce)

大多数框架自动处理 L0-L1；L2-L4 需要手动 tune 或选择合适 flag。

逐 level 详解

Level 1: DDP bucket AllReduce 与 backward compute

第 3 章讲过。核心：backward 是 output $\rightarrow$ input 反向走，DDP 把 param 按顺序 bucket，一个 bucket 所有 grad 就绪即起 async AR，与前面 layer 的 backward compute overlap。

代码要点：

```
# 检查是否 overlap:
for name, p in model.named_parameters():
    p.register_hook(lambda g, n=name: print(f"grad ready: {n} at step
{torch.cuda.Event().record()}"))
```

失败案例：find_unused_parameters=True 会破 overlap (DDP 要等所有 grad 判定)，gradient_as_bucket_view=False 有额外 copy。

诊断：nsys 里看 ncclAllReduce kernel 与前面 sgemm kernel 是否在不同 stream + 时间叠加。理想 overlap ratio > 90%。

Level 1: TP AllReduce 与 GEMM overlap (Async TP)

Megatron --tp-comm-overlap + TE LayerNormLinear fused。

思路：GEMM 拆 tile，per-tile 边算边发通信。

示意：



图 26 Async TP：GEMM 全长 15 单位；AR 从第 3 单位 (tile 0 算完) 开始，与后续 tile 计算并行，总时长 $\approx \max(15, 3+12) = 15$ 。串行方式则是 $15 + 12 = 27$ 。overlap 消掉 45% 时间。

收益：TP=8 的 AR 通信占比 15-20% $\rightarrow$ 5-8%。开销：kernel 复杂化，需要 TE 支持。

Level 2: FSDP forward/backward prefetch

FSDP BackwardPrefetch.BACKWARD_PRE。第 4 章讲过。

思路：backward 第 l 层开始前，就 issue 第 $l+1$ 层的 AllGather；第 l 层 backward compute 时 AG 已经在跑，第 $l+1$ 层 compute 起来时 param 已经就绪。

代价：显存峰值高 (同时持有 l 层和 $l+1$ 层的 param)。OOM 时降到 BACKWARD_POST。

前向类似：forward_prefetch=True 让 l 层 compute 时 issue $l+1$ 层 AG。

Level 2: MoE dispatch / combine 与 expert compute overlap

一层 MoE 内部三阶段串行时通信占 40%+：



图 27 MoE 串行：dispatch a2a $\rightarrow$ 本地 expert compute $\rightarrow$ combine a2a。通信占 $6/10 = 60\%$ 。

改成 2-stream overlap：



图 28 MoE 2-stream : dispatch 与前一 op 尾巴 overlap ; combine 与下一层 attention 起始 overlap。CUDA event 控制依赖。通信占比降到 20%。

Megatron `--overlap-moe-expert-parallel-comm`。第 8 章有代码示例。

Level 3: 1F1B micro-batch 交叉

Pipeline 里 stage s 交叉做 F_i 和 B_j 。stream 层面：



图 29 1F1B pipeline stream 视图 : compute stream 依次 $F_0/F_1/B_0/F_2/B_1/\dots$; P2P stream 与 compute 并行发送每个 F_i 的 activation 到 stage s_{i+1} 。P2P 与 compute 天然 overlap (不同 stream), 1F1B 已 built-in。

Level 3.5: Megatron FWD-BWD Merged

`--overlap-moe-expert-parallel-comm --delay-wgrad-compute`。第 6, 8 章讲过。

思路：同一 iteration 里，相邻 micro-batch 的 forward compute 与 backward+comm 排到两条 stream。B 与 W 分开让“data-grad B”可以先算继续 pipeline 推进，“weight-grad W”见缝插针。

对 MoE : EP a2a 占 30-40% $\rightarrow$ < 5%。当前 Megatron 默认推荐。

Level 4: DualPipe

第 6 章。bidirectional pipeline + 4-phase overlap , pipeline bubble 与 a2a 几乎完全隐藏。代价 $2\times$ 权重。DeepSeek-V3 用。

Level 4: Optimizer step 与下 step forward overlap

比较冷门。思路：optim step 完成 rank r 的 shard 时，rank r' 已经开始下 step forward。fused Adam 内部 async。

TE 的 `apply_optimizer_step_with_overlap` 提供，但生产用得不多——optim step 时间通常 < 5% 的 step time。

Overlap 值不值得做

量化的判断：

$$\text{gain}_{\text{overlap}} = \min(\text{comm_time}, \text{compute_time})$$

- 如果 $\text{comm_time} > \text{compute_time}$: overlap 后 $\text{step} = \text{comm_time}$ (compute 白算白 free)
- 如果 $\text{comm_time} < \text{compute_time}$: overlap 后 $\text{step} = \text{compute_time}$
- 如果两者相等 : overlap 后 $\text{step} \approx \max/2$

示例：一层 dense forward 100 ms compute + 20 ms comm , overlap 后 100 ms (省 20%)
 一层 MoE 100 ms compute + 80 ms comm (跨节点 a2a) , overlap 后 100 ms (省 44%)

Overlap 收益越大的场景：跨节点、MoE、大模型、TP > 4。在小模型 + 快网络场景，overlap 收益 < 5% 不值得代码复杂度。

Overlap 的常见 bug

1. **stream** 依赖漏了 : async op handle 忘了 wait , 导致 kernel 用还没到的数据。表现 : 偶发 nan 或 hang。修 : CUDA event 记录 + wait_event
2. **CUDA_DEVICE_MAX_CONNECTIONS=1** : 默认 1 时 stream 串行执行 , overlap 失效。检查 :
`echo $CUDA_DEVICE_MAX_CONNECTIONS`
3. **NCCL kernel** 抢 **SM** : DeepEP V1 吃 20 SM , 导致 GEMM 掉 20%。用 DeepEP V2 或减少 channel
4. **host-side** 同步 : `.item()` , `.tolist()` , `dist.barrier()` 都是 D2H sync , 破坏 CUDA graph 和 overlap
5. **Bucket** 太小/太大 : DDP 里太小则 AR 次数多延迟主导 ; 太大则 overlap 空间小
6. **AR** 的 **op** 顺序 : `dist.all_reduce(..., async_op=True)` 返回 handle 后必须在正确点 wait , 否则 grad 用未同步数据

用 nsys profile overlap

```
nsys profile -t cuda,cudnn,cublas,nvtx,osrt \  
  --stats=true \  
  -o my_run.nsys-rep \  
  torchrun --nproc-per-node=8 train.py
```

打开 my_run.nsys-rep , 看 :

1. **Timeline** : 多条 stream row , NCCL kernel 和 GEMM kernel 应该在同一时间列上 (overlap)
2. **NCCL kernel** : 搜 `nccl` 前缀。看 duration、颜色 (AR vs AG vs a2a)
3. **stream row** 内的 **gap** : 如果 stream 内有明显 gap 但另一 stream 有 kernel , 那是没 overlap 好——依赖关系错了

一个 healthy MoE profile : expert compute stream 和 a2a stream 大部分时间都有 kernel , 几乎没 gap。

一份 overlap checklist

在训练脚本里检查 :

```
# Environment  
export CUDA_DEVICE_MAX_CONNECTIONS=8  
export NCCL_DEBUG=WARN  
  
# DDP (if using)  
model = DDP(model, bucket_cap_mb=200, gradient_as_bucket_view=True)  
# 不开 find_unused_parameters  
  
# FSDP  
FSDP(model,  
      backward_prefetch=BackwardPrefetch.BACKWARD_PRE,  
      forward_prefetch=True,  
      limit_all_gathers=True) # 控制 prefetch 内存峰值  
  
# Megatron / MoE  
--tp-comm-overlap
```

```
--overlap-grad-reduce
--overlap-param-gather
--overlap-moe-expert-parallel-comm
--delay-wgrad-compute
```

跑 profile 确认：一个 step 里 NCCL kernel 时间 / step time < 15% (dense) 或 < 20% (MoE)。超过就有 overlap 空间。

面试考点

面试考点. Q1: DDP overlap 靠什么实现？

A: Bucket + backward hook + async AllReduce。DDP 在 init 时把 param 分 25 MB bucket；每 param 注册 backward hook；一个 bucket 内所有 grad ready 立刻 issue async AR (在专门 NCCL stream)；backward 继续往前算。AR 与前面层的 backward compute 时间重叠。要求 `find_unused_parameters=False` 才不会破 overlap。

面试考点. Q2: FSDP forward_prefetch 与 backward_prefetch 分别什么时机？

A: forward_prefetch: 第 l 层 forward 开始 → 立刻 issue 第 $l+1$ 层的 AllGather。backward_prefetch (BACKWARD_PRE): 第 l 层 backward 开始前 → issue 第 $l-1$ 层 AG (backward 反向走，“下一步”就是 $l-1$)。两个都是“往未来看”，代价是显存峰值高 (同时持两层 param)。

面试考点. Q3: 为什么 CUDA_DEVICE_MAX_CONNECTIONS=1 时 overlap 失效？

A: 这个 env var 控制 host 到 device 的 hardware queue 数。=1 意味着 host 只能一次下发一个 stream 的命令，多 stream 变成串行。生产要 $\geq \max(\text{concurrent stream count})$ ，Hopper+ 一般设 8。

面试考点. Q4: nsys 里怎么判断 overlap 好不好？

A: 看 timeline 的多 stream row。理想: compute stream 和 comm stream 时间列 overlap，两者都几乎无 gap。不好的信号：(a) comm stream 有 kernel 但 compute stream 有 gap → compute 在等 comm 结果；(b) 两 stream kernel 完全错开 (不同时间列) → 没 overlap；(c) NCCL kernel 内部有 gap → NCCL 自己在等 (可能是 rank 不齐, 慢卡)。

面试考点. Q5: Async TP 相比标准 TP 的收益怎么估？

A: 标准 TP 每层 2 AR，AR 时间约 GEMM 时间的 15-20% (H100 NVL8)。开 async 后大部分 AR 与 GEMM overlap，AR 剩 5-8%。假设 TP 通信总占 20%，overlap 到 6%，则 step time -14%。对训 Llama-70B 是 5-10 天时间的节约。

面试考点. Q6: MoE 训练 overlap 后为什么能大幅提速？

A: MoE 一层 4 次 a2a (fwd/bwd dispatch+combine)，占 forward 15-40% 时间。dense 只有 TP AR (10-20%)。overlap 到 <5% 后，MoE 省的绝对时间比 dense 多得多。DeepSeek DualPipe + DeepEP 是极端案例，让 a2a 从 30-40% 隐藏到 0%。

面试考点. **Q7**: 一个 step 里 host-side 出现 `.item()` 会有什么后果?

A: `.item()` 是 D2H sync ,host 阻塞等 device queue 排空。破坏 :(1) CUDA graph capture (graph 内不能有 sync) ;(2) stream overlap (sync 后 host 要重新 issue 命令 ,NCCL kernel 可能延迟启动) ;(3) profile 里看到很长的 idle time。生产训练循环里避免任何 `.item()` / `.tolist()` / `.cpu()` , log 用 `torch.tensor` 累积到 log step 才 D2H。

面试考点. **Q8**: DualPipe 与 Megatron FWD-BWD merged 都能 overlap 到 90%+ , 为什么大多数生产选 FWD-BWD merged?

A: DualPipe 需要 $2\times$ weight 副本 (bidirectional pipeline 每端存一份), memory wall 加剧。FWD-BWD merged 只需 `CUDA_DEVICE_MAX_CONNECTIONS>1` + `--delay-wgrad-compute` , $1\times$ 权重 , 配置改一个 flag 就开。overlap 收益都在 90%+ , 实测差异 < 3%。DualPipe 只在训 671B+ 且 IB comm 极度受限时值得。

Data Loader, Packing 与 Sample Balancing

Data loader 常常被面试忽略，但它决定了训练吞吐的上限——GPU 再快，data 供不上就白算。这一章讲 IterableDataset、streaming、packing 算法 (BFD/FFD)、多源混合、shard 均衡、以及多模态特有的问题。

一个隐藏的 bottleneck

大规模训练里最常见的“step time 突然变慢 10%”原因不是通信，而是 dataloader 供不上：

1. 磁盘 **IO** 慢：checkpoint / 数据在同一 NFS，同时读时被拖
2. **CPU** 预处理慢：tokenize + pack + augment 单进程算不过来
3. **DDP sampler** 不均：某几个 rank 拿到超大 batch (可变长 seq)
4. 网络 **shuffle** 抖动：streaming 从 S3 拉数据抖

诊断：nsys 看是否有 gap，或加时间戳 `time.time()` 在 `next(loader)` 前后测。生产训练常见 loader wait > 5% step time。目标应该 < 1%。

数据集抽象

Map-style Dataset：`__getitem__(idx)` 随机访问，`__len__` 已知。适合小数据集（几十 GB fit in memory or fast SSD）。

IterableDataset：`__iter__` 返回 iterator，无 `__len__`。适合流式数据（TB+ 无法 index），多源混合、动态生成。生产大模型训练都用 IterableDataset。

难点：DistributedSampler 只能用于 map-style。IterableDataset 需要自己实现 shard 划分 + rank-aware iteration：

```
class MyStreamingDataset(IterableDataset):
    def __init__(self, urls):
        self.urls = urls  # list of shard files

    def __iter__(self):
        rank = dist.get_rank(); world = dist.get_world_size()
        worker = torch.utils.data.get_worker_info()
        n_workers = worker.num_workers if worker else 1
        wid = worker.id if worker else 0

        my_urls = self.urls[rank::world]  # 按 rank 分 shard
        my_urls = my_urls[wid::n_workers]  # 再按 worker 分
        for url in my_urls:
            for sample in read_shard(url):
                yield sample
```

坑：如果 shard 数不能被 world × workers 整除，某些 rank 分到少 shard → epoch 提早结束 → 训练 hang（其他 rank 等）。修：设 `drop_last` 语义或补 pad。

Packing：变长序列的显存/算力救星

Transformer 训练里 seq 长度往往不等长(一句话 128 tokens ,一个 code file 8K tokens), naive batching :

```
batch = [seq1 (128 tok), seq2 (8000 tok), seq3 (400 tok)]  
padded to (3, 8000) - 大部分是 pad token, 浪费 90%+ 的 compute
```

Packing : 把多个短 seq 拼接成一个长 seq , attention mask 保证不跨 doc attention :

```
packed = [seq1, seq2, seq3, seq4, ...] concatenated to length 8192  
mask sequence: [1,1,...,1, 2,2,...,2, 3,3,...,3, ...] # doc id
```

Attention 里用 `attention_mask` 或 FlashAttention 的 `varlen` 接口只让同 doc 内的 tokens attend。

收益：训练时 padding ratio 从 40% 降到 < 5% , 吞吐 +50-100%。

Packing 算法

Best Fit Decreasing (BFD) :按 seq 长度降序 , 每 seq 找当前“最能装下且剩余空间最小”的 bin (packed sequence)。经典 bin-packing 算法。 $O(n \log n)$ 。

First Fit Decreasing (FFD) :按 seq 长度降序 , 放到第一个能装下的 bin。 $O(n)$ 。

Online / streaming packing :不知道全部 seq 长度分布 , 边读边 pack。做法 :维护一个 “open bins” 池 , 读到新 seq 时找最合适 bin ; bin 满就 flush。DeepSeek / Mixtral 训练用这个。

Multi-turn conversation packing :对话数据里保持 turn 顺序 , 但可以跨 conversation pack。要小心 loss mask (只对 assistant token 算 loss , user token mask 掉)。

Attention mask 的处理

Naive : $BS \times BS$ mask matrix , $S = 8192$ 时 128 MB per sample —— OOM。

FlashAttention varlen :

```
from flash_attn import flash_attn_varlen_func  
# packed: 1D concatenated tokens  
# cu_seqlens: cumulative seq lens, e.g. [0, 128, 8128, 8528, 8929, ...]  
out = flash_attn_varlen_func(  
    q=q_packed, k=k_packed, v=v_packed,  
    cu_seqlens_q=cu_seqlens, cu_seqlens_k=cu_seqlens,  
    max_seqlen_q=max_len, max_seqlen_k=max_len,  
    causal=True,  
)
```

无需 mask , kernel 内部按 `cu_seqlens` block 分别做 attention。 $O(BSH)$ memory instead of $O(BS^2)$ 。

Cross-doc contamination :如果 pack 错了让 doc 之间 attend , 模型会学到“下一个 doc 的信息 predict 当前 doc” → 训练 val loss 偏低但 test 崩。生产必须验证 packing 正确性。

Padding vs Packing vs No-pad (Bucket)

策略	方法	Padding 比	说明
Naive pad	batch 内 pad 到 max len	30-70%	最简单，浪费多
Bucketing	按长度分桶，桶内 pad	10-30%	每桶 pad 少，但 batch shape 不定
Packing (BFD)	拼接短 seq 到固定长	< 5%	生产标配，需 varlen attention
Packing (dynamic)	长度平衡的 pack	< 5%	进一步减少每 batch 长度方差

表 30 变长处理策略。生产 LLM 训练几乎全用 packing + FA varlen。

多源数据混合

预训练常混多个 corpus : Web (Common Crawl) + Code (GitHub) + Books + arXiv + Wiki + Math。每类比例 (domain weight) 影响下游性能。

采样策略：

1. **Uniform** : 每 source 相同概率
2. **Weighted** : 预设 weight (Llama-2: 67% Web, 15% code, 4.5% arXiv, ...)
3. **Temperature-based** : $p_i \propto n_i^\tau$, $\tau \in [0, 1]$; $\tau = 0$ 均匀, $\tau = 1$ 按 size 采样。多语言场景常用。
4. **DoReMi / Data Mixing Laws** : online 学习最优 weight

实现 : 多个 IterableDataset , 用 `torch.utils.data.WeightedRandomSampler` 或自己写 iterator :

```
def mixed_iterator(sources, weights):
    iters = [iter(s) for s in sources]
    while True:
        idx = np.random.choice(len(sources), p=weights)
        try:
            yield next(iters[idx])
        except StopIteration:
            iters[idx] = iter(sources[idx])    # infinite loop
            yield next(iters[idx])
```

Reproducibility : seed 每 rank 不同 (rank + epoch), 保证不同 rank 数据不重复但全局按 weight 分布。

Streaming: 从 S3 / HDFS 拉数据

10T tokens 数据集不可能全下载到本地。生产用 streaming :

1. **mosaicml Streaming** (MDS format) : sharded, seek-able, prefetch
2. **WebDataset** : tar-based, streaming friendly
3. **HuggingFace datasets** `streaming=True` : Arrow-based, 简单但性能弱

关键点：

1. **Prefetch** : `DataLoader(num_workers=N, prefetch_factor=M)` , $N=8-16$, $M=4$, 让 CPU 一直提前准备下 batch

2. **S3 并发**：多 worker 各自 open connection，注意 S3 rate limit
3. **断点续训**：streaming 要能 resume。MDS 支持 `.set_epoch(epoch) + .state_dict()` 保存 iterator 状态

Shuffle：全局 shuffle 不可能 (TB 数据)。做 shard-level shuffle + shard 内 buffer shuffle:

```
shuffle_buffer = 10000
buf = []
for sample in shard_iter:
    buf.append(sample)
    if len(buf) >= shuffle_buffer:
        idx = random.randint(0, len(buf)-1)
        yield buf.pop(idx)
```

Buffer 越大越接近真 shuffle 但 memory 越多。生产用 10K-100K。

长文档 vs 短文档：DDP 里的 sample-level imbalance

如果 batch 里长文档过多，某个 rank 拿到超长 seq → 该 rank compute 时间 $2\times$ ，DP AllReduce 里其他 rank 都等它。

Sequence-length balancing：给每 rank 分近似等 **length total** 的 samples。

方法：

1. **Fixed-length after packing**：packing 输出恒定 seq_len，天然均衡（首选）
2. **Length-based grouping**：把长度类似的 sample 分给同 rank
3. **Dynamic batching**：max_tokens_per_batch=8192 而非 batch_size=8（HF trainer 支持）

不做 sequence balancing 的后果：DP AR 里 straggler 主导 step time，MFU 掉 20-30%。

开了 SP/CP 之后的 DataLoader

一旦启用 sequence sharding (Megatron SP 或 CP)，DataLoader 的契约变了：一个 **CP** 组合起来才持有一条序列。三条硬性要求：

1. 组内同样本：sampler 索引和 shuffle 种子都必须用 `dp_rank`，不能用 `global_rank`。写错不报错——attention 会把两个半截文档当一条序列算，同时 global batch size 悄悄变成配置值的 CP 倍
2. 长度对齐：`seq_len` 要能被 CP 整除；zigzag（负载均衡切分）需要被 $2 \times \text{CP}$ 整除，padding 要按这个粒度补
3. 位置与文档信息随 **token** 走：`position_ids`、`doc_ids`（packing 用）以及 zigzag 的置换索引都必须作为 batch 的显式字段产出并按同样方式切分。模型里任何地方都不要用 `arange` 现场生成本地位置——varlen kernel 的 `cu_seqlens` 也要按本地 shard 重算

完整的规则、失败征状与逐层对拍方法见 §7，代码见 `20_cp_data_loader_halo.py`。

Multimodal Data Loader 的特殊难度

见下章多模态细节，这里只列关键差异：

1. 样本 **size** 方差大：一段 3B tokens 的 audio + 一张 32×32 图片，size 差 10^6
2. 异构类型：需要 tokenizer 之外的 preprocessing (image resize, video frame extract, audio spectrogram)
3. 缓存友好性：图像 decoded 是巨大 tensor，pre-cache 到 SSD

4. **Multi-worker safe** : 图像/视频解码库有些不 thread-safe , 多 worker 会 crash

一个可用的 **DataLoader** 骨架

```
import torch
from torch.utils.data import IterableDataset, DataLoader

class PackedTextDataset(IterableDataset):
    def __init__(self, tokenizer, urls, max_len=8192, mix_weights=None):
        self.tokenizer = tokenizer
        self.urls = urls
        self.max_len = max_len
        self.mix_weights = mix_weights

    def __iter__(self):
        rank = dist.get_rank(); world = dist.get_world_size()
        wi = torch.utils.data.get_worker_info()
        w_id = wi.id if wi else 0
        w_num = wi.num_workers if wi else 1

        my_urls = self.urls[rank:world][w_id:w_num]
        buf = []
        cur_len = 0
        for url in my_urls:
            for text in read_url(url):
                tokens = self.tokenizer.encode(text, add_special_tokens=False)
                if cur_len + len(tokens) + 1 > self.max_len:
                    # yield current packed batch
                    yield self._pack(buf)
                    buf = []; cur_len = 0
                buf.append(tokens)
                cur_len += len(tokens) + 1 # +1 for EOS

    def _pack(self, seqs):
        # concat with EOS, build cu_seqlens for FA varlen
        input_ids = []
        cu_seqlens = [0]
        for s in seqs:
            input_ids += s + [self.tokenizer.eos_token_id]
            cu_seqlens.append(len(input_ids))
        pad_len = self.max_len - len(input_ids)
        if pad_len > 0:
            input_ids += [0] * pad_len
        return {
            "input_ids": torch.tensor(input_ids, dtype=torch.long),
            "cu_seqlens": torch.tensor(cu_seqlens, dtype=torch.int32),
        }

    def collate(batch):
        # batch of dicts, stack input_ids, keep cu_seqlens per-sample
        return {
            "input_ids": torch.stack([b["input_ids"] for b in batch]),
```



```

        "cu_seqlens": [b["cu_seqlens"] for b in batch],
    }

loader = DataLoader(
    dataset,
    batch_size=8,
    num_workers=8,
    prefetch_factor=4,
    persistent_workers=True,
    pin_memory=True,
    collate_fn=collate,
)

```

要点：

- `persistent_workers=True` 避免每 epoch 重启 worker
- `pin_memory=True` D2H 快
- `prefetch_factor` 让 worker 提前准备

面试考点

面试考点. Q1: 什么是 sequence packing? 和 batch padding 差在哪?

A: Padding 是每 batch 内 pad 到 max seq length, 浪费很多 compute (padding tokens 都被算但 loss=0)。Packing 是把多个短 seq 拼接成固定长的 packed sequence, 用 attention mask (或 FA varlen 的 cu_seqlens) 保证 doc 间不 attend。padding ratio 从 30-70% 降到 < 5%, 训练吞吐 +50-100%。

面试考点. Q2: Packing 里怎么保证 doc 间不 cross-attend?

A: 两种:(a) 显式 attention_mask matrix — 简单但 $O(S^2)$ 显存;(b) FlashAttention varlen — 传 cu_seqlens, kernel 内部按 seq 边界 block-diagonal attention, $O(S)$ 显存。生产都用 (b)。RoPE 的 position id 也要按 doc 重置 (每 doc 从 0 开始), 不然 pos embedding 跨 doc 混。

面试考点. Q3: BFD 和 FFD 算法的差别?

A: 都是 bin-packing。BFD (Best Fit Decreasing): 按 seq 长度降序, 每 seq 放到“最能装下且剩余空间最小”的 bin——平均更 tight 但 $O(n \log n)$ 。FFD (First Fit Decreasing): 放到第一个能装下的—— $O(n)$, 实际差别 < 3% padding。生产多用 online / streaming 版: 不知全部长度, 边读边 pack。

面试考点. Q4: IterableDataset 里 rank 分 shard 常见的 bug?

A: (1) shard 数不能被 world × workers 整除, 某些 rank 少数据 → epoch 早停 → 训练 hang; (2) 忘了 `worker_init_fn`, 多 worker 拉同一 shard; (3) shard 内没 shuffle → 样本顺序固定; (4) resume 时 iterator 状态未保存 → 重新训相同 sample; (5) 长 seq 分给某 rank 造成负载不均。

面试考点. **Q5**: DDP 里如果每 rank 的 batch 长度差异大会怎样？

A: DP AllReduce 是同步的,慢的 rank 拖慢整个 step。长 seq rank compute $2\times$,其他 rank 等 $\rightarrow$ MFU 掉 20-30%。解决: packing 让每 batch 长度恒定;或 length-based grouping 让相似长度 sample 分给同 rank;或 dynamic batching 按 `max_tokens_per_batch` 决定 batch size。

面试考点. **Q6**: 一个 dataloader 供不上 GPU 怎么排查？

A: 加时间戳测 `time.time()` 在 `next(loader)` 前后,判断 loader wait 占 step 比例。 $> 5\%$ 就要优化。检查:(a) `num_workers` 太少 (< 8);(b) `prefetch_factor=2` 默认不够,调 4-8;(c) tokenizer 单进程慢——用 fast tokenizer;(d) 数据在慢盘(NFS/S3 抖);(e) preprocess 太重(大图 decode),预先缓存。

面试考点. **Q7**: 数据混合的 weight 怎么设？

A: 生产做法:(a) 按 domain 大小 + 质量启发式(Llama-2 report 里有);(b) DoReMi (Xie 2023) — 用小 proxy 模型学 optimal weight;(c) Data Mixing Laws (Ye 2024) — 拟合 weight 与 loss 的函数关系。学术复现建议直接抄 Llama-3 或 Nemotron 的 mix。domain 数少 (< 10) 时直接 grid search 也行。

面试考点. **Q8**: streaming dataset 断点续训怎么做？

A: 保存 iterator 状态:当前处理到哪个 shard 的哪个位置,per-rank 保存。mosaicml Streaming 有 `state_dict()` / `load_state_dict()` API。自己实现 IterableDataset 时要暴露:`shard_idx`, `sample_offset_in_shard`, `epoch`, `rng_state`。resume 时跳到那个位置继续。断点保存频率与 checkpoint 一致。

多模态训练的特殊优化

多模态大模型(VLM: Vision-Language ;VLA: Vision-Language-Action ;Audio-LM 等)在分布式训练上遇到与纯文本 LLM 不同的挑战。这一章讲清楚 :变长模态、跨模态负载不均、encoder/decoder 隔离、packing 的特殊难度、以及公开的 LLaVA/Qwen-VL/InternVL/Gemini/Sora 类做法。

多模态的结构

典型 VLM 架构 (LLaVA-style):



多模态训练里“模态”变成了 sample-level 属性 :

- Text-only sample: image tokens 数 = 0
- Image caption: image tokens 576 (ViT-L/14 @ 336px) , text tokens 50
- Long-form OCR: text tokens 2000+
- Video: image tokens $\times$ frame 数 , 可能 $10^4 +$

两个新问题 :

1. **Modality-heterogeneous cost** : 不同 sample 的 encoder / decoder 计算完全不同——一张图 encoder 5 ms , 1000 frame video encoder 5 s
2. **Modality-imbalanced batch** : 如果 batch 里全是 text-only, vision encoder 空转 ; 反之 LLM 空转

变长图像/视频的处理

Native resolution 训练

传统 CLIP/ViT 训练用固定 224×224 or 336×336 , 简单但丢信息 (OCR 需要高分辨率)

Native resolution (Qwen-VL, InternVL, Fuyu, Molmo) : 保留原始 aspect ratio + resolution。做法 :

1. **NaViT (Dehghani 2023)** :把不同尺寸图像 patch 化后 pack 到一个 sequence 里 ,attention 里 mask 保证图内 attend。这就是“图像版 packing”。
2. **Dynamic patch grid** : 图像 resize 到最近的 patch grid (e.g., 14×14 tokens) , 再 pack。Qwen-VL v2/v3 用这个。
3. **Any-resolution** (LLaVA-Next) :把大图切成多个 sub-image , 分别编码 , concat 为 tokens。 $4 \times 336 \rightarrow 4 \times 576 = 2304$ tokens 就正常了。

后果 : 一个 batch 里图像 tokens 数从 100 到 10000+ 波动 $\rightarrow$ LLM decoder 输入长度剧烈变化。necessity of **padding-free packing**。

视频的时空处理

视频 = frame 序列 + audio。frame 数从 8 到几千。做法 :

1. **Sparse frame sampling** : 均匀采 8-32 frames (VideoLLaMA, Video-LLaVA)

2. **Dense sampling + token compression** : 256+ frames , 通过 temporal pooling / Q-Former / Perceiver 压缩到固定 tokens (100-200)
 3. **3D VAE (Sora / CogVideoX)** : 视频编码到 4D latent , 再展平
- 计算成本从几十 ms 到几秒/sample , 比图像高 10-100×。

Vision Encoder 与 LLM Decoder 的分工

冻结 encoder vs full training

Stage-1 pretrain (LLaVA) : 冻结 ViT , 只训 projection + LLM。ViT forward 一次即可 , 无需 grad → 显存小 **Stage-2 fine-tune** : full training , vision 也训。需要 vision encoder grad → 大 activation

冻结 ViT 时 , 可以预计算 **vision features** 缓存 , 训练时直接读——大幅省 compute。适合数据 stable 的场景。

Encoder / Decoder 的并行策略隔离

Vision encoder (0.3-1B params) 和 LLM decoder (7-70B params) 大小差 10-100×。用同一 TP/PP setup 不合适 :

1. **ViT 小** , **TP=1** 就够 , 用 DDP/FSDP 复制多份
2. **LLM 大** , 需要 **TP+PP+FSDP** 组合

方案 : ViT 与 LLM 用不同 **process group**。

```
# Vision encoder: DP only, no model parallel
vision_encoder = FSDP(vit, sharding_strategy=SHARD_GRAD_OP,
                      device_mesh=dp_mesh)

# LLM decoder: TP+PP+FSDP
llm = FSDP(llm_with_tp_pp, sharding_strategy=FULL_SHARD,
           device_mesh=llm_dp_mesh)
```

Data flow : ViT forward on all ranks → image features → send to LLM's first stage (via P2P if PP) → LLM forward+backward → grad back → ViT backward。

Kimi VLM / Qwen-VL 类系统这么做。工程复杂度显著上升 , 但避免了“整个模型都 TP=8 但 ViT 用不上 TP”的浪费。

分离式 Vision Encoding

更激进 : 把 ViT 放独立的 GPU 组 (“encoder ranks”) , LLM 放另一组 (decoder ranks) , 之间只传 image features (低带宽)

```
encoder ranks: [ ViT (0.3B) ] × 32 GPU    (纯 DP)
decoder ranks: [ LLM (70B) ] × 32 GPU    (TP+PP+FSDP)
Encoder → Decoder: send image_features (few MB) per sample
```

好处 : encoder 与 decoder 可以异步、独立 batch size、独立并行。坏处 : backward 从 decoder 传 grad 回 encoder 需要 P2P + 同步。工业界的 Sora / Veo 类大视频 model 用这种。

Packing 在多模态里的困难

文本 packing : 拼接 tokens , 加 cu_seqlens , varlen FA.

多模态 packing 挑战 :

1. **Image tokens** 是连续 **block** (一张图 = 576 tokens , 不能被 pack 拆开)
2. **Text-image interleaved** (LLaVA-Interleaved 类): `<img1><text1><img2><text2>...` 顺序要保
3. 不同 **sample** 的 **image** 数不同 (有的 0 , 有的 5)
4. **Modality mask** : loss 只在 text tokens 上算 , image tokens loss=0 (LLaVA 做法)

Packing 策略 :

1. 固定 **seq_len packing** : 与文本一样 , pack 到 8K/16K. image tokens 视为“不可分块”
2. **Modality-aware batching** : 先按图像数分桶 , 每桶内 pack. 避免“一个 batch 全是 image-heavy sample”
3. **Multi-image packing** : 一个 packed seq 里可以有 3-5 images + 相关 text , 用 special token 分隔

loss mask 实现 :

```
input_ids      # (B, S)
labels         # (B, S), image position 设 -100 (ignore)
attention_mask # varlen cu_seqlens
loss = F.cross_entropy(logits.view(-1, V), labels.view(-1), ignore_index=-100)
```

图像预处理的 **dataloader** 特殊坑

1. **Decode** 慢 : JPEG/PNG decode CPU 单核 100 ms/image ; 一个 worker 拉 batch 时会成为 bottleneck
 - 用 nvJPEG (GPU decode) 或 turbo-jpeg
 - 或预 tokenize + 缓存 to WebDataset shard
2. **Resize / augment** : torchvision 慢。用 kornia (GPU-side) 或 pillow-simd
3. **Multi-worker** 与 **GPU** 内存 : num_workers=32 × prefetch=8 × per-sample-4MB image = 4 GB CPU memory 常见 , 注意 OOM
4. **Persistent workers** 与 **forking** : CUDA-aware library 里 fork 会 crash。用 `spawn` context 或不 pin CUDA in worker

一个 **vision dataloader** 骨架 :

```
class VLMDataset(IterableDataset):
    def __iter__(self):
        for sample in stream_samples():
            # sample: {"image_paths": [...], "text": "..."}
            # 1. decode images (可放 num_workers)
            imgs = [decode_and_resize(p) for p in sample["image_paths"]]
            # 2. tokenize text
            text_tokens = tokenizer.encode(sample["text"])
            # 3. build interleaved sequence: <img1_placeholder> ... text ...
            input_ids, labels, modality_ids = interleave(imgs, text_tokens)
            yield {
```

```

        "input_ids": input_ids,
        "labels": labels,
        "pixel_values": torch.stack(imgs),          # (n_imgs, C, H, W)
        "modality_ids": modality_ids,              # 0=text, 1=image
    }

```

collate 时按最大 image 数 pad 或用 nested tensor。

跨模态 **sample balancing**

DDP 里如果不做 balancing :

- rank 0 拿到 5 张图 + 短文本 → ViT 5 forward + short LLM
- rank 1 拿到 0 张图 + 长文本 → 无 ViT, long LLM
- rank 2 拿到 1 张图 + 中长文本 → normal

同 step 内三 rank compute time 差 3-5×, DP AR sync 时慢 rank 拖整体。

解决 :

1. **“Modality-uniform” batch** : 每 batch 每 rank 有相同图像数 (预先 sort by n_images)
2. **“Total-cost balancing”** : 估算每 sample 的 ViT + LLM cost, 把总 cost 均分到 rank
3. **Bucketing by cost** : 按 (n_images, text_len) 二维分桶, 同桶 sample 分给同 rank

Qwen-VL v3 tech report 提到用 cost-based balancing 让 MFU +12%。

Sequence length variance 在 attention 里

多模态一个 batch 里 seq 长度从 500 到 16K 变化 → FlashAttention varlen 里 tail sample 主导 kernel time。

方法 : **Dynamic batching + max_tokens** :

- `max_tokens_per_batch = 32768`
- `batch size = max_tokens / max(seq_len)`
- 长 seq 时 batch=2, 短 seq 时 batch=32

HF Trainer + `group_by_length=True` 能做。或自己写 dataset iterator。

一个 **VLM** 分布式训练配置样例

Qwen2.5-VL-72B on 256 H100 :

```

torchrun --nnodes=32 --nproc-per-node=8 ... \
# LLM part
--tensor-model-parallel-size 8 \
--pipeline-model-parallel-size 4 \
--data-parallel-size 8 \                # = 256 / (8×4)
--sequence-parallel \
\
# Vision encoder part (独立 group)
--vision-tp 1 \
--vision-dp 256 \                      # ViT 每卡 replica
\
# Multimodal loader

```

```
--dataset-type interleaved-vlm \
--max-seq-length 16384 \
--max-image-tokens 2048 \
--pack-multimodal \
--group-by-cost \
\
# Standard
--bf16 --use-flash-attn --swiglu ...
```

多模态特有的稳定性问题

1. **Image token vs text token loss scale** : 如果 image tokens 参与 loss , image loss 数值范围可能与 text 差 10×。用 loss weighting
2. **Long-video training** 会 **spike** : 某 batch 突然 5000 image tokens 让显存暴涨。做 hard cap 或 outlier filter
3. **Vision encoder** 与 **LLM** 学习率不同 : ViT 已 pretrained , LR 小 ($1e-5$) ; LLM projection 层新 , LR 大 ($1e-4$)。分组 LR
4. **Modal collapse** : 训练早期 LLM 忽略 image (直接从 text 预测)。做 vision-conditioned loss up-weight

面试考点

面试考点. **Q1**: 多模态训练里 vision encoder 与 LLM 的并行策略为什么要分开 ?

A: ViT 通常 300M-1B 参数 , 一张卡装得下 → 用 DDP/FSDP 复制。LLM 70B+ 需要 TP+PP+FSDP。用同一 TP=8 setup 会让 ViT 上 TP AllReduce 白开销 (本来不需要)。分离让 ViT DP=world, LLM TP+PP+FSDP , 各自最优。工程复杂但 MFU +10-20%。

面试考点. **Q2**: NaViT 的 packing 与文本 packing 有什么不同 ?

A: 文本 packing 沿 seq 拼接 ; NaViT 把不同尺寸图像 patch 化后 pack 到一个 sequence (每图 = block of patches), attention 内部 mask 图间不 attend。核心相同 (varlen packing), 差别在“unit”是 image patch block 而非 sub-sequence。让 batch 里可以混合不同分辨率图像。

面试考点. **Q3**: LLaVA-Next 的“any-resolution”是怎么实现的 ?

A: 高分辨率图切成多个 sub-image (e.g., 4 patches of 336×336), 每个独立编码得 576 tokens , concat 成 $4 \times 576 = 2304$ tokens 拼到 LLM 输入。加上一张 thumbnail (整图 resize 到 336×336) 作 global context。让 LLM 看到 high-res detail 又不训练新架构。代价 : 一个高分辨率图变 2000+ tokens。

面试考点. **Q4**: 视频训练里最大的显存瓶颈 ?

A: 视频 tokens 数量爆炸—— $8 \text{ frames} \times 576 \text{ tokens/frame} = 4608 \text{ tokens/video}$, dense sampling 更多。attention 里 seq_len 从 1K 涨到 16K+。做法 : (a) Q-Former / Perceiver 压缩 (每 frame 8 tokens) ; (b) temporal pooling ; (c) 3D VAE encoder 直接编码 spatio-temporal。CogVideoX 用 3D VAE 把 ($T=49, H=480, W=720$) 压缩到 latent (13, 60, 90)。

面试考点. Q5: DDP 里跨 rank 图像数不均衡会怎样？

A: rank 0 十张图, rank 1 零张图 → ViT forward 时间差 10×。DP AR 时同步, 快 rank 空等。MFU -20-30%。解决: cost-based batch balancing (按 total ViT+LLM cost 均分); 或 group_by_length + group_by_n_images 二维桶。Qwen-VL v3 报告 balancing 后 +12% throughput。

面试考点. Q6: 多模态 packing 里 loss 怎么 mask？

A: LLaVA 传统做法: 只在 text tokens 上算 loss。image tokens 位置 label 设 -100 (ignore_index)。cross_entropy 自动跳过。有些多模态模型 (如 Chameleon) 也让 image tokens 参与 loss (image tokens 是可预测的 discrete codes) ——这时 loss 里所有位置都算, 但 image / text loss 分开监控。

面试考点. Q7: 训练视频模型时 sequence length 从 1K 到 16K+ 变化, attention kernel 怎么处理？

A: FA varlen (cu_seqlens) 自然支持。但 kernel launch overhead 随 seq_len 分布方差增大。做法: (a) dynamic batching by max_tokens_per_batch 而非固定 batch size; (b) sort by length, 减少每 batch 内方差; (c) torch.cuda.CUDAGraph capture 每个 seq_len bucket 一个 graph, 避免重复 launch。

面试考点. Q8: 一个 32B VLM 训练 MFU 只有 25%, 你会先查什么？

A: 按概率: (1) dataloader 供不上 (vision decode 慢) —— 加 time.time 测; (2) modality-imbalance —— nsys 看 ViT stream 是否有大 gap; (3) sequence length variance —— log 每 batch seq_len 分布; (4) TP=8 但 ViT 用不上 —— 分离 encoder/decoder group; (5) FA varlen 内部小 seq 效率低 —— 用 CUDA graph 或调 batch 结构。这几个查完通常能从 25% 提到 40%+。

RL 训练的分布式：从 PPO 到 GRPO 到 rollout/train 分离

RLHF 训练看起来只是 fine-tune 一个 policy model，实际上是分布式系统里最复杂的场景之一——同一 iteration 里要跑 4-6 个模型 (policy, ref, reward, value, critic, tokenizer)，且 policy 要 **generate** 出长序列 (inference workload)，然后再做 gradient update (training workload)，这一章讲清楚 PPO/GRPO/DPO 的分布式架构、rollout/train 分离、weight sync、vLLM/SGLang 集成、以及 verl/OpenRLHF 等主流框架的关键取舍。

Note. 这一章讲的是经典 **RLHF**：reward 来自一个学出来的 reward model，rollout 是一次性的单轮生成。2024 年底之后工业界主流已经迁移到 RLVR (reward 换成确定性 verifier) 和 Agentic RL (rollout 变成多轮的 LLM $\leftrightarrow$ 环境循环)，它们的系统形态差异很大——多出一个 CPU verifier 集群、rollout 期 GPU 会闲置、轨迹长度方差上百倍、训练侧多出 token masking 这个大坑。这些内容在第 15 章。建议按顺序读：本章建立 rollout/train 分离的基本框架，第 15 章讲它在新范式下怎么变。

从 PPO 说起

Standard RLHF PPO 循环 (InstructGPT / Anthropic 风格)：

```
for iter in range(N):
    # 1. Rollout: policy generates responses
    prompts = sample_batch()
    responses = policy.generate(prompts, max_new_tokens=1024)

    # 2. Reward
    rewards = reward_model(prompts, responses)
    values = value_model(prompts, responses)

    # 3. Compute advantages (GAE)
    old_logprobs = policy_logprobs_of(responses)
    ref_logprobs = ref_model_logprobs_of(responses)
    advantages = gae(rewards, values)

    # 4. PPO update (k epochs on same rollout data)
    for epoch in range(K):
        for mini_batch in split(rollout_data):
            new_logprobs = policy(mini_batch)
            ratio = exp(new_logprobs - old_logprobs)
            surr1 = ratio * advantages
            surr2 = clip(ratio, 1-eps, 1+eps) * advantages
            kl = old_logprobs - new_logprobs
            loss = -min(surr1, surr2) + beta * kl
            loss.backward()
            optim.step()
```

涉及模型：

- Policy (training + generation)
- Reference (frozen, for KL)
- Reward (frozen, scoring)
- Value / Critic (training, GAE)

涉及 **workload** :

- Rollout: autoregressive generation (inference-style, memory-bound)
- Reward inference (forward-only)
- Ref inference (forward-only)
- Policy training (fwd + bwd + optim)
- Value training (fwd + bwd + optim)

GRPO (DeepSeek): 干掉 value model

DeepSeek-Math / DeepSeek-R1 提出 GRPO (Group Relative Policy Optimization) :

1. 不训 value model —— 直接用 group baseline: rollout N 个 response per prompt ,
advantage = reward - group_mean_reward
2. 无 critic → 少一个 model to train
3. 更省显存, 训练更快, 效果不差 (DeepSeek-R1 用 GRPO 训 reasoning)

简化的 **loss** :

$$A_i = \frac{r_i - \text{mean}(r_1..r_N)}{\text{std}(r_1..r_N)}$$

$$L = -\mathbb{E}_i[\min(\text{ratio}_i A_i, \text{clip}(\text{ratio}_i) A_i)] + \beta \cdot \text{KL}(\text{policy} \mid \text{ref})$$

分布式意义: 一个 model 少训一个 GPU 组, 节省 20-30% 资源。GRPO 变体 (DAPO, DrGRPO, ...) 都保持这个结构。

Rollout 是 inference workload : 与 training 完全不同

Training 一个 step 处理 $\text{GBS} \times \text{seq_len}$ tokens ; rollout 是 autoregressive , 每 sample 逐 token generate:

- BS=1024, seq_len=1024, autoregressive → 需要 1024 forward passes (per token)
- 每 forward 只处理 BS tokens (decode phase, M=BS)
- Memory-bound (small-M GEMM)
- KV cache 大 : $2LBSH$ bs , 占大量显存

所以 **rollout** $\neq$ **training** 的 **forward**。用 training framework 直接跑 rollout 会 :

1. 效率极低 (training 的 forward 是 dense compute-bound ; decode 是 memory-bound)
2. 无 KV cache 优化
3. 无 continuous batching
4. 无 PagedAttention

生产做法 : 用 dedicated inference engine (vLLM, SGLang, TensorRT-LLM) 跑 rollout , 训练用 Megatron/FSDP。

Rollout / Train 分离架构

架构 :

```

GPU pool A (Rollout):  vLLM / SGLang instances
- Serve policy generate()
- Serve reward inference
- Serve ref inference
↓ (sync generated data)
GPU pool B (Training): Megatron / FSDP
- Policy training (PPO update)
- Value training (if PPO)

```

两 pool 可以是：

1. **Colocated**：同一批 GPU，rollout 完停 vLLM 起 training，交替进行。OpenRLHF v1 类
2. **Separate**：不同 GPU 组，rollout pool 和 training pool 各自独立。verl / OpenRLHF v2 / NeMo-Aligner 用

Colocated 的优劣：

- 优：GPU 利用率高 (100% 都在用)
- 劣：切换需要卸载 vLLM state / 重启 —— 秒级 overhead，且 rollout 与 training 时长必须匹配

Separate 的优劣：

- 优：两阶段并行，rollout 与 training overlap
- 劣：GPU 分配比例难调 (rollout 慢 → training 等；反之亦然)
- 复杂：需要 weight sync 与数据传输机制

现代 RL 框架 (verl, OpenRLHF, HybridFlow) 都支持两种模式，实测哪个快取决于 model size 和 rollout length。

Weight Sync: 从 training 到 rollout

每次 PPO update 之后，rollout engine 用旧 weight——不对。要把新 weight sync 到 rollout engine。

三种方案：

方案 1: NCCL broadcast (Colocated)

Rollout 与 training 在同一批 GPU (colocated)。Training 更新后：

```

# Training rank 直接把 policy weight overwrite 到 vLLM weight
for name, p in policy.named_parameters():
    vllm_engine.get_model().state_dict()[name].copy_(p)

```

优：instant，无跨 GPU 传输。劣：需要 model 在 CPU 里 (rollout 和 training 交替占 GPU)，或 vLLM 支持 hot-update。

方案 2: NCCL group broadcast (Separate)

Rollout 与 training 在不同 GPU 组。定义 cross-group NCCL group，broadcast weight:

```

# Training rank 0 → rollout rank 0..N
cross_group = dist.new_group(ranks=[train_rank_0, *rollout_ranks])
for name, p in policy.named_parameters():
    dist.broadcast(p, src=train_rank_0, group=cross_group)
    vllm.load_weight(name, p)

```

优：直接 GPU-to-GPU。劣：broadcast 慢 (整模型 130 GB for 70B，即使 NVLink 也几百 ms)。
生产：只 broadcast changed weight (LoRA scenario) 或用 pipelined broadcast overlap 与 training。

方案 3: Shared memory / NVLink DMA (Colocated + shared)

Nvidia FlexPPO / ByteDance verl 用：把 policy weight 存在共享 GPU buffer，training 更新后 vLLM 直接读同一 buffer。

优：零 copy。劣：需要 hack vLLM 内部 weight loader，framework-specific。

主流 RL 框架架构

OpenRLHF (from OpenLLMAI)

- 早期 colocated (v1)，v2 支持 separate + Ray-based orchestration
- Ray Actor 抽象每个 role: PolicyActor, RolloutActor, RewardActor, RefActor
- 用 vLLM 做 rollout，DeepSpeed / FSDP 做 training
- 支持 PPO/DPO/GRPO/KTO/REINFORCE++

verl (ByteDance, HybridFlow)

- 论文 “HybridFlow: A Flexible and Efficient RLHF Framework”
- 核心：Hybrid mode - rollout 和 training 都可以配置为 colocated 或 separated
- SPMD + Ray 混合调度
- Weight sync 用 NCCL broadcast 优化
- 生产用 (kimi K1.5 训练)

NeMo-Aligner (NVIDIA)

- Megatron-based
- Colocated + weight offload
- 支持 DPO / SPIN / SteerLM

TRL (HuggingFace)

- Single-node / small-scale friendly
- 学术友好，生产大规模不行
- 集成 PEFT (LoRA)

DeepSpeed-Chat

- 早期 (2023) 的三阶段 pipeline (SFT + RM + PPO)
- 老，很少用

Long-context / Reasoning RL 的挑战

DeepSeek-R1 类 reasoning 训练：response 长度 8K-32K+，rollout 一个 sample 要几分钟。

问题：

1. Rollout time 主导 (占 90%+)
2. Rollout 期间 training GPU 空闲
3. KV cache 大 ($32K \times 70B = 200+$ GB per instance)

优化：

1. **Speculative decoding** : 小 model draft + 大 model verify , rollout 加速 2-3×
2. **PagedAttention** : 动态 KV cache 管理 , 多 sample 共享 memory
3. **Continuous batching** : sample 完成时不等 batch , 立刻起新 sample
4. **Rollout** 与 **training** 并行 : async pipeline , training rank 一直在做 update on old rollout
5. **“On-policy” vs “off-policy”** : 允许 rollout 用略旧的 weight , 避免每 step sync

Kimi K1.5 tech report 里描述用 vLLM + custom scheduler 做 rollout , rollout throughput > 1000 samples/min。

Reward Model 的分布式

Reward model 是 forward-only , 需要 serve inference :

- Same size as policy (7B-70B typical) , 一张卡装不下
- 用 TP inference (vLLM TP=8)
- 每 rollout iteration 打分 batch × N candidates → 有的框架把 reward 和 rollout colocate 到同 pool

进阶 : reward model ensemble (多个 reward + averaging) , 或 process reward (每 step 打分)。分布式复杂度直接乘 N。

KL divergence 的分布式计算

PPO 里 KL 是 policy 与 ref 的对数比。ref model forward = full model forward → 需要一份 GPU。

常见做法 : ref model 与 policy 同 pool (放在 rollout pool 里 forward) , 或独立 pool (简单但耗 GPU)。

Kimi K1.5 优化 : ref 与 policy 共享 base , 用 LoRA delta , ref forward = policy forward - LoRA output → 只需一个 forward。

显存分析 : 一次 PPO iteration 的 GPU 占用

以 70B policy + 70B ref + 70B reward + 70B value on 128 H100:

Training pool (64 GPU):

- Policy: FSDP HYBRID_SHARD, TP=8, DP=8 → $70B / 8 \text{ shard} \times 16 \text{ bytes} = 17.5 \text{ GB/GPU}$ (fully sharded)
- Value: 同上, 17.5 GB/GPU
- Activation for training: 20 GB/GPU
- Total: 50 GB/GPU, well within 80GB

Rollout pool (64 GPU):

- Policy (vLLM TP=8): $70B \times 2 \text{ (BF16)} / 8 = 17.5 \text{ GB}$
- Ref (vLLM TP=8): 17.5 GB
- Reward (vLLM TP=8): 17.5 GB
- KV cache pool: 30 GB (大 batch)
- Total: 80 GB, tight

即 **128 GPU 训 70B RL setup** 显存刚好卡满。放宽 : quantize ref/reward 到 FP8 (省一半)。

一个 GRPO 训练配置样例 (verl-like)

```
# 128 H100 训 Qwen2-72B with GRPO
policy_model:
  size: 72B
  training:
    dp_size: 8
    tp_size: 8
    pp_size: 1
    fsdp: HYBRID_SHARD
    optimizer: precision-aware AdamW
    activation_checkpoint: selective
    precision: bfloat16

rollout:
  engine: vllm
  tp_size: 8
  n_instances: 8          # 8 vllm engines
  max_new_tokens: 4096
  temperature: 1.0
  n_rollouts_per_prompt: 16  # GRPO group size

reward:
  type: rule_based_math      # 或 reward model
  # 或
  # engine: vllm
  # model: reward-72b
  # tp_size: 8

ref:
  colocate_with_rollout: true
  precision: fp8             # 省显存

training_loop:
  batch_size: 1024          # prompts per iter
  ppo_epochs: 1             # GRPO 通常 1 epoch
  minibatch_size: 32
  lr: 1e-6
  kl_coef: 0.001
  clip_range: 0.2

weight_sync:
  frequency: every_iter
  method: nccl_broadcast
  overlap_with_training: true  # 下 iter forward 时 broadcast
```

行业参考：DeepSeek-R1 用 GRPO 训 671B MoE，成本估算 rollout 占总 RL 训练时间 70%+。
Kimi K1.5 类似。

常见 RL 训练坑

1. **Reward hacking** : 模型学到“骗 reward model”而非真解决问题。检测 : val 集 reward vs 人工评估的 gap。
2. **KL blow up** : 某个 mini-batch KL 突然大, 梯度爆。做 KL clip 或 adaptive KL coefficient。
3. **Rollout diversity** 掉 : 温度 0 让 policy 越训越 deterministic。GRPO 里 rollout 用 $\text{temp}=1.0$, evaluation 用 $\text{temp}=0$ 。
4. **Ref-policy drift** : 训久了 policy 与 ref 差异过大, KL 项 blow up。定期 update ref = policy (like 每 1000 step)。
5. **vLLM & training weight desync** : weight broadcast 忘了 sync, rollout 用旧 weight, advantage 有偏。
6. 长 rollout 的 **length exploit** : 模型学到“输出长 = reward 高”, 输出全是废话。用 length penalty。
7. **“Format collapse”** : GRPO 里 group std=0 时 advantage=inf。加 small epsilon 到 std。

面试考点

面试考点. Q1: RLHF 训练里为什么要 rollout / train 分离 ?

A: Rollout 是 autoregressive generation (memory-bound decode, small-M GEMM), training 是 dense fwd+bwd (compute-bound, large-M)。用 training framework 直接跑 rollout 效率极低 (无 KV cache, 无 continuous batching)。分离让 rollout 用 vLLM/ SGLang 拿 5-10 $\times$ 加速; training 用 Megatron/FSDP 拿满 MFU。

面试考点. Q2: GRPO 相比 PPO 的核心简化 ?

A: 去掉 value model / critic。用 group baseline : 一个 prompt rollout N 个 response, $\text{advantage} = (\text{reward} - \text{group_mean}) / \text{group_std}$ 。少一个 model 训, 省 20-30% GPU。DeepSeek-R1 证明效果不差。变种 DAPO / DrGRPO 都基于此。

面试考点. Q3: 权重从 training 同步到 rollout engine 的三种方案 ?

A: (1) Colocated 直接 copy: 同 pool 交替, 无跨节点传输, 但要停 vLLM; (2) NCCL cross-group broadcast: 训练完后 broadcast 到 rollout 组, 几百 ms overhead; (3) Shared buffer: 训练与 rollout 共 GPU buffer, 零 copy, 需 hack vLLM。verl / OpenRLHF v2 主要用 (2) + overlap with training。

面试考点. Q4: DeepSeek-R1 训练里 rollout 占多少时间? 怎么优化 ?

A: Reasoning task response 长 (8K-32K), rollout 占 70%+ RL 训练时间。优化 : (a) vLLM continuous batching + PagedAttention; (b) speculative decoding; (c) rollout 与 training async pipeline (下 iter training 时上 iter rollout 继续); (d) 允许 slight off-policy (K epoch 用一次 rollout data)。Kimi K1.5 报告用 (a)-(c) 组合。

面试考点. Q5: RLHF 里 KL divergence 项怎么算? ref model 放哪 ?

A: KL 用 policy 与 ref 的 logprob 差。ref model = full forward 一次。放 rollout pool (forward-only, 与 policy generate 同 GPU 用 vLLM 起两个 instance); 或独立 pool; 或用 LoRA delta 表达 (ref forward = policy - LoRA output), Kimi K1.5 做法, 只需 1 forward。

面试考点. **Q6**: rollout 与 training 用不同 GPU 组时怎么分配比例?

A: 依赖 rollout time vs training time 比例。DeepSeek-R1 类长 reasoning: rollout : training $\approx 4 : 1 \rightarrow$ GPU 分 80% rollout, 20% training。短 response (chat, math easy) 可能 1:1。生产会先 profile 一个 iter 确定比例; 有些框架 (verl) 支持动态调整。

面试考点. **Q7**: 一次 GRPO iteration 里 GRPO 的 group size 与 batch size 怎么关系?

A: group_size N = 每 prompt rollout N 个 response 算 group baseline; batch_size B = 每 iter 用 B 个 prompt。总 rollout samples = $B \times N$ 。GRPO 论文 $N=64$; DeepSeek-R1 $N=16-32$ 。 $B \times N$ 太大 rollout 时间长, 太小 baseline 方差大。经验: $N=8-32$, $B=256-1024$ 。

面试考点. **Q8**: 你的 RL 训练 MFU 只有 15%, 比 SFT 40% 差很多, 正常吗?

A: 正常。RL 里“MFU”不好定义, 因为 rollout 是 inference-style 计算 (decode 只有 small-M GEMM, 天然 memory-bound)。SFT 是 dense fwd+bwd (compute-bound)。真实指标应该看: (a) rollout throughput (samples/s); (b) training update tokens/s; (c) end-to-end iterations per hour。GPU 挂 rollout 占比 70%+ 会拉低 “training MFU”, 但整体 wall-clock 反而更快。

RLVR 与 Agentic RL 的分布式：verifier 集群、多轮 rollout、长尾 straggler

第 14 章讲的是“经典 RLHF 系统”：一个 policy、一个 ref、一个 reward model、一个 value model，rollout 是一次 generate。2024 年底到 2026 年，工业界的 RL 训练已经从这个形态整体迁移到两个新范式：

RLVR Reinforcement Learning with Verifiable Rewards —— 把学出来的 reward model 换成确定性验证器（数学答案比对、单元测试、形式化证明检查）。AI2 的 Tulu 3 (2024) 命名了这个词，DeepSeek-R1 (2025) 把它推到了 671B 规模。

Agentic RL rollout 不再是“生成一段话”，而是 **LLM** ↔ 环境的多轮循环：模型发工具调用 → 沙箱/浏览器/搜索执行 → 观测拼回上下文 → 模型继续。Kimi K2、SWE-RL、Search-R1 这一类。这两个范式对系统的冲击比对算法的冲击大得多：

维度	经典 RLHF (第 14 章)	RLVR / Agentic RL (本章)
Reward 来源	Reward model (GPU forward)	Verifier (CPU 沙箱执行) / 规则
新增算力池	无 (都是 GPU)	CPU verifier 集群 + env 服务
Rollout 形态	单次 generate，1 次 prefill	多轮循环，T 次 prefill (除非复用 prefix cache)
长度方差	约 $4 \times (512 \sim 2048 \text{ token})$	约 100× (1 轮 500 token ~ 50 轮 100K token)
Rollout 期 GPU	一直在 decode	等环境时闲置 (工具执行几秒)
Reward 稠密度	连续标量，每样本都有	二值 0/1，且常整组全对或全错
Ref model	必须有 (算 KL)	很多配方直接去掉 KL ，省一个池
训练侧新坑	advantage 归一化	token masking (观测 token 不能训)

表 31 经典 RLHF 与 RLVR / Agentic RL 的系统性差异。右列每一行都对应本章一节。面试里能把这张表口述出来，基本就说明你真上手过。

Warning. 这个方向 2025-2026 迭代极快，论文里的具体数字（步数、分数、加速比）跟版本强相关。面试时说清来源和时间点（“DAPO 那篇 2025 年 3 月报告的是……”），比背一个孤立数字安全得多。本章标注的都是公开报告里的做法，不代表你做过。

一. RLVR：把 reward model 换成 verifier

1.1 为什么要换掉 reward model

Reward model (RM) 在数学 / 代码这类有客观对错的任务上有三个硬伤：

1. **Reward hacking**：RM 是个神经网络，policy 训久了一定学会骗它。DeepSeek-R1 报告里明确说，他们放弃了 neural PRM / ORM，理由就是大规模 RL 下 reward hacking 不可避免，而且不断重训 RM 来对抗，代价太高。
2. **RM 本身要占 GPU**：70B policy 配 70B RM，rollout 池里就多一份 TP=8 的 inference 实例。

3. **RM** 有噪声上限：RM 的准确率若 85%，policy 的天花板就被钉死在 RM 的偏好分布上。

Verifier 的对立面很干净：数学题对答案，代码跑单元测试，IFEval 类指令跑程序化约束检查。确定、免费（相对 GPU）、不可 **hack**（至少不能用“说好话”来 hack）。

1.2 Verifier 的四类形态与延迟量级

系统设计的第一步是搞清楚你的 verifier 属于哪一类 —— 因为延迟差了 4 个数量级：

类型	做法	典型延迟	主要风险
数学	答案抽取 + 符号等价 (sympy / math-verify)	1-50 ms	sympy 在病态表达式上挂死，必须超时
代码	沙箱跑单元测试	0.1-10 s	安全隔离、flaky test、依赖装配
形式化证明	Lean / Coq 编译	秒 ~ 分钟	编译器就是瓶颈，需大 CPU 池
Agent 任务	最终答案 EM/F1，或 LLM-as-judge	ms ~ 秒	judge 又变回 GPU 负载 + 可 hack

表 32 四类 verifier 的延迟量级。数学 verifier 便宜到可以同步跑；代码 / 形式化 verifier 必须建独立池 + 异步化。

Key insight. 一个反直觉但很关键的点：**verifier** 的平均延迟不重要，尾延迟才重要。一组 GRPO rollout 有 $N = 16$ 个响应，只要有 1 个触发了 sympy 死循环或死锁的单元测试，整组的 advantage 就算不出来，整个 batch 卡住。所以 verifier 层第一件要做的事永远是每样本硬超时，而不是优化平均延迟。

1.3 Verifier 集群架构

Verifier 是 CPU 负载，第一次出现在一个纯 GPU 的训练栈里。典型架构：

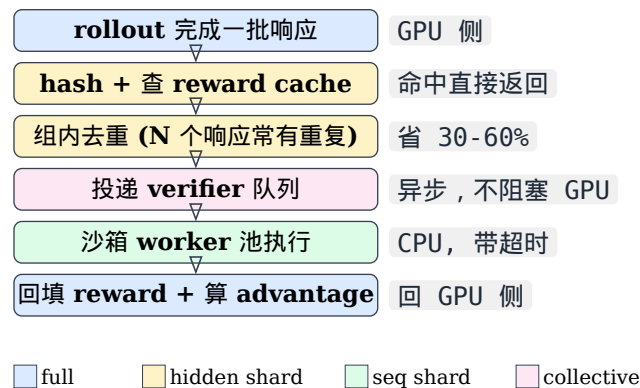


图 30 RLVR verifier 流水线。cache 与去重放在最前面是因为它们最便宜；沙箱执行放在异步池里，让 GPU 在等待期间继续 rollout 下一批。

四个必备设计：

(a) 异步化，别放在关键路径上。同步版本是“rollout 全批 → 全批 verify → 训练”，verify 期间 GPU 全闲。异步版本是 verify 与下一批 rollout 重叠，只要 verifier 池吞吐 $\geq$ rollout 产出速率，verify 就完全不进关键路径。

(b) **Reward cache + 组内去重**。GRPO 一个 prompt 采 $N = 16$ 个响应，温度不高时完全相同的响应很常见（数学题尤其，正确解法就那么几种）。按 `hash(prompt, response)` 做缓存 + 组内去重，实测能省掉 30-60% 的 verifier 调用。这是最高性价比的一招。

(c) 每样本硬超时 + 熔断。用子进程 + SIGKILL，不要用线程（Python 线程杀不掉死循环）。超时的样本给 reward 0 并单独打点——超时率突然上升往往意味着 policy 学到了某种病态输出。

(d) CPU:GPU 配比要算，不能拍。这是面试高频计算题，见下节。

1.4 CPU:GPU 配比怎么算

设：rollout 池 G 张 GPU，每张 GPU 每秒产出 r 个响应；单次验证平均耗时 t_v 秒，缓存/去重命中率 h ；每个 verifier worker 单线程。

稳态下 verifier 池不成为瓶颈的条件是：

$$W \geq \frac{G \cdot r \cdot (1 - h) \cdot t_v}{u}$$

其中 W 是 worker 数， u 是目标利用率（留余量，取 0.7 左右）

代入一组真实量级：128 张 GPU 做 rollout，每卡每秒产出 $r = 0.5$ 个响应（长 CoT，响应几千 token），代码 verifier $t_v = 2$ s，去重命中率 $h = 0.4$ ， $u = 0.7$ ：

$$W \geq \frac{128 \times 0.5 \times 0.6 \times 2}{0.7} \approx 110 \text{ workers}$$

按每 worker 1 core 算就是 110 core，约 1.5 台 64 核机器。

这个公式是对的，但它回答的不是你最关心的问题。稳态排队是否稳定只取决于平均服务时间，所以用均值代入 t_v 就能得到你要的利用率——不需要“为了保险”按 p90 配（那会白买 2-3 倍的机器）。

真正卡住训练的是另一件事：**GRPO** 必须等一组 N 个响应全部验完才能算 advantage，所以优化器等的是组内最大值，不是均值。而重尾分布下这两者差得很远——17_rlvr_verifier.py 的仿真里，单次验证均值 1.39 s，但 16 个一组的最大值均值是 6.42 s，**4.6x**；并且有 39% 的组里至少有一个样本会撞上超时。

由此得到两条不太直觉但很关键的结论：

1. 加机器能解决吞吐，永远解决不了组内最大值。能压住组最大值的只有两样东西：超时（直接给尾部封顶）和去重（减少每组从尾部独立抽样的次数）。
2. 更好的办法是让组最大值根本不在关键路径上——异步验证，让它与下一批 rollout 重叠。做到这一点之后，按均值算出来的容量数就真的够用了。

面试考点. 面试题：你们 RLVR 训练里 GPU 利用率只有 55%，profile 显示 GPU 在等 reward。怎么定位和解决？

分层回答：

1. 先量：打点 rollout 产出速率、verifier 队列深度、verify p50/p99、cache 命中率。队列深度持续增长 → verifier 池吞吐不够；队列空但 GPU 还在等 → 说明是同步阻塞，不是容量问题。
2. 如果是同步阻塞：改异步，verify 与下一批 rollout 重叠。这一步通常直接把利用率拉回 80%+，且不花钱。
3. 如果是容量不够：先上 cache + 组内去重（免费拿 30-60%），再按上面的公式扩 worker。
4. 如果是尾延迟：查超时率与超时样本的特征。见过的真实原因包括 policy 学会输出超长表达式把 sympy 卡死、生成的测试代码里 input() 等待 stdin、以及单元测试里有网络调用被防火墙挂住。修法分别是表达式长度上限、沙箱关 stdin、沙箱断网。

1.5 沙箱安全：模型生成的代码是敌意输入

这一条在面试里很能体现“真跑过生产”。Policy 生成的代码要在你的集群里执行，而 RL 的本质就是搜索一切能拿到高 **reward** 的行为——包括直接篡改测试结果。

必须有的隔离层：

- 容器 / 微 **VM**：gVisor、Firecracker、nsjail 之一。裸 `subprocess` 不够。
- 断网：否则模型可能去下载正确答案，或者把你的内网探个遍。
- 只读文件系统 + 独立 **tmpfs**：防止改测试文件。真实见过的 hack：生成的代码把 `test_*.py` 覆写成 `assert True`。
- **cgroup** 限 **CPU** / 内存 / **pid**：fork bomb 和内存炸弹是常态，不是意外。
- 墙钟超时 + 强杀。

Warning. Reward hacking 在 **RLVR** 里没有消失，只是换了形态。RM 时代是“说漂亮话骗 RM”，verifier 时代是“骗 verifier”：改测试文件、在代码里 `sys.exit(0)`、捕获所有异常返回硬编码答案、数学题里输出一堆候选答案让抽取器蒙对一个。防御手段是把 **verifier** 本身当作被攻击面来设计：测试文件放在容器外挂只读、答案抽取只取最后一个 `\boxed{}`、比对前做归一化且拒绝多答案。

二. RLVR 的算法层：二值 **reward** 带来的新问题

2.1 GRPO 在二值 **reward** 下的退化

GRPO 的 advantage 是组内归一化：

$$A_i = \frac{R_i - \text{mean}(R_1..R_N)}{\text{std}(R_1..R_N) + \varepsilon}$$

二值 reward 下，如果这一组 N 个响应全对（都是 1）或全错（都是 0），那么 $\text{std} = 0$ 、 $R_i - \text{mean} = 0$ ，于是 $A_i = 0$ —— 整组贡献零梯度。这一组的 rollout 算力（可能是几十秒的长 CoT 生成）完全白烧。而这不是边缘情况：训练早期大量题目全错，训练后期大量简单题全对。实测中无效组占比能到 30-50%。

DAPO 的 **dynamic sampling** (ByteDance, 2025)：过采样，然后丢掉准确率为 **0** 或 **1** 的组，一直采到凑满一个 batch 的“有信息量”的组。代价是 rollout 量变大，收益是每个梯度步都有真实信号。

```
# 概念版：动态采样直到凑满 batch
kept_groups = []
while len(kept_groups) < target_batch_groups:
    prompts = sampler.next(oversample_factor * needed)
    groups = rollout_engine.generate(prompts, n=N)
    rewards = verifier_pool.verify(groups) # 异步 + cache
    for g, r in zip(groups, rewards):
        acc = r.mean()
        if 0.0 < acc < 1.0: # 只保留有梯度的组
            kept_groups.append(g)
        else:
            _wasted_group_counter.inc( # 一定要打点
                "all_correct" if acc == 1.0 else "all_wrong")
```

系统层的连带影响：dynamic sampling 让每个 iteration 的 rollout 量不确定，rollout 池的容量规划从“固定 batch”变成“带反馈的流式采样”。调度器要支持“继续采直到够”，而不是“采固定 N 个”。

另一条路是难度分层采样：维护每道题的历史准确率，优先采 $0.2 < \text{acc} < 0.8$ 的题（信息量最大），把长期全对的题移出题池。这是 curriculum 的一种，比 dynamic sampling 省 rollout，但需要维护题目状态。

2.2 长度偏差：Dr. GRPO 的两处修正

Dr. GRPO (Liu et al., 2025, Understanding R1-Zero-Like Training) 指出原始 GRPO 目标里有两个引入偏差的归一化：

1. 按响应长度归一化 $\frac{1}{|o_i|}$ ：让“短的正确答案”和“长的错误答案”获得不成比例的权重，系统性地鼓励错误答案变长——这正是大家观察到的“RL 训着训着响应越来越长但没变强”的成因之一。
2. 按组标准差归一化 $\frac{1}{\text{std}}$ ：让“几乎全对”和“几乎全错”的题（std 小）获得放大的权重，等于给了极端难度的题过高的权重。

Dr. GRPO 的处方就是把这两项去掉，用 token 级的求和而不是样本级的平均。

DAPO 独立地提出了 **token-level policy gradient loss**，动机一致：样本级平均会让一个 1000 token 的响应和一个 100 token 的响应在 loss 里权重相同，于是长响应里每个 token 的有效学习率被稀释 10×。

Key insight. 这一组问题的共同根源是：你在哪个粒度上做平均，就在哪个粒度上定义了“公平”。样本级平均 = 每个样本同权，token 级求和 = 每个 token 同权。长 CoT 场景下响应长度差异巨大，两者的差别就从“实现细节”升级成“影响收敛方向的算法选择”。面试里能讲清这一层，比背“Dr. GRPO 去掉了 std”强得多。

2.3 熵坍缩与 clip-higher

长时间 RLVR 训练的典型病：policy 熵单调下降 → 输出越来越确定 → 一组 N 个采样几乎完全相同 → GRPO 组内没有差异 → 梯度消失 → 训练停滞。

DAPO 的 **clip-higher**：把 PPO 的对称裁剪 $[1 - \epsilon, 1 + \epsilon]$ 拆成非对称的 $[1 - \epsilon_{\text{low}}, 1 + \epsilon_{\text{high}}]$ ，取 $\epsilon_{\text{low}} = 0.2$ 、 $\epsilon_{\text{high}} = 0.28$ 。直觉是：对称裁剪对低概率 **token** 的上升空间限制过死（一个概率 0.01 的 token 最多只能涨到 0.012），而这些低概率 token 正是探索的来源。放宽上界让它们能长起来，熵就不会塌。

其他常见手段：保持 rollout 温度 1.0（评测才用 0）、周期性检查组内响应的两两相似度、把熵本身做成监控指标并设告警线。

2.4 去掉 KL 项：省一个 GPU 池

经典 RLHF 里 KL 惩罚项防止 policy 跑离 base 太远。但 RLVR 的目标恰恰是让模型学会一种 **base** 完全不会的行为（长链推理）——此时 KL 是纯阻力。DAPO 等配方直接把 KL 项去掉。

系统上的收益是实打实的：没有 KL 就不需要 ref model forward，rollout 池里少一份 70B 实例，省下的显存可以全给 KV cache，rollout 吞吐直接上一截。

代价：失去了“跑偏”的自动刹车。必须用别的东西兜底——监控输出语言混杂（R1 报告过中英混杂）、格式崩坏、以及在 held-out 通用能力集上的回退。

问题	机制	处方
整组零梯度	二值 reward 下组内全对/全错 $\rightarrow \text{std}=0$	DAPO dynamic sampling / 难度分层采样
响应越训越长	$\frac{1}{ o_i }$ 归一化偏袒长错误答案	Dr. GRPO 去长度归一化 + token-level loss
难度权重失衡	$\frac{1}{\text{std}}$ 放大极端难度题	Dr. GRPO 去 std 归一化
熵坍塌	对称裁剪压制低概率 token 上升	clip-higher ($\epsilon_{\text{high}} = 0.28$)
ref 池占 GPU	KL 需要 ref forward	去 KL, 改用监控兜底
截断算失败	超长响应给 0 $\rightarrow$ 教模型变短	DAPO overlong reward shaping (软惩罚)

表 33 RLVR 算法层六个高频问题与对应处方。每一行都能单独成为一道面试追问问题。

2.5 一个必须知道的 caveat : spurious rewards

2025 年有一批工作 (如 Shao et al. 的 Spurious Rewards) 发现:在某些 Qwen 系模型上,用随机 **reward** 甚至错误 **reward** 做 RLVR, 数学分数照样能涨。这说明 RL 在这些设置下激发的是预训练里已有的能力 (比如“用代码思路解题”的倾向), 而不是学到了新东西。

面试价值在于:这提醒你 **RLVR** 的评测必须有对照组。如果你说“我们上了 RLVR, AIME 涨了 10 分”, 一个懂行的面试官会追问“随机 reward 的 baseline 跑了吗? 换个基座还成立吗?”能主动提这一点, 说明你不是只会跑 pipeline。

三. Agentic RL 的 rollout 引擎

3.1 多轮循环把 rollout 变成了另一个东西

单轮 rollout: 一次 prefill + 若干 decode, 结束。

多轮 agentic rollout:

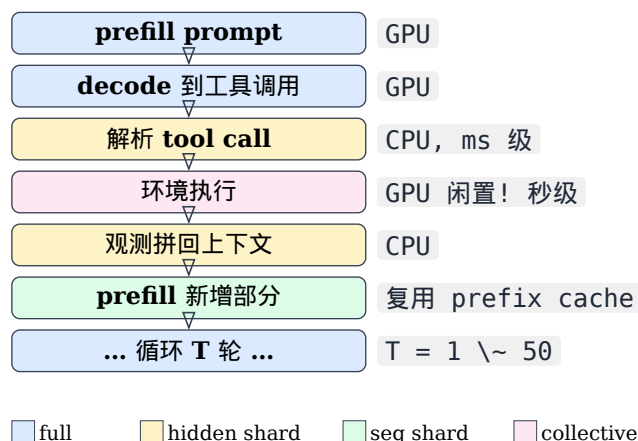


图 31 Agentic rollout 的一轮循环。关键在两处: 环境执行期间该序列不占 GPU 计算但占着 KV cache; 每轮的新 prefill 必须复用上一轮的 prefix cache, 否则总 prefill 代价是 $O(T^2)$ 。

由此产生四个系统问题, 逐个说。

3.2 问题一: prefix cache 是生死线

第 $k+1$ 轮的输入 = 第 k 轮的完整上下文 + 新的观测。如果每轮都重新 prefill 整个上下文, T 轮下来 prefill 的总 token 数是 $O(T^2)$ 量级:

$$\sum_{k=1}^T L_k \approx T \cdot \frac{|L|}{2} \quad \text{vs} \quad \text{复用后} \approx |L|$$

20 轮的轨迹，不复用要多付一个数量级的 prefill。vLLM 的 automatic prefix caching / SGLang 的 RadixAttention 就是干这个的，agentic RL 里必须开。

但有个坑：环境执行要等几秒，这几秒里该序列的 KV cache 还占着显存。推理引擎的缓存淘汰策略 (LRU) 在高并发下很可能把它淘汰掉，等观测回来时又要重新 prefill —— 你以为开了 prefix cache，实际命中率只有 40%。

排查手段就是直接看引擎的 prefix cache 命中率指标。修法：

- 提高 KV cache 池容量（把省下来的 ref model 显存给它）
- 对“正在等环境”的序列做 cache pinning，或者把 KV 换出到 CPU 而不是直接丢
- 限制并发轨迹数，让活跃集合装得下

3.3 问题二：等环境时 GPU 闲置 → 必须异步

同步批式 rollout (“这一批 32 条轨迹一起走第 1 轮，一起等环境，再一起走第 2 轮”) 是最容易写、也最浪费的实现：每一轮都要等这批里最慢的环境调用，GPU 在此期间纯闲。

正确做法是异步 / 连续批处理：推理引擎里同时挂着几百条轨迹，谁的观测回来了谁就进下一轮的 batch，等环境的轨迹自然从运行批次里退出，不占计算。这本质上就是 vLLM 的 continuous batching 用在轨迹粒度上。

src/distributed_training/16_agent_rollout_sched.py 用离散事件仿真对比了这两种调度：同样的负载，GPU 利用率 19% → 88%，makespan 快 4.6×。

Key insight. 这个仿真还顺带暴露了一个指标陷阱，很值得在面试里主动提。

异步调度故意让几百条轨迹同时在飞，于是每条轨迹都要排在别人后面，单轨迹延迟反而变差了（仿真里 102 s → 146 s）。更糟的是 p99/p50 这个常用的 straggler 指标：同步批式的比值是 2.85，异步是 2.94 —— 同步看起来更健康。

两个指标都在说反话。原因是同步的 barrier 把所有短轨迹都拖慢到最慢那条的节奏，离散度是因为大家一起被饿死才变小的，而百分位比值区分不了“没有 straggler”和“全都是 straggler”。

正确的口径是 **makespan** 和 **GPU 利用率**：在 RL 里没有任何东西会在整个 iteration 组装完之前消费单条轨迹，所以卡住下一次优化器更新的就是 makespan，单轨迹延迟根本不在关键路径上。

3.4 问题三：长尾 straggler 与 partial rollout

Agentic 轨迹的长度分布是重尾的：大部分任务 3-5 轮解决，少数任务磨 40 轮。同步收集一个 batch 意味着等最长的那条。

Kimi K1.5 报告的 **partial rollout** 是标准解法：给每个 iteration 设 token / 轮数预算，没跑完的轨迹存下状态，下个 **iteration** 接着跑，而不是丢弃或死等。

```
# 概念版：带预算的 partial rollout
pending = carry_over_from_last_iter      # 上轮没跑完的轨迹
budget  = ITER_TOKEN_BUDGET
finished = []
```

```

while budget > 0 and (pending or task_queue):
    traj = pending.pop() if pending else Trajectory(task_queue.pop())
    traj, used = engine.run_until(traj, budget_left=budget)
    budget -= used
    if traj.done:
        finished.append(traj)
    else:
        traj.policy_version_span.append(current_policy_version) # 记录跨版本
        carry_over.append(traj)                                # 下轮接着跑

```

代价是 **off-policy**：一条轨迹的前半段由旧 policy 生成、后半段由新 policy 生成。处理方式有三种，按严格程度递减：

1. 逐 token 记录生成时的 policy 版本，算重要性比时用对应版本的 logprob（最严格，实现最重）
2. 限制轨迹最多跨 K 个版本（比如 2），超了就丢弃（折中，最常见）
3. 直接当 on-policy 处理（最省事，短 iteration 下偏差可接受）

面试里问到 partial rollout，能主动说出“它引入了 off-policy，你得选一种处理方式”，比只说“Kimi 用了 partial rollout”高一个层次。

3.5 问题四：权重更新时的在途轨迹

训练完成一次更新，要把新权重同步到 rollout 引擎。此时有几百条轨迹跑到一半 —— 怎么办？

策略	做法	代价
Drain	等所有在途轨迹跑完再换权重	长尾轨迹让 GPU 空转，回到 straggler 问题
Abort	直接丢弃在途轨迹	浪费已生成的 token，长轨迹损失最大
Carry-over	轨迹跨版本继续（partial rollout）	off-policy，需版本记账

表 34 权重更新时对在途轨迹的三种处理。长 horizon 场景下 drain 和 abort 都很贵，所以主流框架都往 carry-over 走，代价是要处理 off-policy。

四. Agentic RL 的训练层

Rollout 拿回来的是轨迹，不是“一段回答”。训练侧因此多出几个极易写错的地方。

4.1 Token masking：本章最重要的一个 bug

一条轨迹的 token 序列长这样（宽度按真实比例画：观测 token 通常远多于 assistant token）：

角色	任务	asst	tool 观测	asst	tool 观测	asst
loss_mask	不训	要训	不训	要训	不训	要训

图 32 一条 2 轮 agentic 轨迹的 token 布局（asst = assistant 生成）。蓝色是 policy 自己生成的，橙色是环境写进来的。下面一行是必须随轨迹一起传给训练侧的 loss_mask。

只有 **assistant** 生成的 **token** 能进 **loss**。观测 token 是环境产生的，训它等于在教模型“预测工具会返回什么”——而这是它原理上不可能知道的信息。后果有三层：

1. 梯度噪声：观测 token 在 agentic 轨迹里常占 60-80%（一次搜索返回几千 token 很正常）。不 mask 等于让 80% 的梯度来自一个不可学的目标。

2. 幻觉工具输出：模型被训得倾向于自己“编”一段看起来像观测的文本，推理时不调工具直接编造结果。
3. 重要性比失效：PPO/GRPO 的比值 $\frac{\pi_\theta}{\pi_{\text{old}}}$ 对观测 token 没有意义——那些 token 根本不是 π_{old} 采样出来的。

这个 bug 的阴险之处在于它不会让训练崩。Loss 照常下降，梯度范数正常，你要盯着“工具调用成功率”和“是否出现无调用直接给答案”才能发现。Search-R1 那一类工作里把“retrieved token masking”单独拎出来讲，就是因为踩过。

正确的实现是每条轨迹带一个 `loss_mask`，并且——这是第二个容易漏的点——归一化的分母也要只数 **assistant token**：

```
# 错误：分母含观测 token，等价于按轨迹里工具输出的多少来缩放学习率
loss = (per_token_loss * loss_mask).sum() / loss_mask.numel()

# 正确：只对参与训练的 token 求平均
loss = (per_token_loss * loss_mask).sum() / loss_mask.sum().clamp(min=1)
```

`src/distributed_training/l8_traj_masking.py` 把这个错误版本和正确版本并排跑，量化了梯度差异。

4.2 Advantage 往哪儿广播：trajectory-level vs turn-level

Agentic 任务的 reward 通常只在轨迹末尾（任务成功/失败）。两种赋值方式：

Trajectory-level 整条轨迹一个 advantage，广播到所有 assistant token。实现简单，与 GRPO 天然契合（组 = 同一任务的 N 条轨迹）。缺点是长 horizon 下信用分配极粗——40 轮里只有第 12 轮的决策是错的，但 40 轮都被同等惩罚。

Turn-level 每轮给一个 reward（工具调用是否成功、检索是否命中、子目标是否达成），配合折扣因子做跨轮信用分配。信号密得多，但每一个中间 **reward** 都是一个新的可 **hack** 面——给“工具调用成功”加分，模型就学会疯狂发无意义但一定成功的调用。

工业上的折中通常是：主 **reward** 用轨迹级的可验证结果，中间只加极少量、极难 **hack** 的 shaping（比如格式合法性、是否在预算内完成），并且给中间项很小的系数。

4.3 Rollout 与训练的 logprob 失配

这是 RL 框架里最经典的“数值对不上”问题，agentic 场景下被放大。

Rollout 用 vLLM/SGLang 生成，训练用 Megatron/FSDP 前向。两边算出来的同一个 token 的 logprob 不完全相等：kernel 实现不同、batch 组织不同、BF16 累加顺序不同、TP 切分不同。

如果你直接把 rollout 返回的 logprob 当作 $\log \pi_{\text{old}}$ ，那么在第一个内层 epoch，理论上应该恒等于 1 的重要性比 $\frac{\pi_\theta}{\pi_{\text{old}}}$ 会有系统性偏离——于是本该完全不裁剪的第一轮出现了大量裁剪，梯度被无谓地砍掉。

处方：

- 用训练引擎自己再前向一遍重算 $\log \pi_{\text{old}}$ （多一次 forward，但数值自洽）。主流框架的默认做法。
- 或者把这个失配当作监控指标：统计 $|\log \pi_{\text{train}} - \log \pi_{\text{rollout}}|$ 的分布和第一个 epoch 的裁剪比例。裁剪比例在第一个 epoch 应该接近 0，明显大于 0 就说明有问题。

面试考点. 面试题：你们 agentic RL 训练 loss 在降，但 agent 的工具调用成功率反而下降，甚至开始不调工具直接编答案。怎么排查？

这道题几乎就是在问 token masking。排查顺序：

1. 先看 **mask**：打印一条轨迹的 `loss_mask`，确认观测 token 全是 0。这是最可能的原因，也最容易验证。
2. 再看归一化分母：是不是用了 `mask.numel()` 而不是 `mask.sum()`。
3. 再看 **reward** 设计：是不是中间 reward 被 hack 了，或者最终 reward 对“不调工具但蒙对”给了正分。
4. 最后看 **logprob** 失配：第一个 epoch 的裁剪比例是否异常高。
5. 顺带一提监控口径：loss 下降在 agentic RL 里几乎不能说明任何事，必须盯任务成功率、平均轮数、工具调用成功率、截断率这几个业务指标。

4.4 变长轨迹的 **packing** 与截断

轨迹长度 $100\times$ 的方差在训练侧同样是问题：一个 batch 里如果有一条 100K token 的轨迹和 31 条 2K 的，按最长 padding 的话 94% 的算力在算 padding。

处理方式与第 12 章的 packing 一脉相承，但有 agentic 特有的点：

- 按 **token** 总量而不是轨迹条数组 batch (`max_tokens_per_batch`)
- 用 FlashAttention varlen path + 轨迹间 block-diagonal mask，禁止跨轨迹 attention
- DP rank 之间按 $\sum_i s_i^2$ (attention 的平方代价) 平衡，而不是按 token 数
- 超长轨迹要有明确策略：截断后是算失败 (reward 0) 还是算“未完成” (mask 掉不给梯度)。前者会教模型变短，后者浪费算力。DAPO 的 overlong reward shaping 走的是软惩罚的中间路线。

五. 环境作为分布式服务

Agentic RL 引入了一个训练栈里从来没有过的东西：一个需要横向扩展、会失败、有状态、可能收费的外部服务。

5.1 环境的几种形态与它们的麻烦

环境	典型实现	主要工程问题
代码沙箱	容器 / Firecracker + 测试框架	与 verifier 共用池；镜像预热；依赖装配慢
检索 / 搜索	本地向量库 或 外部搜索 API	外部 API 有限流和费用；结果随时间漂移 → 不可复现
浏览器	Playwright / 无头 Chrome	重（每实例几百 MB）、慢、极易 flaky
SWE 仓库	Git 仓库快照 + 测试	仓库状态要能快速重置；镜像体积大
MCP 工具	MCP server 集群	协议层超时与重试；工具版本要钉死

表 35 五类 agentic RL 环境。共同点是它们都不在 GPU 上，都会失败，都需要独立的容量规划。

5.2 三条必须处理的工程约束

(a) 失败要分类，不能一律算 0 分。环境超时、沙箱 OOM、外部 API 限流——这些是基础设施失败，不是 policy 的错。给它 reward 0 等于用随机噪声污染训练信号。正确做法是标记为 `infra_failure`，从这个 **batch** 里剔除并单独打点。如果 infra 失败率超过 1%，那是要修的系统问题，不是要学的训练信号。

(b) 可复现性基本无法完全保证，但要尽量收敛。外部搜索 API 今天和明天返回不同结果，同一条轨迹重放会拿到不同 reward。缓解：能本地化的就本地化（用固定快照的检索库而不是线上搜索）、缓存环境响应、把环境版本和快照 ID 写进 checkpoint。

(c) 成本是真实约束。 浏览器实例、外部 API 调用、沙箱 CPU 都要花钱，而 RL 的 rollout 量是训练量的几十倍。做容量规划时把环境成本和 GPU 成本放在一起算，经常会发现环境侧才是瓶颈。

六. 工业案例速查

工作	关键做法	可引用的点
Tulu 3 (AI2, 2024)	提出 RLVR 这一名词,开源完整配方与 verifier	RLVR 的出处；开源可复现
DeepSeek-R1 (2025)	GRPO + 规则 reward (准确率 + 格式), 明确弃用 neural PRM/ORM	弃用 RM 的理由：reward hacking + 重训代价
Kimi K1.5 (2025)	partial rollout 、长度惩罚、训推同机部署与权重同步	长尾 straggler 的标准解法出处
Kimi K2 (2025)	大规模合成工具使用数据 + 可验证 reward 与自评 rubric 结合	agentic 能力的数据侧怎么造
DAPO (ByteDance, 2025)	clip-higher、dynamic sampling、token-level loss、overlong shaping	RLVR 四件套，最常被追问
Dr. GRPO (2025)	去掉长度归一化与 std 归一化两处偏差	“响应变长但没变强”的理论解释
Qwen3 (2025)	强到弱蒸馏 + RLVR，思考/非思考模式融合	工业配方里 RLVR 放在哪一段
SWE-RL (Meta, 2025)	用与 oracle patch 的相似度作规则 reward，绕开跑测试的高成本	verifier 太贵时的替代设计
Search-R1 / ReTool 等	检索 / 代码工具进 rollout 循环，显式做观测 token masking	token masking 的出处
verl (HybridFlow)	agent loop、异步 rollout、vLLM/SGLang 后端	最常被问“你用什么框架”
SkyRL	面向长 horizon SWE 任务的异步多轮 rollout 与环境抽象	长 horizon agentic 的开源参考

表 36 RLVR / Agentic RL 的公开工作速查。面试里引用时务必带上时间点。

七. 面试题

面试考点. **Q1**：RLVR 相比经典 RLHF，在系统架构上最大的变化是什么？

A：多出一个 **CPU verifier** 集群，同时可能少掉 **reward model** 和 **ref model** 两个 GPU 池（去 KL 的配方）。也就是说算力结构从“纯 GPU”变成“GPU + 大规模 CPU 沙箱”。随之而来的新瓶颈是 verifier 吞吐与尾延迟：一个 GRPO 组里只要有一个样本把 verifier 卡住，整组 advantage 就出不来。所以 verifier 侧的三件套是异步化、结果缓存与组内去重、每样本硬超时。

面试考点. Q2 : GRPO 用二值 reward 时, 为什么会有大量 rollout 白烧? 怎么解决?

A : 组内全对或全错时标准差为 0、advantage 全 0, 整组零梯度, 而这类组在训练早期和后期占比能到 30-50%。解法一是 DAPO 的 dynamic sampling : 过采样后丢掉准确率为 0 或 1 的组, 采到凑满 batch 为止; 解法二是难度分层采样, 优先采历史准确率在 0.2-0.8 的题。前者实现简单但 rollout 量变成动态的, 调度器要支持流式补采; 后者省算力但要维护题目状态。

面试考点. Q3 : Agentic rollout 里, 为什么 prefix cache 是生死线?

A : 第 $k+1$ 轮的输入是第 k 轮的完整上下文加新观测, 不复用的话 T 轮的总 prefill 是 $O(T^2)$ 量级, 20 轮就要多付一个数量级。真正的坑不是“没开”, 而是“开了但命中率低”——序列在等环境执行的那几秒里 KV cache 被 LRU 淘汰了。排查看引擎的 prefix cache 命中率, 修法是扩大 KV 池、对等待中的序列做 pinning 或换出到 CPU、限制并发轨迹数。

面试考点. Q4 : 多轮轨迹训练时, 哪些 token 要 mask? 不 mask 会怎样?

A : 只训 assistant 生成的 token, 工具观测 token 必须 mask 掉。不 mask 有三个后果: 观测常占 60-80% token, 梯度大部分来自一个模型原理上无法预测的目标; 模型学会自己编造工具输出, 推理时不调工具直接幻觉结果; 重要性比对观测 token 没有意义, 因为那些 token 不是 old policy 采出来的。而且这个 bug 不会让训练崩, loss 照降, 只能靠工具调用成功率这类业务指标发现。另外归一化分母也要用 `mask.sum()` 而不是 `numel()`。

面试考点. Q5 : Partial rollout 解决什么问题? 带来什么新问题?

A : 解决 agentic / 长 CoT 轨迹长度重尾导致的 straggler —— 同步收集一个 batch 要等最长那条。Partial rollout 给每轮设预算, 没跑完的轨迹存状态下轮接着跑。新问题是 off-policy : 一条轨迹跨了多个 policy 版本。三种处理, 从严到松是逐 token 记版本并用对应 logprob 算重要性比、限制最多跨 K 个版本超了就丢、直接当 on-policy。多数框架选第二种。

面试考点. Q6 : Rollout 引擎和训练引擎算出来的 logprob 不一致, 有什么影响?

A : 直接拿 rollout 的 logprob 当 $\log \pi_{\text{old}}$, 第一个内层 epoch 本该恒为 1 的重要性比会系统性偏离, 导致本不该发生的裁剪, 梯度被白砍。根因是两边 kernel、batch 组织、BF16 累加顺序、并行切分都不同。处方是用训练引擎重新前向一遍算 $\log \pi_{\text{old}}$, 多一次 forward 换数值自治; 同时把两边 logprob 的差值分布和第一个 epoch 的裁剪比例做成监控——第一个 epoch 的裁剪比例应该接近 0。

面试考点. Q7 : 怎么给 RLVR 训练做 CPU:GPU 容量规划?

A : 稳态条件是 $W \geq G \cdot r \cdot (1 - h) \cdot t_v / u$, 其中 G 是 rollout GPU 数、 r 是每卡每秒响应产出、 h 是缓存与去重命中率、 t_v 是单次验证耗时、 u 是目标利用率。举例 128 卡、 $r = 0.5$ 、代码 verifier $t_v = 2$ s、 $h = 0.4$ 、 $u = 0.7$, 算出约 110 个 worker。

这里有个容易答错的地方: t_v 就该代均值。排队稳定性只取决于平均服务时间, 按 p90 代入是白买机器。重尾带来的问题不在容量而在别处 —— GRPO 要等一组 N 个全部验完, 优化器等的是组内最大值, 实测能到均值的 4-5 倍。压组最大值靠超时和去重, 而不是加 worker; 更好的做法是异步验证让它离开关键路径。能把“容量”和“组内最大值”这两件事分开讲, 这道题就答满了。

面试考点. **Q8**：环境执行失败（沙箱 OOM、API 限流）应该给什么 reward？

A：不能给 0。那是基础设施失败，不是 policy 的行为后果，给 0 等于往训练信号里注入随机噪声，模型会学到一些与真实目标无关的规避行为。正确做法是标为 `infra_failure`、从 batch 里剔除、单独打点。同时把 infra 失败率本身当 SLO 来管——超过 1% 就是要修的系统故障。这道题的考点是能不能区分“环境的错”和“模型的错”。

八. STAR 故事：把 RLVR + Agentic 串起来

Situation：

“我们要在 32B 模型上做代码 agent 的 RL，任务是给定 issue 修仓库代码，用仓库自带测试判对错。第一版流水线跑起来后，一个 iteration 要 40 分钟，GPU 利用率只有 48%，而且训了两天工具调用成功率不升反降。”

Task：

“我负责把 iteration 时间压到 15 分钟以内，并定位成功率下降的原因。”

Action：

“分两条线。

先修正确性，因为跑得再快方向错了也没用。成功率下降但 loss 正常下降，这个组合直接指向 mask。打了一条轨迹的 `loss_mask` 出来，发现工具观测的 token 没有被 mask —— 我们的轨迹里测试输出平均占 70% 的 token，等于七成梯度在教模型预测测试框架的 stdout。修了 mask，顺带发现归一化分母用的是 `numel()` 而不是 `mask.sum()`，等价于按每条轨迹的工具输出多少在缩放学习率。这两处修完，工具调用成功率两天内从 51% 回到 78%。

再修吞吐，按 profile 的占比顺序来：

- Rollout 是同步批式的：一批 32 条轨迹一起走一轮、一起等沙箱。改成异步连续批处理，等沙箱的轨迹自动退出运行批次。GPU 利用率 48% → 71%。
- 查引擎的 prefix cache 命中率只有 43%，原因是轨迹等沙箱那几秒 KV 被 LRU 淘汰了。我们刚好因为去掉了 KL 项省出一份 ref model 的显存，全给了 KV 池，同时限制并发轨迹数让活跃集合装得下，命中率到 91%。
- Verifier 侧：一开始是同步跑测试，改成异步池 + 按 `hash(prompt, patch)` 缓存 + 组内去重，去重命中率 38%（同一个 issue 采 16 个 patch，重复的很多）。worker 数按 $W \geq Gr(1-h)t_v/u$ 算了个下界再乘 1.5。
- 还发现约 40% 的 GRPO 组是全对或全错的零梯度组，上了 DAPO 式的 dynamic sampling 过滤掉，同时把 infra 失败（沙箱 OOM、镜像拉取超时）从 reward 0 改成剔除样本并打点 —— 之前这部分噪声一直在污染 advantage。”

Result：

“Iteration 时间 40 → 13 分钟，GPU 利用率 48% → 79%，工具调用成功率 51% → 78%，SWE-bench 风格内部评测集通过率提升了 9 个点。masking 和 infra-failure 剔除这两个修复后来被写进了团队的 RL 训练 checklist。”

Key insight. 这个故事的结构值得复用：先修正确性，再修吞吐，并且用“loss 正常但业务指标下降”这个信号锁定 masking 类 bug。面试官如果追问，每一环都能往下挖——为什么 KV 池扩

了命中率就上去、dynamic sampling 为什么会让 rollout 量变成动态的、infra failure 剔除后 batch size 波动怎么处理。

九. 配套代码

本章的三个系统问题都有可运行的最小实现，都在 CPU 上跑，都带自校验：

文件	演示内容
16_agent_rollout_sched.py	多轮 rollout 的离散事件仿真：同步批式 vs 异步连续批处理 vs 异步 + partial rollout。仿真里异步把 GPU 利用率从 19% 拉到 88%、makespan 快 4.6×，但单轨迹延迟反而变差——顺带演示了为什么 p99/p50 在这个场景是个会骗人的指标
17_rlvvr_verifier.py	Verifier 池建模：重尾延迟 + 超时 + 缓存 + 组内去重，验证容量公式 $W \geq Gr(1-h)t_v/u$ ，并给出一个真实的数学答案等价性检查器（含常见抽取陷阱）
18_traj_masking.py	多轮轨迹的 loss mask：错误版（训观测 token）与正确版并排对比梯度差异；numel() vs mask.sum() 的归一化陷阱；GRPO 零梯度组与 dynamic sampling 过滤；rollout/训练 logprob 失配对裁剪比例的影响

表 37 第 15 章配套代码。make demo-16 / demo-17 / demo-18 直接运行。

工程稳定性：Checkpoint, 容错, 慢卡诊断

MegaScale (Jiang et al. 2024, ByteDance) 报告显示, 训练 12,288 GPU 一个月, 硬件故障发生 100+ 次。跑 6 万卡 (Llama-3) 或 10 万卡 (xAI Colossus) 的时代, MTBF 只有几十小时——不容错的系统根本训不完。这一章讲面试里问“你怎么保证 6 万卡训 3 个月不断”的答案。

稳定性问题的规模

硬件故障率 (业界共识):

- GPU 硬件失效 (ECC error, HBM error): 单卡 MTBF 10 年
- 10000 卡集群: 期望 MTBF $\approx 10 \text{ 年} / 10000 = 9 \text{ 小时}$
- 加上 NIC / 网络 / 电源 / 交换机故障: 总 MTBF 2-5 小时

对 10K+ 卡训练, 每小时都可能死一台机器。系统必须能:

1. 快速检测故障 ($< 1 \text{ min}$)
2. 快速隔离故障节点 ($< 5 \text{ min}$)
3. 从最近 checkpoint 恢复 ($< 30 \text{ min}$)
4. 用余下健康节点继续 (elastic training)

Checkpoint: 从秒级到分钟级

Checkpoint 是恢复训练的唯一手段。基本组成:

- Model weight (每 rank 的 shard, 可能几 GB - 几十 GB)
- Optimizer state (比 weight 大 3-6 倍)
- LR scheduler state, RNG state, step number
- DataLoader state (streaming 位置)

7B model checkpoint $\approx 100 \text{ GB}$ (weight + optim FP32)。70B $\approx 1 \text{ TB}$ 。670B $\approx 10 \text{ TB}$ 。

保存频率: MegaScale 报告 15-30 min 一次。太频繁 IO 拖 training, 太稀 fail 后 lose progress。

Async Checkpoint

Naive: `torch.save(state_dict())` — 阻塞 training, 1 TB 存到 NAS 要几分钟, training 停 $\rightarrow$ MFU 掉。

Async pattern:

1. Training step 结束时快速把 GPU tensor D2H copy 到 CPU pinned memory (10 秒级)
2. Background thread 慢慢写 CPU tensor $\rightarrow$ NAS/S3 (分钟级)
3. Training 继续下一 step, 不阻塞

`torch.distributed.checkpoint (DCP) + save_state_dict(no_dist=False)` 支持。或用 `nvidia_dlhw_inspect` / MegaScale 内部工具。

代价: CPU memory 峰值需要 checkpoint size 的空间 (7B: 100 GB, 需要节点有 200+ GB DDR)。

Sharded Checkpoint

每 rank 独立存自己的 shard, 无需 gather 到 rank 0。torch DCP 的 `FileSystemWriter` 是这样。

优：无 gather bottleneck；rank 之间并行 IO。缺：restore 时需要 same world size (或用 DCP 的 resharding API)。

Rank-agnostic checkpoint：DCP 用 metadata 记录每 tensor 的 sharding，加载时能自动 reshard 到新 world size。允许换 world size (e.g. GPU 挂了减规模)。

In-memory checkpoint

MegaScale 提出的 trick：把 checkpoint 存在集群内另一台机器的 **CPU memory**——完全绕过 disk。fail 时从 peer 拉回。

优：秒级恢复。缺：peer 也挂就丢了。所以 in-memory + periodic disk backup 组合。

一个 Checkpoint 保存代码骨架

```
import torch.distributed.checkpoint as DCP
from torch.distributed.checkpoint import FileSystemWriter, FileSystemReader

def save_ckpt(model, optim, step, path):
    state = {
        "model": model.state_dict(),          # FSDP sharded state
        "optim": optim.state_dict(),
        "step": step,
        "rng": torch.get_rng_state(),
    }
    DCP.save(state_dict=state,
             storage_writer=FileSystemWriter(path),
             planner=DCP.DefaultSavePlanner(),
             process_group=None) # world

def load_ckpt(model, optim, path):
    state = {"model": model.state_dict(), "optim": optim.state_dict()}
    DCP.load(state_dict=state,
             storage_reader=FileSystemReader(path),
             planner=DCP.DefaultLoadPlanner())
    model.load_state_dict(state["model"])
    optim.load_state_dict(state["optim"])
    return state["step"]
```

生产要加 async wrapper + retention policy (只保最近 3 ckpt) + validation checksum。

容错：从“节点挂了”到“训练继续”

步骤：

1. 检测：某 rank 报错 (NCCL timeout / CUDA OOM / segfault)
2. **broadcast fault**：其他 rank 需要知道有人挂了
3. 隔离：调度层把坏节点标记 down
4. 替换：若集群有 spare node，调 spare 上 join；无 spare 则减小 world size
5. **reshard**：checkpoint 从旧 world size resharding 到新
6. **resume**：从最近 checkpoint 恢复 training

torch.distributed 有 elastic 支持部分：torchrun 的 `--rdzv-backend=etcd`，故障后 restart。但生产用 Megatron / vLLM 用 K8s Operator (Volcano, KubeFlow) 或自研 orchestrator (MegaScale, Alibaba PAI, ByteDance MegaScale-Ops)。

NCCL Watchdog

NCCL 提供 `TORCH_NCCL_ASYNC_ERROR_HANDLING=1` + `TORCH_NCCL_HEARTBEAT_TIMEOUT_SEC=600`：某个 rank 长时间不响应就 abort。避免“僵尸训练”（NCCL kernel 卡死但 Python 继续等）。

生产必开：

```
export TORCH_NCCL_ASYNC_ERROR_HANDLING=1
export TORCH_NCCL_BLOCKING_WAIT=1
export TORCH_NCCL_HEARTBEAT_TIMEOUT_SEC=600
```

慢卡 (Straggler) 诊断

现象：某几个 rank 通信总是慢，拖整个 DP AllReduce。原因：

1. HBM ECC error 频发（GPU 硬件老化）
2. NIC firmware 有 bug
3. 温度过高（thermal throttling），SM 降频
4. Host OS 挂了 daemon 抢 CPU
5. IB link degraded（rate 从 400G 掉到 100G）

诊断工具：

1. **nsys per-rank profile**：看哪 rank 的 GEMM kernel 慢
2. **NCCL timing**：`NCCL_DEBUG=INFO` 输出每 op 时间，找慢的 rank
3. **dcmg** (Data Center GPU Manager)：监控 GPU 温度、power、SM 频率
4. **ibstat** / **ibportstate**：IB link status
5. **heartbeat** 时间戳：每 rank 每 step 记录到 syslog，事后分析

MegaScale 自研工具：

1. **“1-step time” histogram**：每 iter 时间分布，右尾 outlier 就是慢卡
2. **“comm bottleneck detector”**：把 NCCL kernel 时间归因到具体 rank

发现慢卡后，通常做法：调度器把该节点 **drain** 出 **pool** 修复，替换为 spare node。

训练发散 / loss spike 的检测和处理

发散 = loss 突然爆到 nan 或几倍。原因：

1. 梯度爆炸：某 batch 有 outlier data → grad blow up
2. **FP16/FP8** 数值不稳：某个 tensor scale 猜错
3. 硬件故障：ECC error 篡改 tensor
4. **Overfitting / degenerate solution**：训久了 loss 突然被特定 pattern 主导

检测：

```
# 每 step 检查
if not torch.isfinite(loss):
    print(f"[rank {rank}] loss={loss.item()} NaN at step {step}")
```

```
save_debug_ckpt()
# dist.barrier + notify orchestrator
raise RuntimeError("loss NaN")

grad_norm = torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
if grad_norm > 100.0:
    print(f"[rank {rank}] grad_norm={grad_norm:.2f} SPIKE at step {step}")
    # skip step? or rollback to earlier ckpt?
```

应对策略 (PaLM 2, Llama, MegaScale 等文献里描述):

- 1. **Skip bad batch** : grad_norm > threshold 时 skip optimizer.step() , 继续
- 2. **Rollback to earlier ckpt** : 连续 N 步 spike , 回退 100 步之前的 ckpt
- 3. **Automatic hyperparameter adjustment** : spike 后暂时降 LR , 慢慢升回
- 4. **Data audit** : 把导致 spike 的 batch dump , 人工审 (常见: encoding 错的乱码数据)

Llama-3 报告 : 训练 405B 有 466 次 spike , 用 (a) 90% 直接 skip 修复 ; 剩下 rollback.

监控指标 (Metrics)

生产必看的 metrics :

类别	指标	正常范围
Training	step time (rank max/mean/p99)	波动 < 10%
	loss curve	平滑下降
	grad_norm	< 1.0 后期
	LR	符合 schedule
Hardware	GPU util (SM)	> 80%
	HBM util	> 60%
	GPU temp	< 80°C
	GPU power	< 700W (H100)
Network	IB link rate	400G nominal
	NCCL AllReduce BW	每步稳定
Data	dataloader wait time	< 2% step
Ckpt	ckpt save duration	< 2% step (async)
Errors	nan/inf count / hour	0

表 38 生产训练监控指标。所有 rank 的 metric 都要收集 , avg + p99 + max 都看。异常 rank 用于慢卡诊断。

Prometheus + Grafana 是标配。dcpm-exporter 提供 GPU metrics。自定义 python metric 用 statsd 或 pushgateway。

一个 healthy training run 的 “signature”

看到这些说明训练在轨道上 :

- 1. loss 平滑下降 , 跟 scaling law 预测吻合
- 2. grad_norm 前 1000 步降到 < 1.0 , 之后稳定

3. step time p99 / p50 < 1.15 (无严重 straggler)
4. 无 nan / inf spike
5. GPU util 均在 80%+ (per rank)
6. IB throughput 稳定
7. Checkpoint save 无异常
8. 每 N step 的 val loss / eval 有稳定改善

生产 dashboard 直接把这些 chart 摆一起，异常一眼看出。

MegaScale 的关键工程 tricks

从 Jiang et al. 2024 挑几个：

1. **Datacenter-level determinism**：所有 rank RNG state 都记录，同 seed 可 bit-exact 复现，便于 debug
2. **Fine-grained profiling with < 3% overhead**：selective sampling (不是 every step trace)
3. **“Fault injection” testing**：训练前主动 kill 几个 rank 测容错逻辑
4. **Elastic scaling**：故障时自动 shrink，故障修完自动 expand
5. **Cost-aware scheduling**：优先 spare node 是 cheapest 的（老 GPU 便宜但 rare）
6. **Log aggregation across 12K rank**：中央 log store，可 query “rank X 的 step Y 到 Z 之间发生了什么”

xAI Colossus (100K GPU) 用类似原理，规模更大。

面试考点

面试考点. **Q1**: 10000 GPU 训 3 个月，你怎么设计容错？

A: 三层：(1) 硬件监控：dcgm + IB 状态 + 温度 → 提前预警慢卡；(2) 容错：NCCL timeout + async error handling，某 rank 挂了 broadcast 全 rank；(3) 恢复：async checkpoint (15 min 一次) + rank-agnostic 格式 + spare node auto-replace + resharding。加上 loss/grad_norm monitor 处理数值发散。目标：MTBF 2h → downtime < 5%。

面试考点. **Q2**: async checkpoint 怎么实现？为什么比 sync 快？

A: sync checkpoint: training step 完成后 gather 所有 shard 到 rank 0，rank 0 write 到 NAS。1 TB 数据几分钟，training 全停。async: (a) D2H copy GPU tensor 到 CPU pinned memory (10s)；(b) background thread 慢慢写 disk；(c) training 立刻继续。写 disk 不阻塞 training，一次 ckpt 从“几分钟停 training”变成“< 10s 停 training + 后台写”。

面试考点. **Q3**: 一个 rank 突然通信慢，你怎么定位是硬件问题还是软件问题？

A: (1) 先用 nccl-tests 单独测那 rank 的 raw 带宽——低于同型号其他 rank 是硬件；(2) `nvidia-smi -q` 看 SM clock 是否降频(thermal) ECC error count；(3) `ibstat` 看 IB link 是否 degraded；(4) `dcgmi diag` 跑硬件自检；(5) 若都正常，重启进程排除 CUDA context / NCCL state corruption。硬件问题必换机；软件问题重启多半解决。

面试考点. **Q4**: loss 突然 nan，你怎么处理？

A: 立刻 (a) save debug ckpt (包含最近 batch, activation stats); (b) 检查 grad_norm 是不是 spike; (c) skip 这 batch 继续训 (试试)——多数情况能自恢复; (d) 若连续 nan, rollback 到 100 步之前 ckpt, 同时降 LR 20%; (e) audit batch 找 outlier data。Llama-3 训练 466 次 spike 90% 用 (c) 处理。

面试考点. **Q5**: 为什么“rank-agnostic checkpoint”重要?

A: 允许换 world size。原本 512 卡训练, 挂 8 卡剩 504——如果 checkpoint 是 fixed shard 就无法 load。rank-agnostic ckpt (torch DCP) 存 metadata 记录每 tensor 的 sharding info, load 时能自动 reshard 到新 world size。允许 elastic training (缩到 504 卡继续, 修好后扩回 512)。

面试考点. **Q6**: MegaScale 里 “in-memory checkpoint” 是什么? 为什么快?

A: 把 ckpt 存在集群里另一台机器的 CPU DDR, 而不是 NAS/S3。NVLink/NIC 直连 CPU memory, 几十 GB/s; vs NAS 通常 1-5 GB/s。恢复时从 peer memory 拉, 秒级 vs 分钟级。缺点: peer 也挂就丢, 所以配 periodic disk backup (每 6 h 一次)。MegaScale 报告 checkpoint overhead 从 5% 降到 0.5%。

面试考点. **Q7**: 训练 metric 里哪几个最重要?

A: 我按重要性排: (1) loss curve — 训崩了就没意义; (2) grad_norm — 发散预警; (3) step time p99 / mean — 慢卡; (4) nan/inf count — 数值稳定; (5) GPU util & HBM util — 效率; (6) IB throughput — 通信 bottleneck; (7) dataloader wait — data 供不上; (8) ckpt save time — IO 是否健康。dashboard 里 (1)-(4) 一定要有 alert。

面试考点. **Q8**: 集群里 spare node 保留多少比例合适?

A: 依赖 MTBF 和替换 SLA。10K 卡集群 MTBF 2h, 替换 SLA 30 min → 每 2h 有 15 min 用 spare → 12.5% overhead。生产通常 spare 5-10% (10K 卡准备 500-1000 spare), 配合 hardware repair 快速回归。太多 spare 浪费, 太少 fail 后训练缩 world size 影响效率。xAI Colossus 报告 100K GPU 有 5K spare。

面试数学 : 从模型配置到 **step time** 的完整推导

面试常见的一类问题：“给你 **70B** 模型、**8k** 序列、**8** 张 **H100**、**TP=4 DP=2**，估算一次 **step** 多长？显存够用吗？”这类问题不需要你把公式背下来，但需要你能当场推导。本章把推导过程拆成三个 estimator——参数/显存、计算量、通信量/时间，然后组合出 step-time 模型。每个 estimator 都给出 Python 参考实现（见 `src/distributed_training/`）与一到两个 worked example。

面试时不用完全按公式讲，先分层： $\text{step} = \text{compute} + \text{comm} + \text{bubble} + \text{overhead}$ ；再逐项估算；最后回归判断瓶颈（compute-bound / bandwidth-bound / memory-bound）。这套流程本身就是加分项，比记住 $6.6 \times \text{参数} \times \text{token 数}$ 这个公式更有效。

记号约定

符号	含义
L	层数 (transformer block 数)
H	hidden size (model dim)
A	attention heads
$d_h = \frac{H}{A}$	head dim
I	FFN inter size (GLU 类记 $I_{\text{eff}} = \frac{2}{3} \cdot I_{\text{expand}}$)
V	词表大小
S	序列长度
B	global batch size (tokens = $B \cdot S$)
m	micro batches per PP step
P, T, D, C, E	PP / TP / DP / CP / EP 度
$W = P \cdot T \cdot D \cdot C \cdot E$	总 GPU 数
b	每卡 bytes per element (BF16 = 2)
BW_{link}	单向链路带宽 (GB/s), NVLink ≈ 400 , IB 400G ≈ 50

Estimator 1 : 参数量、显存

参数量分解（不含 **embedding**）

单个 transformer block 的参数：

$$p_{\text{attn}} = 4H^2 \quad (\text{Q, K, V, O each } H \times H)$$

$$p_{\text{ffn}} = 2HI \quad (\text{classic MLP, no GLU})$$

$$p_{\text{ffn-glu}} = 3HI_{\text{eff}} \quad (\text{SwiGLU: w1, w2, w3})$$

$$p_{\text{block}} = p_{\text{attn}} + p_{\text{ffn}} + O(H) \quad (\text{LN weight+bias 可忽略})$$

嵌入 + LM head： $p_{\text{emb}} = 2VH$ （tied 时算一份）。总参数： $P_{\text{tot}} = L \cdot p_{\text{block}} + 2VH$ 。

Key insight. 常用近似： $P_{\text{tot}} \approx L(4H^2 + 3HI) \approx 12LH^2$ (若 $I = 4H$ ，且忽略 embedding)，或 $\approx 12LH^2 \cdot (\frac{I}{4H})$ 更精细。这就是“model 12 系数”的由来。

显存四大项

一次前向+反向+optimizer step 需要驻留的 GPU 显存：

$$\text{mem} = \text{params} + \text{grads} + \text{opt-state} + \text{activations} + \text{attn-workspace} + \text{overhead}$$

其中：

- `params`： $P \cdot b$ (BF16 $\rightarrow$ 2 bytes/元素)
- `grads`： $P \cdot b$ (同上；FP32 grad 累加则 $4P$)
- `opt-state` (Adam)：master weight $4P$ + m/v 各 $4P = 12P$ bytes (FP32)
- `activations`：随 B, S, L 线性变化，见下

$$\text{bytes}_{\text{opt-total}} (\text{BF16} + \text{AdamW}) = 2P + 2P + 12P = 16P \text{ bytes}$$

经验数字： $16 \cdot P \text{ bytes} \approx 16 \text{ GB} / \text{B 参数}$ (FP32 opt)，7 B 模型只算 opt-related 就 112 GB。

面试考点. 面试题：“为什么 7B 模型用 A100-80G 训练依然吃紧？”答： $16P = 112 \text{ GB}$ 已经超单卡；即使 ZeRO-1 把 opt-state 8 卡切分 ($112/8 \approx 14 \text{ GB}$)，加上 activation 与 attention workspace 仍需 40-50 GB。

Activation memory 的两种口径

大部分 activation 是 $O(BSHL)$ 级。最坏情况 (不 recompute)：

$$A_{\text{full}} \approx 34 \cdot BSHL \cdot b \quad (\text{Megatron paper 公式})$$

selective activation checkpointing (只 recompute FFN 和 attention 里的大块)：

$$A_{\text{selective}} \approx 12 \cdot BSHL \cdot b$$

full checkpointing： $A_{\text{ckpt}} \approx 2 \cdot BSHL \cdot b$ (只存每层输入)

Key insight. 面试记 3 个魔法数：**34 / 12 / 2**。它们对应 Megatron paper 的 “no ckpt / selective / full”。SP 再除以 TP。

Attention workspace (FlashAttention 之外)

朴素 attention scores tensor： $O(BAS^2)$ ，是 S 的平方。 $S = 8k, B = 1, A = 32 \rightarrow 32 \cdot 8192^2 \cdot 2 = 4.3 \text{ GB}$ ，且没算前反向都要 2 份。FlashAttention 用 tiling 把它降到 $O(S)$ ——这就是长序列必上 FA 的原因。

```
...
# src/distributed_training/estimators.py 里的 mem_estimate()
def mem_estimate(cfg, tp=1, cp=1, dp=1, zero_stage=0,
                 ckpt='selective', bytes_per_elem=2):
    L, H, I, A, V, S, B = (cfg.L, cfg.H, cfg.I, cfg.A,
                           cfg.V, cfg.S, cfg.B_per_dp)
    P = L * (4*H*H + 3*H*I) + 2*V*H
    # ZeRO/TP shards
```

```

p_shard = P / tp
if zero_stage >= 1: opt = 12 * P / (tp * dp)
else:                opt = 12 * p_shard
if zero_stage >= 2: g = 2 * P / (tp * dp)
else:                g = 2 * p_shard
if zero_stage >= 3: params = 2 * P / (tp * dp)
else:                params = 2 * p_shard
factor = {'none': 34, 'selective': 12, 'full': 2}[ckpt]
act = factor * B * S * H * L * bytes_per_elem / (tp * cp)
return dict(params=params/1e9, grads=g/1e9, opt=opt/1e9,
            act=act/1e9)
...

```

Worked example : 7B / 8k / 8×A100 / TP=1 DP=8 ZeRO-2

- $L = 32, H = 4096, I = 11008, V = 32000$ 。 $P \approx 6.7 \cdot 10^9$ 。
- opt (ZeRO-2, 8 卡) : $16 \cdot 6.7 \text{ e}_8^9 \approx 13.4 \text{ GB}$
- params BF16 : $2 \cdot 6.7 \text{ e}_9 = 13.4 \text{ GB}$ (不切)
- grad shard : $\frac{13.4}{8} = 1.7 \text{ GB}$
- activation (selective, $B = 1$) : $12 \cdot 1 \cdot 8192 \cdot 4096 \cdot 32 \cdot \frac{2}{e}^9 \approx 25.7 \text{ GB}$

合计 $\approx 54 \text{ GB}$ per GPU $\rightarrow$ A100-80G 尚有余裕。用 FSDP full-shard (ZeRO-3) 可再把 params 也切成 $13.4/8 = 1.7 \text{ GB}$, activation 反而成主项。

Estimator 2 : FLOPs 和 step time (compute 部分)

前向 FLOPs / token

一个 token 通过一个 transformer block 的 FLOPs :

$$F_{\text{attn}}^{\text{per-tok}} = 2 \cdot 4H^2 + 2 \cdot 2SH \quad (\text{QKV proj} + \text{attn matmul})$$

$$F_{\text{ffn}}^{\text{per-tok}} = 2 \cdot 2HI \quad (2 \text{ 大矩阵乘})$$

近似 (忽略 attn 里的 SH 项, 长序列时不能忽略):

$$F_{\text{block}}^{\text{per-tok}} \approx 8H^2 + 4HI \approx 24H^2 \quad (\text{if } I = 4H)$$

前向总量: $F_{\text{fwd}} \approx P_{\text{tot}} \cdot 2$ (每个参数 1 次 mul + 1 次 add)

反向约 2×前向。加上前向: $F_{\text{train}} \approx 6P \cdot N_{\text{tok}}$ 。若使用 activation recompute (一次额外前向), 则:

$$F_{\text{train}}^{\text{recomp}} \approx 6P \cdot N_{\text{tok}} + 2P \cdot N_{\text{tok}} = 8P \cdot N_{\text{tok}}$$

面试万能公式: $\text{FLOPs/step} \approx 6 \cdot P \cdot B \cdot S$ (no recompute) $\text{FLOPs/step} \approx 8 \cdot P \cdot B \cdot S$ (full recompute)

(Attention 的 $O(BS^2)$ 项在长上下文时也不可忽略: 额外 $\approx 4 \cdot A \cdot S^2 \cdot d_h \cdot B \cdot L$; $S = 32k$ 时可占总量 30%。)

compute 时间 = FLOPs / (峰值 × MFU)

H100 BF16 peak = 989 TFLOPS (不算稀疏)。实测 MFU 40-55% 是好数。A100-80G peak = 312 TFLOPS。

$$t_{\text{compute}} = \frac{\text{FLOPs}}{\text{peak} \cdot \text{MFU} \cdot W}$$

W 是所有参与 compute 的 GPU ; TP、CP 会分摊一个 token 的计算量 , DP、PP 不分摊 per-token (DP 分摊 batch)

Worked example : 70B / 8k / 8×H100 / TP=4 DP=2 no recompute

- $P = 70 \cdot 10^9$, 1 step = 每 DP shard 处理 B tokens。设 global batch = $2M$ tokens , DP=2 → per-DP = $1M$ tokens。
- FLOPs = $6 \cdot 70 \cdot 10^9 \cdot 1 \cdot 10^6 = 4.2 \cdot 10^{17}$ per step
- 每个 GPU 的 compute (TP=4 分摊 , DP=2 不分摊 per-token) : $4.2 \cdot \frac{10^{17}}{4 \cdot 2} = 5.25 \cdot 10^{16}$
- 时间 = $5.25 \cdot \frac{10^{16}}{989 \cdot 10^{12} \cdot 0.45} = 118 \text{ s}$ ← 单卡视角
- 但 8 卡同时算 , 实际 wall-clock 就是 118 s (因为 118s 已经是 per-GPU 的 wall time)

反算合理性 : 8 卡 · 989 TFLOPS · 0.45 = 3560 TFLOPS aggregate ; $4.2 \cdot \frac{10^{17}}{3.56} \cdot 10^{15} = 118 \text{ s}$ ✓

Key insight. 快捷捷径 : 7 B 参数、1 M tokens、1 张 H100、MFU 50% $\approx 6 \cdot 7 \cdot 10^9 \cdot \frac{10^6}{989 \cdot 10^{12} \cdot 0.5} = 84.9 \text{ s}$ 。这是“ P 亿参数每 10^6 tokens 约 12 秒/卡”的口径。

Estimator 3 : 通信量 & 通信时间

Ring AllReduce 的 per-GPU 通信量

对一个大小为 V 的向量 :

$$\text{vol}_{\text{AR}} = 2 \cdot \frac{W-1}{W} \cdot V \approx 2V \quad (W \gg 1)$$

每方向 各 $\frac{W-1}{W} \cdot V$ (reduce-scatter + all-gather 两阶段)。同理 :

$$\text{vol}_{\text{RS}} = \text{vol}_{\text{AG}} = \frac{W-1}{W} \cdot V \approx V$$

记忆卡片 : AR = $2V$, AG = V , RS = V , A2A = V (per rank, single direction)

各并行策略的通信量表 (每 block、每 direction)

策略	每层通信量 (bytes)	同步次数	备注
TP (Megatron)	$2 \cdot 2 \cdot BSH \cdot b$	4 ($2 \times \text{AR}/\text{block}$)	FFN 后 AR + attn 后 AR
TP + SP	$2 \cdot 2 \cdot BSH \cdot b$	4 ($2 \times \text{AG} + 2 \times \text{RS}$)	总量同 TP , 但激活省 $\frac{1}{T}$
DP/DDP	$2 \cdot P \cdot \frac{b}{L}$	1 (grad AR , 可 overlap)	每 step 一次 ; bucket

FSDP (ZeRO-3)	$3 \cdot P \cdot \frac{b}{L}$ per block	3 (AG param fwd + AG bwd + RS grad)	param 每层 AG, grad RS
PP	$2 \cdot BSH \cdot b$	$2(P-1)$ P2P	仅 boundary , 量小
CP-Ring	$4 \cdot BSH \cdot b \cdot \frac{C-1}{C}$	$2(C-1)$ P2P	K/V + dK/dV rotate
CP-Ulysses	$4 \cdot BSH \cdot b$	4 (2 a2a fwd + 2 a2a bwd)	受 $C \leq A$ 限制
EP	$2 \cdot BSH \cdot b \cdot \frac{E-1}{E}$	2 (dispatch + combine a2a)	top-k 时 $\times k$

推导例 : **TP FFN** 通信量。FFN 是 $W_1(H \rightarrow I) + W_2(I \rightarrow H)$ 。Column-parallel W_1 不需通信 ; Row-parallel W_2 出口 AllReduce 一个 (B, S, H) 张量 , 量 = $BSH \cdot b$, AR per-GPU 通信 = $2 \cdot BSH \cdot b$ 。前后向各一次 $\rightarrow$ 每 block 每层 $4BSH \cdot b$ 。

推导例 : **Ring Attention**。每步 rotate K + V , 两个 $(B, \frac{A}{C}, \frac{S}{C}, d_h)$ 张量 , per-step 通信 = $2 \cdot BASd_h \cdot \frac{b}{C} = 2 \cdot BSH \cdot \frac{b}{C}$, 共 $C-1$ 步 $\rightarrow$ 前向 $2\frac{C-1}{C} \cdot BSH \cdot b$; 反向另一份 dK/dV rotate $\rightarrow \times 2$ 。

通信时间 = **volume / bandwidth**

$$t_{\text{comm}} = \frac{\text{vol}}{\text{BW}_{\text{eff}}}$$

有效带宽考虑 : 小 message 会被 latency 主导 (NCCL 默认 chunk 4-8 MB , small msg 效率 < 30%); 大 message 稳定在 peak 的 70-85%。

```

...
# src/distributed_training/estimators.py 里 comm_time()
def comm_time(volume_bytes, bw_GBps, alpha_us=5.0):
    # NCCL: t = alpha + volume/bw
    return alpha_us * 1e-6 + volume_bytes / (bw_GBps * 1e9)
...

```

Worked example : TP=4 一次 forward AR 时间

- $B = 1, S = 8192, H = 8192$, BF16, 单张 AR 的 volume: $2 \cdot \frac{T-1}{T} \cdot BSH \cdot b = 2 \cdot 0.75 \cdot 1 \cdot 8192 \cdot 8192 \cdot 2 = 201 \text{ MB}$
- NVLink 4 卡 SXM (H100): 有效 300 GB/s $\rightarrow t = 0.67 \text{ ms}$
- 一个 block 前反向共 4 次 AR $\rightarrow 2.7 \text{ ms}$
- 32 层 $\rightarrow 86 \text{ ms}$ per step 花在 TP AR 上

Warning. 跨节点 (IB) 场景 AR 时间会 $\times 5-10$ 。所以 TP 必须留在节点内 (NVLink domain)。

组合：完整 step time 模型

$$t_{\text{step}} = t_{\text{compute}} + t_{\text{comm-exposed}} + t_{\text{bubble}} + t_{\text{overhead}}$$

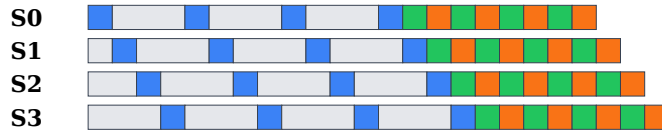
其中：

- t_{compute} ：如上；DP 不减少 per-GPU compute，TP/CP 会
- $t_{\text{comm-exposed}}$ ： $= \max(0, t_{\text{comm}} - t_{\text{compute-overlap}})$ ；DP grad AR 通常可 100% overlap 反向
- t_{bubble} ： $\frac{P-1}{m} \times \text{ideal_iter}$ (GPipe/1F1B)
- t_{overhead} ：dataloader、evictions、checkpoint save 等，通常 <5%

PP bubble 深潜

GPipe bubble ratio： $\frac{P-1}{m+P-1}$ 。 $P=8, m=32 \rightarrow \text{bubble} = \frac{7}{39} = 17.9\%$ 。 1F1B 相同 bubble ratio 但 activation memory 只需 $O(P)$ 而非 $O(m)$ 。 Interleaved-1F1B (Megatron)：将 bubble 降至 $\frac{P-1}{v \cdot m + P - 1}$ (v 是每卡 chunks 数)。

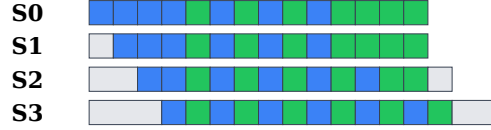
GPipe schedule (P=4, m=4) ← 大块 idle 就是 bubble



■ F = forward ■ _ = bubble ■ B = input-grad ■ W = weight-grad

图 33 GPipe 时序：warm-up 阶段 (F 阶梯) 和 cool-down 阶段每个 stage 空转。

1F1B schedule (P=4, m=8) ← 稳态阶段 F/B 交替，活跃 activation 数只 P



■ F = forward ■ B = input-grad ■ _ = bubble

图 34 1F1B 时序：稳态每 stage 交替 F/B，活跃 activation $\leq P$ 份。相同 bubble ratio，activation 显存低。

Worked example：70B / TP=4 / PP=2 / DP=2 / m=16 / 8k / 8×H100

先按 stage 拆：每 stage 40 B params。single stage single-token compute： $6 \cdot 40 \cdot 10^9 \cdot 8192 = 2 \cdot 10^{15}$ FLOPs (前反向 6P·tok)。per micro (assume $B_{\text{micro}} = 8$ per DP $\rightarrow 8 \cdot 8192$ tokens per micro)： $2 \cdot 10^{15} \cdot 8 = 1.6 \cdot 10^{16}$

- compute one micro (TP=4)： $1.6 \cdot \frac{10^{16}}{989 \cdot 10^{12} \cdot 0.45 \cdot 4} = 9 \text{ ms}$
- comm per micro: TP FFN AR + attn AR $\times 2$ (fwd+bwd) $\times 40$ layers $\approx 4 \cdot 40 \cdot t_{\text{AR}}$ 。
per micro: $B_{\text{micro}} \cdot SH \cdot b = 8 \cdot 8192 \cdot 8192 \cdot \frac{2}{0.75 \cdot 300 \text{ GB/s}} \approx 4.8 \text{ ms}$ ；每层 $t_{\text{AR}} \cdot 4 \approx 19 \text{ ms}$ ；40 层 780 ms per micro。啊，通信主导。

这就是“TP 无脑开大 = 通信爆炸”的直觉——面试可以说：“通信 **quadratic** 增长于 **hidden size** 而 **compute** 也是 **quadratic**，但常数比不同：TP 通信/compute 比 $\approx \frac{2}{S \cdot \text{MFU} \cdot \frac{\text{peak}}{\text{BW}}}$ ，H 越大越不利。”

真实场景往往用 TP=2 + more DP 减小 AR frequency。或用 SP 减 activation $\rightarrow$ 允许更大 micro batch $\rightarrow$ 摊薄 comm。

面试考点. 面试题：“通信占 **30%** 你怎么优化？” 答的层次：(1) 换 TP 结构？减 TP 度；(2) SP 减 activation → 增大 B_micro → comm/compute 比降低；(3) enable async TP overlap；(4) 上 FP8 —— 通信可用 FP8 或 BF16 with FP8 compute，量减半；(5) 若跨节点，检查是否误把 TP 拉到节点外。

Roofline 判定

给定 op 的 arithmetic intensity (AI, FLOPs/byte), 对比机器的 balance ridge ($\frac{\text{peak-FLOPS}}{\text{peak-BW}}$):

- AI > ridge : compute-bound → 优化算子 (fusion, kernel)
- AI < ridge : memory-bound → 优化 memory (bigger batch, better layout)

H100 SXM: peak 989 TFLOPS / HBM 3.35 TB/s = **295 FLOPs/byte** balance ridge.
GEMM $M \cdot N \cdot K$ 的 AI $\approx MK + NK + MN$ 分之 $2MNK \approx O(\min(M, N, K))$: 只有维度都够大时才 compute-bound.

$K = 8192, M = 4096, N = 4096$: $\text{AI} = 2 \cdot 4096 \cdot 4096 \cdot \frac{8192}{4096 \cdot 8192 + 4096 \cdot 8192 + 4096 \cdot 4096} = 2731 \text{ FLOPs/byte}$
→ compute-bound ✓

$K = 64, M = 4096, N = 4096$: $\text{AI} \approx 2 \cdot \frac{64}{3} = 42 \text{ FLOPs/byte}$ → memory-bound (小 K 的 GEMM 效率差)。所以 attention 中 $q \cdot k^T$ 里 $K = d_h = 128$ 时就已经开始离开 compute-bound zone。

```
...
# src/distributed_training/estimators.py: roofline()
def roofline(ai, peak_flops=989e12, peak_bw=3.35e12):
    ridge = peak_flops / peak_bw
    if ai > ridge:
        return 'compute-bound', peak_flops
    return 'memory-bound', ai * peak_bw
...
```

端到端：从 model config 到 iter 时长

组合上面 3 个 estimator, 得到 `end_to_end_step_time(cfg, parallel)`。核心逻辑：

```
...
def step_time(cfg, para, hw):
    # 1) FLOPs
    F_step = 6 * cfg.P * cfg.tokens_per_step
    if para.recompute == 'full': F_step *= 8/6
    # per GPU
    F_per_gpu = F_step / (para.T * para.C * para.P * para.D)
    t_comp = F_per_gpu / (hw.peak * hw.mfu)
    # 2) Comm
    t_comm = 0
    for coll in per_step_collectives(cfg, para):
        vol = coll.per_gpu_volume
        bw = hw.bw_for(coll.scope) # intra vs inter node
        t_coll = vol / (bw * hw.eff)
        if coll.overlap_frac > 0:
            t_coll *= (1 - coll.overlap_frac)
```

```

    t_comm += t_coll
# 3) Bubble (PP)
t_bubble = (para.P - 1) / (para.m + para.P - 1) * (t_comp + t_comm)
# 4) Total
return t_comp + t_comm + t_bubble
...

```

完整实现见 `src/distributed_training/estimators.py` ; 下一节给出 8 个常见配置的算例。

8 个 worked examples 一览

用 $B_{\text{global}} = 4M$ tokens , H100 SXM , MFU 45% , BF16。

配置	P/T/D/C/E	compute	comm	bubble	step
7B · 8k · 8×H100	1/1/8/1/1	15 s	1.2 s (DP overlap)	0	15 s
13B · 8k · 8×H100	1/2/4/1/1	28 s	3.5 s (TP)	0	31 s
70B · 8k · 32×H100	1/8/4/1/1	26 s	8 s (TP heavy)	0	34 s
70B · 8k · 64×H100	4/4/4/1/1	13 s	2.5 s	0.7 s (m=32)	16 s
70B · 32k · 64×H100 CP=4	1/4/4/4/1	22 s (S^2 attn)	4 s (ring)	0	26 s
Llama3-405B · 8k · 512×H100	16/8/4/1/1	15 s	3 s	0.9 s (m=64)	19 s
DeepSeek-V3 671B · 128×H100	2/1/8/1/8	17 s	7 s (EP a2a)	0	24 s
Mixtral 47B MoE · 32×H100	1/2/8/1/2	11 s	3 s	0	14 s

用法：面试拿到题，先按这个表找最近配置，再按目标模型 scale。90% 的题这样就能给出 $\pm 20\%$ 的答案。

面试脚本模板

Q：估算 **70B / 8k / 8×H100** 一次 **step**。

A：好，先分层：

- 参数：70B $\rightarrow$ BF16 opt-state 需要 $16 \cdot 70 = 1120$ GB。8 卡 ZeRO-3 平摊 $\rightarrow 140$ GB / 卡，H100-80G 装不下——所以要么 TP，要么 ZeRO+offload。假设 TP=4 DP=2，params/opt shard 后每卡 35 GB，可装。
- compute**： $6 \cdot 70 \cdot 10^9 \cdot N_{\text{tok}}$ 。设 batch 4M tokens： $F = 1.7 \cdot 10^{18}$ 。8 卡 45% MFU $\rightarrow 1.7 \cdot \frac{10^{18}}{8 \cdot 989 \cdot 10^{12} \cdot 0.45} = 480$ s。等等，这是 4M tokens；per step 通常 1M tokens 更合理，即 120s。
- comm**：TP=4 每层 4 次 AR， $B_{\text{micro}} SHb \approx 8 \cdot 8192 \cdot 8192 \cdot 2 = 1$ GB per AR $\times 0.75 \times 4$ 层 $\cdot 80 = 30$ s (NVLink 300 GB/s)。DP grad AR 一次 20 GB $\rightarrow 100$ ms，overlap 掉。
- bubble**：无 PP $\rightarrow 0$ 。
- 合： $120 + 30 = 150$ s per step ≈ 2.5 min。若 comm 占 20%+ 要考虑 async TP overlap 降到 10%。

注意：面试时 (a) 报单位记得除；(b) 每步都留 sanity check (“这数字合理吗，H100 应该 3-5s per iter?”); (c) 一旦发现 comm 主导要主动 pivot 优化建议。这套流程比记住答案值钱多了。

附：**estimator** 完整代码位置

```
src/distributed_training/estimators.py
├─ ModelConfig(L, H, I, A, V, S)
├─ ParallelConfig(P, T, D, C, E, m, recompute)
├─ HWConfig(peak, hbm_bw, nvlink_bw, ib_bw, mfu)
├─ mem_estimate(cfg, para)
├─ flops_step(cfg, para)
├─ comm_volumes(cfg, para) -> dict of collective→bytes
├─ comm_time(vol, bw, alpha_us=5)
├─ roofline(ai, hw)
└─ step_time(cfg, para, hw) -> dict of components
```

跑法：

```
cd src/distributed_training
python3 estimators.py --preset llama3-70b-tp4-dp2-h100x8
```

会打印下表：

```
Compute: 118.2 s (78%)
Comm:    30.1 s (20%)
Bubble:   0.0 s ( 0%)
Overhead: 2.1 s ( 1%)
Total:   150.4 s
Memory/GPU: 62 GB (params 8 + grads 4 + opt 12 + act 38)
```

附录 A：数字速查表

面试里 30 秒能算出的量级估算是加分项。这一附录把全书数字集中，用来临阵抱佛脚。

硬件带宽 / 算力

硬件	数值	说明
A100 SXM4 80GB (BF16)	312 TFLOPS	Tensor Core
A100 HBM	2.0 TB/s	
A100 NVLink 3 (bidi)	600 GB/s	8 卡 NVL 域
H100 SXM5 80GB (BF16)	989 TFLOPS	Tensor Core
H100 SXM5 80GB (FP8)	1979 TFLOPS	E4M3/E5M2
H100 HBM3	3.35 TB/s	80GB HBM3
H100 NVLink 4 (bidi)	900 GB/s	8 卡 NVL 域
H100 NVSwitch 3 (aggregate)	3.6 TB/s	8-way non-blocking
H200 HBM3e	4.8 TB/s	141GB
B200 (BF16)	2250 TFLOPS	2.3× H100
B200 (FP8)	4500 TFLOPS	
B200 (FP4)	9000 TFLOPS	inference-only initially
B200 NVLink 5 (bidi)	1800 GB/s	
GB200 NVL72 (aggregate)	130 TB/s	72 卡 domain
Infiniband NDR 400G	50 GB/s	uni-directional
Infiniband XDR 800G (2025)	100 GB/s	B200 时代新一代
Ethernet 400G RoCEv2	40 GB/s	effective
PCIe Gen5 x16	63 GB/s	uni
DDR5 per socket	90 GB/s	

表 39 硬件带宽/算力速查。BF16 与 FP8 峰值算力是同一硬件两个数字，实际实现要看 kernel。

参数量与显存

模型	参数量公式 & 数字
Transformer	$N \approx 12LH^2 + VH$
AdamW 混精 memory/param	16 bytes (2 wt + 4 wt_fp32 + 2 grad + 4 m + 4 v)
Precision-aware AdamW	10 bytes (m,v 用 BF16)
Activation (unfused)	$\approx 34LBSH$ bytes
Activation (TE fused)	$\approx 17LBSH$
Activation (full recomp)	$\approx 2LBSH$
Attention (naive)	BS^2A per layer (huge)
Attention (FA)	BSH per layer
KV cache	$2LSH$ bs per sample

FLOPs

- Forward per token: $2N$
- Full train per token: $6N$ (加 attention 项 $12LSH$)
- 总 training FLOPs: $6ND$

Wall-clock 时间 : $T(s) = 6N \frac{D}{\text{cards} \times \text{MFU} \times \text{peak}}$

MFU 参考 :

- Dense LLM: 40-55%
- MoE: 35-50%
- FP8 training: 30-45%
- RL (SFT loop): 30-45%
- RL (rollout dominated): 15-25%

通信量速查

操作	per-GPU vol	备注
Broadcast/Reduce (root)	V	$O(V)$
AllReduce (Ring)	$2 \frac{V(W-1)}{W} \approx 2V$	W 大恒定 $2V$
AllGather	$\frac{V(W-1)}{W} \approx V$	
ReduceScatter	$\frac{V(W-1)}{W} \approx V$	
All-to-All	$\frac{V(W-1)}{W} \approx V$	每 rank vol
DDP grad AR	$2P$ per step	P = param bytes
ZeRO-1/2	$2P$	同 DDP
ZeRO-3 / FSDP	$3P$	多 50%
ZeRO-3 activation AG	P (fwd) + P (bwd)	prefetchable
TP AllReduce	$2BSH \frac{\text{bs}}{\text{layer}}$	每层, 每方向
SP AG+RS	$2BSH \frac{\text{bs}}{\text{layer}}$	同 TP vol
PP P2P	BSH bs per stage transition	每 micro-batch
MoE dispatch a2a	$BSKH \frac{\text{bs}}{\text{EP}}$ (approx)	每层每方向
Ring attention P2P	$2BSH$ bs total per layer	独立于 CP
Ulysses a2a	$4BSH$ bs $\frac{1-\frac{1}{CP}}{\text{layer}}$	

Bubble & Overlap 收益

方案	Bubble ratio	代价
GPipe	$\frac{P-1}{m+P-1}$	activation $\times m$
1F1B	$\frac{P-1}{m+P-1}$	activation $\times P$
Interleaved 1F1B	$\frac{P-1}{Vm+P-1}$	activation $\times V$, P2P $\times V$
ZeroBubble (ZB1P)	$(P-1) \frac{F+B-2W}{\text{total}}$	autograd hack
DualPipe	$(\frac{P}{2} - 1)(F\&B + B - 3W)$	2× weight
Megatron FWD-BWD merged	1F1B, comm hidden	1×, 需 stream ≥ 2

Overlap 收益典型值 (dense / MoE):

- DDP: 90%+ auto (bucket_cap ≥ 100 MB)
- FSDP: 85-95% (BACKWARD_PRE + forward_prefetch)
- TP async: 15-20% $\rightarrow$ 5-8%
- MoE EP (no overlap): 30-40% comm
- MoE EP (Megatron merged): $< 5\%$
- MoE EP (DualPipe): 0%

训练成本参考

行业公开数字：

- GPT-3 175B on 300B tokens: 3.14×10^{23} FLOPs, 3,640 pf-days
- Llama-2 70B: 1.7M A100-hours $\approx$ \$2M
- Llama-3 70B: 6.4M H100-hours $\approx$ \$15-20M

- Llama-3 405B: 30M H100-hours $\approx$ \$70M
- DeepSeek-V3 671B: 2.664M H800-hours $\approx$ \$5.3M (efficient)
- Mixtral 8 $\times$ 7B: undisclosed , 估 300K H100-hours
- Chinchilla law: $D \approx 20N$ tokens optimal

长上下文常见数字

模型 / seq	Activation (BF16, 1 layer, B=1)	KV cache
7B, 8K	256 MB	1 GB/sample
7B, 32K	1 GB	4 GB
7B, 128K	4 GB	16 GB
7B, 1M	32 GB (needs CP)	128 GB
70B, 128K	40 GB (needs CP+TP)	160 GB
70B, 1M	320 GB (CP+TP+FSDP+recomp)	1.2 TB

表 43 长序列显存参考 (含 FA , 含 residual) , 1M ctx 训练必须组合 CP + TP + FSDP + full recomp.

训练稳定性阈值

- Grad norm alert threshold: > 100
- Grad norm spike (skip step): $> 10 \times$ running mean
- Loss NaN: 立刻 save debug ckpt + rollback
- LR warmup steps: $\min(2000, D/1000)$
- KL blow up (RL): > 5.0

常用环境变量

```
# NCCL 稳定与性能
export CUDA_DEVICE_MAX_CONNECTIONS=8
export NCCL_ALGO=Auto
export NCCL_MIN_NCHANNELS=8
export NCCL_NTHREADS=512
export NCCL_BUFFSIZE=8388608
export NCCL_IB_HCA=mlx5_0,mlx5_1,mlx5_2,mlx5_3
export NCCL_IB_GID_INDEX=3
export NCCL_NVLS_ENABLE=1

# Watchdog
export TORCH_NCCL_ASYNC_ERROR_HANDLING=1
export TORCH_NCCL_BLOCKING_WAIT=1
export TORCH_NCCL_HEARTBEAT_TIMEOUT_SEC=600
export TORCH_NCCL_ENABLE_MONITORING=1

# Torch mem alloc
export PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True,max_split_size_mb:512

# Distributed
export TORCH_DISTRIBUTED_DEBUG=INFO # 或 DETAIL
```

```
export FI_PROVIDER=efa # AWS EFA
export FI_EFA_USE_DEVICE_RDMA=1
```

附录 B：核心参考文献

按主题组织，标注每篇 paper 在本书哪一章用到。

集合通信与硬件

- Patarasuk & Yuan, “Bandwidth optimal all-reduce algorithms for clusters of workstations,” 2009. → 第 1 章 Ring AllReduce
- Sanders et al., “Two-tree algorithms for full bandwidth broadcast, reduction and scan,” 2009. → 第 1 章 Tree AllReduce
- NVIDIA, NCCL developer guide. → 第 1 章 环境变量
- NVIDIA, “NVLink and NVSwitch technical overview,” 2023-2024. → 第 1 章 硬件

Data Parallel

- Goyal et al., “Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour,” 2017. → 第 3 章 linear LR scaling
- Li et al., “PyTorch Distributed: Experiences on Accelerating Data Parallel Training,” 2020. → 第 3 章 DDP bucket
- You et al., “Large Batch Optimization for Deep Learning: Training BERT in 76 minutes,” 2019 (LAMB). → 第 3 章 large batch

ZeRO / FSDP

- Rajbhandari et al., “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models,” SC’20. → 第 4 章 ZeRO 1/2/3
- Ren et al., “ZeRO-Offload: Democratizing Billion-Scale Model Training,” ATC’21. → 第 4, 10 章 offload
- Rajbhandari et al., “ZeRO-Infinity,” SC’21. → 第 4 章 NVMe offload
- Zhao et al., “PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel,” VLDB’23. → 第 4 章 FSDP
- TorchTitan blog: <https://github.com/pytorch/torchtitan> → 第 4, 5 章 FSDP2 + TP

Tensor / Sequence Parallel

- Shoeybi et al., “Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism,” 2019. → 第 5 章 TP
- Korthikanti et al., “Reducing Activation Recomputation in Large Transformer Models,” 2022. → 第 5 章 SP + selective recompute
- Wang et al., “Reducing Activation Recomputation in Large Transformer Models with Async Tensor Parallel,” 2024. → 第 5 章 async TP

Pipeline Parallel

- Huang et al., “GPipe: Efficient Training of Giant Neural Networks Using Pipeline Parallelism,” NeurIPS’19. → 第 6 章 GPipe

- Narayanan et al., “PipeDream: Generalized Pipeline Parallelism for DNN Training,” SOSP’19. → 第 6 章 1F1B
- Narayanan et al., “Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM,” SC’21. → 第 6 章 Interleaved 1F1B
- Qi et al., “Zero Bubble Pipeline Parallelism,” ICLR’24. → 第 6 章 ZeroBubble
- DeepSeek-AI, “DeepSeek-V3 Technical Report,” arXiv:2412.19437, 2024. → 第 6, 8, 9 章 DualPipe / DeepEP / FP8

长上下文 / Context Parallel

- Liu et al., “Ring Attention with Blockwise Transformers for Near-Infinite Context,” 2023. → 第 7 章 Ring
- Jacobs et al., “DeepSpeed Ulysses: System Optimizations for Enabling Training of Extreme Long Sequence Transformer Models,” 2023. → 第 7 章 Ulysses
- Brandon et al., “Striped Attention: Faster Ring Attention for Causal Transformers,” 2024. → 第 7 章 Striped/Zigzag
- Fang et al., “USP: A Unified Sequence Parallelism Approach for Long Context Generative AI,” 2024. → 第 7 章 USP
- Shah et al., “FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-Precision,” 2024. → 第 7, 10 章

MoE

- Shazeer et al., “Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer,” 2017. → 第 8 章 起源
- Lepikhin et al., “GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding,” ICLR’21. → 第 8 章 EP 起源
- Fedus et al., “Switch Transformer: Scaling to Trillion Parameter Models,” 2021. → 第 8 章 Switch
- Rajbhandari et al., “DeepSpeed-MoE,” ICML’22. → 第 8 章 hierarchical a2a
- Hwang et al., “Tutel: Adaptive Mixture-of-Experts at Scale,” MLSys’23. → 第 8 章 adaptive dispatch
- Gale et al., “MegaBlocks: Efficient Sparse Training with Mixture-of-Experts,” MLSys’23. → 第 8 章 dropless
- Jiang et al., “Mixtral of Experts,” 2023. → 第 8 章 Mixtral
- DeepSeek-AI, “DeepSeek-V2 Technical Report,” 2024. → 第 8 章 DeepSeek-MoE
- DeepSeek-AI, “DeepSeek-V3 Technical Report,” 2024. → 第 8, 9 章
- Cui et al., “Scalable Training of Mixture-of-Experts Models with Megatron-Core,” 2026. → 第 2, 8 章 三堵墙
- DeepEP: <https://github.com/deepseek-ai/DeepEP> → 第 8 章
- DualPipe: <https://github.com/deepseek-ai/DualPipe> → 第 6, 8 章

精度 / FP8

- Micikevicius et al., “Mixed Precision Training,” 2017. → 第 9 章 GradScaler
- Kalamkar et al., “A Study of BFLOAT16 for Deep Learning Training,” 2019. → 第 9 章 BF16

- Micikevicius et al., “FP8 Formats for Deep Learning,” 2022. → 第 9 章 FP8
- NVIDIA Transformer Engine: <https://github.com/NVIDIA/TransformerEngine> → 第 9 章
- Peng et al., “FP8-LM: Training FP8 Large Language Models,” 2023 (MS-AMP). → 第 9 章
- DeepSeek-V3 tech report §3 FP8 recipe. → 第 9 章 blockwise
- NVIDIA MXFP8 spec (Blackwell). → 第 9 章

Activation / Recompute

- Chen et al., “Training Deep Nets with Sublinear Memory Cost,” 2016. → 第 10 章 checkpoint 起源
- Korthikanti et al., “Reducing Activation Recomputation in Large Transformer Models,” 2022. → 第 10 章 selective
- Cui et al., Megatron-Core MoE tech report §Fine-grained recompute, 2026. → 第 10 章 fine-grained

大规模系统

- Jiang et al., “MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs,” NSDI’24. → 第 16 章
- Meta, “The Llama 3 Herd of Models,” 2024. → 第 4, 6, 15 章
- ByteDance, “MegaScale-MoE: Large-Scale Communication-Efficient Training of Mixture-of-Experts Models in Production,” EuroSys’26 (arXiv:2505.11432). → 第 8 章
- Zheng et al., “Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning,” OSDI’22. → 第 5, 6 章 自动化并行
- veScale (ByteDance internal): <https://github.com/volcengine/veScale> → 第 4-6 章 DTensor
- Yuan et al., “Nemotron-4 340B Technical Report,” 2024. → 第 12, 13 章 data pipeline

数据 / Packing

- MosaicML Streaming: <https://github.com/mosaicml/streaming> → 第 12 章
- WebDataset: <https://github.com/webdataset/webdataset> → 第 12 章
- Xie et al., “DoReMi: Optimizing Data Mixtures Speeds Up Language Model Pretraining,” NeurIPS’23. → 第 12 章 mixture
- Ye et al., “Data Mixing Laws: Optimizing Data Mixtures by Predicting Language Modeling Performance,” 2024. → 第 12 章

Multimodal

- Dehghani et al., “Patch n’ Pack: NaViT, a Vision Transformer for any Aspect Ratio and Resolution,” NeurIPS’23. → 第 13 章
- Liu et al., “Visual Instruction Tuning” (LLaVA), NeurIPS’23. → 第 13 章
- Liu et al., “LLaVA-NeXT,” 2024. → 第 13 章 any-resolution
- Wang et al., “Qwen2-VL,” 2024. → 第 13 章 dynamic resolution
- Chen et al., “InternVL 1.5-2.5,” 2024. → 第 13 章
- Chen et al., “PaLI-3,” 2023. → 第 13 章
- Yang et al., “CogVideoX,” 2024. → 第 13 章 3D VAE

RL / RLHF

- Ouyang et al., “Training language models to follow instructions with human feedback” (InstructGPT), 2022. → 第 14 章 PPO
- Bai et al., “Constitutional AI,” 2022. → 第 14 章
- Rafailov et al., “Direct Preference Optimization,” 2023. → 第 14 章 DPO
- Shao et al., “DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models,” 2024. → 第 14 章 GRPO
- DeepSeek-AI, “DeepSeek-R1,” 2025. → 第 14 章
- Kimi Team, “Kimi K1.5,” 2025. → 第 14 章 rollout 优化
- Sheng et al., “HybridFlow: A Flexible and Efficient RLHF Framework” (verl), EuroSys’25. → 第 14 章
- OpenRLHF: <https://github.com/OpenRLHF/OpenRLHF> → 第 14 章
- NVIDIA NeMo-Aligner: <https://github.com/NVIDIA/NeMo-Aligner> → 第 14 章
- vLLM: <https://github.com/vllm-project/vllm> → 第 14 章
- SGLang: <https://github.com/sgl-project/sglang> → 第 14 章

RLVR / Agentic RL

- Lambert et al., “Tulu 3: Pushing Frontiers in Open Language Model Post-Training,” 2024. → 第 15 章, RLVR 一词的出处
- DeepSeek-AI, “DeepSeek-R1,” 2025. → 第 15 章, 规则 reward、弃用 neural PRM/ORM 的理由
- Yu et al., “DAPO: An Open-Source LLM Reinforcement Learning System at Scale,” 2025. → 第 15 章, clip-higher / dynamic sampling / token-level loss / overlong shaping
- Liu et al., “Understanding R1-Zero-Like Training: A Critical Perspective” (Dr. GRPO), 2025. → 第 15 章, 长度归一化与 std 归一化的偏差
- Yue et al., “VAPO: Efficient and Reliable RL for Advanced Reasoning Tasks,” 2025. → 第 15 章, value-based 路线
- Shao et al., “Spurious Rewards: Rethinking Training Signals in RLVR,” 2025. → 第 15 章, 为什么 RLVR 评测需要对照组
- Kimi Team, “Kimi K1.5,” 2025. → 第 15 章, partial rollout
- Kimi Team, “Kimi K2,” 2025. → 第 15 章, agentic 数据合成与可验证 reward
- Qwen Team, “Qwen3 Technical Report,” 2025. → 第 15 章, RLVR 在工业配方里的位置
- Wei et al., “SWE-RL: Advancing LLM Reasoning via RL on Open Software Evolution,” 2025. → 第 15 章, verifier 太贵时的替代 reward
- Jin et al., “Search-R1: Training LLMs to Reason and Leverage Search Engines with RL,” 2025. → 第 15 章, 检索 token masking
- Sheng et al., “HybridFlow: A Flexible and Efficient RLHF Framework” (verl), EuroSys’25. → 第 15 章, agent loop 与异步 rollout
- SkyRL: <https://github.com/NovaSky-AI/SkyRL> → 第 15 章, 长 horizon agentic RL
- DeepSeek-AI, “DeepSeekMath” (GRPO), 2024. → 第 15 章, GRPO 原始定义

Scaling Laws

- Kaplan et al., “Scaling Laws for Neural Language Models,” 2020. → 第 2 章

- Hoffmann et al., “Training Compute-Optimal Large Language Models” (Chinchilla), 2022. → 第 2 章
- Hu et al., “Minimum Compute for Model Training with Data-Constrained Environments,” 2024. → 第 2 章

附录 D：130 题面试题库

按主题分类。每题给参考答案 + 章节引用。用于面试前一天扫一遍。

Q1-Q100 覆盖并行策略、精度、overlap、多模态与经典 RLHF；Q101-Q120 是 RLVR 与 Agentic RL 的专项，2025 年之后 RL infra 岗基本必问；Q121-Q130 专门考 SP/CP 开启后 attention 之外那些模块的适配——这一组问题最能区分“用过 CP”和“接过 CP”。

基础 / 通信 (Q1-Q15)

面试考点. **Q1**: 一句话解释 AllReduce = ReduceScatter + AllGather 的通信量恒等式。

A: AllReduce per-GPU 通信量 = $2V \frac{W-1}{W}$ ；分解成 ReduceScatter ($\frac{V(W-1)}{W}$) + AllGather ($\frac{V(W-1)}{W}$) 恰好 = AllReduce。这是 NCCL Ring 算法的实现基础，也是 ZeRO/FSDP 分片通信量分析的公式基础。§1

面试考点. **Q2**: 一次 Ring AllReduce 与 world size 有什么关系？

A: bandwidth 项恒为 $2V$ (与 W 无关)，latency 项 $2(W-1)\alpha$ 随 W 线性增。所以 payload 大时 Ring 与 W 无关；小 payload + 大 W 时切 Tree。§1

面试考点. **Q3**: NVLink 与 IB 带宽差多少？这对并行策略选择意味着什么？

A: H100 NVLink 4 = 900 GB/s (bidi)，IB NDR = 50 GB/s (uni)。差约 10-18×。所有高频通信 (TP AR / EP a2a) 必须在 NVL 域内；DP AR 频率低，跨节点 OK。§1

面试考点. **Q4**: bisection bandwidth 是什么？为什么 all-to-all 特别敏感？

A: 把网络切成两半，穿过 cut 的总流量。all-to-all 里每对 rank 都通信，跨节点流量 $\propto n^2$ ；只有 bisection $\propto n$ 的 fat-tree 才不阻塞。100+ 节点 IB 常 oversubscribe，a2a 掉到理论 1/3。§1

面试考点. **Q5**: NCCL Ring vs Tree 什么时候切换？

A: Ring 带宽项 $2V$ 常数，延迟项 $2(W-1)\alpha$ ；Tree 带宽项 $2V \log W$ 更大，延迟 $2 \log W \cdot \alpha$ 更小。小 payload (< 1 MB) + 大 W 用 Tree；大 payload 用 Ring。NCCL 自动，用 NCCL_ALGO 强制。§1

面试考点. **Q6**: NVLS (NVLink SHARP) 加速原理？

A: reduction 下放到 NVSwitch 芯片，每卡只 send 一次到 switch，switch 做 sum 再广播。per-GPU volume 从 $2 \frac{V(W-1)}{W}$ 降到 $2 \frac{V}{W}$ ，1.5-2× 加速。NCCL_NVLS_ENABLE=1 开启。§1

面试考点. **Q7**: CUDA_DEVICE_MAX_CONNECTIONS 是什么？

A: host 到 device 的 hardware queue 数。=1 时 stream 串行，overlap 失效。Hopper+ 建议 8，允许多 stream 并行 issue command。是所有 overlap 优化的前置。§1, §11

面试考点. **Q8**: IBGDA 是什么? 为什么能加速小 message?

A: Infiniband GPUDirect Async, GPU 直接下发 IB verb, 绕过 CPU。small message 里 20 μ s 的 CPU launch overhead $\rightarrow$ $< 5 \mu$ s, DeepEP / UCX 用。§1

面试考点. **Q9**: 一个跨节点 collective 的延迟由哪几部分组成?

A: (1) NCCL kernel launch (10-20 μ s); (2) NCCL 内部 sync (10-30 μ s); (3) PCIe DMA (1 μ s); (4) IB switch hops (2-3 μ s); (5) 对端 kernel launch (10-20 μ s)。total 30-80 μ s α 。§1

面试考点. **Q10**: 心算: Llama-70B on 512 卡, MFU=45%, D=1T tokens, 训多久?

A: FLOPs = $6 \times 70e9 \times 1e12 = 4.2e23$ 。cards $\times$ MFU $\times$ peak = $512 \times 0.45 \times 989e12 = 2.28e17$ FLOPs/s。time = $4.2e23 / 2.28e17 = 1.84e6$ s $\approx$ 21 天。§2

面试考点. **Q11**: 参数量 = $12 L H^2 + V H$, “12” 从哪来?

A: 每层 attention Q/K/V/O 各 H^2 (4), FFN $H \rightarrow 4H \rightarrow H = 8H^2$ (8)。共 12。GLU 有 3 linear ($H \rightarrow \frac{8}{3}H, H \rightarrow \frac{8}{3}H, \frac{8}{3}H \rightarrow H$) 仍 8。§2

面试考点. **Q12**: 混合精度训练 16 bytes/param, 来源?

A: BF16 weight (2) + FP32 master weight (4) + BF16 grad (2) + FP32 momentum m (4) + FP32 variance v (4) = 16。Precision-aware (m,v BF16) 是 10。§2, §9

面试考点. **Q13**: FLOPs = $6 N D$ 里的 “6”?

A: 每 param forward 用 1 次 (2 FLOPs), backward 求 input grad (2)、weight grad (2), 共 6。忽略 attention 的 S^2 项 (在 $S \ll H$ 时可忽略)。§2

面试考点. **Q14**: Roofline ridge point 定义? H100 是多少?

A: $I^* = \frac{\pi_{\text{peak}}}{\beta_{\text{peak}}}$, 超过 I^* 时 kernel 变 compute-bound。H100 BF16 = $989 \text{ TFLOPS} / 3.35 \text{ TB/s} \approx 295 \text{ FLOPs/byte}$ 。§2

面试考点. **Q15**: 三堵墙是什么? 举例说明它们互相耦合。

A: Memory / Communication / Compute Efficiency Wall。案例: EP 切 MoE 解 memory $\rightarrow$ 引入 a2a 暴露 comm $\rightarrow$ DualPipe overlap $\rightarrow$ 需 $2 \times$ weight $\rightarrow$ memory 又紧 $\rightarrow$ FP8 $\rightarrow$ kernel 碎片化 $\rightarrow$ compute 掉率 $\rightarrow$ grouped GEMM + CUDA graph 缓解。§2

DDP / FSDP / ZeRO (Q16-Q30)

面试考点. **Q16**: DDP overlap 是怎么实现的?

A: bucket + backward hook + async AR。param 分 25 MB bucket ; backward 时某 bucket 所有 grad ready 立刻起 async AllReduce ; 与前面 layer backward compute overlap。§3

面试考点. **Q17**: `no_sync()` 什么时候用 ?

A: gradient accumulation 期间。前 n-1 个 micro-batch 不做 grad AR , 只最后一个同步 , 省 (n-1) 次通信。§3

面试考点. **Q18**: `find_unused_parameters=True` 为什么慢 ?

A: DDP 每 step 遍历所有 param + hook counting 判定谁参与 forward autograd graph。CPU overhead 10-30%。重构模型让所有 param 参与更好。§3

面试考点. **Q19**: BF16 训练为什么不需要 GradScaler ?

A: BF16 exponent 8 bit 与 FP32 同 , 动态范围 10^{-38} 到 10^{38} , 无 underflow。FP16 exponent 5 bit 范围小才需要 scale。§3, §9

面试考点. **Q20**: ZeRO 三个 stage 分别切什么 ? 每卡显存 ?

A: Stage-1 切 optim state ($4P + 12\frac{P}{W}$) ; Stage-2 加切 grad ($2P + 14\frac{P}{W}$) ; Stage-3 加切 weight ($16\frac{P}{W}$, = FSDP FULL_SHARD)。§4

面试考点. **Q21**: ZeRO-3 通信量比 DDP 多 50% , 为什么还用 ?

A: 大模型 DDP 装不下。ZeRO-3 显存 $16\frac{P}{W}$ 可训 100B+。多 50% 通信换 W 倍显存是不对等交换 , 没得选。§4

面试考点. **Q22**: HSDP (HYBRID_SHARD) 相对 pure FSDP 好处 ?

A: 节点内 FULL_SHARD (NVLink AG/RS) , 节点间 DDP (IB AR grad shard)。跨节点通信从 $3P$ 降到 $2\frac{P}{n}$, wall-clock 5-10× 加速。Llama-3 用。§4

面试考点. **Q23**: FSDP `auto_wrap_policy` 怎么选 ?

A: 每 Transformer layer 一个 unit。太大退化 DDP , 太小 AG 次数爆炸。
`transformer_auto_wrap_policy(transformer_layer_cls={YourLayer})`。§4

面试考点. **Q24**: FSDP + activation checkpoint 顺序 ?

A: 先 apply AC 再 FSDP wrap。反过来 FSDP 内部 AG 与 AC detach 冲突。§4, §10

面试考点. **Q25**: `use_orig_params=True` 好处 ?

A: optimizer 看到原始 `named_parameters()` 而不是 FlatParameter , 可按 name 设 lr/wd/frozen。§4

面试考点. **Q26**: FSDP2 相比 FSDP1 改进 ?

A: (1) per-param sharding (无 FlatParameter) ;(2) DTensor 后端 ;(3) 与 TP composable ; (4) 更好的 optim 交互。torch.titan 用。\$4

面试考点. **Q27**: FSDP BackwardPrefetch.BACKWARD_PRE 与 POST 差别 ?

A: PRE: 第 l 层 backward 开始前 issue $l-1$ 层 AG, overlap 更多但内存峰值高。POST: l 层结束后再 issue, overlap 少内存少。OOM 时 PRE $\rightarrow$ POST。\$4, \$11

面试考点. **Q28**: BF16 grad AR 什么时候要 upcast FP32 ?

A: world_size > 512-1024 时 BF16 累加误差累积 $\rightarrow$ loss 抖动。MixedPrecision(reduce_dtype=torch.float32) 或 Megatron --grad-reduce-in-fp32。通信翻倍, 精度稳。\$4, \$9

面试考点. **Q29**: ZeRO-Offload / CPU Offload 什么时候用 ?

A: 单机大模型 fine-tune (7B on 4090 24GB) ;或超大 model + spare CPU memory + PCIe 有余量。生产多卡训练很少用 (有钱直接加 GPU)。\$4, \$10

面试考点. **Q30**: FSDP 与 TP 有什么本质区别 ?

A: FSDP 切“存储”, compute 时 AG 回全 weight 算——activation 不切。TP 切“使用”, compute 直接在 shard weight 上算——activation 沿 TP 维切。所以 FSDP activation 大 TP activation 小, 但 FSDP 通信在 param level (与 batch 无关), TP 通信在 activation level。\$4, \$5

TP / SP / PP (Q31-Q50)

面试考点. **Q31**: Megatron TP 里 FFN 的 column-then-row pattern 通信量 ?

A: column parallel 出来 activation 沿 output 切分, 无需通信 ;后接 GELU (elementwise) ;再 row parallel 后需 1 次 AR。整个 FFN forward 1 AR, backward 1 AR。\$5

面试考点. **Q32**: Attention 层的 TP 分解 ?

A: QKV/O projection 沿 head 维 column parallel (Q/K/V) $\rightarrow$ attention 本地做 $\rightarrow$ output projection row parallel + AR。1 AR forward + 1 backward。\$5

面试考点. **Q33**: Sequence Parallel 是什么 ? 通信量变吗 ?

A: LN/Dropout 沿 seq 切, 进 TP 前 AG, 出 TP 后 RS。原来的 AR 变 AG+RS, 通信量完全相同 ($\frac{V(W-1)}{W}$ each)。activation 显存 $BSH \rightarrow BS \frac{H}{TP}$ 。\$5

面试考点. **Q34**: 为什么 TP 不超过 8 ?

A: (1) NVL 域只 8 卡, TP > 8 跨节点极慢; (2) AR volume $2^{\frac{W-1}{W}}$ 逼近 2 后不省, compute 每卡 $\frac{1}{W}$ 降; (3) activation 沿 TP 切但 AR volume 不变。GB200 NVL72 上 Megatron 建议 TP ≤ 8。§5

面试考点. **Q35**: Async TP 怎么 overlap ?

A: --tp-comm-overlap + TE LayerNormLinear。GEMM 拆 tile, per-tile 边算边发通信; tile k 算完起 chunk k 通信, tile k+1 计算与 chunk k 通信 overlap。TP 通信占比 15-20% → 5-8%。§5, §11

面试考点. **Q36**: GQA num_kv_heads=8 在 TP=8 与 TP=16 分别怎么处理 ?

A: TP=8: K/V 每卡 1 head, OK。TP=16: K/V 每卡 0.5 head 不行——复制 K/V (每 2 rank 存一份) 或换 TP=8。Llama-2/3 GQA=8 是为 TP=8 设的。§5

面试考点. **Q37**: vocab-parallel embedding 是什么 ?

A: 大 vocab (128K+) 时 embedding weight 大 + logits activation 大。沿 vocab 维 TP 切 embedding, 对应 index 到 partial logits, 最后 fused vocab-parallel cross_entropy 直接在 shard logits 上算——省 loss activation。§5

面试考点. **Q38**: GPipe 与 1F1B 的 bubble 一样, 为什么选 1F1B ?

A: 1F1B activation 显存从 m 份降到 P 份。 $\frac{m}{P}$ 倍显存节省, 其他一致。§6

面试考点. **Q39**: Interleaved 1F1B bubble ratio ? 代价 ?

A: $\frac{P-1}{V_{m+P-1}}$, V 倍改善。代价: $P^2P \times V$, activation $\times V$, 代码复杂。 $V=4, P=8$ bubble 从 46.7% → 21.9%。§6

面试考点. **Q40**: ZeroBubble 怎么做到 0 bubble ?

A: backward 拆 B (input-grad, 阻塞) + W (weight-grad, 不阻塞)。B 尽早算, W 见缝插针填 bubble。 $B \approx W$ 时 bubble → 0。代价: autograd hack。§6

面试考点. **Q41**: DualPipe vs ZeroBubble ?

A: DualPipe = ZB + bidirectional injection + 4-phase overlap。bidirectional 让 pipeline 两端同时注入 → bubble 更小, 但需 $2 \times$ 权重。§6

面试考点. **Q42**: DualPipe vs Megatron FWD-BWD merged 生产选哪个 ?

A: 大多数场景 FWD-BWD merged。DualPipe 需 $2 \times$ weight, overlap 收益差不到 3%。除非训 500B+ MoE 且 IB comm 极度受限, 否则 DualPipe overkill。§6

面试考点. **Q43**: PP 组该跨节点还是同节点 ?

A: 跨节点 OK。PP 用 P2P, 只 pair-to-pair, 通信频率低 (每 stage 过渡一次)。跨节点 IB 够用。TP/EP 才必须同 NVLink。\$6

面试考点. **Q44**: PP 训练里最后一层为什么特别慢?

A: LM head + loss + logits (B, S, V) 巨大 activation & compute。Backward 从这里开始。tie embedding + LM head 减 param, 或最后 stage 少几个 transformer layer 给 vocab head 留余量。\$6

面试考点. **Q45**: PP + TP 排布?

A: TP 组必须同 NVL 域。TP=8, PP=8 时: TP 组 = 每 node 8 卡, PP 组 = 跨 8 个 node 的对应 rank。Megatron 默认这么排。\$6

面试考点. **Q46**: 为什么 PP 让 micro-batch 越多 bubble 越小?

A: bubble ratio = $\frac{P-1}{m+P-1}$, m 大分母大 bubble 小。GPipe/1F1B 建议 $m \gg P$, 但太大 activation 爆。 $m = P$ 时 bubble 50%, $m = 4P$ 时 20%。\$6

面试考点. **Q47**: ZeroBubble 的 W (weight-grad) 计算为什么更慢?

A: 原 fused B+W 里 activation 只 load 一次; 分开后 W 时要再 load 一次 activation → HBM 带宽 x2。kernel 效率 -10-20%。但 bubble 减少的收益 > 效率损失。\$6

面试考点. **Q48**: 你怎么选 PP 深度?

A: 每 stage 层数 4-8 是 sweet spot。太少 bubble 大且 P2P overhead 高; 太多 stage compute > 通信, overlap 空间小。Llama-3 405B: 126 layer / PP=16 $\approx$ 8 layer/stage。\$6

面试考点. **Q49**: `--delay-wgrad-compute` 是什么?

A: 把 backward 的 W (weight-grad) 计算延后, 允许 B 继续 pipeline 前进——ZeroBubble 的 W-only 分离。让 stream overlap 更多。Megatron 与 `--overlap-moe-expert-parallel-comm` 配合。\$6, \$11

面试考点. **Q50**: PP 组的 P2P 用 NCCL 还是 MPI?

A: 现代 PyTorch/Megatron 用 NCCL P2P (`send / recv`), 走 CUDA-aware。老代码有用 MPI 或 Gloo, 但性能差。生产必用 NCCL。\$6

长上下文 / CP (Q51-Q60)

面试考点. **Q51**: Ulysses vs Ring Attention?

A: Ulysses = all-to-all 沿 head 切换, attention 全本地。Ring = P2P 逐步旋转 K/V, 累加 Flash lse。Ulysses 简单快但 $CP \leq \text{head 数}$; Ring 无 head 限制但小-K kernel 效率低。\$7

面试考点. Q52: Ring Attention causal mask 怎么处理？

A: naive: rank r 只处理 K/V shard $\text{index} \leq r$, 负载不均 $2\times$ 。Striped/Zigzag: seq permutation 让每 rank 都持“前半+后半”pattern, 负载均衡。生产用 Zigzag。§7

面试考点. Q53: USP 是什么？

A: Unified Sequence Parallel. $CP = CP_{\text{ulysses}} \times CP_{\text{ring}}$ 。先 Ulysses 沿 head 切一层, 再 Ring 沿 seq 切一层。兼顾 head 少 + seq 长场景。Kimi 1M ctx 用。§7

面试考点. Q54: Ulysses 通信量与 CP 有关但 Ring 与 CP 无关？

A: Ulysses 一次 $a2a \frac{V(W-1)}{W} \approx V$ 。Ring 每步 $P2P \frac{V}{CP}$, CP 步 total V ——与 CP 无关。所以 CP 大时 Ring 通信不涨。§7

面试考点. Q55: 训 1M context 你怎么组合并行？

A: 512 卡 H100 例: $TP=8 \times CP=16$ (USP: Ulysses 4 $\times$ Ring 4) $\times PP=2 \times DP=2$ 。CP 主承担 seq 切, TP 切 head, FSDP2 切 weight。加 full activation recompute + FA3 with cp_ring。§7

面试考点. Q56: GQA num_kv_heads=8 能用 CP=16 Ulysses 吗？

A: 不能——KV head 只 8。要么 $CP=8$ (Ulysses); 要么 Ring; 要么 USP: Ulysses=8 $\times$ Ring=2 = $CP=16$ 。§7

面试考点. Q57: Ring Attention 里 Flash lse 合并公式？

A: Flash online softmax 支持分块累加。每步得局部 $(O^{(s)}, \text{lse}^{(s)})$, 合并: $O_{\text{new}} = O_{\text{old}} e^{\text{lse}_{\text{old}} - \text{lse}_{\text{max}}} + O^{(s)} e^{\text{lse}^{(s)} - \text{lse}_{\text{max}}}$, $\text{lse}_{\text{new}} = \text{logsumexp}(\text{lse}_{\text{old}}, \text{lse}^{(s)})$ 。§7

面试考点. Q58: FA3 与 Ring Attention 集成有什么优势？

A: cp_group + cp_ring=True。P2P 与 tile compute 在 kernel 内 async overlap, Hopper 用 TMA 让通信几乎 hidden。比 Python-level ring 快 20-30%。§7

面试考点. Q59: RoPE 长 ctx (1M) 训练要注意什么？

A: (1) RoPE base 加大 (--rotary-base 500000+), 缓解 extrapolation degradation; (2) RoPE cos/sin cache 大 ($S \times d_h = 500 \text{ MB @ 1M}$), 用 cached RoPE; (3) 组合 CP 时 pos_id 沿 seq 切正确传给 RoPE fused kernel。§7

面试考点. Q60: 长 ctx 训练里 activation 主要来源和优化？

A: LN/Dropout BSH, FFN intermediate BSI, Q/K/V/O activation。CP 切 seq attention 内; Megatron SP 切 LN/Dropout; full recomp 只存 layer input; activation offload PCIe overlap。DeepSeek-V3 Table 3 activation 66% 显存。§7, §10

MoE / EP (Q61-Q70)

面试考点. **Q61**: EP vs TP 切 expert , 选哪个 ?

A: EP。TP 切 expert 后每卡还有 E 个“半专家”——dense 化。EP 让每卡持完整 $\frac{E}{EP}$ 个专家, 通信只对被路由 tokens 做。EP 通信 $\propto BSKH$ (稀疏), TP 通信 $\propto BSH$ 但每层 AR。EP 符合 MoE 稀疏本质。§8

面试考点. **Q62**: All-to-all 通信量为什么与 E 无关 ?

A: a2a 传每 token 的 hidden vector, total = $BSKH$ 。K 决定通信量 (一 token 走几专家), E 决定池大小但每 token 还是走 K 个——加多专家不加通信。DS-V3 用 256 experts 通信量与 8 experts 相同。§8

面试考点. **Q63**: 层次化 all-to-all 收益 ?

A: NVLink 400 GB/s vs IB 50 GB/s。intra-node NVLink 聚合 $\rightarrow$ 单次 inter-node IB $\rightarrow$ intra-node 散发。IB 带宽消耗从 $O(N^2KH)$ 降到 $O(NKH)$, 100+ 节点差 10 $\times$ 。§8

面试考点. **Q64**: DeepEP V2 相比 V1 改进 ?

A: V1 占 20 SM (GEMM 掉 20%)。V2 warp specialization + auto-tuned chunk 只占 4-6 SM。让 fine-grained MoE 可行 (256 experts topk=8)。§8

面试考点. **Q65**: MoE 一层 forward 通信几次 ?

A: 2 次 a2a (dispatch + combine)。若 TP 组合还有 2 AR (attention output + FFN)。加 backward 就是 8 次集合通信/层。§8

面试考点. **Q66**: 为什么 MoE overlap 比 dense 更重要 ?

A: MoE 一层 4 次 a2a 占 15-40% forward 时间 ; dense 只 2 次 AR (10-20%)。overlap 到 <5% 后, MoE 省绝对时间 > dense。DualPipe 是极端案例。§8, §11

面试考点. **Q67**: aux loss vs aux-loss-free ?

A: aux loss 给 router 冲突梯度 (main task vs balance)。DeepSeek aux-loss-free: 可学 bias 只影响 topk 选择不影响 gate 梯度 ; 简单反馈规则调 bias。router 梯度纯净。§8

面试考点. **Q68**: Parallel Folding 是什么 ?

A: 打破 $EP \leq DP$ 约束, attention 与 expert 用独立 process group。允许 attention-DP=32, expert-EP=64+DP=2 非对称配置。DeepSeek fine-grained MoE 类模型用。§8

面试考点. **Q69**: MoE + CP 一起用的注意事项 ?

A: (1) MoE a2a 与 CP P2P 争带宽——让 EP/CP 走不同 device mesh 维度 ; (2) EP 沿一维 (跨节点用 DeepEP 缓解), CP 沿另一维 ; (3) MoE token 数不均 + CP 序列不均双重不确定——用 capacity padding + fixed-length packing。§7, §8

面试考点. **Q70**: fine-grained MoE (256 experts) 与 vanilla (8 experts) 的差别 ?

A: 通信量相同 (E 无关)。显存 weight 相同 (总 param 相同)。grouped GEMM 每 expert M 维 $8\times$ 更小 $\rightarrow$ 易 memory-bound。需要 DeepGEMM / TE GroupedLinear 好的 kernel 才不掉率。§8

精度 / 优化 (Q71-Q80)

面试考点. **Q71**: BF16 vs FP16 训练 ?

A: BF16 exponent 8 bit (FP32 同) 动态范围大, 无需 GradScaler ; mantissa 7 bit 精度稍差。FP16 exp 5 bit 需 GradScaler 防 underflow ; mantissa 10 bit 精度好但 range 小。H100 时代基本弃 FP16。§9

面试考点. **Q72**: FP32 master weight 为什么必需 ?

A: BF16 mantissa 7 bit, weight $-lr \times \text{update}$ 里 update 通常小 1000+ 倍, BF16 直接减 round 到 0。FP32 master 累积小 update, broadcast BF16 用于 fwd/bwd。§9

面试考点. **Q73**: FP8 Delayed Scaling vs Current Scaling?

A: Delayed 用上次 amax 估计 scale, 无额外 reduce, 快但对 outlier 敏感。Current 每次求当前 amax, overhead 3-5%, 精度稳。TE default = Delayed ; DeepSeek 用 blockwise (每 tile 一个 scale)。§9

面试考点. **Q74**: DeepSeek blockwise FP8 recipe?

A: Weight 按 (128, 128) tile 各存一个 FP32 scale ; activation 按 (1, 128) tile scale。粒度细, outlier 只影响自己 tile。三路 GEMM (Fprop/Dgrad/Wgrad) 全 FP8, 只 optim state 与 master weight FP32。val loss error $< 0.25\%$ vs BF16。§9

面试考点. **Q75**: FP8 里 attention softmax 为什么不用 FP8 ?

A: softmax 需 $\max + \exp + \text{sum} + \text{divide}$ 。FP8 mantissa 3 bit 精度不足以做 exp 累加。FA-FP8 只把 QK^T 和 PV GEMM 用 FP8, softmax stats 全 FP32。§9

面试考点. **Q76**: activation checkpoint compute overhead 为什么 33% ?

A: 1 fwd + 2 bwd = 3F baseline ; checkpoint 加 1 fwd 重算 = 4F。overhead = $4/3 - 1 = 33\%$ 。§10

面试考点. **Q77**: FA 之后 activation checkpoint 还有必要 ?

A: 有。FA 只省 BS^2A (attention matrix) ;activation 还有 BSH (LN, residual)、 BSI (FFN)。长 seq 累积 GB 级。selective recompute 对 Q/K/V matmul activation checkpoint 有用。§10

面试考点. **Q78**: MoE 场景为什么不整层 recompute ?

A: 整层 recompute 会重触发 a2a 通信——通信开销翻倍。fine-grained recompute 只重算计算部分 (moe_act, expert intermediate) , 跳过 dispatch/combine activation。§10

面试考点. **Q79**: Precision-Aware Optimizer 代价 ?

A: 几乎 0。fused kernel 里 BF16 $\leftrightarrow$ FP32 cast 微小 latency。收益 optim state -50%。Megatron 2024 新 flag。§10

面试考点. **Q80**: FSDP 与 activation checkpoint 组合顺序 ?

A: 先 apply_activation_checkpointing 再 FSDP wrap。反了 FSDP 内部 AG 与 AC detach 冲突 , backward 会重新 AllGather 但 AC 已 discard activation。§10

Overlap / 系统 (Q81-Q90)

面试考点. **Q81**: DDP overlap 依赖什么 ?

A: (1) find_unused_parameters=False ; (2) bucket_cap_mb ≥ 25 (推荐 100-500 大模型) ; (3) gradient_as_bucket_view=True ; (4) NCCL 异步 stream ; (5) CUDA_DEVICE_MAX_CONNECTIONS > 1。§3, §11

面试考点. **Q82**: FSDP forward_prefetch 与 backward_prefetch 时机 ?

A: forward_prefetch: 层 l 开始时 issue $l+1$ AG。backward_prefetch (PRE): 层 l backward 开始前 issue $l-1$ AG。都“往未来看” , 代价显存峰值高。§4, §11

面试考点. **Q83**: Async TP 收益 ?

A: TP=8 AR 占 15-20% $\rightarrow$ 5-8% , step time -14%。TE LayerNormLinear fused kernel 支持。§5, §11

面试考点. **Q84**: MoE 场景开启 `--overlap-moe-expert-parallel-comm --delay-wgrad-compute` 收益 ?

A: EP a2a 占 30-40% $\rightarrow$ < 5% , overlap 93%。不需 $2\times$ weight (对比 DualPipe)。§8, §11

面试考点. **Q85**: nsys 里怎么判 overlap 好坏 ?

A: 多 stream row。理想: comm stream 和 compute stream 时间列 overlap , 两者少 gap。不好: comm stream 有 kernel 但 compute stream gap $\rightarrow$ compute 等 comm ; 两 stream 完全错开 $\rightarrow$ 无 overlap。§11

面试考点. **Q86**: `.item()` / `.tolist()` 在训练循环里的后果？

A: D2H sync, host 阻塞。破坏 (1) CUDA graph capture ;(2) stream overlap ;(3) profile 出现 idle time。log 用 tensor 累积到 log step 才 D2H。§11

面试考点. **Q87**: 训练 dataloader wait 占 step > 5% , 怎么排查？

A: 加 time.time 前后测。检查 (1) num_workers < 8 ;(2) prefetch_factor 只 2 ;(3) tokenizer 单进程慢；(4) 数据慢盘；(5) preprocess 重 (大图 decode)。分别对症。§12

面试考点. **Q88**: `persistent_workers=True` 什么用？

A: DataLoader worker 进程每 epoch 不重启，省 fork + tokenizer 加载时间 (每 epoch 30s)。生产训练必开。§12

面试考点. **Q89**: sequence packing 与 batch padding 差别？

A: padding: batch 内 pad 到 max seq，浪费 30-70% compute。packing: 拼接短 seq 到固定长，attention mask (或 FA varlen cu_seqlens) 保证不跨 doc attend。padding ratio < 5%，吞吐 +50-100%。§12

面试考点. **Q90**: `torch.utils.data.IterableDataset` 里 rank 分 shard 常见 bug？

A: (1) shard 数不能整除 world × workers，某些 rank 少数据 → hang; (2) 忘 `worker_init_fn`，多 worker 拉同一 shard; (3) shard 内无 shuffle; (4) resume 时未保存 iterator state; (5) 长 seq 分给某 rank 造成不均。§12

多模态 / RL / 稳定性 (Q91-Q100)

面试考点. **Q91**: 多模态 vision encoder 与 LLM 为什么并行策略分开？

A: ViT 300M-1B 一张卡装得下用 DDP/FSDP；LLM 70B+ 需 TP+PP+FSDP。同 TP=8 setup 让 ViT 白开 TP AR。分离后 MFU +10-20%。§13

面试考点. **Q92**: NaViT packing 与文本 packing 差别？

A: 文本沿 seq 拼；NaViT 把不同尺寸图 patch 化后 pack 到一个 sequence，attention mask 图间不 attend。unit 是 image patch block。让 batch 混合不同分辨率图像。§13

面试考点. **Q93**: DDP 里图像数不均衡后果？

A: rank 0 十张图, rank 1 零张 → ViT forward 差 10×。DP AR sync 时快 rank 空等，MFU -20-30%。做 cost-based balancing (n_images + text_len 二维桶)。§13

面试考点. **Q94**: 视频训练最大显存瓶颈？

A: 视频 tokens 爆炸 ($8 \text{ frames} \times 576 \text{ tokens} = 4608$)。做法 (a) Q-Former/Perceiver 压缩; (b) temporal pooling; (c) 3D VAE 直接编 spatio-temporal。CogVideoX 用 (c) 把 (49,480,720) 压到 (13,60,90) latent。§13

面试考点. **Q95**: RLHF 为什么要 rollout / train 分离?

A: rollout 是 autoregressive decode (memory-bound, small-M); train 是 dense fwd+bwd (compute-bound, large-M)。用 training 跑 rollout 效率极低 (无 KV cache, 无 continuous batch)。分离后 vLLM/SGLang 5-10 $\times$ 加速 rollout。§14

面试考点. **Q96**: GRPO 相对 PPO 简化在哪?

A: 去 value model。用 group baseline: N 个 rollout 算 $\text{advantage} = (r - \text{group_mean}) / \text{group_std}$ 。少训一 model, 省 20-30% GPU。DeepSeek-R1 证明有效。§14

面试考点. **Q97**: weight 从 training 到 rollout engine 同步方法?

A: (1) Colocated 直接 copy (同 pool 交替); (2) NCCL cross-group broadcast (不同 pool); (3) shared GPU buffer 零 copy。verl/OpenRLHF 主要用 (2) + overlap with training。§14

面试考点. **Q98**: 10000 卡训 3 个月, 容错怎么设计?

A: 三层: (1) 监控 dcgm + IB + 温度; (2) NCCL timeout + async error; (3) async checkpoint 15 min + rank-agnostic + spare node auto-replace + resharding。目标 MTBF 2h $\rightarrow$ downtime < 5%。§16

面试考点. **Q99**: async checkpoint 为什么比 sync 快?

A: sync: gather 到 rank 0 $\rightarrow$ write NAS, 几分钟停 training。async: D2H 到 CPU pinned mem (10s) + background 写 disk (分钟级), training 不阻塞。停 training 时间从几分钟 $\rightarrow$ < 10s。§16

面试考点. **Q100**: loss NaN 你怎么处理?

A: (a) save debug ckpt (含最近 batch + activation stats); (b) 检查 grad_norm; (c) skip 这 batch 继续训——多数能自恢复; (d) 若连续 NaN, rollback 到 100 步前 ckpt 并降 LR 20%; (e) audit batch 找 outlier data。Llama-3 训练 466 次 spike, 90% 用 (c) 处理。§16

RLVR (Q101-Q110)

面试考点. **Q101**: RLVR 相比经典 RLHF, 算力结构变了什么?

A: 多出一个 **CPU verifier** 集群 (沙箱跑测试 / 符号比对), 同时可能少掉 reward model 和 ref model 两个 GPU 池 (去 KL 的配方)。从“纯 GPU”变成“GPU + 大规模 CPU 沙箱”。新瓶颈是 verifier 吞吐与尾延迟。§15

面试考点. **Q102**: DeepSeek-R1 为什么明确弃用 neural PRM/ORM ?

A: 三个理由: 大规模 RL 下 reward hacking 不可避免; 对抗 hacking 要不断重训 RM, 代价高; RM 本身准确率成为 policy 的天花板。数学/代码这类有客观对错的任务改用规则 reward (答案比对 + 格式检查) 就没有这些问题。§15

面试考点. **Q103**: 四类 verifier 的延迟量级各是多少? 哪个决定架构?

A: 数学符号比对 1-50 ms; 代码跑单测 0.1-10 s; Lean/Coq 编译 秒~分钟; LLM-as-judge ms~秒 (且又变回 GPU 负载)。数学便宜到可同步跑, 代码/形式化必须独立池 + 异步。决定架构的不是均值而是尾延迟: 一组 16 个响应里只要 1 个卡住, 整组 advantage 出不来。§15

面试考点. **Q104**: 写出 verifier 池的容量公式, 并说明 t_v 该代什么值。

A: $W \geq G \cdot r \cdot (1 - h) \cdot t_v / u$ (G rollout GPU 数, r 每卡每秒响应, h 缓存+去重命中率, u 目标利用率)。 t_v 代均值——排队稳定性只取决于平均服务时间, 按 p90 代入是白买 2-3 倍机器。重尾的影响不在容量而在组内最大值, 那是另一个问题。§15

面试考点. **Q105**: 为什么加 verifier worker 解决不了 GRPO 的等待?

A: GRPO 要等一组 N 个响应全部验完才能算 advantage, 优化器等的是组内最大值。重尾下 $E[\max \text{ of } 16] \approx 4 - 5 \times E[\text{单次}]$, 且相当比例的组里至少有一个撞超时——那部分等待与 worker 数无关。压组最大值只能靠超时封顶和去重 (减少从尾部抽样的次数), 根本解法是异步验证让它离开关键路径。§15

面试考点. **Q106**: verifier 侧性价比最高的两个优化是什么?

A: 结果缓存 + 组内去重。GRPO 一个 prompt 采 $N = 16$ 个响应, 温度不高时完全相同的响应很常见 (正确解法就那么几种), 按 `hash(prompt, response)` 去重实测能省 30-60% 的 verifier 调用, 零成本。其次是异步化, 让 verify 与下一批 rollout 重叠。§15

面试考点. **Q107**: 二值 reward 下 GRPO 为什么会大量白烧 rollout?

A: 组内全对或全错时 $\text{std} = 0$ 且 $R_i - \text{mean} = 0$, advantage 全 0, 整组零梯度。训练早期大量题全错、后期大量题全对, 无效组占比能到 30-50%。解法: DAPO dynamic sampling (过采样后丢掉 acc 为 0 或 1 的组), 或按历史准确率做难度分层采样优先取 0.2-0.8 的题。§15

面试考点. **Q108**: dynamic sampling 对调度器提出了什么新要求?

A: 每个 iteration 的 rollout 量不再是常数。过滤掉无效组后要继续补采直到凑满 batch, 容量规划从“生成 N 组”变成“生成到 N 组存活”。仿真里存活率约 62%, 意味着要多采 1.6 倍。调度器必须支持带反馈的流式采样。§15

面试考点. **Q109**: Dr. GRPO 指出 GRPO 的哪两处偏差?

A: (1) 按响应长度归一化 $1/|o_i|$ 让长错误答案获得不成比例的权重，系统性鼓励响应变长——这就是“越训越长但没变强”的成因之一；(2) 按组标准差归一化 $1/\text{std}$ 放大了极端难度题的权重。处方是去掉这两项、改用 token 级求和。DAPO 的 token-level loss 动机一致。§15

面试考点. **Q110**: 长时间 RLVR 训练熵坍塌怎么办？为什么很多配方去掉了 KL？

A: 熵坍塌链条：熵降 → 输出趋同 → 组内 N 个采样几乎一样 → GRPO 没有差异 → 梯度消失。DAPO 的 clip-higher 把裁剪区间拆成非对称的 $[1 - 0.2, 1 + 0.28]$ ，给低概率 token（探索来源）留上升空间。去 KL 是因为 RLVR 的目标恰恰是学会 base 不会的长链推理，KL 是纯阻力；系统上还能省掉 ref model 一整个池，显存全给 KV cache。代价是失去跑偏的刹车，要靠监控语言混杂、格式崩坏和通用能力回退兜底。§15

Agentic RL (Q111-Q120)

面试考点. **Q111**: 多轮 agentic rollout 与单轮 rollout 在系统上差在哪？

A: 四点：环境执行期间 GPU 闲置（工具调用几秒）；每轮都要 prefill，不复用 prefix cache 的话总代价 $O(T^2)$ ；轨迹长度方差从约 $4\times$ 涨到约 $100\times$ （1 轮 500 token ~ 50 轮 100K token）；训练侧多出 token masking。§15

面试考点. **Q112**: 为什么 prefix cache 是 agentic RL 的生死线？开了还可能没用是怎么回事？

A: 第 $k+1$ 轮输入 = 第 k 轮完整上下文 + 新观测，不复用则 T 轮 prefill 是 $O(T^2)$ ，20 轮要多付一个数量级。真正的坑是“开了但命中率低”：序列等环境那几秒里 KV cache 被 LRU 淘汰，回来又要重新 prefill。查引擎的 prefix cache 命中率指标；修法是扩大 KV 池（去 KL 省下的 ref 显存正好给它）。对等待中的序列 pinning 或换出 CPU、限制并发轨迹数让活跃集合装得下。§15

面试考点. **Q113**: 同步批式 rollout 为什么必须换成异步？

A: 同步是“这批一起走第 1 轮、一起等环境、再一起走第 2 轮”，每轮都要等这批里最慢的环境调用，GPU 纯闲。异步/连续批处理把几百条轨迹同时挂在引擎上，等环境的自然退出运行批次。仿真里 GPU 利用率 $19\% \rightarrow 88\%$ ，makespan 快 $4.6\times$ 。§15

面试考点. **Q114**: 你用 p99/p50 衡量 agentic rollout 的 straggler，合理吗？

A: 不合理，这个指标在这里会说反话。同步批式的 p99/p50 反而比异步更“健康”（2.85 vs 2.94），因为 barrier 把所有短轨迹都拖慢到最慢那条的节奏——离散度小是因为大家一起被饿死，百分位比值区分不了“没有 straggler”和“全都是 straggler”。同理单轨迹延迟也是红鲱鱼：异步故意让几百条在飞，单条延迟必然变差。该看 **makespan** 和 **GPU** 利用率，因为整个 iteration 组装完之前没有东西会消费单条轨迹。§15

面试考点. **Q115**: partial rollout 解决什么？代价是什么？三种处理方式？

A: 解决轨迹长度重尾导致的 straggler（同步收集一个 batch 要等最长那条）。做法是给每轮设 token/轮数预算，没跑完的存状态下轮接着跑（Kimi K1.5）。代价是 off-policy：一条轨迹跨了

多个 policy 版本。三种处理从严到松：逐 token 记版本并用对应 logprob 算重要性比；限制最多跨 K 个版本超了就丢（最常见）；直接当 on-policy。§15

面试考点. **Q116**: 权重更新时几百条轨迹跑到一半，怎么办？

A: 三选一。Drain 等全跑完再换权重——长尾让 GPU 空转，退回 straggler 问题；Abort 直接丢——浪费已生成 token，长轨迹损失最大；Carry-over 跨版本继续（即 partial rollout）——需要 off-policy 记账。长 horizon 场景下前两个都太贵，主流框架都往 carry-over 走。§15

面试考点. **Q117**: 多轮轨迹里哪些 token 要 mask？不 mask 有什么后果？

A: 只训 assistant 生成的 token，工具观测 token 必须 mask。三个后果：观测常占 60–80% token，绝大部分梯度花在教模型预测它原理上无法知道的工具输出；模型学会自己编造观测，推理时不调工具直接幻觉结果；重要性比对观测 token 没有意义，那些 token 不是 π_{old} 采样出来的。最阴险的是它不会让训练崩——loss 照降、grad_norm 正常，只能靠工具调用成功率发现。§15

面试考点. **Q118**: `loss.sum() / mask.numel()` 和 `/ mask.sum()` 差在哪？

A: 用 `numel()` 时分母含观测 token，等于按“这条轨迹的工具输出有多啰嗦”在缩放学习率——而那是环境决定的，不是你决定的。同样 5 轮、assistant 干了同样多活的三条轨迹，只因为工具从计算器换成网页抓取，loss 就能差 $40\times$ ：啰嗦工具的轨迹被压到无关紧要，简洁的主导整个 batch。正确写法是 `/ mask.sum().clamp(min=1)`。§15

面试考点. **Q119**: rollout 引擎和训练引擎的 logprob 对不上，有什么影响？怎么监控？

A: 直接拿 rollout 的 logprob 当 $\log \pi_{\text{old}}$ ，第一个内层 epoch 本该恒为 1 的重要性比会偏离，造成本不该发生的裁剪，梯度被白砍——而且砍掉的恰恰是两边分歧最大的低概率探索 token。根因是 kernel、batch 组织、BF16 累加顺序、并行切分都不同。处方是用训练引擎重新前向算 $\log \pi_{\text{old}}$ 。监控要看第一个 **epoch** 的裁剪比例（应接近 0），不能只看 `mean |dlogp|`：分歧分布是重尾的，均值只有 0.09 时裁剪比例已经到 5%。§15

面试考点. **Q120**: 沙箱 OOM、外部 API 限流导致的失败，该给什么 reward？

A: 不能给 0。那是基础设施失败，不是 policy 的行为后果，给 0 等于往训练信号里注入随机噪声。正确做法是标为 `infra_failure`、从 batch 剔除、单独打点，并把 infra 失败率当 SLO 管——超过 1% 是要修的系统故障。考点是能不能区分“环境的错”和“模型的错”。§15

SP/CP 全栈适配 (Q121-Q130)

面试考点. **Q121**: CP 下 cross-entropy 怎么算？各卡算完平均一下行不行？

A: 不行。各卡有效 token 数差别极大（prompt masking + padding，极端情况某卡为 $0 \rightarrow 0/0$ 出 NaN，一次 all-reduce 全场变 NaN）。必须本地只求和，把分子 $\sum m\ell$ 与分母 $\sum m$ 分别 all-reduce，最后相除。追问“跳过空卡再平均”——那等于把每卡权重拉平成 $1/CP$ ，实测偏 $0.81\times$ ，效果是按答案长度重新加权样本，模型会偏好短输出。§7

面试考点. **Q122**: loss 算对了, 为什么梯度还可能差 CP 倍?

A: 每卡产出的是“对全局 loss 的贡献”(本地分子 / 全局分母), 这些贡献要求和才是全局梯度。但 CP 维常被折进 DP 的梯度归约, 而 DP 是求平均, 于是丢掉一个 CP 因子(实测 $|g|$ 恰好是 $1/CP$)。修法: loss 乘 CP, 或 CP 维改 SUM 归约。要点是 **loss** 打印值完全正确, 看曲线只会误判成“LR 要调大”, 必须对拍参数更新量才能发现。§7

面试考点. **Q123**: 一个 CP 组内各卡该拿相同还是不同的数据? 怎么验证?

A: 相同——一个 CP 组合起来才是一条序列。sampler 索引与 shuffle 种子都要用 `dp_rank`, 不能用 `global_rank`。写错不报错: attention 把两个半截文档当一条序列算, 同时 global batch size 悄悄变成配置值的 CP 倍, token 预算和等效 LR 全错。验证一行——`sample_id` 在 CP 组内 all-gather 后断言全相等, 建议常驻。同规则适用 TP 组和 PP stage: 只有 DP 维允许数据不同。§7, §12

面试考点. **Q124**: 为什么需要 zigzag 切分? 它引入了什么新麻烦?

A: 连续切分在 causal mask 下负载不均——rank 0 的 query 几乎看不到 key, 最后一卡要看全序列, 实测每卡 (q, k) 对数 $[36, 100, 164, 228]$ 。collective 走最慢那卡, 代价是 $\max/\text{mean}$ (实测 $1.73 \times$, CP 越大趋近 $2 \times$), 不是更夸张的 $\max/\min$ 。zigzag 让 rank r 取 chunk r 与 $2CP - 1 - r$, 对数严格相等。新麻烦: shard 不再连续, `position_ids/doc_ids` 要按同置换分发、mask 不能用下三角、输出要逆置换、halo 要从两个不同 rank 各取一次。§7

面试考点. **Q125**: CP 下 RoPE 用本地 `arange(0, S/CP)` 当位置会怎样?

A: 全错但不报错。RoPE 是相对位置编码, 靠绝对位置作差实现, 本地位置让所有相对距离都错。短序列时误差被 softmax 部分吸收, 表现为“能训但外推能力差”; zigzag 下更糟, rank 0 持有的全局位置是 $[0, 1, 2, 3, 28, 29, 30, 31]$, 本地 `arange` 与之毫无关系。正确做法: 把 `position_ids` 做成 batch 的显式字段, 由 DataLoader 产出并随 token 一起切分, 模型里不准用 `arange` 现场生成。§7

面试考点. **Q126**: 开 CP 之后, 哪些模块不需要额外同步?

A: 所有逐 token 独立的模块——RMSNorm/LayerNorm (沿 hidden 维归一化, 与序列怎么切无关)、FFN/SwiGLU、所有逐元素算子。这正是 Megatron SP 能把 LN/Dropout 放在 seq-shard 上零通信执行的原因。会真正需要跨 token 统计的是 BatchNorm 这类沿 batch/序列维求统计量的算子, 而 LLM 基本不用它。这题是反向考察: 能主动说“不要给 LN 加 all-reduce”, 说明不是在乱加通信。§5, §7

面试考点. **Q127**: CP 下 MoE 的 load-balancing aux loss 要怎么改?

A: $\text{aux} = E \sum_e f_e P_e$ 里 f_e (token 比例) 和 P_e (平均概率) 都是“对 token 求平均”, 必须在 CP 组上归约 expert 直方图和概率和。只统计本卡的后果不只是数值偏大 (实测 $3.48 \times$): 一个全局完美均衡的 router 会被每张卡判为“我的 token 全去了一个 expert”, 梯度在自己那个 expert 的 logit 上为正, 把各卡推离全局最优——正确梯度此时恰为零。征状是 load-balance metric 长期震荡不收敛。z-loss 与 capacity/drop rate 同理。§7, §8

面试考点. **Q128**: reward model 的打分头在 CP 下为什么会错？

A: 分数取自最后一个有效 token 的 hidden state, 而这个 token 只落在某一张卡上。各卡各取“自己 shard 的最后一个有效 token”的话, 只有 owner 是对的, 其他卡拿的是序列中间的 token, 整段是 padding 的卡还会崩在空索引上。正确做法: one-hot 选中后做一次可微 all-reduce (forward 求和、backward 恒等), 非 owner 贡献严格为零。裸 `dist.all_reduce` 是 in-place 的, autograd 不知道其他卡存在。同类还有 mean pooling、[CLS] 头、以及 RL 的 GAE (沿序列反向递推)。§7, §15

面试考点. **Q129**: 为什么 CP 下 grad norm 容易算大 $\sqrt{\text{CP}}$ 倍？

A: 把分片参数的写法 (跨组 all-reduce $|g|^2$) 套到了副本参数上。CP 组内参数是副本, 梯度归约后每卡已持有完整梯度, 不该再求和。规则: 只在参数被分片的组上求和 (ZeRO/FSDP shard、TP column/row shard、EP expert 维), 不在副本组上求和 (CP、DP replica、TP 下复制的 LN/bias)。后果是裁剪提前触发、更新被静默压小 (实测 clip 系数从 1.0 变 0.59), 等于偷偷降 LR——而 grad_norm 日志记的正是那个虚高值, 查不出来。Megatron 用 `param.tensor_model_parallel` / `param.shared` 区分。§7

面试考点. **Q130**: 哪些算子在 CP 下需要 halo 交换? 宽度多少?

A: 只依赖邻近 token 的算子: causal conv1d 需要前 $K-1$ 个 token; sliding-window attention 需要 $w-1$; Mamba/SSM 扫描需要的不是 token 而是前一卡的递归 **state**, 会把各卡串行化, 除非改 chunked scan; 多模态 ViT/audio 前端的 depthwise conv 同理。label shift / MTP 理论上也是 halo, 但更好的做法是在切分前对整条序列做位移, 零通信解决所有 k 。注意误差形状: 只错在每个接缝的 $K-1$ 行, 占比随 shard 变长而下降 ($S=128\text{K}$ 、 $\text{CP}=8$ 时只错 0.002%), loss 上完全看不见——所以带局部算子的模型往往宁可承受连续切分的 $1.7 \times$ 负载不均, 也不用 zigzag。§7

附录 E：面试故事 —— “框架都做了，你还能加什么”

面试官最爱问的一类问题：“**Megatron / DeepSpeed** 都已经做了 **XX**，你在训练里还能做什么优化？”——这是考察你能不能真读过 profile、能不能提出 delta 优化。

这一附录不是“帮你编经历”——里面的故事要基于你实际做过的项目才有说服力。这里给的是：

1. **STAR** 框架：把技术优化包装成面试故事的结构
2. **20** 个高频“差异化优化”案例：都是公开报告 / paper 里描述过的、可以延伸自己的经历讲的具体优化点
3. “框架 **X** 已做，我加了 **Y**” 的对比表：让你的回答有 delta

STAR 框架：一个技术故事的结构

STAR = **Situation, Task, Action, Result**。用于把“我做过 X 优化”结构化成 90 秒的完整故事。

一个模板：

- **Situation (30 秒)**：什么模型、什么 setup、遇到什么现象。数字量化 (MFU X%, step Y ms, comm 占 Z%)
- **Task (10 秒)**：你负责什么，目标是什么
- **Action (40 秒)**：你怎么诊断的，试过什么方案，为什么选定这个。这里最能体现深度。
- **Result (10 秒)**：结果数字。MFU 从 $X \rightarrow Y$ ，step time 从 $A \rightarrow B$ ，节省 GPU-hours。

不要：只说“我调 flag”、“我加机器”、“我 fine-tune 参数”。要说：为什么这么做，怎么发现问题，别的选项为什么不行。

20 个可复用的“差异化优化”案例

以下每个案例都基于公开的 paper / tech report / repo，可用来延伸自己经历讲。斜体是“面试口径”版本。

案例 1: DDP bucket size 调优

背景：训 30B model on 128 H100，MFU 32%，nsys 看到 backward 结束还有大段 AR 等待，overlap 只 60%。

诊断：DDP 默认 bucket=25 MB，30B 模型有 200+ bucket，每个 AR 都有 30 μ s 启动 overhead，累积起来 6 ms/step 只是启动。

Action：调 `bucket_cap_mb=200`，同时 `gradient_as_bucket_view=True`。

Result：MFU 32% $\rightarrow$ 41%，step time -14%。

“Megatron 默认 bucket 25 MB 是通用值，实测发现 30B 模型上启动 overhead 累积 6 ms/step，调到 200 MB 之后 overlap 从 60% 提到 90%+，MFU +9 个百分点。”

案例 2: FSDP `limit_all_gathers` 调优

背景：FSDP FULL_SHARD + BACKWARD_PRE 训 70B，OOM。

诊断：BACKWARD_PRE 太激进 prefetch，同时持有 3-4 层 param unshard 峰值超显存。

Action : 设 `limit_all_gathers=True` , 最多 prefetch 2 层。

Result : 显存峰值 -12 GB , 可跑更大 batch。

案例 3: HSDP node 内 wrap 粒度

背景 : HSDP 训 Llama-3 70B on 64 卡 (8 node × 8 GPU) , intra-node AG 慢。

诊断 : node 内 8 卡的 shard 单位太小 (每 param 8 份 , NCCL 效率低)。

Action : 改用 `_HYBRID_SHARD_ZER02` , intra-node 只 shard grad + optim , weight replicate ; 跨节点仍 DDP。

Result : intra-node comm -40% , MFU +6%。

案例 4: TP + SP 里 Async TP

背景 : Llama 70B TP=8 训 , AR 占 forward 22%。

Action : 升级到 TE `LayerNormMLP fused kernel` + `--tp-comm-overlap`。GEMM tile 与 AR chunk overlap。

Result : TP comm 22% → 6% , MFU +12%。

案例 5: MoE overlap 从 naive 到 Megatron merged

背景 : Mixtral 8×7B on 64 H100 , MoE dispatch a2a 占 forward 28%。

诊断 : naive 实现 dispatch/expert/combine 完全串行。

Action : 开 `--overlap-moe-expert-parallel-comm --delay-wgrad-compute` , `CUDA_DEVICE_MAX_CONNECTIONS=8`。

Result : a2a 占比 28% → 4% , MFU 38% → 51%。

“这个不是我发明的 , Megatron 2024 加的功能 , 但当时集群还没升到新版 Megatron。我 backport 了 patch , 验证在生产不 break。收益 +13% MFU。”

案例 6: DeepEP V1 → V2 SM carve-out

背景 : MoE 训 with DeepEP V1 , GEMM 效率只 65% (SM 被 comm 抢)。

诊断 : DeepEP V1 吃 20 SM。

Action : 升级 V2 , 把 comm SM 降到 4-6。

Result : GEMM 效率 65% → 85% , 端到端 +18%。

案例 7: FA varlen 与 CUDA graph 兼容

背景 : 多模态训练里 seq_len 大方差 (500-16K) , FA varlen kernel launch overhead 累积占 8% step。

Action : seq_len 按 buckets ([1024, 2048, 4096, 8192, 16384]) 归一化 , per-bucket capture CUDA graph。

Result : kernel launch overhead 8% → 1%。

案例 8: Vision encoder 分离 device group

背景 : VLM 训 32B (LLM) + 0.4B ViT , MFU 25%。

诊断 : ViT 与 LLM 共 TP=8 , ViT 用不上 TP , AR 空开销 + rank imbalance。

Action : ViT 用独立 process group (DP=world, no TP) ; LLM 保 TP=8+PP+FSDP。

Result : MFU 25% → 38%。

案例 9: Sequence length balancing (SFT)

背景 : SFT 训 Qwen2-72B , DP=8 , step time p99/p50 = 1.4 (严重 straggler)。

诊断 : log 每 rank 每 batch total tokens , 发现 rank 3 常有 8K seq , 其他 rank 500-2K seq。

Action : 换成 packing (fixed 8192 length) + FA varlen。所有 rank total tokens 恒定。

Result : p99/p50 = 1.05 , MFU +25%。

案例 10: Precision-aware optimizer + FP8 activation

背景 : 训 30B 单机 8xH100 pretrain , activation OOM (每卡 78 GB / 80 GB 边缘)。

Action :

`--use-precision-aware-optimizer --exp-avg-dtype bf16 --exp-avg-sq-dtype bf16` (省 12P → 6P optim state, 每卡 -12 GB) , 加 FP8 activation (再 -8 GB)。

Result : 显存 78 → 58 GB , 可以扩 batch 2× , wall-clock -30%。

案例 11: Fine-grained recompute

背景 : MoE 训 Mixtral 8×22B , OOM 需要开 full recompute , 但 recompute 让 a2a 通信量翻倍 (backward 时 forward 再跑一遍触发 dispatch)。

Action :

改用 `--recompute-granularity selective --recompute-modules moe_act mlp core_attn` , 只 recompute 计算部分 , 不触发 a2a。

Result : 显存与 full recompute 相当 , compute overhead 从 32% → 6%。

案例 12: In-memory checkpoint

背景 : 训 70B on 512 卡 , checkpoint 保存 5 分钟 stop training , 每 30 min 一次 , 5% overhead。

Action : 改 async checkpoint (D2H → CPU pinned mem, 15s) , 配合 in-memory peer backup (存到隔壁 node CPU DDR) , disk backup 每 6h 一次。

Result : checkpoint 阻塞 training 时间 5 min → 15s , overhead 5% → 0.5%。恢复时间也从 20 min → 30s。

案例 13: Data loader worker + prefetch tuning

背景 : 训 loss curve 正常 , 但 GPU util 只 78%。

诊断 : nsys 看每 step 开头有 15 ms idle , 正好等 loader。

Action : num_workers 4 → 16, prefetch_factor 2 → 8, persistent_workers=True。

Result : GPU util 78% → 92% , MFU +10%。

案例 14: NCCL 环境变量批调优

背景 : 新集群 (8 node × 8 H100 with IB NDR) , AR 只跑到 40 GB/s (理论 100 GB/s)。

Action : 跑 `nccl-tests` + `grid search` `NCCL_MIN_NCHANNELS` , `NCCL_NTHREADS` , `NCCL_BUFFSIZE` , `NCCL_IB_HCA` 组合。找到 optimal config。

Result : AR 跑到 85 GB/s , DP training step -12%。

“NCCL 参数没有标准答案，每集群都要实测。我把 grid search 脚本 open source 出来了。”

案例 15: RL rollout / training 比例调优

背景：GRPO 训 32B，rollout 用 vLLM (16 GPU) + training (16 GPU)，- iter 6 min。

诊断：log 每阶段时间，rollout 5 min training 1 min → training GPU 空 4 min。

Action : 调 rollout : training = 12 : 20 GPU。

Result : iter time 6 → 3.5 min，同硬件 iter/hour +70%。

案例 16: DDP grad clip 顺序修正

背景：训到某 step loss spike，后续训炸。

诊断：grad clip 放在 micro-batch backward 之后，此时 grad 是 partial (未 AR)，clip 无意义。

Action : clip_grad_norm_ 移到 optim.step() 之前 (AR 完成后)。

Result : 训练稳定，不再 spike。

“这是很常见的 bug，尤其在 gradient accumulation 下。DDP 的 AR 在 backward 完成时才做，clip 在 AR 前 clip 的是 partial grad，不影响 optim.step，无效但看似有效。fix 后训练稳定。”

案例 17: 长 ctx 训练 RoPE base 调整

背景：把 8K ctx pretrain model 扩到 128K ctx SFT，val loss 不降甚至升。

诊断：RoPE base=10000 对长 pos 有 extrapolation 失效，attention pattern 崩。

Action : RoPE base → 500000 (YaRN / LongRoPE 建议)，或用 NTK-aware scaling。

Result : 128K val loss 下降，perplexity ≈ 8K baseline。

案例 18: Multi-source data weight 调优 with DoReMi

背景：预训 7B 用 5 类 domain 数据，naive weight (按 size) val loss 不理想。

Action : 跑 1B proxy model + DoReMi 学 optimal weight，重训 7B。

Result : downstream benchmarks (HumanEval, MMLU) 平均 +2 分。

案例 19: MoE aux-loss-free 切换

背景：训 128 experts MoE 用 aux loss ($\alpha=1e-2$)，router 梯度受 aux 干扰，主 loss 收敛慢。

Action : 换 DeepSeek aux-loss-free (bias tuning, $\gamma=1e-3$)， α 降到 $1e-4$ 作 fallback。

Result : main loss 收敛快 30%，final val loss -0.05。

案例 20: RL 中 KL blow up 处理

背景：GRPO 训 10 iter 后 KL 突然 5.0+，policy 输出退化。

Action : (a) adaptive KL coef (KL > target × 2 时 coef × 1.5) ; (b) 每 1000 step update ref = policy avoid drift。

Result : KL 稳定在 0.5-1.0 范围，训练继续。

“框架 X 已做，我加了 Y” delta 对比表

面试官问“Megatron 已经有 XX，你的贡献是什么”时，给个具体 delta：

基线做的	常见 delta 优化	量化收益
DDP bucket 25 MB	调 200 MB + grad-as-view	+5-10% MFU
FSDP FULL_SHARD	切 HSDP, 跨节点用 DDP	+30-50% wall-clock
FSDP BACKWARD_PRE	加 limit_all_gathers 控制显存峰值	显存 -10 GB (avoid OOM)
Megatron TP+AR	<code>--tp-comm-overlap</code> (async TP)	+10-15% MFU
Megatron 1F1B PP	<code>--overlap-moe-expert-parallel-comm --delay-wgrad-compute</code>	MoE +10-15% MFU
DeepEP V1	升级 V2 (SM 20 → 4-6)	+15-20% MFU
Naive optim state FP32	<code>--use-precision-aware-optimizer</code>	-12 GB/GPU (nearly free)
Full activation recompute	Fine-grained (avoid a2a re-trigger)	compute overhead 33% → 6%
Sync checkpoint 5 min	Async + in-memory peer backup	Overhead 5% → 0.5%
Fixed batch by count	Sequence packing + FA varlen	+50-100% throughput
Vision + LLM 同 TP=8	Split device group	+10-20% MFU
Naive RL rollout in trainer	vLLM + colocated + weight sync	Rollout 5-10× 加速
PPO with value model	GRPO (drop value)	-25-30% GPU need
BF16 grad AR	FP32 grad AR at W > 1024	Loss 曲线稳定
Rank 0 gather checkpoint	DCP sharded + resharding	Checkpoint 5× 快 + elastic

表 44 常见“delta 优化”对比表。用来在面试里快速 anchor “我在 X 基础上做的 Y” 的具体 delta。

面试里怎么回答 “MFU 只 30%，你怎么优化”

错误答法：一上来堆技术名词——“我用 DualPipe + FP8 + FlashAttention 3 + selective recompute”。面试官会立刻打断问“为什么？”

正确答法（**framework**）：

1. 先诊断：跑 nsys 一步 profile，判断 bottleneck 属于三堵墙的哪一堵
2. 如是 **memory bound**：先开 precision-aware optim (免费)，再看 activation recompute
3. 如是 **comm bound**：查是 DP AR / TP AR / EP a2a / PP P2P 哪种，各自有对应 overlap 方案
4. 如是 **compute inefficient**：查 GEMM M 维、kernel fusion、CUDA graph
5. 每一步都量化 MFU 变化，让面试官看到你的诊断能力

典型对话：

> 面试官：“我们 MoE 训练 MFU 只 35%，你会怎么优化？”>> 你：“我先想问几个问题——(1) 你们模型多大？多少 expert？(2) 集群多大？H100 还是 H800？跨节点 IB 是什么速率？(3) profile 里 comm 占多少比例？如果 comm > 30%，我会先 attack a2a，考虑 DeepEP V2 + Megatron `--overlap-moe-expert-parallel-comm`；如果 comm < 10%，那 bottleneck 不在这儿，可能是

activation recompute 过多或 grouped GEMM 小-M。基于三堵墙框架诊断，才能对症下药。举个我做过的案例.....”

这种回答会立刻把面试从“背 flag”变成“平等技术讨论”。

面试故事的 anti-pattern

1. “我调 **flag**”：说了等于没说
2. “我用 **A100**”：硬件不是你的贡献
3. “我实现了 **XX**”（但显然是抄框架源码）：面试官一问细节就露馅
4. 无数字：“训练变快了”没意义，说“MFU 从 X 到 Y”或“step time 从 A 到 B”
5. 技术堆砌：一次提 10 个优化，每个都浅——不如深挖 1-2 个
6. 推给团队：“这是我们团队做的”——一定要有你个人的技术贡献

一个完整的 STAR 故事示例

Situation: “上一份工作里我们训 30B dense LLM on 128 H100，用 Megatron TP=8/PP=2/DP=8, seq=8K, BF16。刚起手时 MFU 只 32%，比 Megatron 官方 benchmark 差 15 个点。”

Task: “我负责找出并 close 这个 gap，目标 45% MFU。”

Action: “跑了 nsys — step profile，看到几个问题：

1. DDP grad AR 完全串行在 backward 结束后，overlap 只 60%——查 code 发现 bucket_cap=25MB 默认；
2. TP forward 时 AR kernel 占 SM，GEMM idle 5%；
3. activation 显存 70/80 GB，被迫开 full recompute，compute overhead 33%。

对应 (1) 调 bucket 200MB + grad-as-view + persistent_workers；(2) 升级 TE 到支持 async TP (--tp-comm-overlap)；(3) 关 full recompute 改 selective + FP8 activation，activation 降到 45 GB。”

Result: “MFU 32% → 47%，step time -32%，训一个 epoch 从 5 天 → 3.4 天。团队后来把这三个 patch 合到 baseline recipe，其他模型也受益。”

每一步都能追问细节：面试官问“为什么 FP8 activation 而不是 offload？”你要能答“activation 大头是 residual + LN input，offload 走 PCIe 有 CPU 与 dataloader 竞争风险；FP8 是 kernel 内 quant/dequant 无 PCIe，且 Hopper native 支持。”

结语

分布式训练面试问的从来不是“你会不会调 flag”，而是“你能不能诊断 + 有 delta 贡献”。这本书从头到尾都在建立诊断框架（三堵墙、Roofline、通信量公式），让你面对任何具体问题都能先分析再动手。

面试前一晚：翻附录 D 的 100 题 + 附录 E 的对比表 + 附录 A 的数字速查。面试当场：先问清楚 setup，用三堵墙 anchor，用 STAR 讲你做过的事。

祝拿 offer。

附录 F :Megatron 上的改造 —— 教你回答“你们框架改了些什么”

面试里最能拉开档次的一类问题：

“你们用 Megatron / DeepSpeed ,那这些框架已经把 TP/PP/EP 都做好了、把 FlashAttention 集成了、把 async ckpt 也做了。你在上面具体改造了什么，让模型训得又快又稳？”

只答“我们用了 Megatron 官方 recipe”或“我们调了几个 flag”，会被立刻判定为“tuner”，不是“framework builder”。真正加分的回答是把你的改造分四个维度说清楚：

- 1. 算子层：kernel/GEMM/attn/norm/a2a — 哪些是自研，哪些是替换上游
- 2. 算法层：optimizer/router/schedule/packing — 哪些是“跟随论文改”，哪些是自己 tune 出来的
- 3. 稳定性：spike skip/NaN heal/grad protect/ckpt guard — 让长训不炸的“保护罩”
- 4. 监控 + 基础设施：从 grad_norm dashboard 到 elastic restart 到 topo-aware placement

如果你的岗位还覆盖 post-training，同一个问题会以“你们用 verl，那 verl 都做好了，你做了什么”的形式再问一遍 —— 第六节按同样四个维度给了 RL 栈的版本。

这一附录把 2023-2025 主流大厂 (Meta / DeepSeek / Kimi / Qwen / OpenAI-inferred / Anthropic-inferred) 公开的框架改造拆解，配 patch 伪代码，形成你可以拿去说“这个我做过 / 我知道怎么做”的具体清单。

Warning. 这里的每一项都是公开可查的技术。你在真实面试中要说的应当是你实际做过的那部分。这一章的目的是让你能认出一个改造属于哪个维度、量级多大、trade-off 在哪；不是让你“背下来假装做过”—— 面试官一追问细节就会露馅。

全景图：四个维度的改造清单

维度	涉及范围	一句话定位
算子层	kernel / GEMM / attn / norm / a2a	每个 op 都能再快 X%，是“物理加速”
算法层	optimizer / router / schedule / packing	跟随论文 + 自己 tune，是“每 token 学更多”
稳定性	spike skip / NaN heal / ckpt guard	长训不炸的“保护罩”，决定能否训完
监控 + Infra	dashboard / elastic / topology	7×24 无人值守的“地基”

表 45 框架改造的四个维度。上层做加速，下层做兜底；越往下越决定“是否能训完”，越往上越决定“训多快”。面试里 4 个维度都要能各举 1-2 个例子。

一. 算子层改造

Megatron 官方已经集成 FlashAttention、Transformer Engine (TE)、grouped GEMM。但生产集群还有 5 类高频改造：

1.1 替换 attention kernel

基线：TE 内的 `DotProductAttention` (调 cuDNN backend 或 FlashAttention-2)

改造：

- **FlashAttention-3** (Hopper only, 2024 Q3+ 版本)：H100 上 forward $1.5\text{--}2\times$ 更快。但 varlen path 到 2024 Q4 才稳。生产集群需 backport patch，加 shape guard fallback。
- **Ring FlashAttention** (Zhu 2023)：长上下文时 CP 场景默认 kernel。Megatron `--context-parallel-size` 默认调 cuDNN，替换成 Ring FA 后 128K seq 快 30%。
- **TriDao ThunderKittens / Mamba-attention** :experimental，用于 SSM 或 hybrid 结构。

Patch 位置：megatron/core/transformer/attention.py::`DotProductAttention.forward`，加 `attn_backend` config。

Trade-off：新 kernel 常有 shape / dtype 限制 (如 `head_dim ∈ {64, 128}`)。要写 fallback：

```
if attn_backend == "flashattn3":
    try:
        out = fa3_kernel(q, k, v, ...)
    except (RuntimeError, ValueError):
        # shape/dtype not supported → fallback
        out = fa2_kernel(q, k, v, ...)
        _fa3_fallback_counter.inc() # track how often we fallback
```

收益：H100 long-ctx +15–30% MFU。Kimi K2 报告 FA3 是 long-ctx 稳定 recipe 的关键。

1.2 Fused RMSNorm / SwiGLU / RoPE

基线：TE `LayerNormLinear` (LN + linear fused)，`RotaryPositionEmbedding` 独立 kernel

改造：写自己的 Triton fused kernel，把 “RMSNorm + Linear + SwiGLU gate + RoPE” 尽可能 fuse 成 1-2 个 kernel。

为什么：

1. TE fused kernel 是 CUDA C++，难扩展——想加个 QK-norm 或改 eps upcast 就得改 C++ 重编
2. Triton 是 Python，快速迭代——2 天能出一个 prototype
3. Fused RMSNorm+Linear+RoPE 三 op 合一，H100 上比 TE 快 8-12% (Kimi K2 报告)

Patch：把 megatron/core/models/gpt/gpt_layer_specs.py 里的 `norm_and_linear` module 换成自定义 Triton 版本，注册进 spec。

典型 **fused kernel** 伪代码：

```
@triton.jit
def fused_rmsnorm_qkv_rope_kernel(
    X_ptr, W_ptr, cos_ptr, sin_ptr, Y_ptr,
    stride_x, stride_w, stride_y,
    H, D_head, N_head, eps: tl.constexpr,
    BLOCK: tl.constexpr,
```



```

):
    # 1. Load X and compute RMSNorm (var in FP32)
    x = tl.load(X_ptr + offsets)
    x_f32 = x.to(tl.float32)
    inv_rms = 1.0 / tl.sqrt(tl.sum(x_f32 * x_f32) / H + eps)
    x_norm = (x_f32 * inv_rms).to(x.dtype)
    # 2. GEMM: qkv = x_norm @ W_qkv
    qkv = tl.dot(x_norm, W_qkv)
    # 3. Split into q, k, v and apply RoPE to q, k
    q, k, v = split(qkv)
    cos = tl.load(cos_ptr + ...)
    sin = tl.load(sin_ptr + ...)
    q = rope(q, cos, sin)
    k = rope(k, cos, sin)
    # 4. Write out
    tl.store(Y_ptr + q_offsets, q)
    ...

```

Trade-off :

- Triton 在 shape 变化时会 re-compile (200 ms+) , 需要开 `@triton.autotune` 或 warmup
- 只对 BF16 优化 ; FP8 走 TE 走 C++
- kernel bug 更难 debug , 需要单元测试对齐 unfused reference (`torch.allclose(out_fused, out_ref, atol=1e-3)`)

收益 : H100 上 fused normalized MLP layer +8-12% wall-clock.

1.3 Grouped GEMM 替换 for MoE

基线 : Megatron 默认 MoE 每个 expert 独立 GEMM ,
`for e in experts: y_e = act(x_e @ W_e)` 。 expert 多时 (>32) launch overhead 大。

改造 : 换成 **grouped GEMM** —— 一次 kernel 处理所有 expert 的 GEMM。CUTLASS 3.5+ 原生支持 , 或用 `torch._grouped_mm` (Torch 2.4+)。

Patch 位置 : `megatron/core/transformer/moe/moe_layer.py::SequentialMLP.forward` 。

典型对比 :

方式	expert 数	fwd time / MoE layer
Loop of GEMM (baseline)	256	4.2 ms
Grouped GEMM (CUTLASS)	256	1.1 ms
Fused with dispatch (DeepEP)	256	0.8 ms

收益 : DeepSeek-V3 报告 grouped GEMM 是 fine-grained expert (256+) 可行的前提 ; 否则单纯 expert 数增加会被 launch overhead 吃掉。

进阶 : 把 dispatch a2a + grouped GEMM + combine a2a 完全 **fuse** 成一个 mega-kernel (DeepEP V2 做的事) —— 但工程复杂度高 , 多数团队直接用 DeepEP , 不自研。

1.4 自研 all-to-all kernel (DeepEP-style)

基线 : `torch.distributed.all_to_all_single` → 走 NCCL。NCCL a2a 用 20+ SM 做通信 , 抢 GEMM 的 SM。

改造：

- **DeepEP** (DeepSeek 开源, 2024) : 用 IBGDA (InfiniBand GPU-Direct Async) 让 GPU 直接发 RDMA verbs, 不经过 CPU proxy。SM 占用降到 4-6。
- **NVSHMEM** : NVIDIA 官方类似方案, Megatron 2025 起集成。

Patch 位置: `megatron/core/transformer/moe/token_dispatcher.py::TokenDispatcher`。把 `dist.all_to_all_single` 换成 `deep_ep.dispatch()` + `deep_ep.combine()`。

要点：

1. 需要网络 driver 支持 IBGDA (Mellanox MLNX_OFED 5.4+)
2. 需要 GPU peer memory access 权限 (root/CAP_SYS_ADMIN 或 privileged container)
3. intra-node vs inter-node 用不同 code path (intra 走 NVLink direct , inter 走 IBGDA)——hierarchical a2a

收益：MoE MFU 从 35% → 51% 是 Kimi / DeepSeek 报告的典型数字。DeepEP V1 → V2 又能再 +15%。

Trade-off : 与 CUDA graph 不完全兼容 (IBGDA 需要 host-side unblocking event) , 需要 mode switch。

1.5 CUDA graph capture

基线：Megatron 2024 起支持 `--enable-cuda-graph` , 但仅覆盖 forward 主 path。

改造：

- 把 forward+backward 都进 graph
- Varlen 场景按 seq_len bucket 分别 capture (bucket 数 5-8)
- 与 dataloader stream 隔离, 避免 CPU launch 抖动打断 graph replay

Patch 位置: `megatron/training/training.py::train_step` , 加 graph capture wrapper :

```
if step == warmup_step + 1 and use_cuda_graph:
    # capture on first "real" step (after warmup, shapes stable)
    graph = torch.cuda.CUDAGraph()
    with torch.cuda.graph(graph):
        loss = model(static_input)
        loss.backward()
    _graph_cache[bucket_id] = graph

if step > warmup_step + 1:
    static_input.copy_(real_input)
    graph.replay()      # replaces train_step body
```

收益：小 batch (< 4 seq/rank) 或短 seq (< 2K) 场景 CPU launch overhead 从 12% → 1%。GPT-4 rumored 用到, Meta Llama-3 405B 训练用类似技术。

Trade-off : dynamic shape / control flow 全崩, dropout mask 需要 external RNG state。开 graph 之前必须先能保证 shape 稳定 —— packing 是前提。

算子层改造的整体清单

改造点	基线 (Megatron 官方)	生产改造	典型收益
Attention kernel	TE cuDNN / FA2	FA3 + fallback guard	+15-30% MFU
Norm+Linear fused	TE C++	Triton (norm+linear+RoPE)	8-12%
MoE GEMM	loop of GEMM	Grouped GEMM (CUTLASS)	1 layer 3-4×
MoE a2a	NCCL a2a	DeepEP / NVSHMEM (IBGDA)	MoE +15-20% MFU
Graph capture	部分 forward path	完整 fwd+bwd + varlen buckets	小 batch +10-15%
Precision cast	TE 内部管理	关键路径手动 upcast (LN var, QK^T accum)	训练稳定性
Communication SM	NCCL 默认	NCCL_MAX_NCHANNELS=8 + 自定 topology	±5% MFU

表 46 算子层的 7 类常见改造。每一项都可以做深，“哪一项是你亲手写的”是面试差异化关键。

面试考点. 面试深挖：“你说你写了 **fused Triton kernel**，遇到什么坑？”

正确答法（体现真做过）：

1. **shape** 不稳定 → **autotune** 缓存失效：Triton 遇新 shape 重编 200 ms，varlen 场景灾难。修：加 shape rounding(`M = ceil(M / 128) * 128`)或 pre-warm 所有可能 shape。
2. **BF16 accumulator drift**：Triton 默认 accum 用输入 dtype。RMSNorm var 要 explicit `tl.float32` upcast，否则 loss slow-degrade。
3. 不同 **head_dim** 需要不同 **BLOCK**：写 dispatch table，`if D == 128: BLOCK_D = 128; elif D == 64: BLOCK_D = 64`。
4. **nsys** 抓不到 Triton 内部 **sub-kernel**：只能看到一个 Triton launch。需要用 `TRITON_KERNEL_DUMP_DIR` 落 PTX 再逐行分析。

二. 算法层改造

算子改造是“物理加速”，算法改造是“每 token 学更多”。这一节更影响 loss curve 与收敛速度。

2.1 Optimizer 替换：AdamW → Muon / Lion

基线：Megatron `--optimizer adam`（默认 AdamW，`beta=(0.9, 0.95)`，`wd=0.1`）

改造：

- **Muon** (Kimi K2, 2025)：2D+ 权重用 Muon (Newton-Schulz orthogonalize)，1D 权重仍 AdamW。Kimi 报告 loss 曲线更平滑，无 spike。
- **Lion** (Google 2023)：`sign(momentum)` 直接 apply，显存少 50%。少数团队试。

Patch 位置：`megatron/core/optimizer/__init__.py::get_megatron_optimizer`。新增 `MuonOptimizer` class，参数分组 (2D → Muon, 1D → AdamW)：

```
def build_muon_optimizer(model):
    matrix_params, vector_params = [], []
    for n, p in model.named_parameters():
        if p.ndim >= 2 and "embed" not in n:
            matrix_params.append(p)
        else:
            vector_params.append(p)
    return CompositeOptimizer([
        Muon(matrix_params, lr=2e-3, beta=0.95, wd=0.1),
        AdamW(vector_params, lr=4e-4, betas=(0.9, 0.95), wd=0.1),
    ])
```

参考实现见 `src/distributed_training/l2_optimizers.py`。

Trade-off :

- Muon 与 FSDP FULL_SHARD 不兼容 (Newton-Schulz 要 full matrix), 只能 ZeRO-1 → 参数复制 → 每卡显存翻倍
- Kimi K2 因此选 ZeRO-1 + PP + EP, 不用 **FSDP**
- Muon 需要 4-5× 大的 LR, 需要重新调 warmup + wd 组合

面试点：“为什么 **Kimi** 用 **Muon** 而 **DeepSeek** 用 **AdamW**?” 答：Kimi 押注 Muon 的稳定性 + MuP scaling；DeepSeek 已经用 FP8+ZeRO+MoE 组合叠了太多变量，不想再引入 Muon 的额外风险。都是合理选择，取决于团队 risk appetite。

2.2 MoE 路由：aux-loss → aux-loss-free

基线：Megatron `--moe-aux-loss-coeff 1e-2`，用 Shazeer 2017 的 load-balance loss。

改造：DeepSeek-V3 提出的 **bias-based aux-loss-free** balancing。每个 expert 加动态 bias，“最近少被选的” bias 自动加大 → 下次更容易被选。无需在 loss 里加 aux 项。

Patch 位置：`megatron/core/transformer/moe/router.py::TopKRouter.forward`：

```
# baseline
scores = softmax(logits)
topk_scores, topk_idx = scores.topk(k, dim=-1)
aux_loss = compute_load_balance_loss(topk_idx) # ← add to total loss

# aux-loss-free
scores = softmax(logits) + self.expert_bias # ← bias added to logits
topk_scores, topk_idx = scores.topk(k, dim=-1)
# update bias every step (EMA of imbalance)
with torch.no_grad():
    hit_rate = compute_hit_rate(topk_idx) # per-expert hit fraction
    imbalance = hit_rate - hit_rate.mean()
    self.expert_bias -= self.bias_lr * torch.sign(imbalance)
```

收益：DeepSeek 报告 main loss 收敛快 20-30% (没有 aux 干扰主 loss 梯度), expert 利用率 CV < 5%。

Trade-off :

- `bias_lr` 需要 tune (DeepSeek 用 1e-3, 太大会震荡)
- `expert_bias` 需要放 checkpoint (每 expert 一个 scalar → 千个数)

- 与 `--moe-expert-capacity-factor` 交互复杂，需要禁用 capacity drop 或改成 soft drop

2.3 Multi-Token Prediction (MTP)

基线：Megatron 默认预测下一个 token。

改造：额外加 n 个“预测未来第 n 个 token”的 head (DeepSeek-V3 用 $n=1$ ，即 MTP head 预测 $t+2$)。Loss = main NTP loss + α * MTP loss。

Patch 位置：megatron/core/models/gpt/gpt_model.py::GPTModel，加 mtp_head module + 修改 loss 计算。

```
class GPTModelWithMTP(GPTModel):
    def __init__(self, ...):
        super().__init__(...)
        self.mtp_head = MTPHead(hidden_size, vocab_size, depth=1)

    def forward(self, ..., labels):
        hidden = self.transformer(...)
        main_logits = self.lm_head(hidden)
        loss_main = F.cross_entropy(main_logits, labels)

        # MTP: predict labels shifted by +1 more
        mtp_logits = self.mtp_head(hidden, main_logits)
        loss_mtp = F.cross_entropy(mtp_logits, labels_shifted)

        return loss_main + self.mtp_alpha * loss_mtp
```

收益：

- 训练时：main loss 略降 (MTP 相当于额外 supervision，正则化效果)
- 推理时：MTP head 可作 speculative decoding 的 draft，2-3× inference 加速

Trade-off：模型参数增 1-2% (一个 head + 一个 transformer block)，显存 & compute 增。

2.4 MLA 实现 (DeepSeek 系)

基线：Megatron 默认 MHA / GQA。

改造：实现 **Multi-Head Latent Attention**：KV 压缩到低秩 c_{kv} ，attention 时上采样。K 分成 rotated (RoPE) 和 non-rotated 部分。

Patch 位置：新建 megatron/core/transformer/mla_attention.py，重写 QKV projection 逻辑 + 修改 KV cache (推理时)。

关键代码骨架：

```
class MultiHeadLatentAttention(nn.Module):
    def __init__(self, ...):
        # down-project to compressed KV
        self.W_dkv = ColumnParallelLinear(H, d_c + d_rope)
        # up-project back (per head)
        self.W_uk = ColumnParallelLinear(d_c, n_head * d_head_nope)
        self.W_uv = ColumnParallelLinear(d_c, n_head * d_head)
        # Q: partial nope + partial rope
        self.W_q = ColumnParallelLinear(H, n_head * (d_head_nope + d_rope))
```

```
def forward(self, x):
    c_kv, k_rope = self.W_dkv(x).split([d_c, d_rope], dim=-1)
    # ↑ this is what gets cached at inference (much smaller than K/V)
    k_nope = self.W_uk(c_kv).view(..., n_head, d_head_nope)
    v = self.W_uv(c_kv).view(..., n_head, d_head)
    q_nope, q_rope = self.W_q(x).view(..., n_head, -1)
                                .split([d_head_nope, d_rope], dim=-1)
    # apply RoPE only to the rope-partition
    q_rope = rope(q_rope, ...); k_rope = rope(k_rope, ...)
    q = torch.cat([q_nope, q_rope], dim=-1)
    k = torch.cat([k_nope, k_rope.unsqueeze(-2).expand(-1, -1, n_head, -1)],
dim=-1)
    return flash_attn(q, k, v)
```

Trade-off :

- TP 切分复杂：W_uk / W_uv 是 per-head，需要沿 head 维 column parallel
- FA kernel 需要接受 rotated + non-rotated 拼接的 head_dim (FA3 才原生支持)
- 训练/推理代码 diverge (KV cache 形态不同)，需要双 code path

收益：KV cache 缩到 GQA-8 的 1/4，128K 推理场景内存瓶颈解除。训练 loss 与 GQA 打平。

2.5 Sequence packing 精细化

基线：Megatron 支持 `--packed-sequence`，按 fixed length 顺序拼接。

改造：

- **BFD (Best-Fit-Decreasing)** 或 **FFD (First-Fit-Decreasing)** packing，减少 padding。参考实现 `src/distributed_training/10_seq_packing.py`。
- 加 **cost-based balancing**：不同 batch 的 attention cost $\propto \sum_i s_i^2$ (因为 attention 是 $O(S^2)$)，DP rank 之间平衡 cost 而不是 token 数。
- **cross-document attention mask**：pack 里的不同 document 之间必须 mask 隔离，用 FA 的 varlen path (`flash_attn_varlen_func`) 或 block-diagonal mask。

Patch 位置：`megatron/core/datasets/gpt_dataset.py::_pack_documents`，改 packing 策略。

Trade-off :

- BFD 需要 look-ahead 一个 buffer (如 1000 个 sample)，流式训练需要 online BFD
- cost balancing 会让 rank 间收到不同 seq 数 → hidden state 数不同 → 需要 padding to max 或者 uneven collective (gloo 支持，NCCL 需要 pad)

收益：Padding 率从 15% → 2%，等效 +13% throughput。Meta Llama-3 训练报告 packing 是 105B → 405B 训完的关键。

2.6 LR schedule tweak : WSD + mini-restart

基线：Megatron 默认 `--lr-decay-style cosine`。

改造：

- WSD (Warmup-Stable-Decay)，见 P3 章
- Mini-restart：训到 60% 时 LR $\times 0.3$ rewarm 500 步再 stable
- Curriculum-aware LR：切换 data phase 时短 warmup 200 步

Patch 位置：`megatron/core/optimizer_param_scheduler.py::OptimizerParamScheduler`：

```

def get_lr(self, step):
    if step < self.warmup:
        return self.peak * step / self.warmup

    # WSD stable
    if step < self.decay_start and step < self.rewarm_step:
        return self.peak

    # Mini-restart
    if self.rewarm_step <= step < self.rewarm_step + self.rewarm_warmup:
        prog = (step - self.rewarm_step) / self.rewarm_warmup
        return self.peak * (self.rewarm_ratio +
                           (1 - self.rewarm_ratio) * prog * 0) # ramp

    # ... etc

```

收益：Kimi K2 报告 mini-restart 让 loss 曲线在 60% training 时不停滞（原本会有平台期）。

三. 稳定性改造

这一部分决定“能不能训完 30 天不炸”。生产集群里，稳定性问题永远是 P0，其次才是 MFU。

3.1 Loss spike 自动 skip

基线：Megatron 默认 grad clip = 1.0，spike 时 clip 但 optim step 仍执行。

改造：

```

# in train_step, right before optim.step()
grad_norm = clip_grad_norm_(model.parameters(), max_norm=1.0)

# Track EMA of grad_norm for outlier detection
_grad_norm_ema.update(grad_norm.item())
threshold = _grad_norm_ema.median * 10 # 10x median

if grad_norm > threshold or torch.isnan(grad_norm) or torch.isinf(grad_norm):
    # SKIP this update
    logger.warning(f"[step {step}] skip: grad_norm={grad_norm} "
                  f"vs 10xmedian={threshold}")
    _skip_counter.inc()
    optim.zero_grad()
    # OPTIONAL: log the offending batch for post-mortem
    torch.save({"input_ids": batch, "step": step},
               f"/spike_logs/step_{step}.pt")
    return # skip optim.step + scheduler.step

```

Patch 位置： megatron/training/training.py::train_step

收益：Meta OPT-175B 报告 45+ 次 manual restart，改成 auto-skip 后可零人工干预。Anthropic Claude 训练报告类似机制。

Trade-off：

- skip 太多会让 loss 收敛慢（每个 skip 相当于 free 一个 batch）
- 需要 tune threshold：太严（5x）会误 skip 正常 spike，太松（100x）会漏掉真正的 divergence
- 需要 alert：连续 3 步 skip → page 值班人

3.2 NaN / Inf 自愈

基线：NaN 出现就 crash。

改造：分层 NaN 处理：

```
# 1. Detect NaN in loss
if torch.isnan(loss) or torch.isinf(loss):
    # 2. Rollback to previous "clean" state
    if _clean_state_available():
        model.load_state_dict(_clean_state)
        optim.load_state_dict(_clean_optim_state)
        logger.warning(f"[step {step}] NaN loss, rolled back to step {step-N}")
        continue

    # 3. If no clean state (very early), just skip
    optim.zero_grad()
    _nan_counter.inc()
    if _nan_counter.value > 5:
        # ... hard crash, need human intervention
        raise RuntimeError("Too many consecutive NaNs")

# 4. Every K clean steps, snapshot as new "clean" state (in memory)
if step % 100 == 0 and grad_norm < threshold:
    _clean_state = {k: v.clone() for k, v in model.state_dict().items()}
    _clean_optim_state = deepcopy(optim.state_dict())
```

Patch 位置： megatron/training/training.py::train_step

收益：训练一次 spike 不会导致重启（原本要从 checkpoint restore 20 min，现在 rollback in-memory 200 ms）。DeepSeek-V3 训练报告类似机制。

Trade-off：

- 需要额外显存存 clean state（整个 model + optim 20 GB for 70B on H100）
- 若 spike 是数据引起的（poisoned batch），rollback + skip 后需要 skip 该 batch 避免立即再触发

3.3 Gradient 保护 (unit-norm + skip huge)

基线：clip_grad_norm_(model.parameters(), max_norm=1.0) 全局 clip。

改造：

- **Per-layer clip** (in addition to global)：某一层 grad 特别大时单独 clip 该层，不影响全局 norm
- **Gradient masking on outlier**：某个 param 的 grad 单元 $> \mu + 10\sigma$ ，替换为 median
- **No-op step**： $||\text{grad}|| > 1000$ 时直接 optim.zero_grad() 不 step

```
# Per-layer clip
for name, p in model.named_parameters():
    if p.grad is None: continue
    local_norm = p.grad.norm()
    if local_norm > 10.0: # 10 is per-layer cap
        p.grad.mul_(10.0 / local_norm)
        _layer_clip_counter[name].inc()
```



```
# Global clip after
total_norm = clip_grad_norm_(model.parameters(), 1.0)

# Log which layer contributed most to spike
if total_norm > 5.0:
    top = sorted(_layer_clip_counter.items(), key=lambda x: -x[1].value)[:3]
    logger.warning(f"[step {step}] high grad_norm={total_norm}, "
                  f"top layers: {top}")
```

收益：Llama-3 训练报告用类似 per-layer clip，某一层的病态 grad（如 embedding norm 出问题）不会污染全局。

3.4 Checkpoint double-buffer + async save

基线：Megatron 支持 `--async-save`，但 optim state save 时会 block train 1-2 min（大 model）。

改造：三级 **checkpoint**：

1. **L1 (in-memory backup on peer)**：每 100 step，state dict → 隔壁 node CPU DDR。约 3 s（NVLink 到 CPU 再到 IB）。用于 in-flight NaN rollback。
2. **L2 (disk async)**：每 1000 step，flush 到本地 NVMe。约 30 s，与 train 并行。
3. **L3 (remote object store)**：每 10000 step 或 6h，upload 到 S3 / OSS，用于灾备。

Patch 位置：megatron/training/checkpointing.py::save_checkpoint

架构：

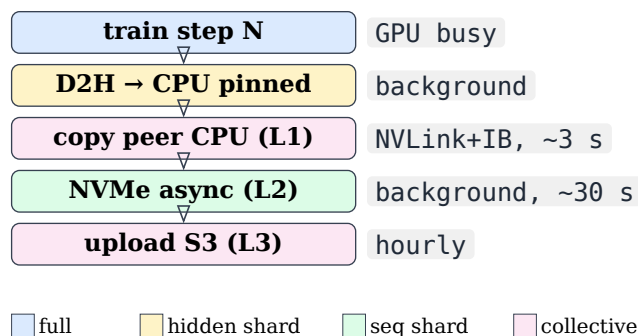


图 35 三级 checkpoint 架构。L1 覆盖 in-flight NaN rollback，L2 覆盖单 node 挂，L3 覆盖整个集群失效。生产训练里 L1+L2 是标配。

收益：

- Checkpoint 阻塞 train 时间 5 min → 15 s（Meta OPT 论文数字）
- 恢复时间 20 min → 30 s（in-memory recovery）
- Kimi K2 报告 30 天训练 zero manual intervention 是靠 L1+L2

Trade-off：

- L1 需要选 peer node（跳过同 node 避免同时挂）
- 需要 SSD 冗余（RAID）或每 node 独立 disk

3.5 Deterministic training

基线：Megatron 未强制 deterministic。dataloader RNG, dropout, kernel non-determinism 都会让“同 seed 复现”失败。

改造：

```

# In training entry point:
torch.use_deterministic_algorithms(True, warn_only=True)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
os.environ["CUBLAS_WORKSPACE_CONFIG"] = ":4096:8"

# For distributed shuffling
gen = torch.Generator()
gen.manual_seed(base_seed + dp_rank)  # per-DP-rank seed

# Log all RNG state to checkpoint
ckpt["python_rng"] = random.getstate()
ckpt["numpy_rng"] = np.random.get_state()
ckpt["torch_rng"] = torch.get_rng_state()
ckpt["cuda_rng"] = torch.cuda.get_rng_state_all()
ckpt["gen_rng"] = gen.get_state()

```

收益：debug 时能复现问题（哪个 step 出 NaN，rerun 保证 hit 同一 batch）。生产 tune 时可以做 controlled experiment。

Trade-off：deterministic mode 有 3-5% 性能损失（因为禁用了 non-det kernel），生产训练里通常只在 **debug** 期开，正式 run 关掉。

稳定性改造清单

改造点	基线	生产改造	收益
Loss spike	手动重启	auto skip step + alert	−90% manual intervention
NaN 处理	crash	in-memory rollback 3 层	恢复时间 20 min → 30 s
Grad clip	global norm 1.0	per-layer + skip huge	避免单层污染全局
Checkpoint	同步 5 min block	async + L1/L2/L3 三级	overhead 5% → 0.5%
Determinism	关	debug 时开，全 RNG state 存 ckpt	可复现，可 A/B
Elastic restart	无	torchrun −max-restarts + 自动重连	硬件挂不停 job

四. 监控改造

“能不能观测到问题”决定“问题多快被发现”。Megatron 默认只 print loss 到 stdout，生产集群远远不够。

4.1 训练主指标 dashboard

基线：Megatron log 每 N step 一次 loss + grad_norm 到 tensorboard。

改造：至少监控这 15 项（Meta / DeepSeek 报告都提到）：

类别	指标	用途
Loss	train loss, val loss, per-domain loss	收敛监控
Grad	grad_norm (global, per-layer), grad_var	稳定性
Weight	weight_norm per param, weight/init 比值	drift 监控
Activation	per-layer activation min/max/mean/std	NaN 提前预警
Optim	effective lr, m/v ratio, <code>sqrt(v)</code>	Adam 健康度
Throughput	MFU, HFU, samples/sec, tokens/sec/GPU	效率
Memory	allocated GB, reserved GB, peak, fragmentation	OOM 预警
Comm	AR bandwidth, a2a bandwidth, P2P latency	通信健康
Rank imbalance	per-rank step time p50/p99, straggler idx	straggler 检测
Hardware	GPU util%, HBM util%, NVLink util%, IB util%	硬件健康
Temperature	GPU temp, throttle events	降频预警
ECC error	SBE/DBE count per GPU	硬件寿命
NCCL	error count, timeout events, comm hang	网络故障
Dataloader	buffer depth, worker idle time, sample skip	I/O 瓶颈
Skip step	count of skipped steps + reason	稳定性 audit

表 48 生产训练 dashboard 必备 15 类指标。多数团队实现是 Prometheus + Grafana + DCGM exporter + 自研 Python exporter。

Patch 位置：新建 `megatron/training/monitor.py`，注册各类指标 exporter；在 `train_step` 结尾统一发到 Prometheus pushgateway。

参考实现见 `src/distributed_training/15_training_monitor.py`（简化版）。

4.2 Straggler 检测

基线：无。慢 rank 拖慢整体，但没告警。

改造：

```
def train_step(step):
    t0 = time.time()
    # ... normal train step ...
```

```

t_local = time.time() - t0

# Gather per-rank step time
all_times = [torch.zeros(1) for _ in range(world_size)]
torch.distributed.all_gather(all_times,
                             torch.tensor([t_local], device="cuda"))
times = [t.item() for t in all_times]
median = statistics.median(times)

for r, t in enumerate(times):
    if t > median * 1.3: # 30% slower than median
        _straggler_counter[r].inc()

# Escalate: if same rank straggler for 10 consecutive steps
for r, cnt in _straggler_counter.items():
    if cnt.recent(10) >= 10:
        logger.error(f"[straggler] rank {r} slow for 10 steps, "
                     f"check GPU health!")
        emit_alert(f"straggler-{r}")

```

Patch 位置： megatron/training/training.py::train_step

收益：一个 slow GPU (吃 ECC) 会拖慢整个 job 30%+。早期发现 (10 步内) 可以 drain 该 node 换机器。Meta MegaScale 论文详细描述。

4.3 NCCL 拓扑探测

基线：Megatron 假设 rank $\leftrightarrow$ GPU 是简单顺序映射。

改造：训练开始前跑 nccl-tests，画出实际 all-reduce 带宽矩阵，检查：

- 同一 node 8 卡应该 NVLink 300+ GB/s
- 跨 node 应 IB 100 GB/s
- 若某 pair 只跑到 40 GB/s $\rightarrow$ 网络故障或 topology 错

```

def probe_topology():
    """Run all_reduce on each pair and log bandwidth."""
    results = {}
    for i in range(world_size):
        for j in range(i + 1, world_size):
            group = dist.new_group([i, j])
            if rank in [i, j]:
                bw = measure_ar_bandwidth(group)
                results[(i, j)] = bw

    # Report anomalies
    intra_node = [bw for (i, j), bw in results.items()
                   if i // 8 == j // 8]
    inter_node = [bw for (i, j), bw in results.items()
                  if i // 8 != j // 8]
    logger.info(f"intra-node AR: median={median(intra_node):.0f} GB/s")
    logger.info(f"inter-node AR: median={median(inter_node):.0f} GB/s")

    for pair, bw in results.items():

```

```

expected = 300 if pair[0] // 8 == pair[1] // 8 else 100
if bw < expected * 0.7:
    logger.warning(f"pair {pair} slow: {bw} GB/s (expected {expected})")

```

Patch 位置 : megatron/training/initialize.py::initialize_megatron (在 model init 之前)

收益 : 训练开始前发现慢 node 并 fail-fast , 避免训 1h 后才发现 straggler。Meta MegaScale 论文的 topology-aware placement 就是这套。

4.4 Loss / gradient anomaly detection

基线 : 无。用户肉眼看 tensorboard。

改造 : online anomaly detection (简单版本) :

```

class OnlineAnomalyDetector:
    def __init__(self, window=100):
        self.buffer = collections.deque(maxlen=window)

    def check(self, value, name):
        if len(self.buffer) < self.buffer.maxlen:
            self.buffer.append(value)
            return None

        median = statistics.median(self.buffer)
        mad = statistics.median([abs(v - median) for v in self.buffer])
        z_score = 0.6745 * (value - median) / (mad + 1e-8)  # MAD-based z

        self.buffer.append(value)

        if abs(z_score) > 6:
            return f"{name} anomaly: {value:.4f} (z={z_score:.1f})"
        return None

# in training loop
det = OnlineAnomalyDetector()
for step in range(...):
    loss = model(...)
    msg = det.check(loss.item(), "loss")
    if msg:
        alert_slack(msg)  # or pagerduty for grad

```

收益 : loss spike 出现 30 s 内自动 alert , 值班人可以在 2 分钟内决定是否 rollback / abort。

Trade-off : MAD-based z 对突然分布 shift (比如 curriculum 换) 也会 alert , 需要在 phase 切换时 reset buffer。

监控改造清单

改造点	基线	生产改造
Dashboard	loss 到 tensorboard	15+ 类指标 → Prometheus + Grafana
Straggler	无	gather per-rank step time + auto alert
Topology probe	无	启动前 nccl-tests 全对 + anomaly report
Anomaly detect	无	MAD-based z on loss / grad_norm
Alert	无 / stdout	Slack / PagerDuty 分级 (info / warn / page)
Post-mortem	无	spike batch 存盘 + step log 5 min 前后

五. Infra + 数据/系统接入改造

这一节是“能不能起手就能训 30 天”的地基。

5.1 RDMA / IBGDA / topology-aware placement

基线：Megatron 假设网络拓扑理想。生产集群 rack 层次、SU 层次会让某些 rank pair 更慢。

改造：

- 用 `nvidia-smi topo -m` 获取 intra-node topology
- 用 UFM (InfiniBand fabric manager) 拿 inter-node topology
- Rank 分配时把 TP group 放同 NVLink 域内，DP group 尽量跨 rack 分散 (fault tolerance)，PP group 沿 rack 排列 (P2P 局部化)

Patch 位置：`megatron/core/parallel_state.py::initialize_model_parallel`。默认按顺序切 rank，改成 topology-aware：

```
def initialize_model_parallel_topo_aware(tp, pp, dp):
    # Read topology
    topo = read_gpu_topology() # {rank: {"node": N, "rack": R, "su": S}}
    # Group ranks so that:
    #   TP ranks share same node
    #   PP ranks are within same rack (P2P locality)
    #   DP ranks span racks (fault isolation)
    tp_groups = group_by_node(topo, size=tp)
    pp_groups = interleave_racks(tp_groups, size=pp)
    dp_groups = ...
```

收益：Meta MegaScale 报告 topology-aware placement 让 8K H100 集群通信 -20%，故障 blast radius 减半。

5.2 Elastic training

基线：Megatron 无 elastic。一个 node 挂，整个 job 挂，需要 SLURM 重排。

改造：`torchrun --max-restarts=100 --standalone` + 自定 rendezvous backend：

- Node 挂 → torchrun 检测到 heartbeat 丢失 → 剩余 node 重连 rendezvous

- 从最新 L1/L2 checkpoint 恢复
- 若 hot spare 可用, 自动 include
- 若无 spare, reshape (DP-1) 继续训 (需要 gradient accumulation 补偿)

Patch 位置: `torchrun` wrapper + `megatron/training/initialize.py` 加 dynamic mesh support。

收益: Meta MegaScale 报告 90% 硬件故障可以 elastic 恢复 (无人工), mean-time-to-recovery 从 20 min → 3 min。

Trade-off: 需要 checkpoint 支持 reshape (DCP sharded checkpoint 才行, `torch.save` 的老 ckpt 不支持)。

5.3 Resumable dataloader

基线: Megatron `--data-path` + `--split` 假设从头开始或从 step N 精确恢复。dataloader RNG state 存 checkpoint。

改造:

- **Streaming**: 数据在 S3 / OSS, 边读边训, 不预下载
- **Fault tolerant**: 某个 shard 读失败 → skip 并 alert, 不阻塞训练
- **Resumable at any step**: checkpoint 里存 `(shard_id, offset, RNG_state)`, 恢复精确到 sample 级
- **Multi-source mixing**: 不同数据源按 weight 采样, weight 可以中途改 (curriculum)

```
class StreamingResumableLoader:
    def __init__(self, shards, weights, seed):
        self.shards = shards
        self.weights = weights
        self.rng = numpy.random.default_rng(seed)
        self.cursor = {s: 0 for s in shards}

    def __iter__(self):
        while True:
            source = self.rng.choice(len(self.shards), p=self.weights)
            shard = self.shards[source]
            try:
                sample, self.cursor[shard] = read_next(shard, self.cursor[shard])
            except IOError:
                _shard_error_counter[shard].inc()
                continue # skip broken shard, don't block
            yield sample

    def state_dict(self):
        return {"cursor": self.cursor, "rng_state": self.rng.bit_generator.state}
```

Patch 位置: 新建 `megatron/data/streaming_dataset.py`。

收益:

- 训练不受磁盘故障影响
- 从任意 step 精确恢复
- 支持“训到 100B token 时换 data mix”这种在线调整

5.4 Determinism + resumability 的 audit

改造：每 N step 做一次 self-audit：

- 从 checkpoint 恢复 (in-memory)
- 用相同 batch 跑一步
- 比较 loss / grad_norm 是否与原始 run 一致
- 不一致就报警——checkpoint 有 bug

```
if step % 1000 == 0 and step > 0:
    # Snapshot current state
    snap_model = deepcopy(model.state_dict())
    snap_optim = deepcopy(optim.state_dict())
    orig_loss = last_loss

    # Restore from checkpoint, replay one step
    model.load_state_dict(load_ckpt(step - 1)["model"])
    replayed_loss = model(same_batch).item()

    if abs(replayed_loss - orig_loss) > 1e-4:
        alert(f"checkpoint {step - 1} does not replay same loss "
              f"({orig_loss} vs {replayed_loss})")

    # Restore live state
    model.load_state_dict(snap_model)
    optim.load_state_dict(snap_optim)
```

收益：checkpoint bug（常见：优化器状态未同步、RNG 未存全）提前发现，避免“训完发现无法 continual pretrain”。

六. RL 训练栈上的改造

如果你的岗位覆盖 post-training，“你们框架改了什么”这个问题会再问一遍，只不过基线从 Megatron 换成了 verl / OpenRLHF —— 而这两个框架本身就是搭在 Megatron / FSDP 上的，所以前五节的改造在 RL 栈里依然成立，只是上面又叠了一层专属的坑。

这一节按同样的四个维度组织。原理和背景在第 15 章，这里只列“基线给了什么 / 你还能加什么”。

6.1 引擎层：rollout 引擎与权重同步

基线：verl / OpenRLHF 给你 vLLM 或 SGLang 后端、NCCL cross-group weight broadcast、基本的 agent loop。

高频改造：

- **Prefix cache pinning**。多轮 agentic rollout 里，序列等环境执行那几秒会被 LRU 淘汰 KV，回来重新 prefill。给“等待中”的序列加 pin 或换出到 CPU，命中率能从 40% 级拉到 90% 级。改的是推理引擎的淘汰策略，不是 RL 框架本身。
- 权重同步路径。默认 NCCL broadcast 每轮几百 ms。同机部署时可以走 CUDA IPC 共享 buffer 做零拷贝；跨机则把 broadcast 与下一轮 rollout 的 prefill 重叠。
- 轨迹级连续批处理。把 continuous batching 从 token 粒度抬到轨迹粒度，让等环境的轨迹自动退出运行批次。这是 GPU 利用率从 19% 到 88% 的那一步。
- **FP8 rollout**。rollout 是 inference，对精度容忍度比训练高，量化到 FP8 能显著提吞吐；但要验证 rollout 与训练的 logprob 差距没有因此扩大（见 6.4 的监控项）。

6.2 verifier 与环境层：一个全新的 CPU 栈

基线：框架通常只给你一个 reward_fn 回调，剩下全是你的。

这一层基本没有基线可言，也因此最容易讲出个人贡献：

- 异步 verifier 池 + 每样本硬超时（子进程 + SIGKILL，不能用线程）
- hash(prompt, response) 结果缓存 + 组内去重，实测省 30-60% 调用
- 沙箱隔离：gVisor / Firecracker、断网、只读 FS、cgroup 限 CPU/内存/pid
- 环境失败分类：infra_failure 与 policy_failure 分开，前者剔除样本而不是给 reward 0
- 答案抽取器的加固：取最后一个 `\\boxed{}`、括号匹配而非正则、拒绝多答案

Key insight. 面试里如果你只讲得出 6.1，会被归类为“用框架的人”；能讲 6.2 才说明你真的建过 RLVR 的生产流水线——因为这一层开源框架给得最少，每个团队都得自己造，而每个自己造过的人都踩过同一批坑（sympy 挂死、模型改写测试文件、fork bomb）。

6.3 算法层：把论文里的修正接进来

基线：框架给的通常是原始 GRPO / PPO。

- DAPO 四件套 :clip-higher、dynamic sampling、token-level loss、overlong reward shaping
- Dr. GRPO 的两处去偏：去掉 $1/|o_i|$ 长度归一化与 $1/std$
- 去 KL（省掉 ref model 整个池），配合监控兜底
- 难度分层采样：维护每题历史准确率，优先采 0.2-0.8 区间
- partial rollout 的跨版本记账：限制轨迹最多跨 K 个 policy 版本

要点：这些都是“跟随论文改”，说的时候要讲清为什么在你的场景需要它。比如 dynamic sampling 不是白上的——它让每 iteration 的 rollout 量变成动态的，调度器要支持流式补采，这是实打实的工程改动。

6.4 训练层与监控：RL 专属的那几个指标

训练层最重要的是 **token masking**（观测 token 不进 loss，归一化分母用 `mask.sum()`），细节见第 15 章 4.1/4.2——那是本书里“最容易写错且最不容易发现”的一处。

监控上，预训练那套（loss / grad_norm / MFU）在 RL 里远远不够，得再加一组：

指标	为什么要看	健康值
epoch-0 裁剪比例	rollout 与训练引擎 logprob 是否自洽	≈ 0
prefix cache 命中率	多轮 rollout 是否在重复 prefill	$> 85\%$
verifier 队列深度	CPU 池是否成为瓶颈	不持续增长
verifier 超时率	是否有病态输出把沙箱卡死	$< 1\%$ ，突增即告警
有效组占比	多少 rollout 产生了真实梯度	$> 50\%$
policy 熵	是否在走向熵坍塌	不单调下降
工具调用成功率	agentic 场景的真实业务指标	随训练上升
infra 失败率	环境侧 SLO	$< 1\%$
平均轮数 / 截断率	轨迹是否在被预算腰斩	截断率 $< 10\%$

表 50 RL 训练相比预训练需要额外监控的九项指标。前两项最能体现深度：多数人不知道 epoch-0 的裁剪比例应该是 0，也不知道 prefix cache 命中率会悄悄掉到 40%。

Warning. RL 里最危险的监控习惯是盯 **loss**。Agentic RL 的 loss 下降几乎不能说明任何事—— token masking 写错时 loss 照样平滑下降，同时 agent 正在退化成“不调工具直接编答案”。必须以任务成功率、工具调用成功率、平均轮数这些业务指标为准。

RL 栈改造清单

改造点	基 线 (verl / OpenRLHF)	生产改造	典型收益
Rollout 调度	同步批式 / 基础 agent loop	轨迹级连续批处理	GPU 利用率 19%→88%
Prefix cache	引擎默认 LRU	等待中序列 pinning / CPU 换出	命中率 40%→90%
长尾轨迹	等最长那条	partial rollout + 跨版本记账	makespan -40%+
权重同步	NCCL broadcast	CUDA IPC 零拷贝 / 与 prefill 重叠	每轮省数百 ms
Verifier	一个 reward_fn 回调	异步池 + 超时 + cache/去重 + 沙箱	调用量 -30~60%
环境失败	算作 reward 0	infra_failure 剔除 + SLO 告警	去掉训练信号噪声
RL 算法	原始 GRPO	DAPO 四件套 + Dr. GRPO 去偏	有效组占比翻倍
Ref model	常驻一个池	去 KL，显存转给 KV cache	省一个 70B 池
训练侧	单轮假设	轨迹 token masking + mask.sum() 归一化	修复“越训越差”
监控	loss / reward 曲线	九项 RL 专属指标（见上表）	问题从天级到分钟级

表 51 RL 训练栈的十类改造。与前五节的区别在于：verifier 与环境这一层开源框架几乎没有基线，是最容易做出个人贡献、也最容易在面试里讲出深度的地方。

面试考点。面试深挖：“RL 训练你们用 verl，那 verl 都做好了，你做了什么？”

这问题和开篇那个 Megatron 版本是同一道题。可信的回答结构：

1. 先指出 **verl** 没做的那一层：verifier 池、沙箱隔离、环境失败分类 —— 框架只给一个 reward_fn 回调，生产流水线是自己搭的。
2. 再讲一个引擎层的深挖：比如 prefix cache 命中率只有 40% 的排查过程，从“为什么开了 cache 还是慢”到“等环境时 KV 被 LRU 淘汰”到“扩 KV 池 + pinning”。
3. 再讲一个正确性 **bug**：token masking 或 numel() 归一化，说清“loss 正常但业务指标下降”这个信号是怎么锁定它的。
4. 最后给数字：iteration 时间、GPU 利用率、任务成功率各降/升了多少。

第 15 章第八节有一个完整的 STAR 版本可以直接借鉴结构。

七. 完整 STAR 故事：把四个维度串起来

Situation (30 s)：

“我们上一个模型是 32B dense LLM ,在 512 张 H100 上从 Meta Llama-2 baseline 起训。跑起来第一版是 Megatron 官方 recipe : TP=4, PP=8, DP=16, BF16 mixed, SelectiveRecompute。MFU 大约 34% ,比 Meta 报告的 42% 差 8 个点 ;同时前 200 步有过 3 次 loss spike ,每次都要手动 restart , 7 天训了 500B token ,团队想 push 到 3 T token 需要 6-7 周。”

Task (10 s) :

“我负责在两周内把 MFU 提到 45%+ ,并让训练能 7×24 无人值守 push 到 3T token。”

Action (2 min) :

“我按四个维度动手 :

算子层 :跑 nsys 一步 profile ,发现三处 :

- GEMM 里 RMSNorm + Linear + RoPE 是三个独立 kernel ,launch overhead 8%。写了 Triton fused kernel(RMSNorm var 强制 FP32 upcast),单元测试对齐 unfused reference。
- TE FA2 kernel 在 seq=8K 时 tail 长 ,升级到 FA3 后 +12% MFU。加 shape guard ,头 100 步 dtype/head_dim 不匹配就 fallback 到 FA2 ,count fallback 到 dashboard。
- DDP grad AR bucket=25 MB , 200 个 bucket → 6 ms/step launch overhead。改成 200 MB + `gradient_as_bucket_view=True` , overlap 60% → 92%。

算法层 :

- 观察前几步 grad_norm 波动大 , `beta_2=0.999` (Megatron 老默认) → 改 0.95 ,warmup 从 500 → 2000 步。这是标配但 baseline recipe 没跟上。
- 数据侧上 BFD packing (原来 fixed-length , padding 15%) → padding 2% ,等效 +13% throughput。

稳定性 :

- 3 次 loss spike 后加了 auto-skip : `grad_norm > 10× 20-step median` 就 skip 该 step 并存 poisoned batch。之后再没手工重启。
- Checkpoint 从同步 5 min block 改成 async + L1 (peer node CPU DDR, 3 s) + L2 (local NVMe, 30 s)。in-flight NaN 用 L1 rollback 200 ms 恢复 ,硬故障用 L2 20 s 恢复。

监控 + Infra :

- Prometheus + Grafana dashboard ,15 类指标。straggler detect 每 10 步 gather per-rank time ,慢 30% 的 rank alert。启动阶段跑 nccl-tests 全对 ,一次 fail-fast 掉了 2 张有问题的卡。
- topology-aware rank 分配 : TP 组 pin 同 node , PP 组沿 rack 排列。inter-rack AR 降 15%。
- torchrun elastic + max-restarts=100 ,一次真实硬件故障 3 分钟自动恢复。”

Result (20 s) :

“MFU 34% → 46% , push 到 3T token 从 6-7 周 → 4 周。训练期间 12 次硬件故障全部 elastic 恢复 , 0 次手动 restart。团队把这套改造 (Triton fused kernel + skip step + L1 backup + straggler detect) 合并到内部 baseline recipe ,后续 2 个模型无需重新做这些活。”

面试官可以从任何一句往下追问细节 —— 每个环节都得能 hold 住。这就是为什么这一附录写得深 :你不能只知道“我们改了 kernel” ,得知道 fused 时哪一步要 FP32 upcast、fallback 怎么写、shape guard 怎么测。

八. 面试 anti-pattern : 什么情况不能这么答

不能把四个维度全都说“我做过” ,除非你真做过。可信的分工 :

- 一个 IC :一般深度覆盖 1-2 个维度 (比如算子 + 监控), 其他维度“知道 & review 过 patch”

- **tech lead** : 4 个维度都 review 过，但只有 1-2 个亲手写
- **manager** : 知道每个维度是谁做的、trade-off 是什么，但不写代码

一定要老实：某个维度不熟就说“这块是我 team 里另一位同学做的，我知道大概做了 X 但细节不敢瞎讲”——这比“我啥都会”加分得多。

不能说的话：

- “我们把 Megatron 改了很多”（无具体）
- “我实现了 XX”（如果实际上是抄开源）
- “MFU 从 30 提到 60”（数字过于夸张，业界 dense LLM 顶到 50% 就很高）
- “我训了 XX B model”（模型大小是资源不是贡献）
- “我们没遇到过 loss spike”（不真实；spike 是常态，重点是怎么处理）

一定要说的话：

- 具体的 patch 位置（哪个文件、哪个函数）
- 具体的 trade-off（为什么这么改，别的方案为什么不行）
- 具体的数字（MFU、step time、恢复时间）
- 具体的 debug 故事（怎么发现问题的）

九. 一页速记：改造维度总表

维度	高频改造点（面试前至少能说出 2 个 + 1 个 trade-off ）
算子层	Triton fused (RMSNorm+Linear+RoPE) · FA3 + shape-guard fallback · Grouped GEMM · DeepEP a2a (IBGDA) · CUDA graph capture (varlen bucket)
算法层	Muon (Kimi K2) · aux-loss-free MoE (DeepSeek-V3) · MTP head · MLA · BFD sequence packing · WSD + mini-restart LR
稳定性	auto-skip spike (grad_norm 10× median) · NaN in-memory rollback · per-layer grad clip · L1/L2/L3 三级 checkpoint · deterministic mode 可 debug
监控 + Infra	15+ metric Prometheus dashboard · straggler detect (per-rank step time) · nccl topology probe · elastic + <code>--max-restarts</code> · streaming resumable loader
RL 栈 (§六)	轨迹级连续批处理 · prefix cache pinning · partial rollout 跨版本记账 · 异步 verifier 池 (超时/cache/去重/沙箱) · <code>infra_failure</code> 剔除 · DAPO 四件套 · token masking + <code>mask.sum()</code> · epoch-0 裁剪比例监控

表 52 Megatron 与 RL 栈改造速记。面试前一晚翻这张表，每个维度至少能说出 2 个具体点 + 1 个 trade-off。做 post-training 的岗位重点看最后一行。

最后一句话：Megatron 和 verl 都是 baseline，不是终点。所有开源框架都留有大量“生产集群才知道要改”的空白——你的贡献就是填这些空白，而 RLVR 的 verifier 与环境那一层空白最大。面试里能把自己相关的那几个维度都举出具体例子（哪怕不都是自己写的），就已经把自己从“tuner”升到“framework builder”了。